

Prepared By : Ravi



Source: spark.apache.org & www.databricks.com

What is data and different types?

Un-Structured Data

```
department number 10 department name ACCOUNTING
department location NEW YORK
department number 20 department name RESEARCH
department location DALLAS
department number 30 department name SALES
department location CHICAGO
department number 40 department name OPERATIONS
department location BOSTON
```

Semi-Structured Data

```
{"DEPTNO": "10", "DNAME": "ACCOUNTING", "LOCATION": "NEW YORK"}
{"DEPTNO": "20", "DNAME": "RESEARCH", "LOCATION": "DALLAS"}
{"DEPTNO": "30", "DNAME": "SALES", "LOCATION": "CHICAGO"}
{"DEPTNO": "40", "DNAME": "OPERATIONS", "LOCATION": "BOSTON"}
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<ROWS>
  <ROW>
    <DEPTNO>10</DEPTNO>
    <DNAME>ACCOUNTING</DNAME>
    <LOCATION>NEW YORK</LOCATION>
  </ROW>
  <ROW>
    <DEPTNO>20</DEPTNO>
    <DNAME>RESEARCH</DNAME>
    <LOCATION>DALLAS</LOCATION>
  </ROW>
  <ROW>
    <DEPTNO>30</DEPTNO>
    <DNAME>SALES</DNAME>
    <LOCATION>CHICAGO</LOCATION>
  </ROW>
  <ROW>
    <DEPTNO>40</DEPTNO>
    <DNAME>OPERATIONS</DNAME>
    <LOCATION>BOSTON</LOCATION>
  </ROW>
</ROWS>
```

Structured Data

	DEPTNO	DNAME	LOCATION
1	10	ACCOUNTING	NEW YORK
2	20	RESEARCH	DALLAS
3	30	SALES	CHICAGO
4	40	OPERATIONS	BOSTON

Unstructured data types

 Text files and documents	 Server, website and application logs	 Sensor data	 Images
 Video files	 Audio files	 Emails	 Social media data

SEMI-STRUCTURED



Databases and Files



What is Data & Metadata?

Youtube: TECHLAKE

Un-Structured Data

```
department number 10 department name ACCOUNTING
department location NEW YORK
department number 20 department name RESEARCH
department location DALLAS
department number 30 department name SALES
department location CHICAGO
department number 40 department name OPERATIONS
department location BOSTON
```

Semi-Structured Data

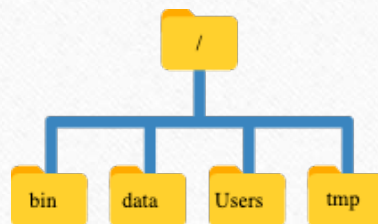
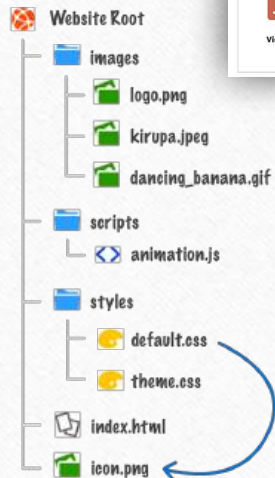
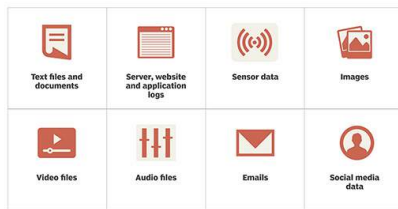
```
{"DEPTNO": "10", "DNAME": "ACCOUNTING", "LOCATION": "NEW YORK"}
{"DEPTNO": "20", "DNAME": "RESEARCH", "LOCATION": "DALLAS"}
{"DEPTNO": "30", "DNAME": "SALES", "LOCATION": "CHICAGO"}
{"DEPTNO": "40", "DNAME": "OPERATIONS", "LOCATION": "BOSTON"}
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<ROWS>
  <ROW>
    <DEPTNO>10</DEPTNO>
    <DNAME>ACCOUNTING</DNAME>
    <LOCATION>NEW YORK</LOCATION>
  </ROW>
  <ROW>
    <DEPTNO>20</DEPTNO>
    <DNAME>RESEARCH</DNAME>
    <LOCATION>DALLAS</LOCATION>
  </ROW>
  <ROW>
    <DEPTNO>30</DEPTNO>
    <DNAME>SALES</DNAME>
    <LOCATION>CHICAGO</LOCATION>
  </ROW>
  <ROW>
    <DEPTNO>40</DEPTNO>
    <DNAME>OPERATIONS</DNAME>
    <LOCATION>BOSTON</LOCATION>
  </ROW>
</ROWS>
```

Structured Data

	DEPTNO	DNAME	LOCATION
1	10	ACCOUNTING	NEW YORK
2	20	RESEARCH	DALLAS
3	30	SALES	CHICAGO
4	40	OPERATIONS	BOSTON

Unstructured data types



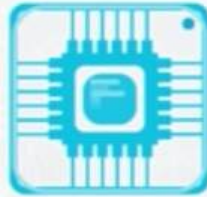
Database



Tables and cols



Fastest



CPU

200x faster than memory



MEMORY

15x faster than SSD



SSD

20x faster than network



NETWORK

Slowest



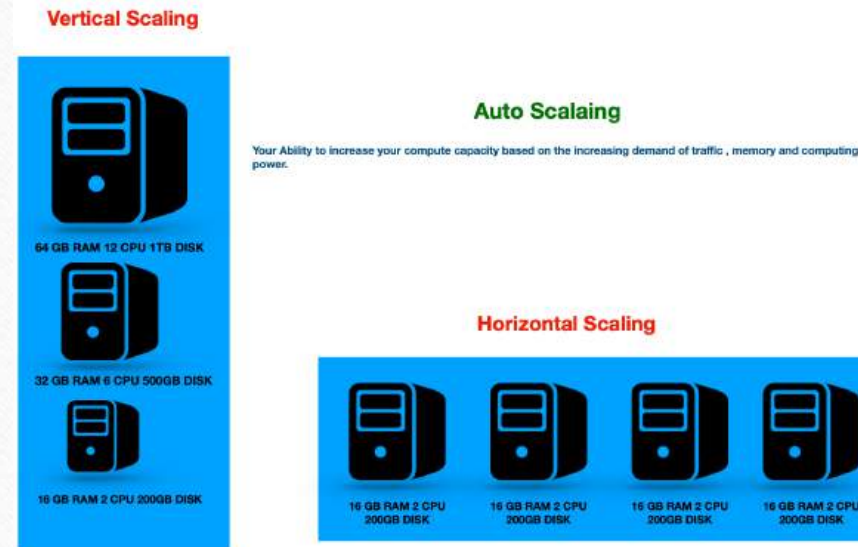
Data Transfer stats

Data size \ bandwidth	50 Mbps	100 Mbps	200 Mbps	500 Mbps	1 Gbps	10 Gbps
1GB	2.7 min	1.4 min	0.7 min	0.3 min	0.1 min	0.0 min
10GB	27.3 min	13.7 min	6.8 min	2.7 min	1.3 min	0.1 min
100GB	4.6 hrs	2.3 hrs	1.1 hrs	0.5 hrs	0.2 hrs	0.0 hrs
1TB	46.6 hrs	23.3 hrs	11.7 hrs	4.7 hrs	2.3 hrs	0.2 hrs
10TB	19.4 days	9.7 days	4.9 days	1.9 days	0.9 days	0.1 days
100TB	194.2 days	97.1 days	48.5 days	19.4 days	9.5 days	0.9 days
1PB	64.7 mo	32.4 mo	16.2 mo	6.5 mo	3.2 mo	0.3 mo
10PB	647.3 mo	323.6 mo	161.8 mo	64.7 mo	31.6 mo	3.2 mo

Online

Offline

Vertical Scaling Vs Horizontal Scaling

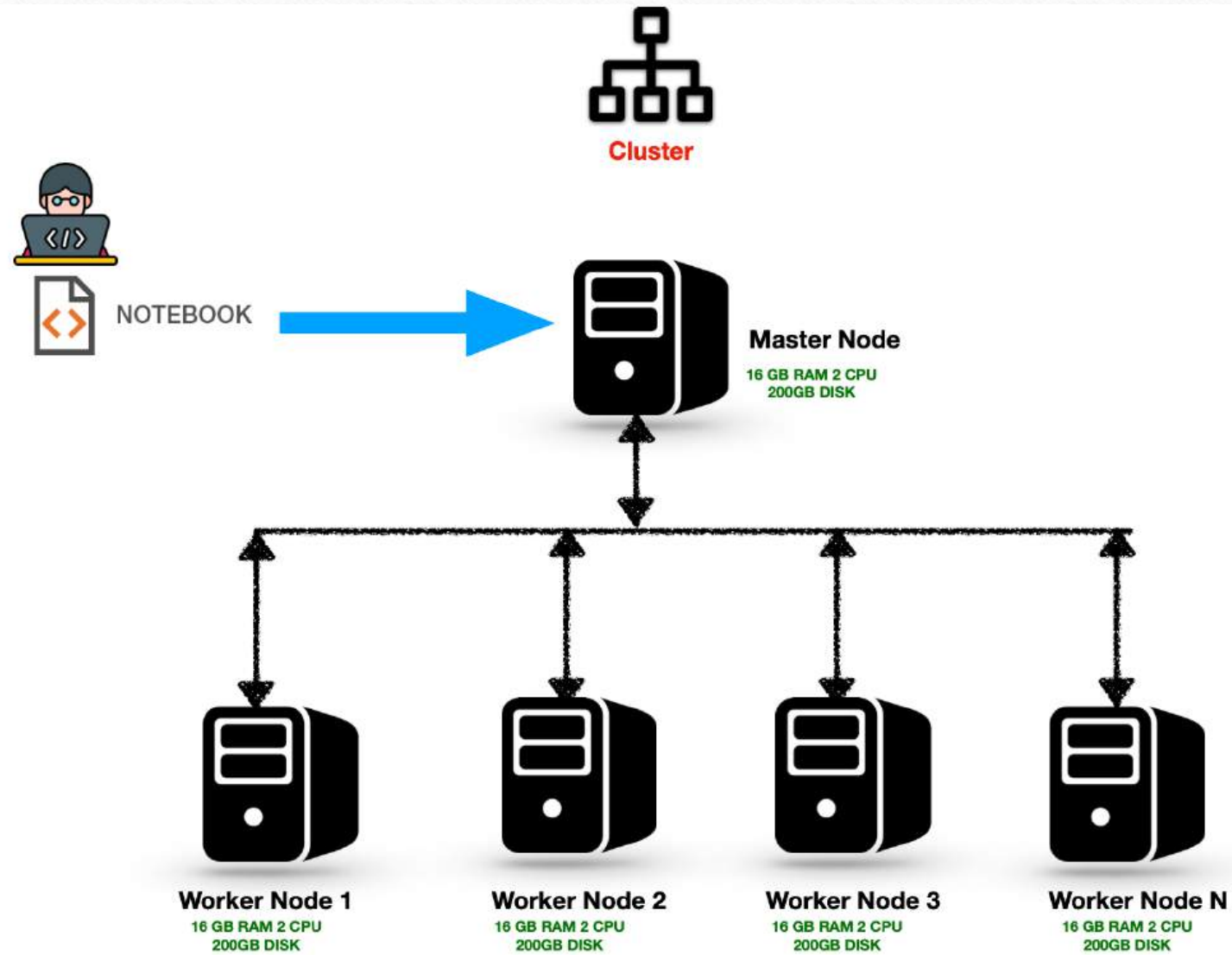


Vertical scaling (also known as scaling up) is the process of adding more resources to a single machine. This can include adding more CPU cores, RAM, storage, or network bandwidth. Vertical scaling is typically easier to implement than horizontal scaling, but it is also more limited. There is a physical limit to how much hardware can be added to a single machine.

Horizontal scaling (also known as scaling out) is the process of adding more machines to a system. This can be done by adding more servers to a load balancer, or by using a distributed computing framework such as Apache Spark or Hadoop. Horizontal scaling is more scalable than vertical scaling, but it can be more complex to implement.

Auto scaling in the cloud for horizontal scaling is the process of automatically adding or removing compute resources based on demand. This can be done in response to changes in traffic, CPU usage, memory usage, or other metrics. Auto scaling can help to improve performance and reliability, while also reducing costs.

All major cloud providers offer auto scaling features. For example, **Amazon Web Services (AWS)** offers Auto Scaling, **Microsoft Azure** offers Azure Autoscale, and **Google Cloud Platform (GCP)** offers Autoscaling.



CPU - CORES - THREADS

✓ What is CPU?

👉 Central Processing Unit (CPU) The CPU is the brain of a computer, containing all the circuitry needed to process input, store data, and output results. The CPU performs basic arithmetic, logic, controlling, and input/output (I/O) operations specified by the instructions in the program.

✓ What is Core?

👉 a core is a small CPU or processor built into a big CPU or CPU socket. It can independently perform or process all computational tasks. From this perspective, we can consider a core to be a smaller CPU or a smaller processor.

✓ What is Thread?

👉 Threads have become a vital part of the computing as they allow the processor to perform multiple tasks at the same time making the tasks faster.

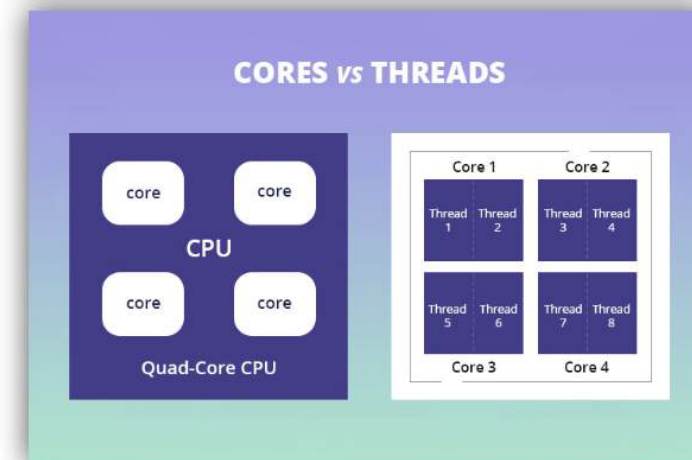
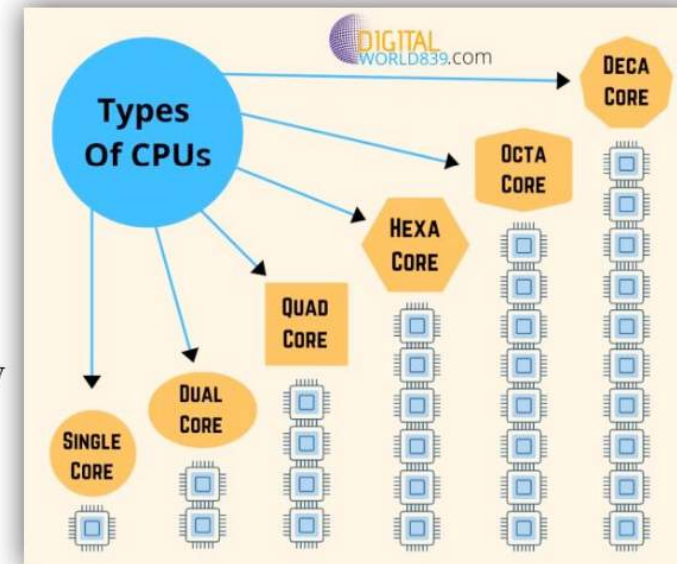
✓ What is vCPU?

👉 a vCPU represents a portion or share of the underlying, physical CPU that is assigned to a particular virtual machine (VM). Each vCPU is a thread of a CPU core

✓ What is SLOT?

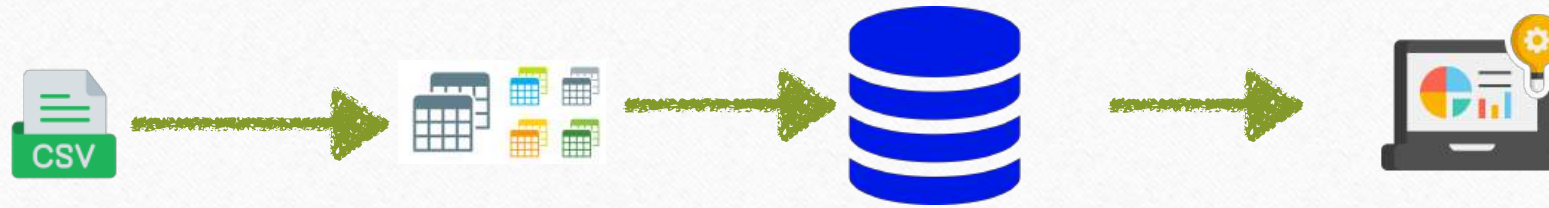
👉 These slots are classified as available threads on Driver Node/ Worker Node in Clusters. A Slot is equal to threads. No of Available threads for execution in executor is called slot.

👉 So if we have 4 available slots, then we can execute 4 tasks on those slots. like running 4 threads on 4 available slots.



Schema OnWrite Vs Schema On-Read

Schema On-Write is Less flexible, Less scalable, Less cost-effective, Typically performs better and manage better data quality. this option is better suitable for OLTP Databases, OLAP databases and Lakehouse architecture metadata API's.



Create Schema Before writing into database.

Schema On-Read is More flexible, More Scalable(datalakes are un-limited storage), More cost-effective ,Can impact performance and less data quality(need to depend on Lakehouse API to get better performance using Delta Lake/Hudi/IceBerg).



Store in Native format in Datalake

Create Schema While Reading from Data Lake

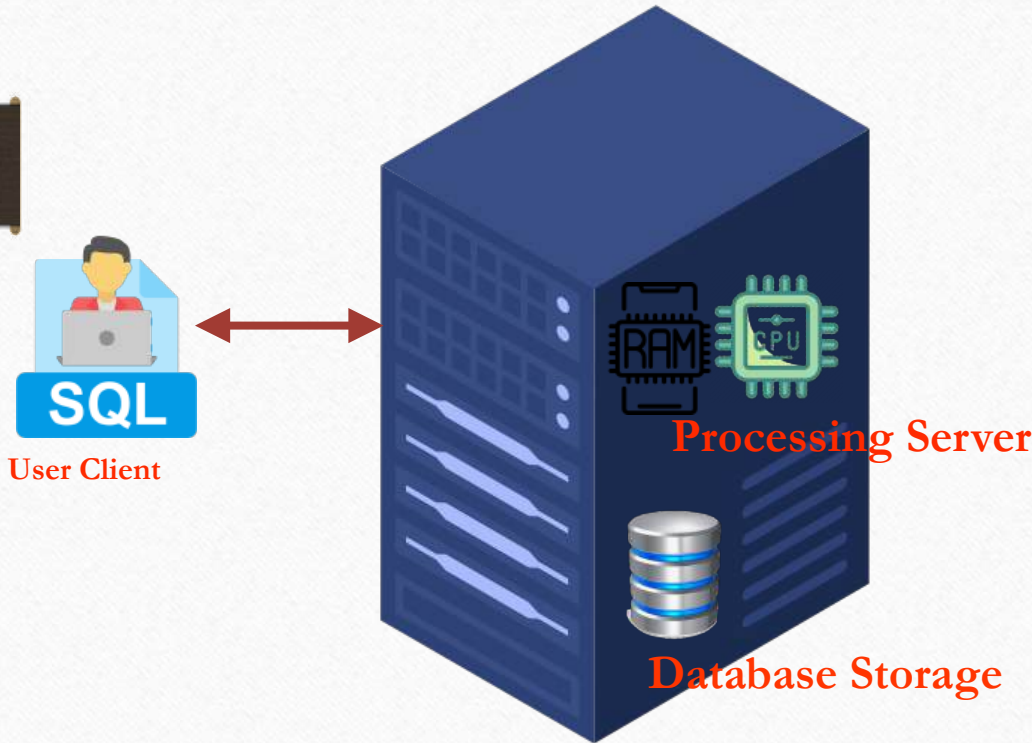


Coupled Storage and Compute

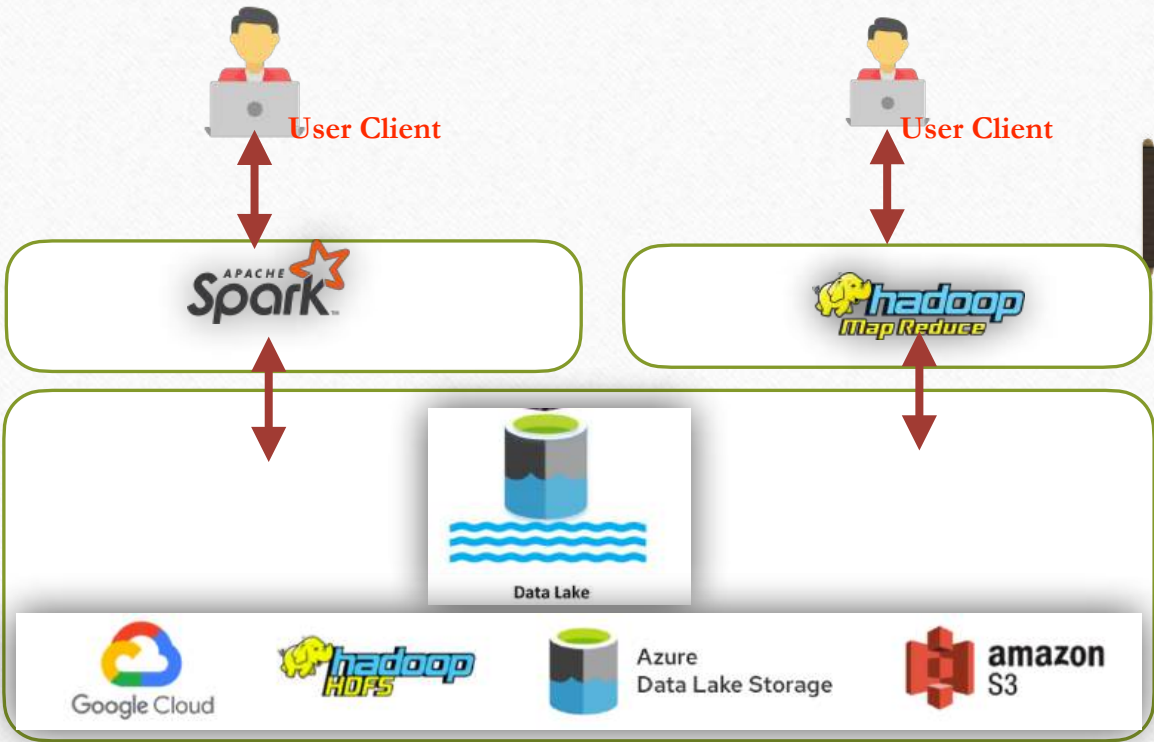


DeCoupled Storage and Compute

InBuilt Storage and Compute(Server/processing Engine)



Separate Storage and Separate Compute

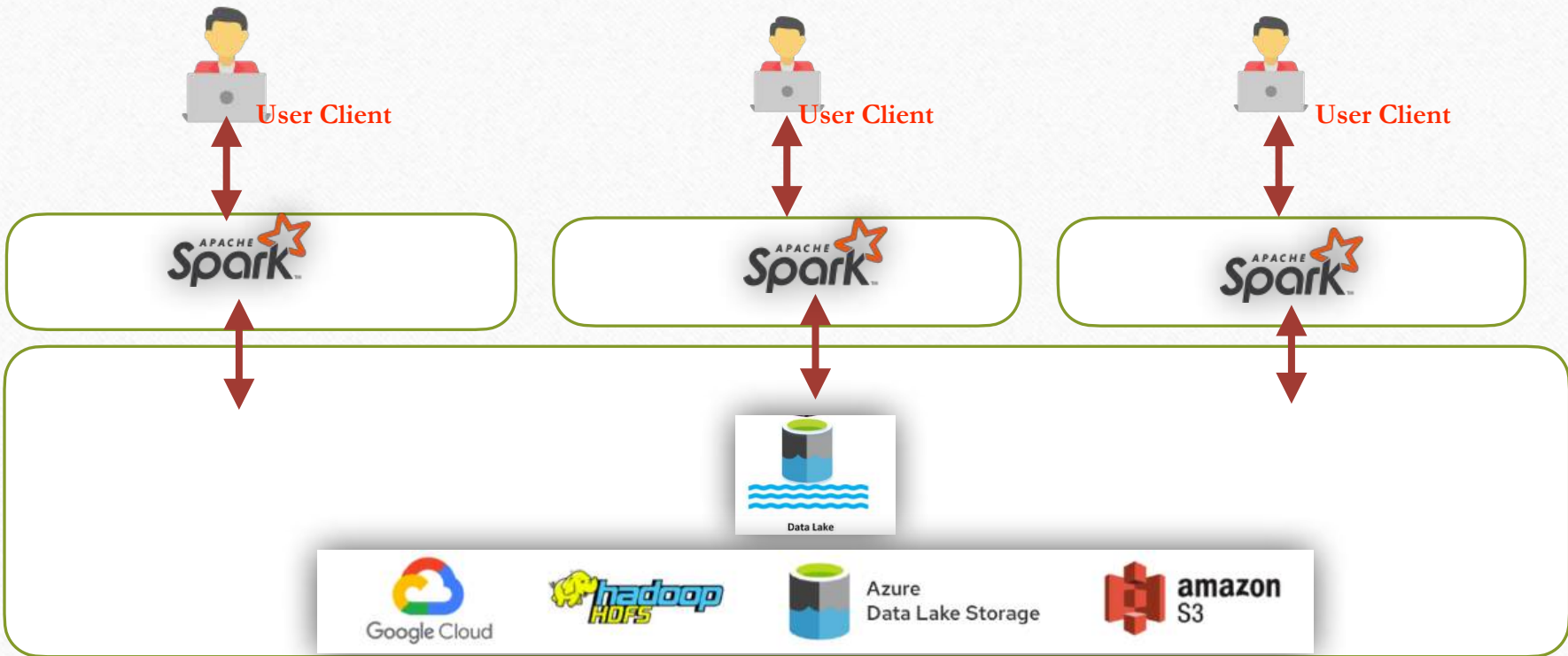


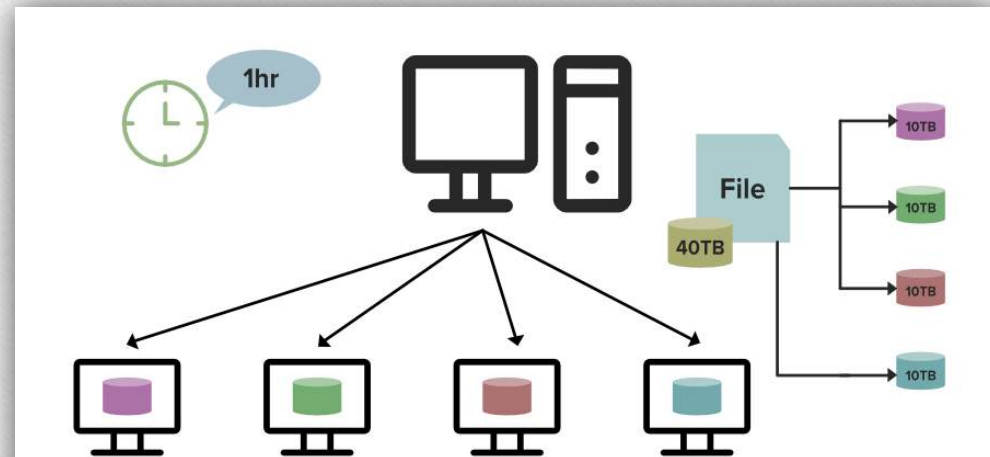
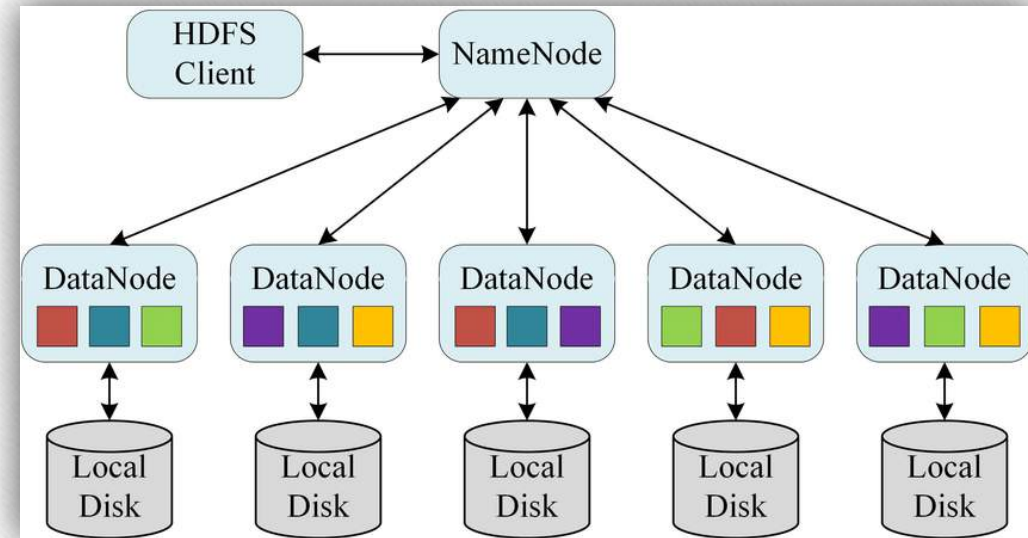
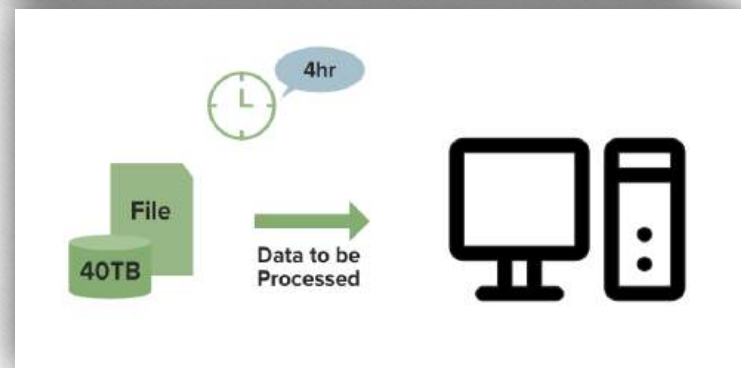
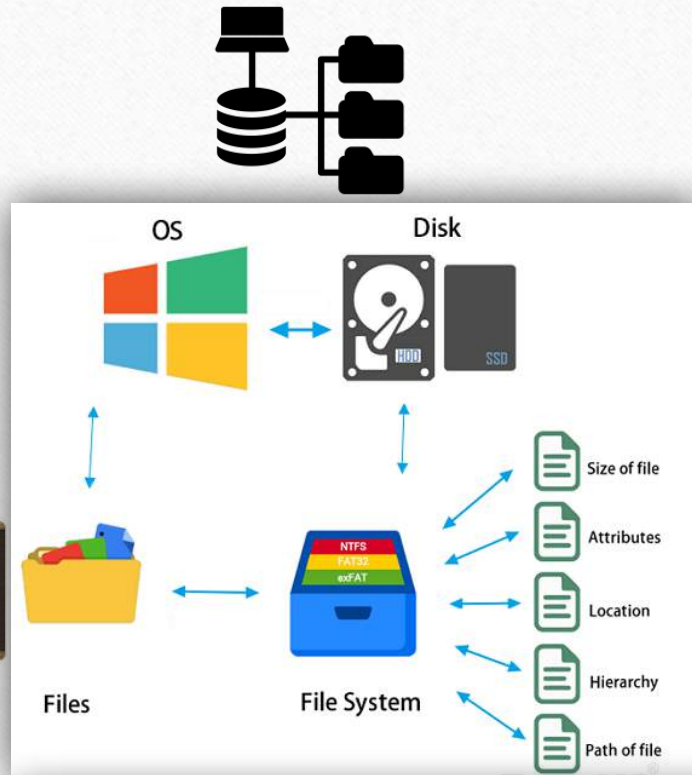


DeCoupled Storage and Compute



Separate Storage and Separate Compute





Databricks DBFS

Databricks File System

The Databricks File System (DBFS) is a distributed file system mounted into an Azure Databricks workspace and available on Azure Databricks clusters. DBFS is an abstraction on top of scalable object storage that maps Unix-like filesystem calls to native cloud storage API calls.

Databricks File System (DBFS)

Databricks File System (DBFS) is a distributed file system mounted into a Databricks workspace and available on Databricks clusters. DBFS is an abstraction on top of scalable object storage and offers the following benefits

- Allows you to **mount** storage objects so that you can seamlessly access data without requiring credentials.
- Allows you to interact with object storage using directory and file semantics instead of storage URLs.
- Persists files to object storage, so you won't lose data after you terminate a cluster.

DBFS root

The default storage location in DBFS is known as the *DBFS root*. Several types of data are stored in the following DBFS root locations:

- `/FileStore`: Imported data files, generated plots, and uploaded libraries. See [Special DBFS root locations](#).
- `/databricks-datasets`: Sample public datasets. See [Special DBFS root locations](#).
- `/databricks-results`: Files generated by downloading the [full results](#) of a query.
- `/databricks/init`: Global and cluster-named (deprecated) [init scripts](#).
- `/user/hive/warehouse`: Data and metadata for non-external Hive tables.

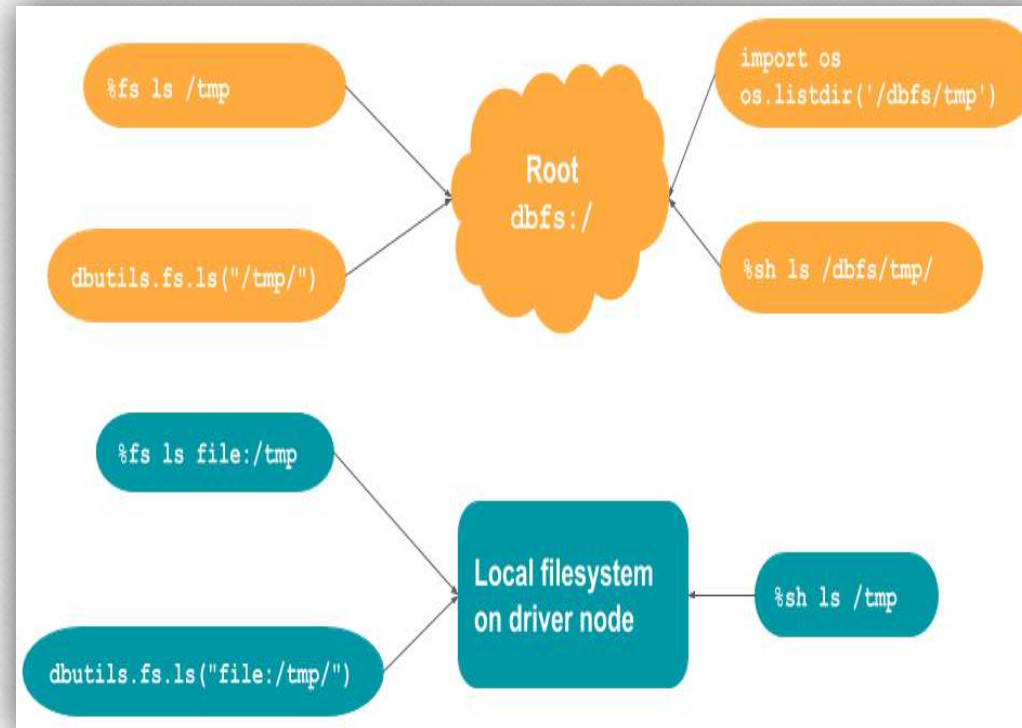
In a new workspace, the DBFS root has the following default folders:

path	name	size
dbfs/FileStore/	FileStore/	0
dbfs/databricks-datasets/	databricks-datasets/	0
dbfs/databricks-results/	databricks-results/	0
dbfs/tmp/	tmp/	0

What is the DBFS root?

The DBFS root is the default storage location for an Azure Databricks workspace, provisioned as part of workspace creation in the cloud account containing the Azure Databricks workspace. For details on DBFS root configuration and deployment, see the Azure Databricks quickstart.

Some users of Azure Databricks may refer to the DBFS root as “DBFS” or “the DBFS”; it is important to differentiate that DBFS is a file system used for interacting with data in cloud object storage, and the DBFS root is a cloud object storage location. You use DBFS to interact with the DBFS root, but they are distinct concepts, and DBFS has many applications beyond the DBFS root.



```
1 dbutils.fs.help()
```



dbutils.fs provides utilities for working with FileSystems. Most methods in this package can take either a DBFS path (e.g., `/foo` or `dbfs:/foo`), or another FileSystem URI. For more info about a method, use **dbutils.fs.help("methodName")**. In notebooks, you can also use the `%fs` shorthand to access DBFS. The `%fs` shorthand maps straightforwardly onto `dbutils` calls. For example, `%fs head --maxBytes=10000 /file/path` translates into `dbutils.fs.head("/file/path", maxBytes = 10000)`.

fsutils

cp(from: String, to: String, recurse: boolean = false): boolean -> Copies a file or directory, possibly across FileSystems

head(file: String, maxBytes: int = 65536): String -> Returns up to the first 'maxBytes' bytes of the given file as a String encoded in UTF-8

ls(dir: String): Seq -> Lists the contents of a directory

mkdirs(dir: String): boolean -> Creates the given directory if it does not exist, also creating any necessary parent directories

mv(from: String, to: String, recurse: boolean = false): boolean -> Moves a file or directory, possibly across FileSystems

put(file: String, contents: String, overwrite: boolean = false): boolean -> Writes the given String out to a file, encoded in UTF-8

rm(dir: String, recurse: boolean = false): boolean -> Removes a file or directory

- mkdirs() - For Creating Directory
- cp() - For copying files from source location to target location and both locations will have files.
- ls - listing files
- mv() - moving from source to target location. after moving only target location will have the file.
- put() - we can place any file in DBFS Location using put()
- rm() - removing files or folders - For multiple folders we need to use recursive True
- head() - reading header data from files..

Creating Directory using `mkdirs()`

```
cmd 11
```

```
1 dbutils.fs.mkdirs("/FileStore/Training_Databricks/")
```

ls listing files using `dbutils.fs.ls('path')`

Cmd 13

```
1 dbutils.fs.ls('/FileStore/tables/')
```

```
Out[5]: [FileInfo(path='dbfs:/FileStore/tables/5xml/', name='5xml/', size=0),
FileInfo(path='dbfs:/FileStore/tables/Samplepdf.pdf', name='Samplepdf.pdf', size=388044),
FileInfo(path='dbfs:/FileStore/tables/bigdata/', name='bigdata/', size=0),
FileInfo(path='dbfs:/FileStore/tables/departments.csv', name='departments.csv', size=170),
FileInfo(path='dbfs:/FileStore/tables/dept-1.csv', name='dept-1.csv', size=97),
```

Creating file using `put` in `dbutils.fs.put('filepath','use True for overwrite')`

Cmd 35

```
1 dbutils.fs.put("dbfs:/tmp/mydir/sample.txt","this is sample text file creation",True)
```

Wrote 33 bytes.

Out[14]: True

Command took 0.54 seconds -- by pysparktelugu@gmail.com at 11/18/2020, 9:43:46 AM on datacluster

Reading file using `head` method.

Cmd 37

```
1 dbutils.fs.head("dbfs:/tmp/mydir/sample.txt")
```

Out[15]: 'this is sample text file creation'

Command took 0.23 seconds -- by pysparktelugu@gmail.com at 11/18/2020,

Removing Directory or File using `rm` method in `dbutils.fs.rm('file or directory',True)`

- Recursive parameter `True` for deleting recursively subfolders and non empty folders

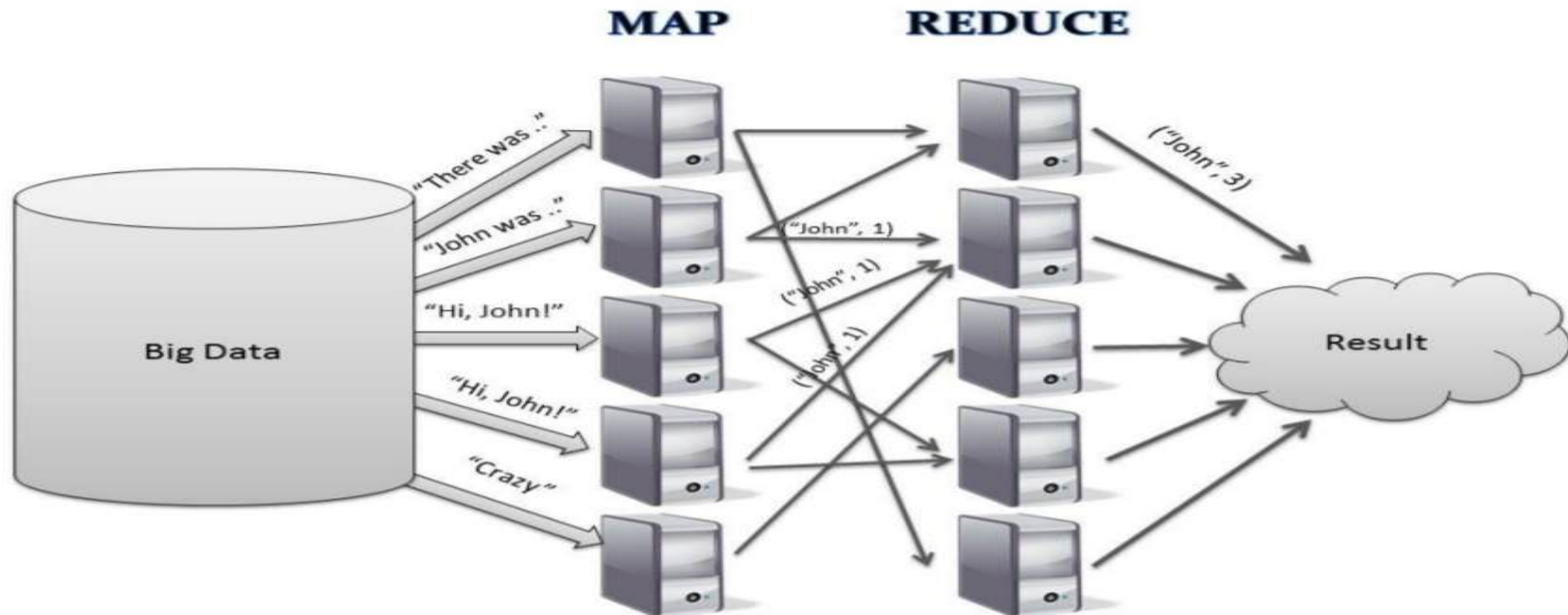
Cmd 40

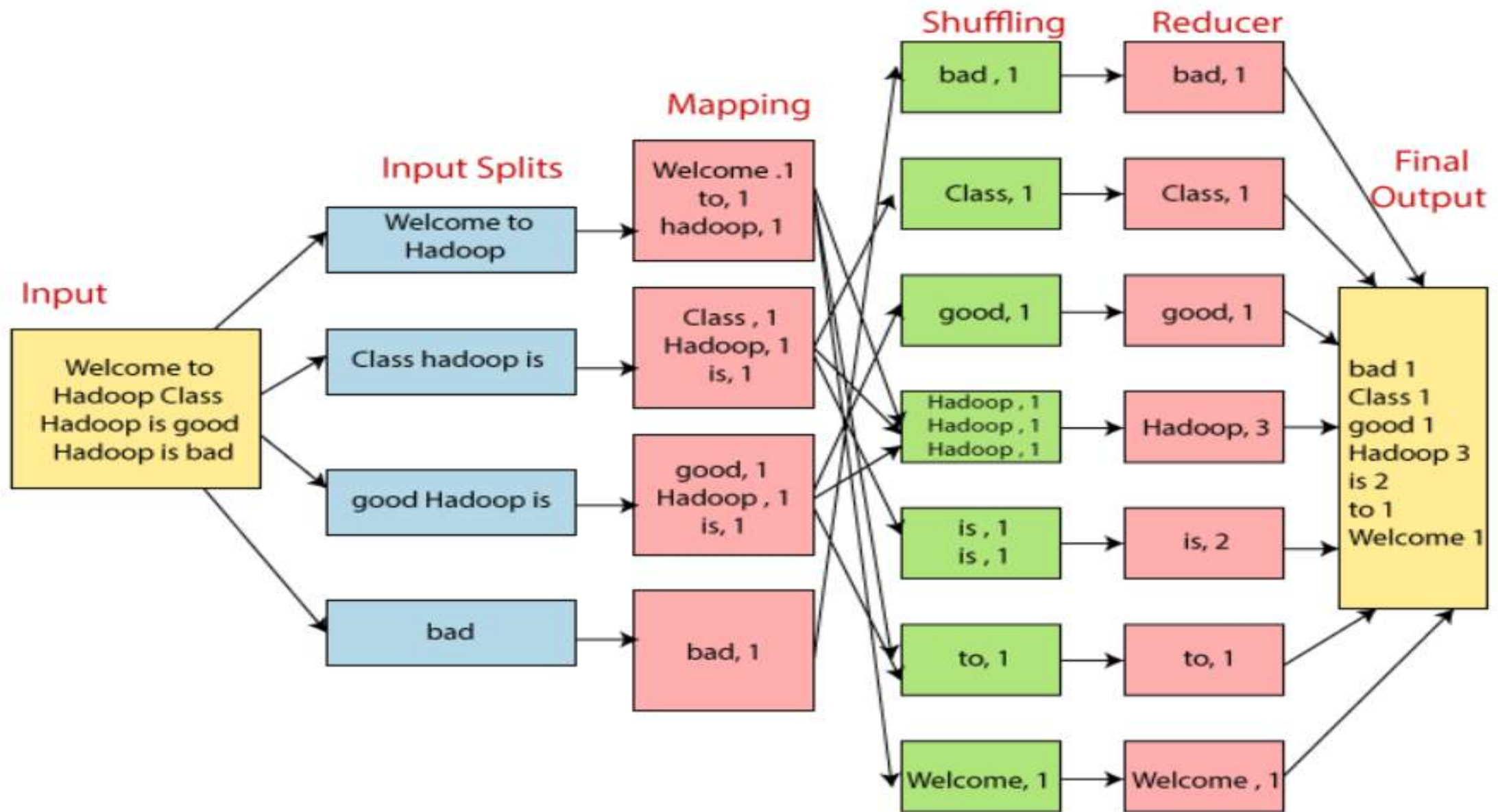
```
1 dbutils.fs.rm('dbfs:/tmp/mydir/',True)
```

Out[16]: True

Command took 1.66 seconds -- by pysparktelugu@gmail.com at 11/18/2020, 9:47:33 AM on datacluster

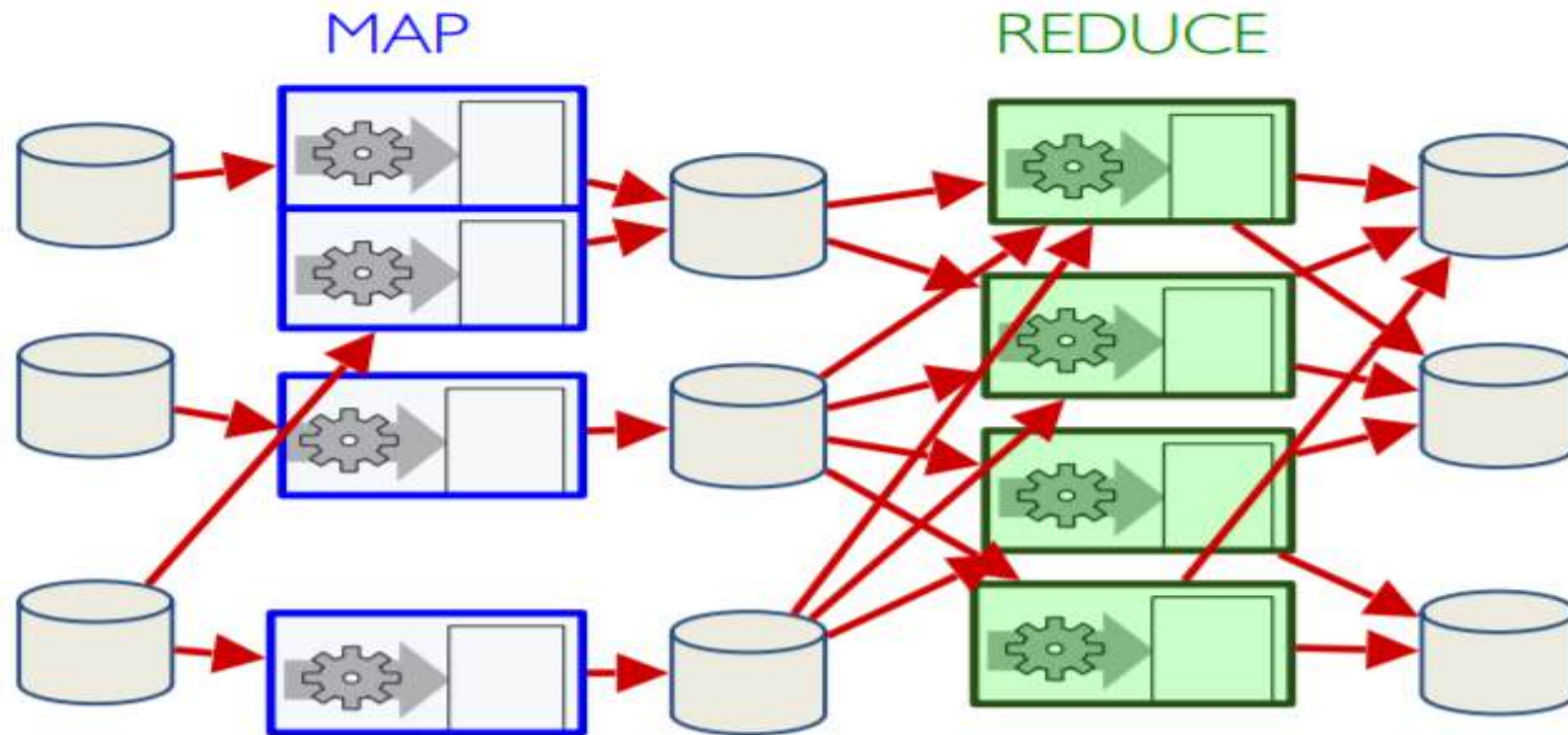
Map/Reduce





Why Spark?

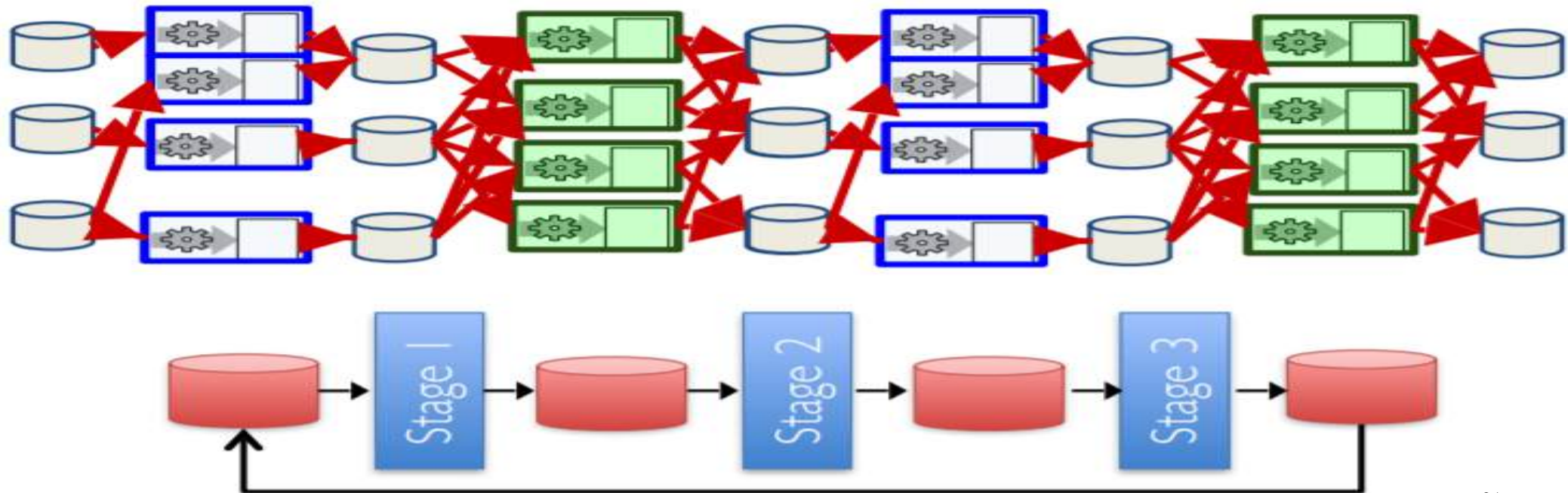
MapReduce Execution



Why Spark?

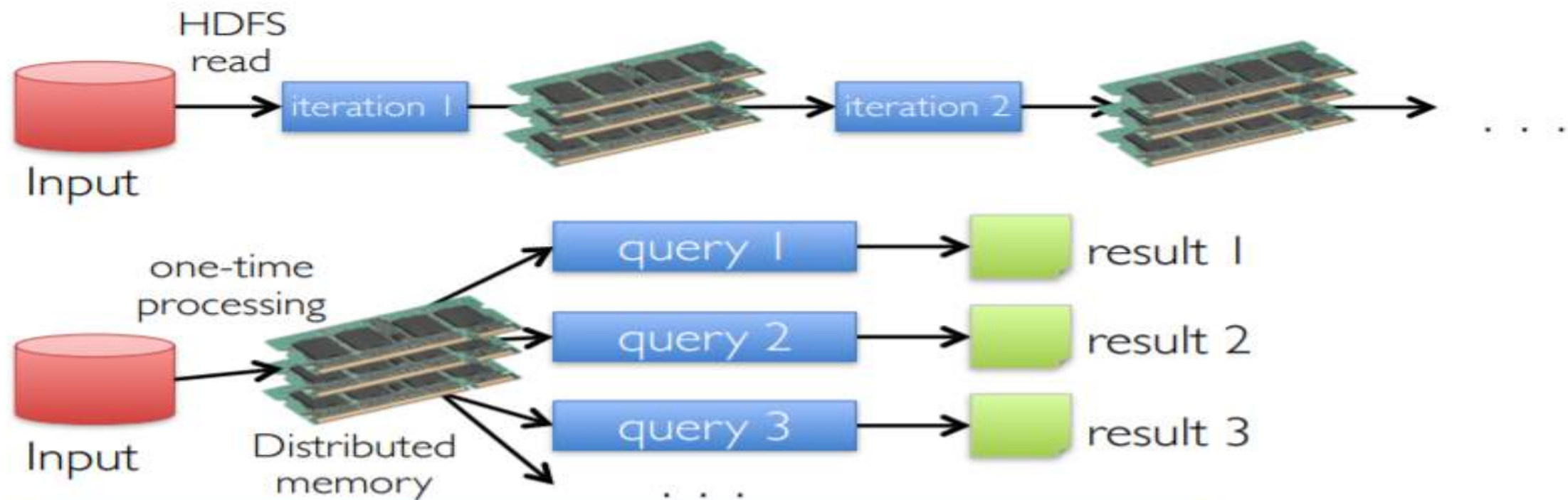
Iterative Jobs

- Disk I/O for each repetition
→ Slow when executing many small iterations



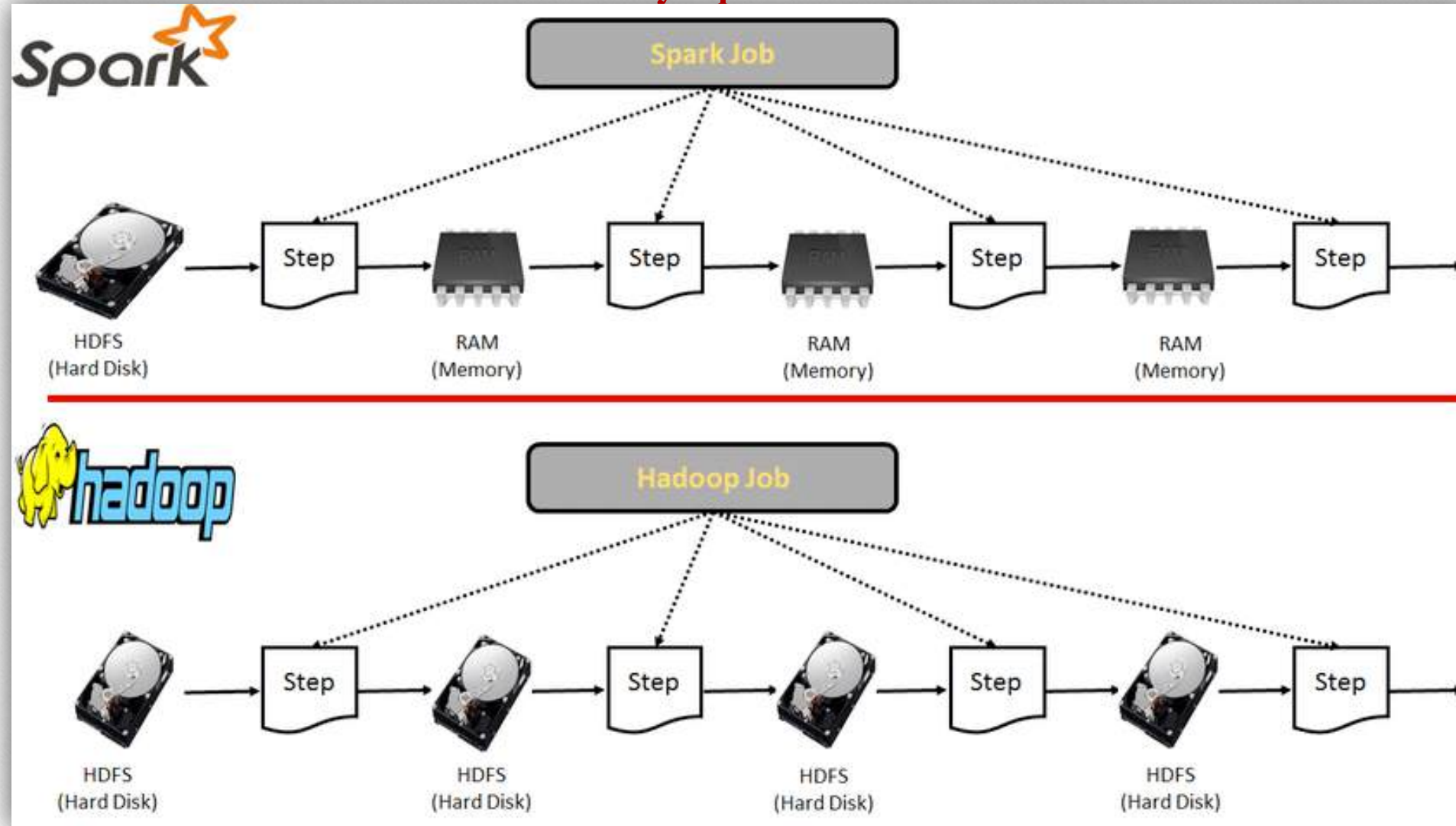
Replace Disk with Memory

In-Memory Data Sharing



10-100x faster than network and disk

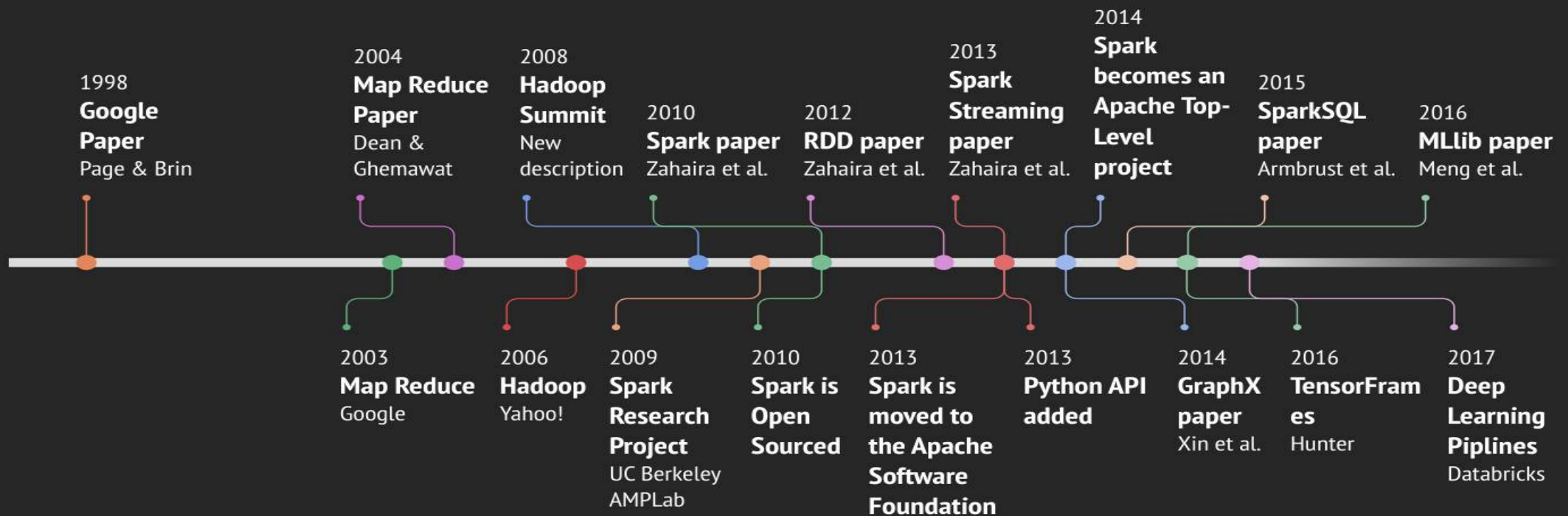
Why Spark?



World Record Data Process in 2014

	Hadoop MR Record	Spark Record	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400 physical	6592 virtualized	6080 virtualized
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	570 GB/s
Sort Benchmark Daytona Rules	Yes	Yes	No
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	virtualized (EC2) 10Gbps network
Sort rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Sort rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min

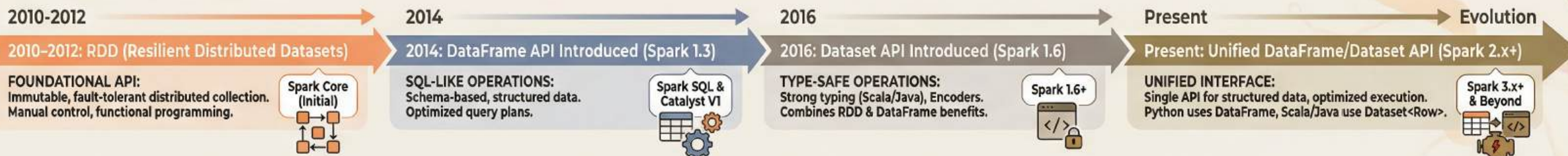
Apache Spark Timeline



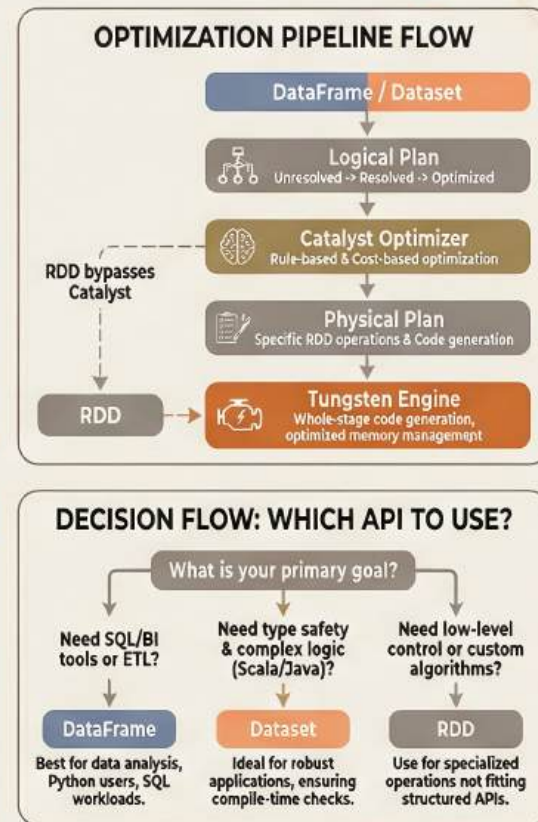
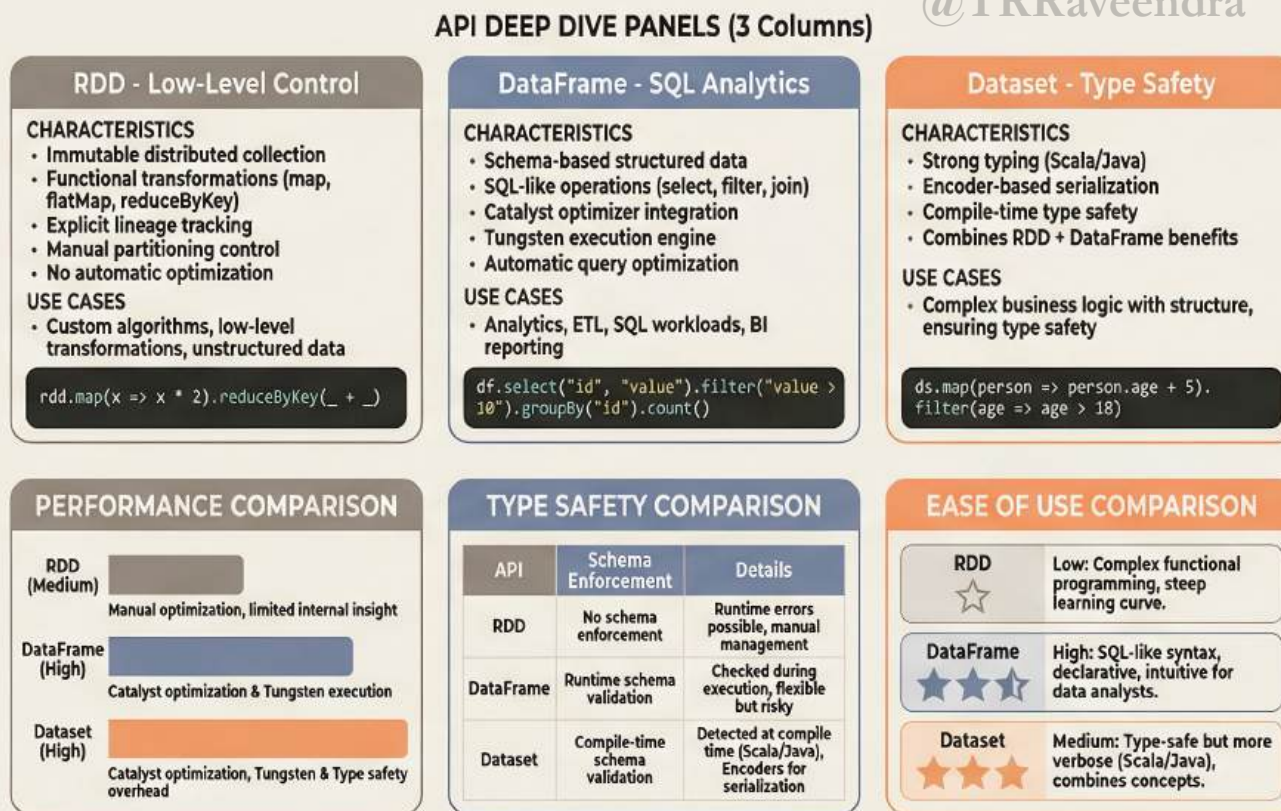
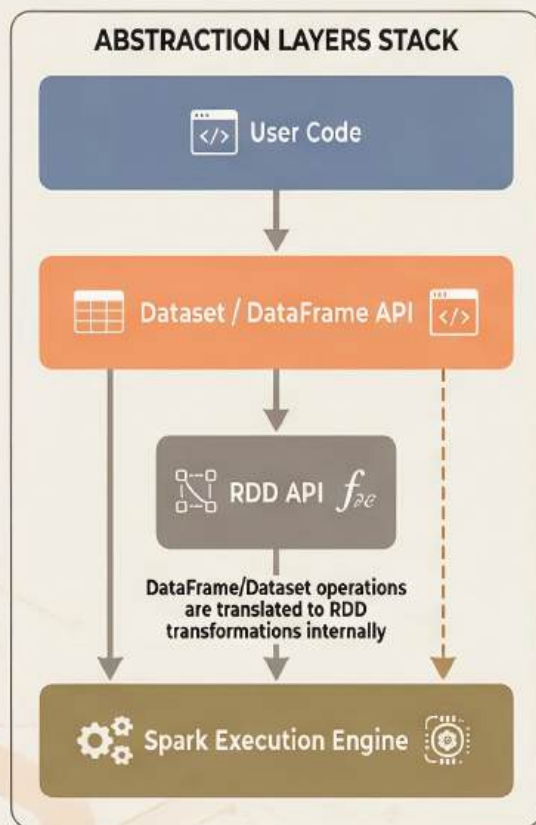
APACHE SPARK APIs EVOLUTION & ARCHITECTURE

A Journey from RDD to Unified DataFrame/Dataset in Spark's Execution Engine

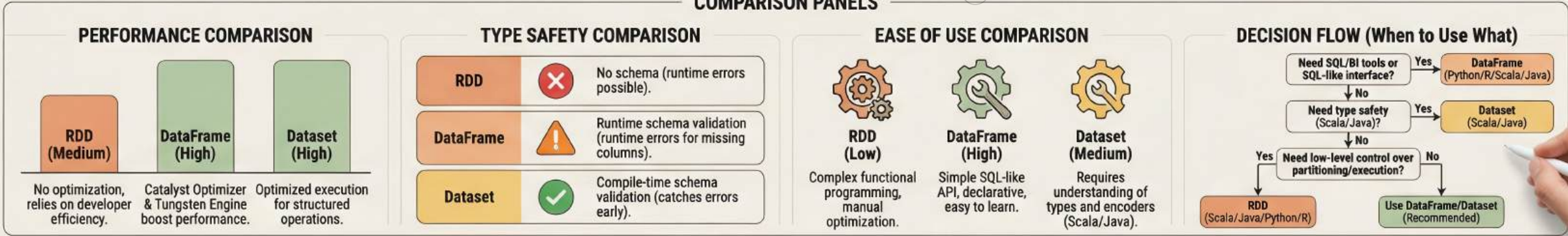
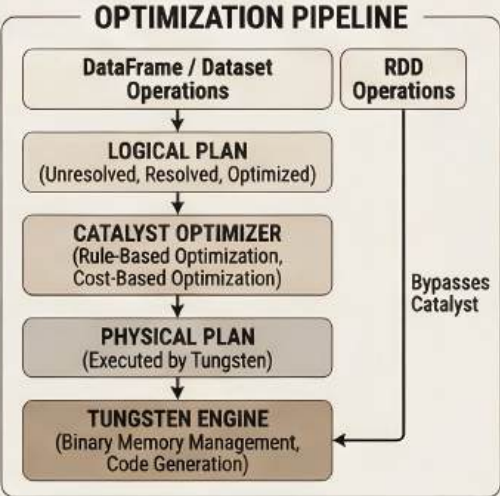
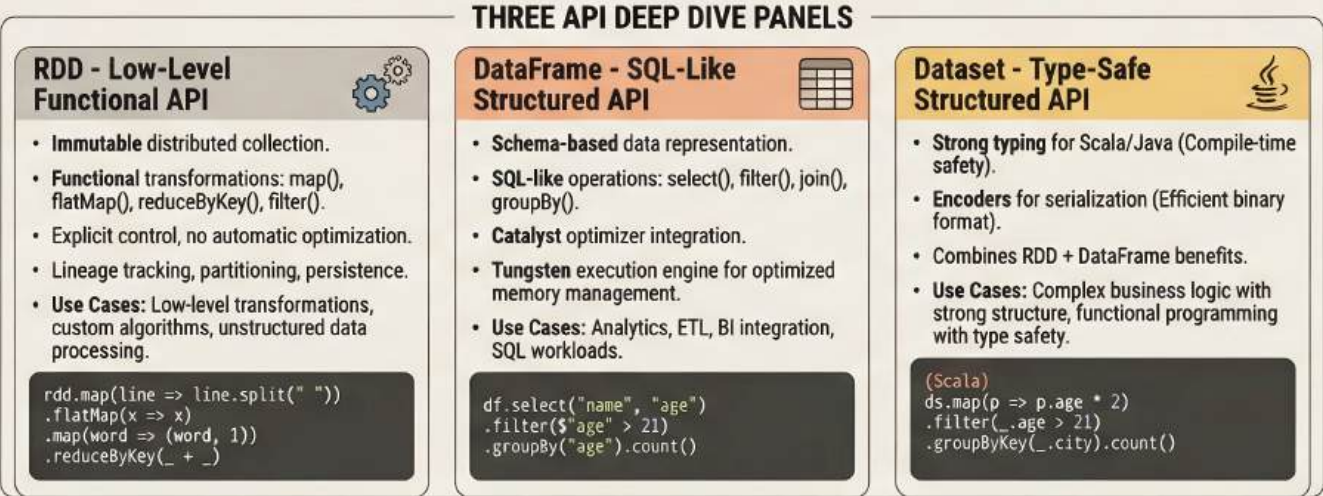
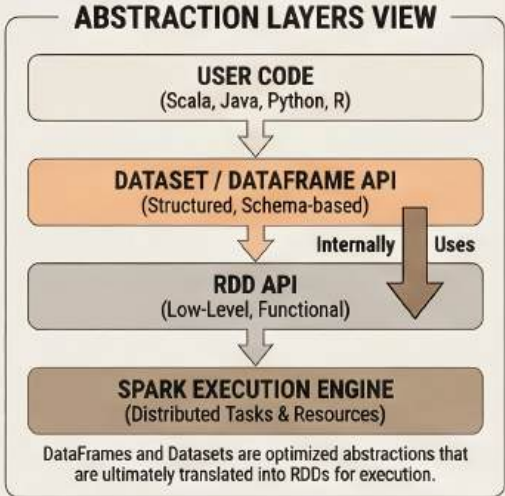
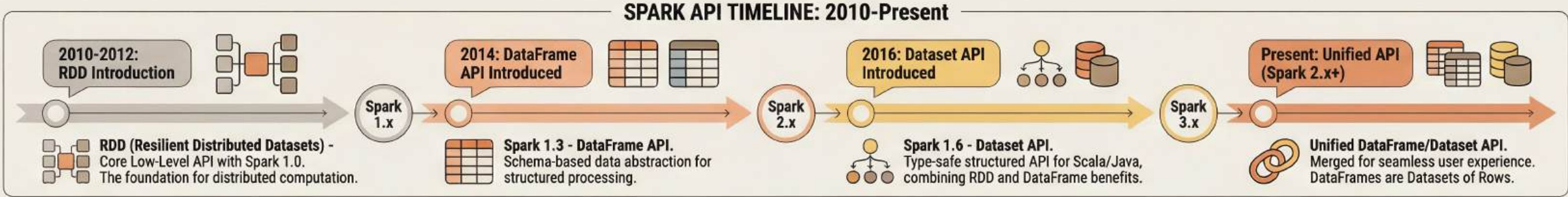
TOP SECTION - SPARK API TIMELINE

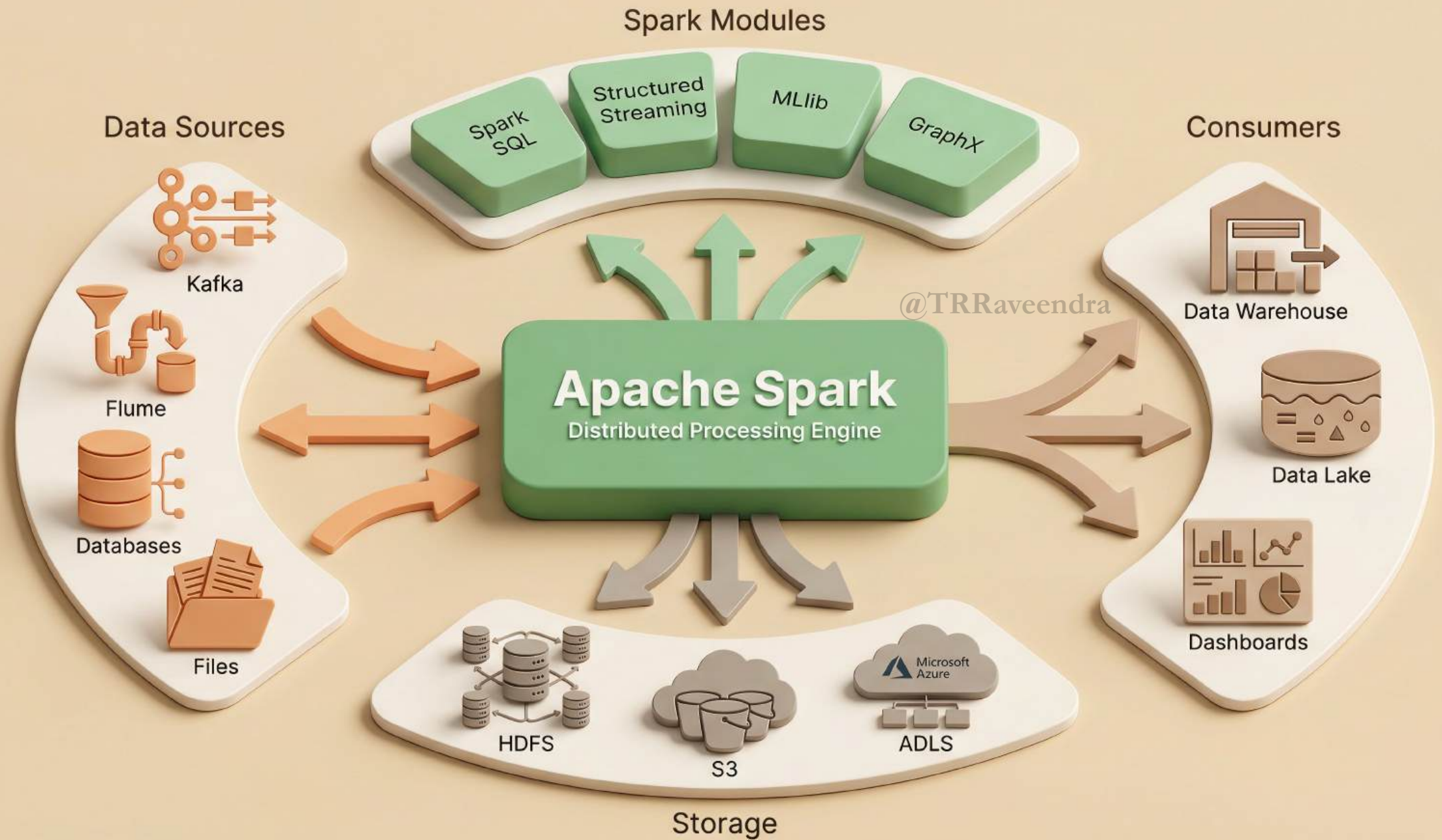


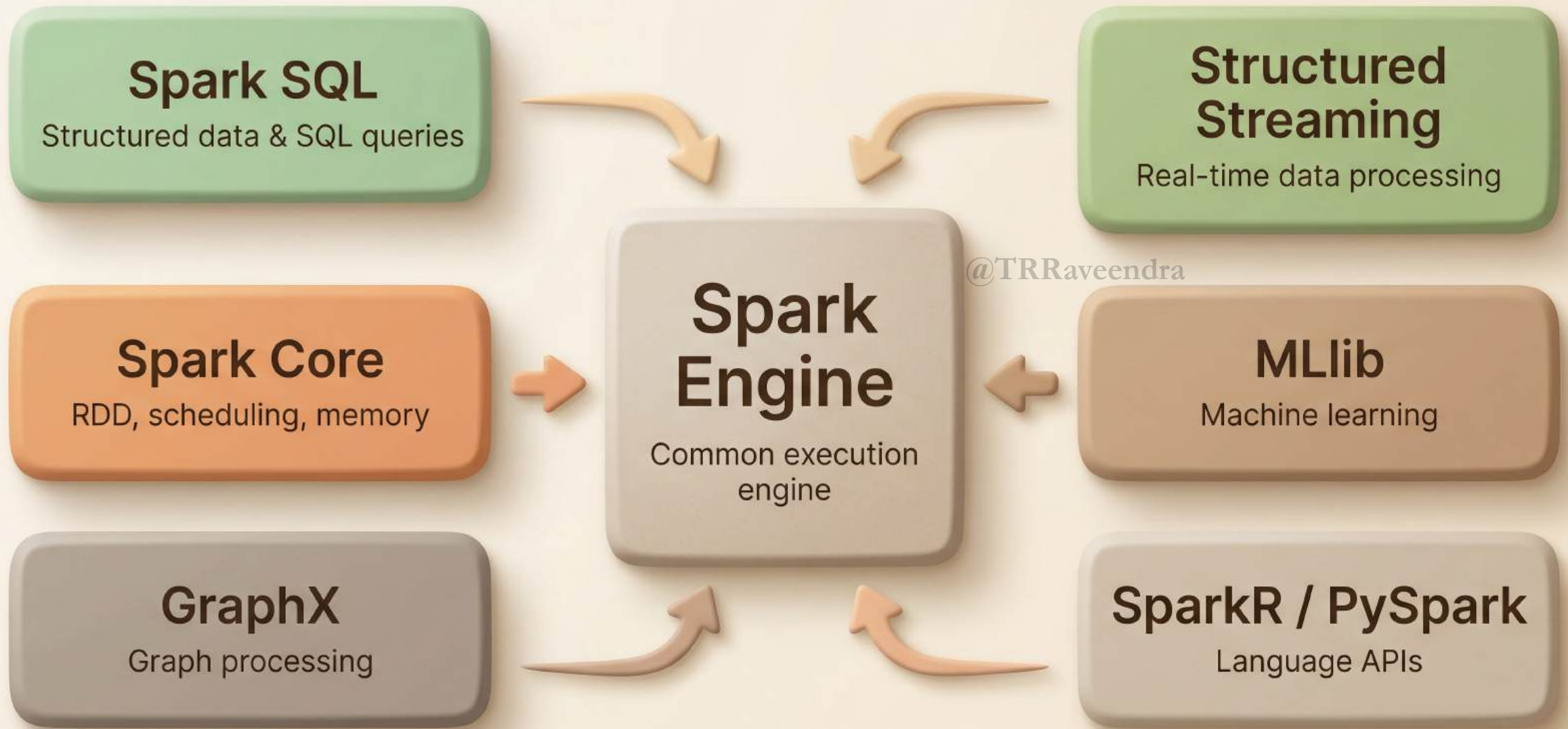
@TRRaveendra



APACHE SPARK APIs EVOLUTION: From RDD to Unified Structured APIs







Identify Spark modules • Know what each module does • Understand they share same engine

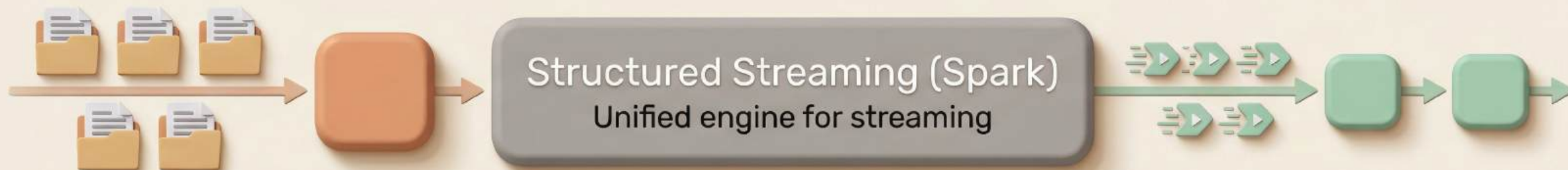
Batch Processing

- Large chunks of data
- Periodic runs
- High throughput

@TRRaveendra

Examples

- Daily ETL
- Reports



Streaming Processing

- Continuous data
- Near real-time
- Low latency

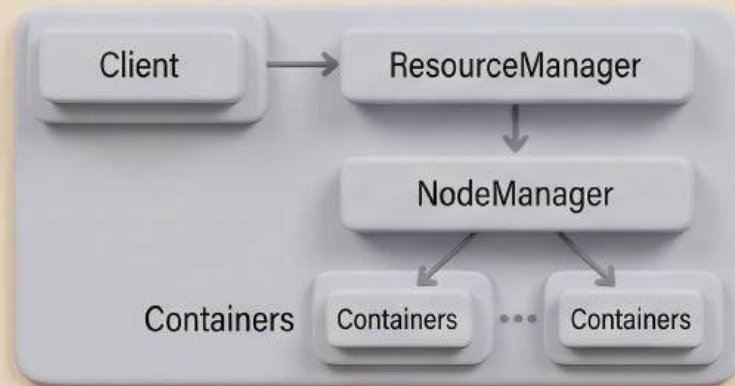
Examples

- Clickstream
- IoT



Hadoop MapReduce

- Disk-based processing
- Two-stage model (Map → Reduce)
- Batch oriented
- High latency



MapReduce Flow

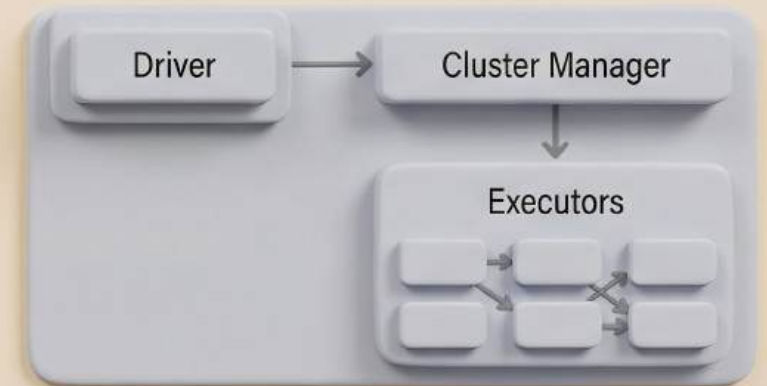


Spark Flow

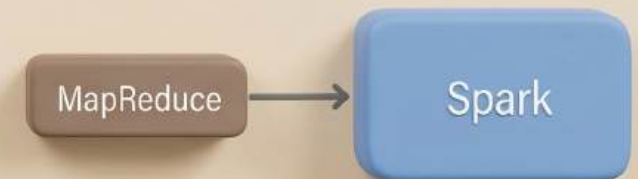


Apache Spark

- In-memory processing
- DAG-based execution
- Batch + Streaming + ML + SQL
- Low latency

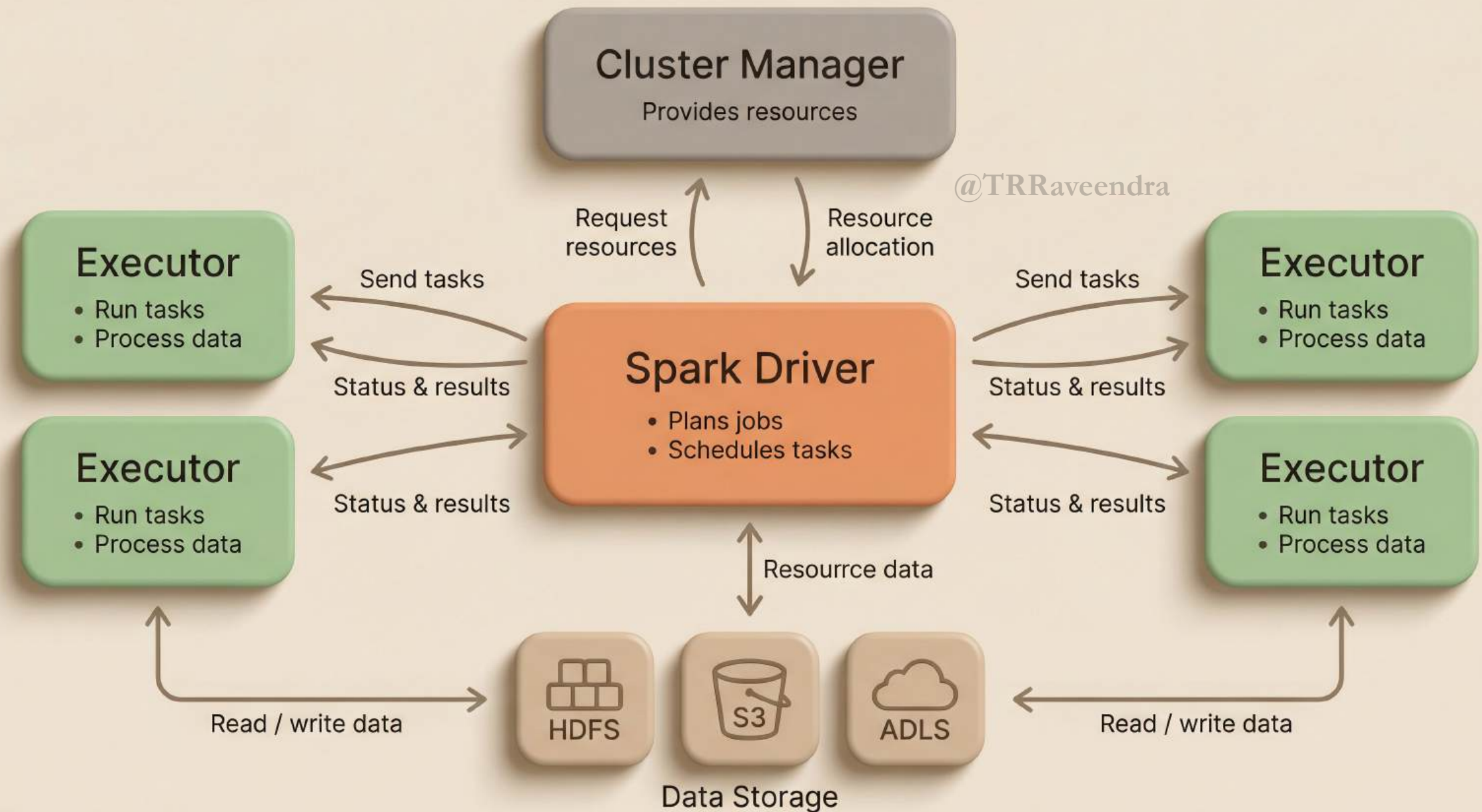


	Hadoop MapReduce	Apache Spark
Processing Speed	Slow	Fast
Fault Tolerance	Replication	Lineage
Ease of Use	Complex	Simple APIs
Use Cases	Large batch ETL	Interactive, Streaming, ML



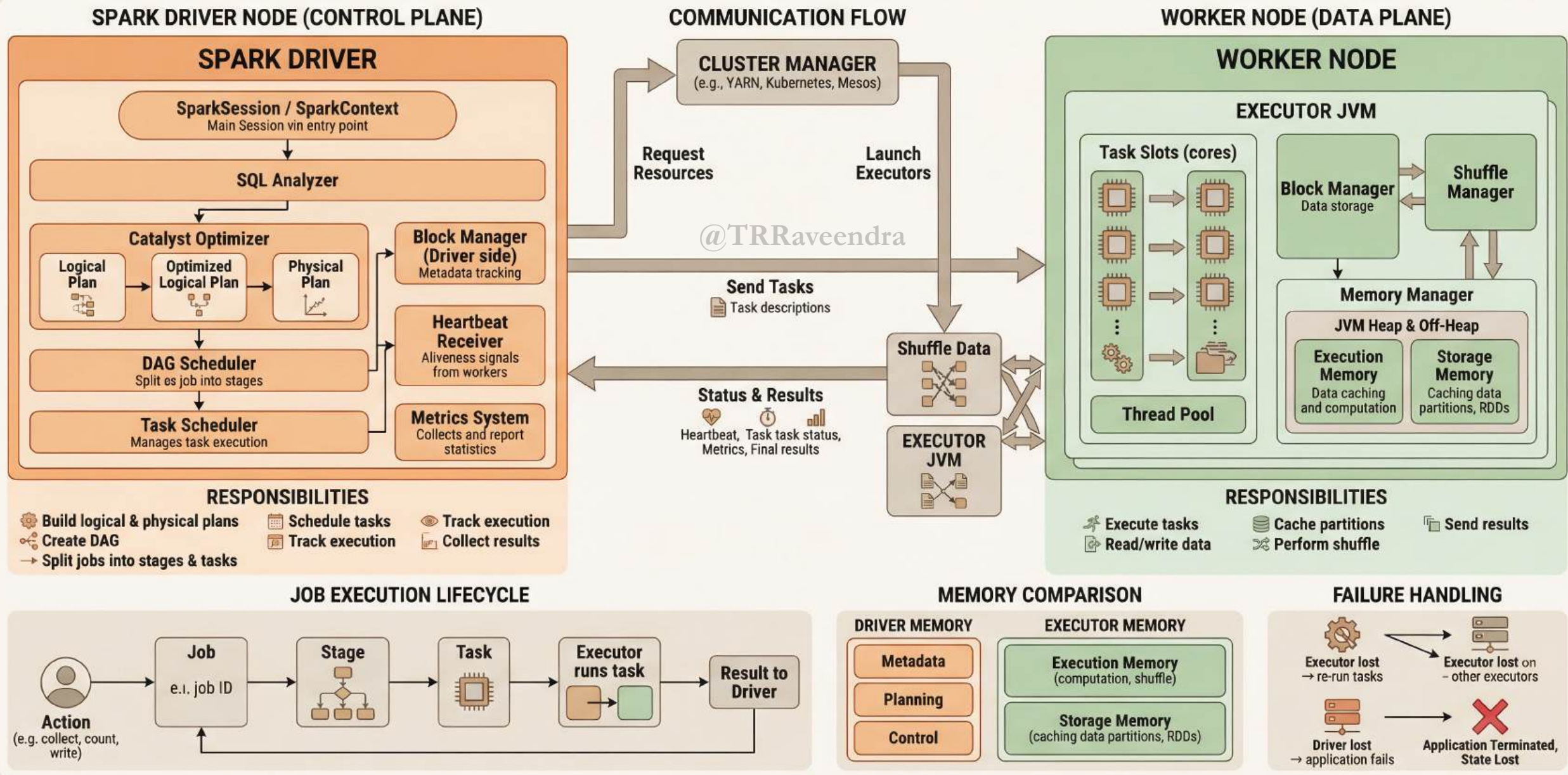
Spark evolved to overcome MapReduce limitations

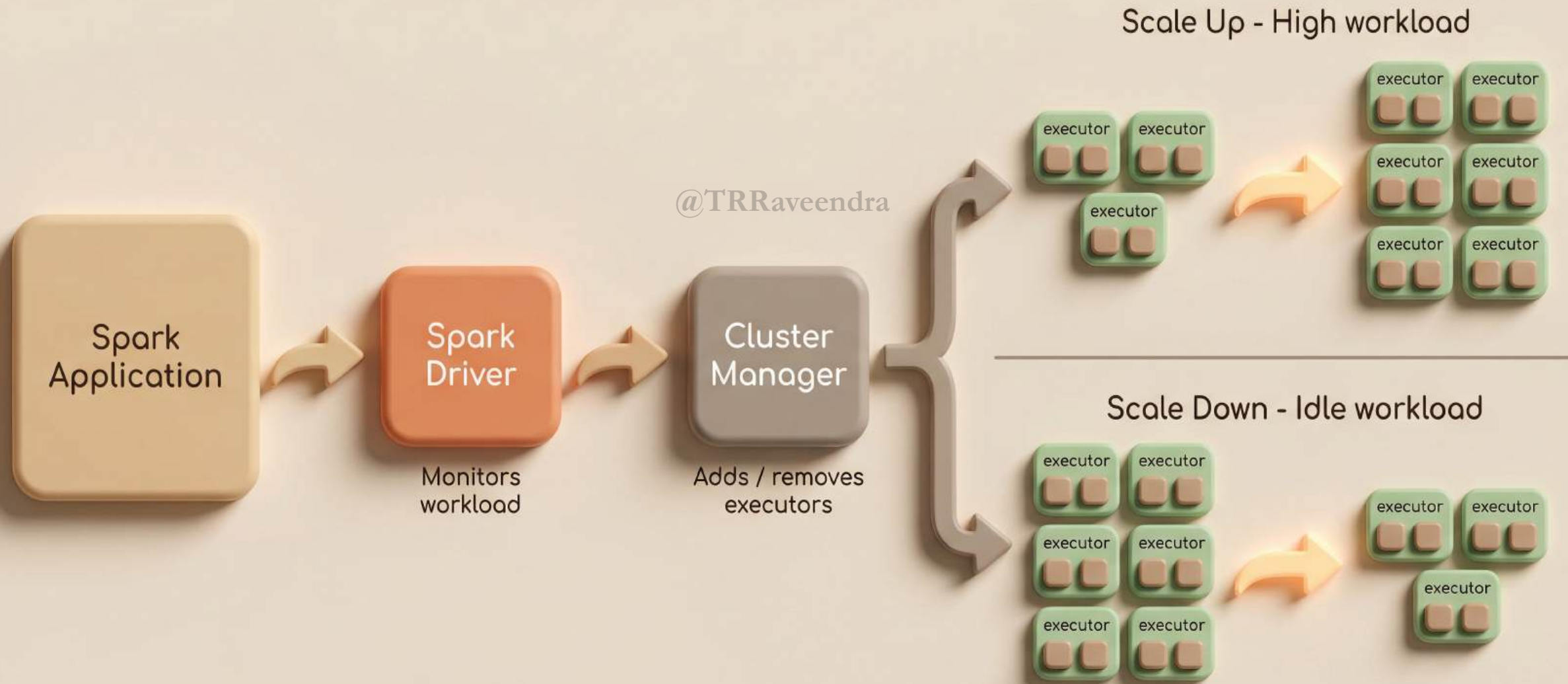
Understand architectural differences • Know why Spark is faster • Know when to use each • Understand evolution of big data engines



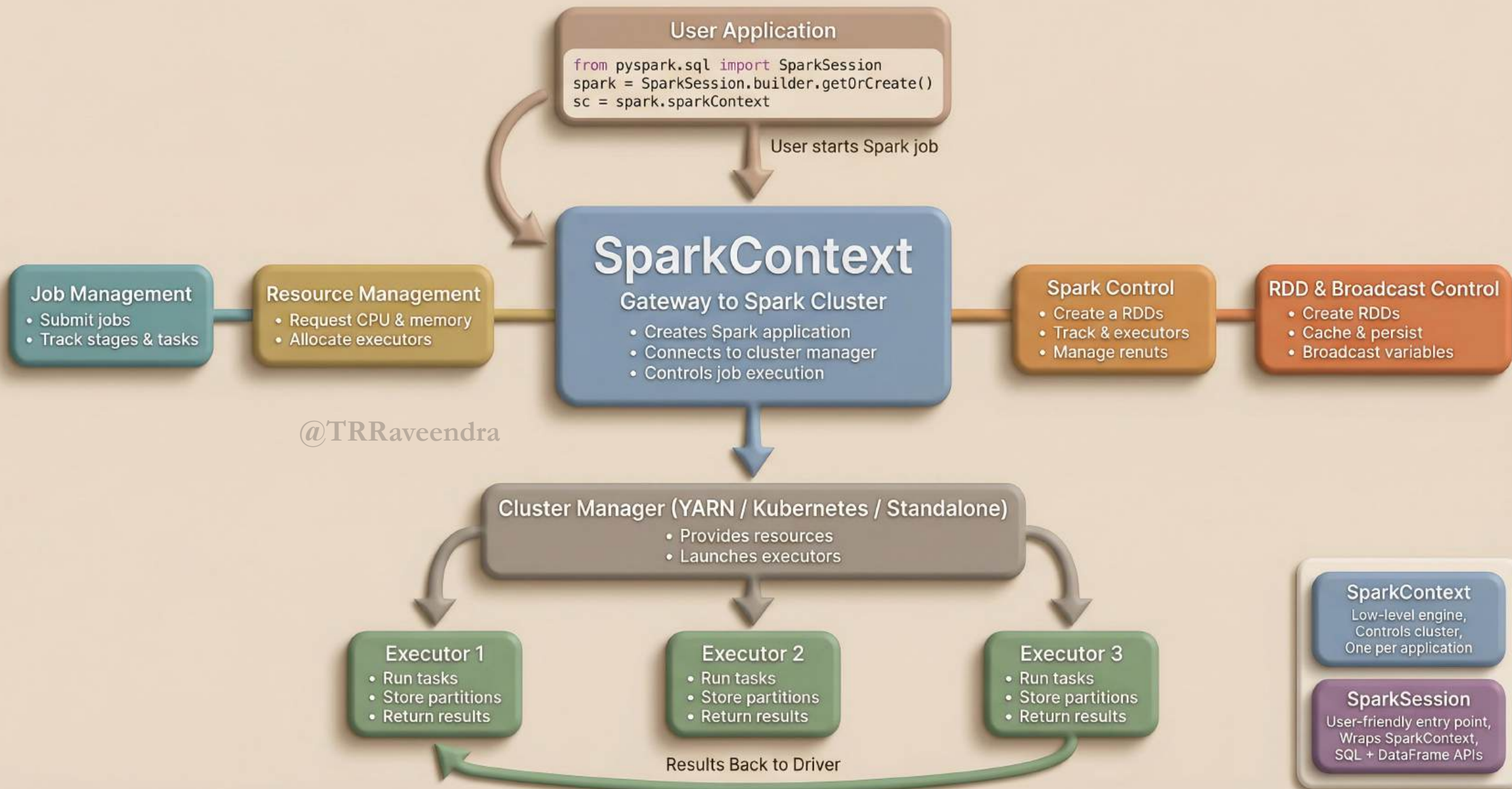
@TRRaveendra

APACHE SPARK DRIVER NODE VS WORKER NODE ARCHITECTURE

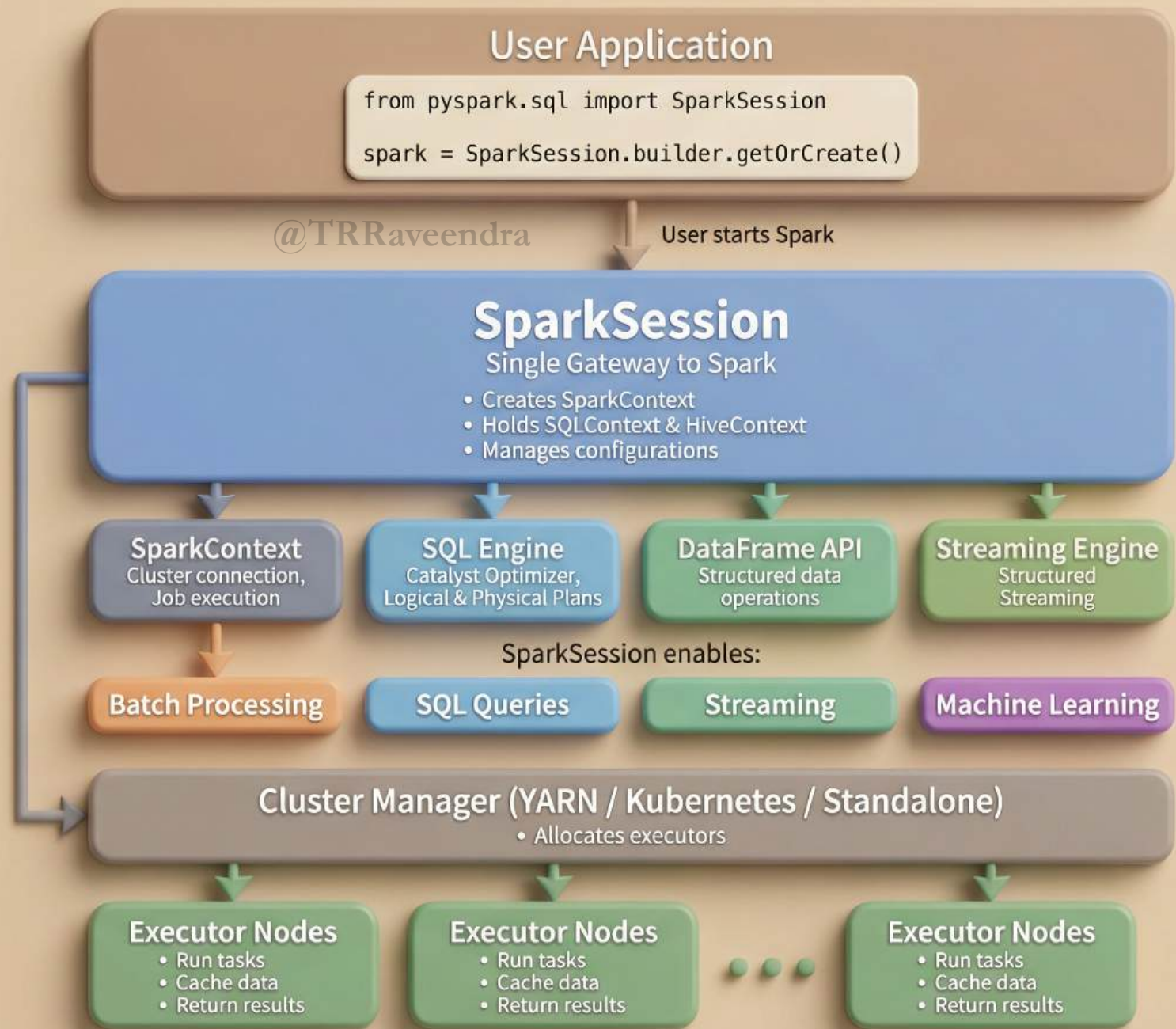




- Understand what dynamic allocation is
- See how Spark scales executors
- Know benefits of dynamic allocation



Understand what SparkContext is • Know its responsibilities • Understand how Spark connects to cluster • See how execution starts



@TRRaveendra

User starts Spark

SparkSession

Single Gateway to Spark

- Creates SparkContext
- Holds SQLContext & HiveContext
- Manages configurations

SparkContext

Cluster connection,
Job execution

SQL Engine

Catalyst Optimizer,
Logical & Physical Plans

DataFrame API

Structured data
operations

Streaming Engine

Structured
Streaming

SparkSession enables:

Batch Processing

SQL Queries

Streaming

Machine Learning

Cluster Manager (YARN / Kubernetes / Standalone)

- Allocates executors

Executor Nodes

- Run tasks
- Cache data
- Return results

Executor Nodes

- Run tasks
- Cache data
- Return results

...

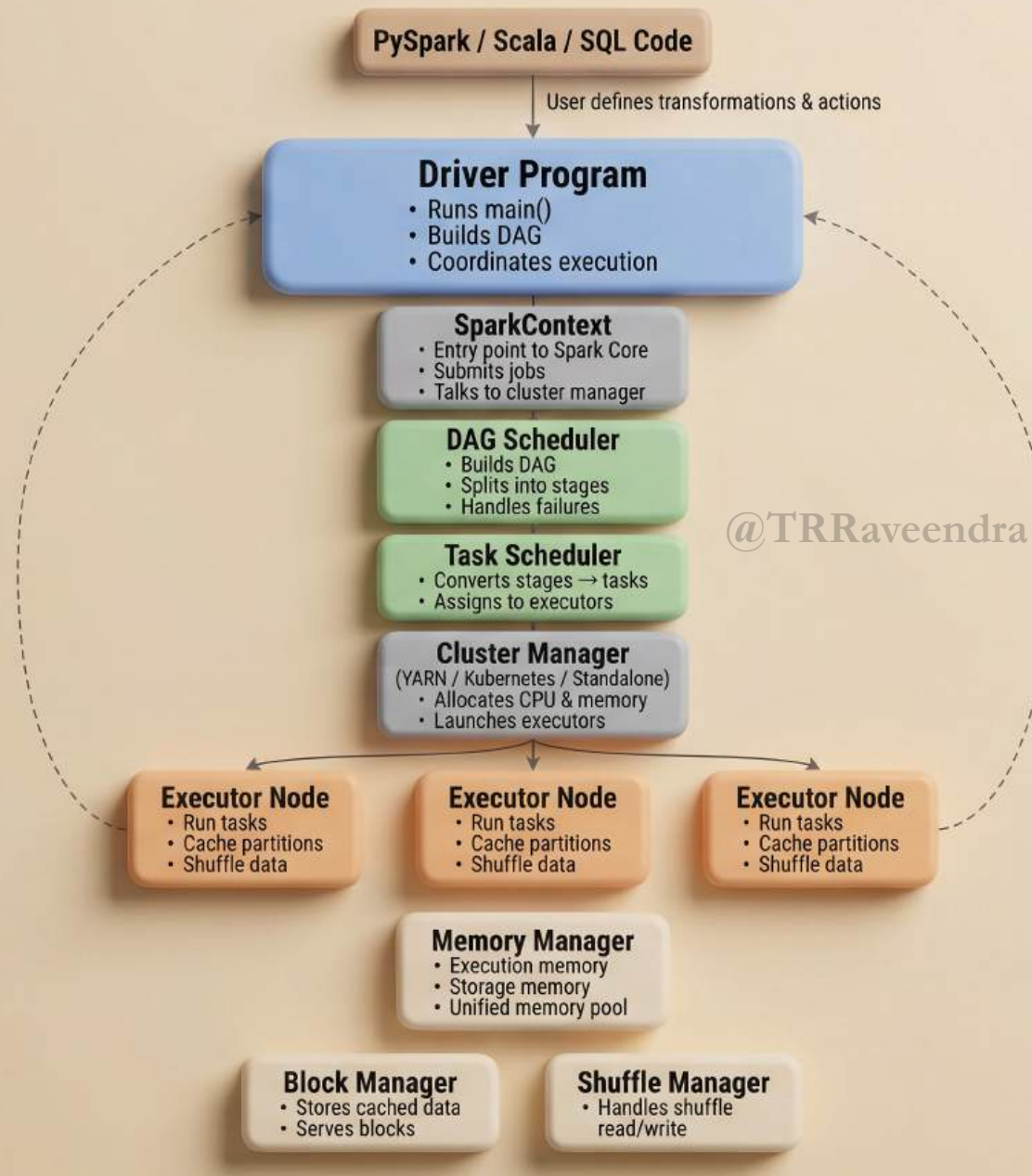
Executor Nodes

- Run tasks
- Cache data
- Return results

SparkSession vs SparkContext

- **SparkSession:**
 - Unified entry point
 - Recommended API
- **SparkContext:**
 - Low-level engine
 - Mostly internal

Learning Goals: Understand SparkSession role • Know what it contains • Understand user access • See execution flow



Mini-comparison

Driver:
Controls
execution

Executor:
Runs
computation

USER
APPLICATION

```
spark.sql('SELECT country, SUM(sales) FROM orders GROUP BY country')
```

User writes SQL

SQLContext

Structured Query Engine

- Executes SQL
- Creates DataFrames
- Parses queries

INTERNAL
ENGINES

Parser

Converts SQL → Logical Plan

Catalyst Optimizer

Optimizes plan

Planner

Creates Physical Plan

DATA SOURCES

Parquet

CSV

JSON

ORC

JDBC

- Schema inference
- Column pruning

EXECUTION LAYER

SparkContext

- Submits jobs
- Talks to cluster

EXECUTORS

Executor Nodes

- Run tasks
- Return results

@TRRaveendra

MINI COMPARISON

SQLContext:

SQL + DataFrame support,
Legacy interface

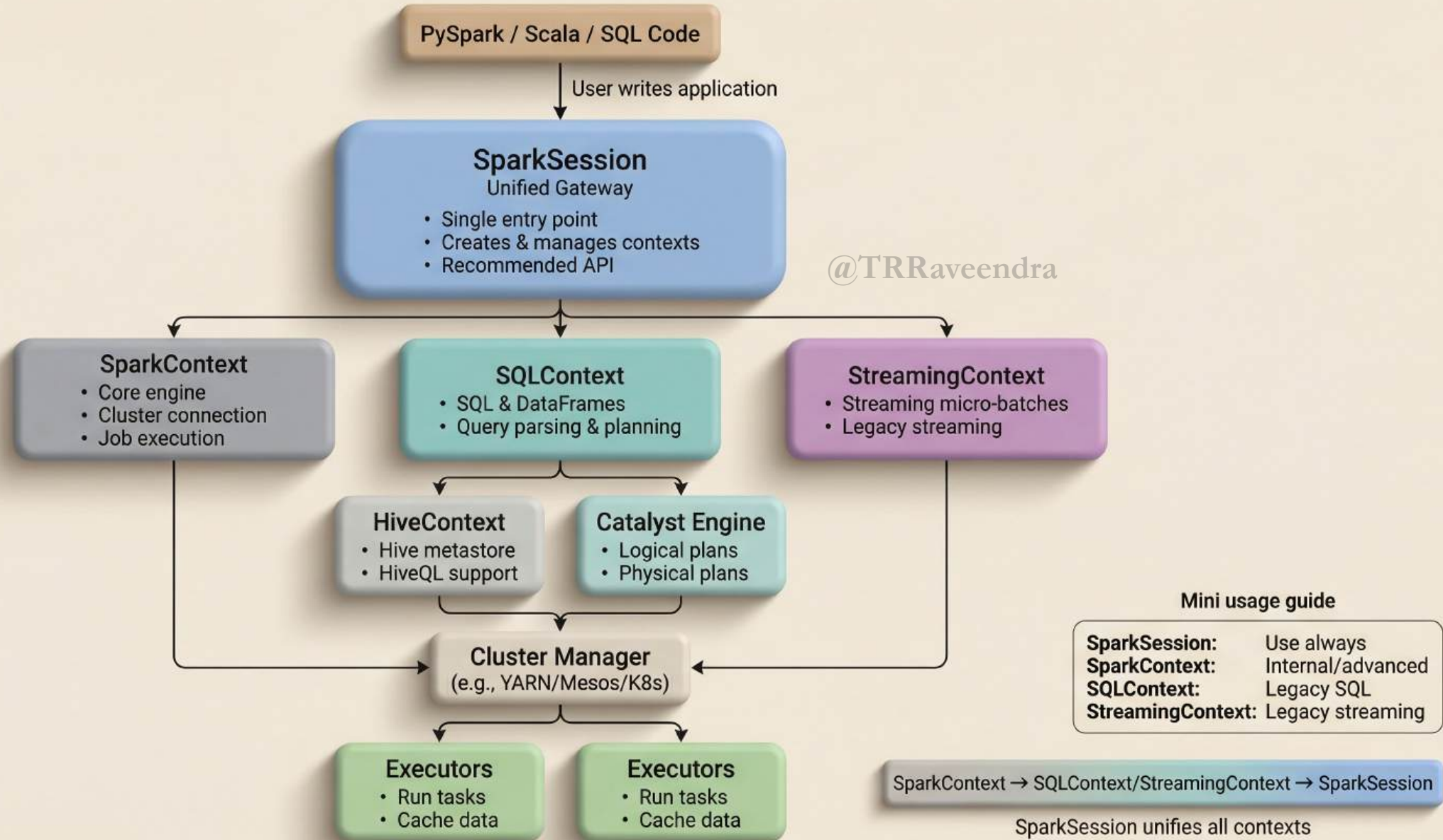
SparkSession:

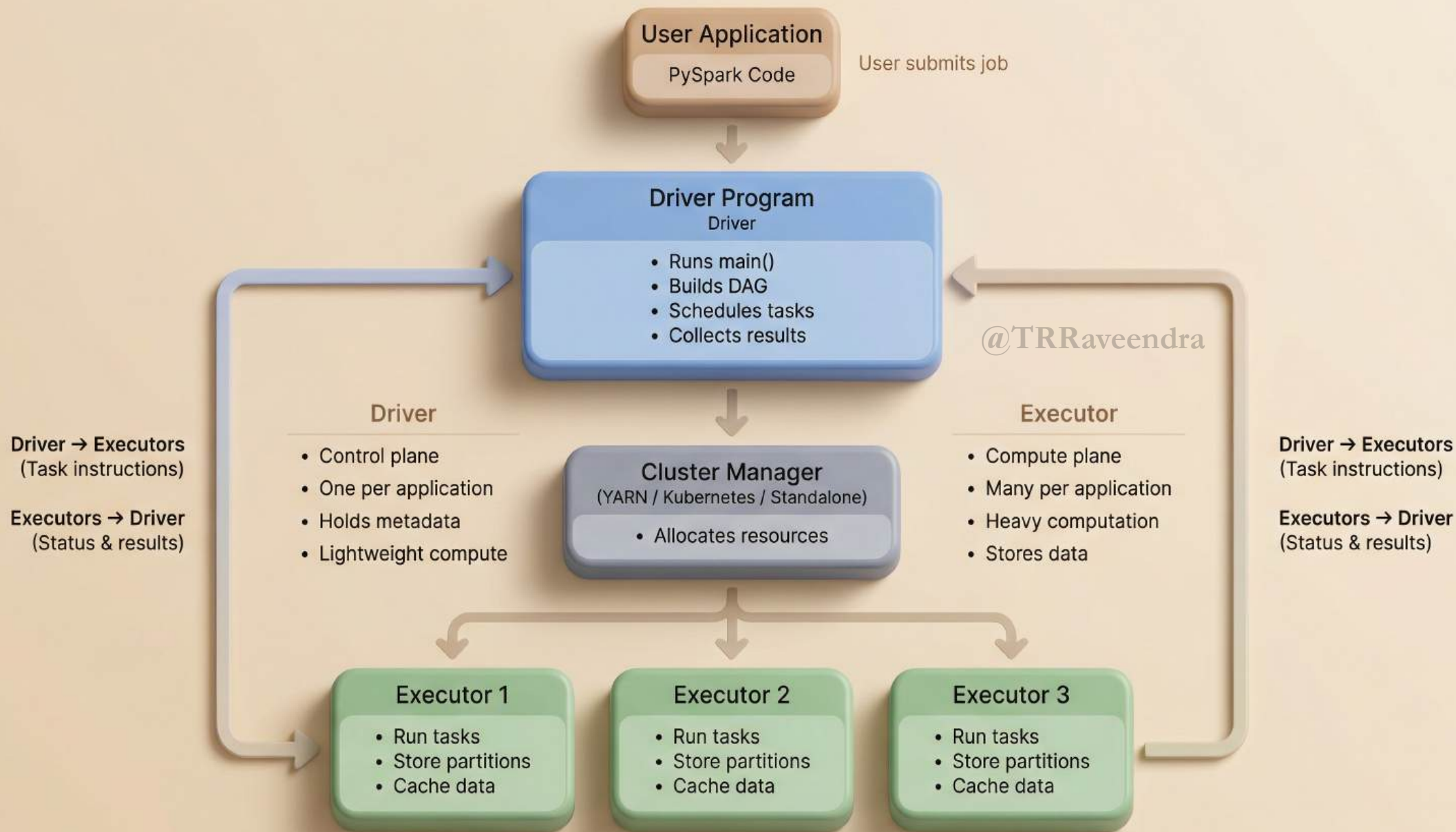
Unified interface,
Modern API

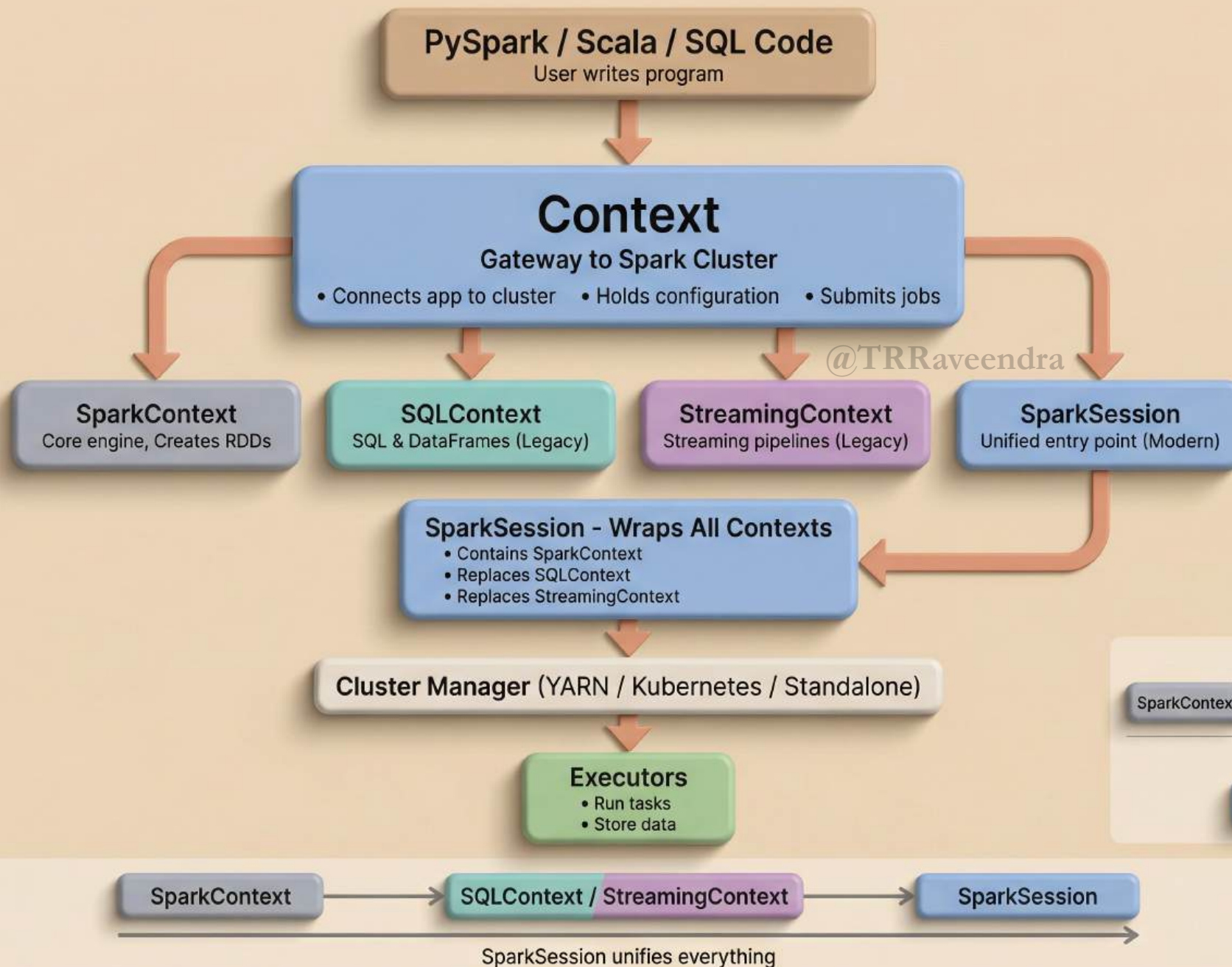
LEARNING GOALS

Understand SQLContext role | See how SQL becomes execution | Know relation to SparkSession | Understand optimization flow

Apache Spark Contexts Hierarchy







✓ Understand what Context means

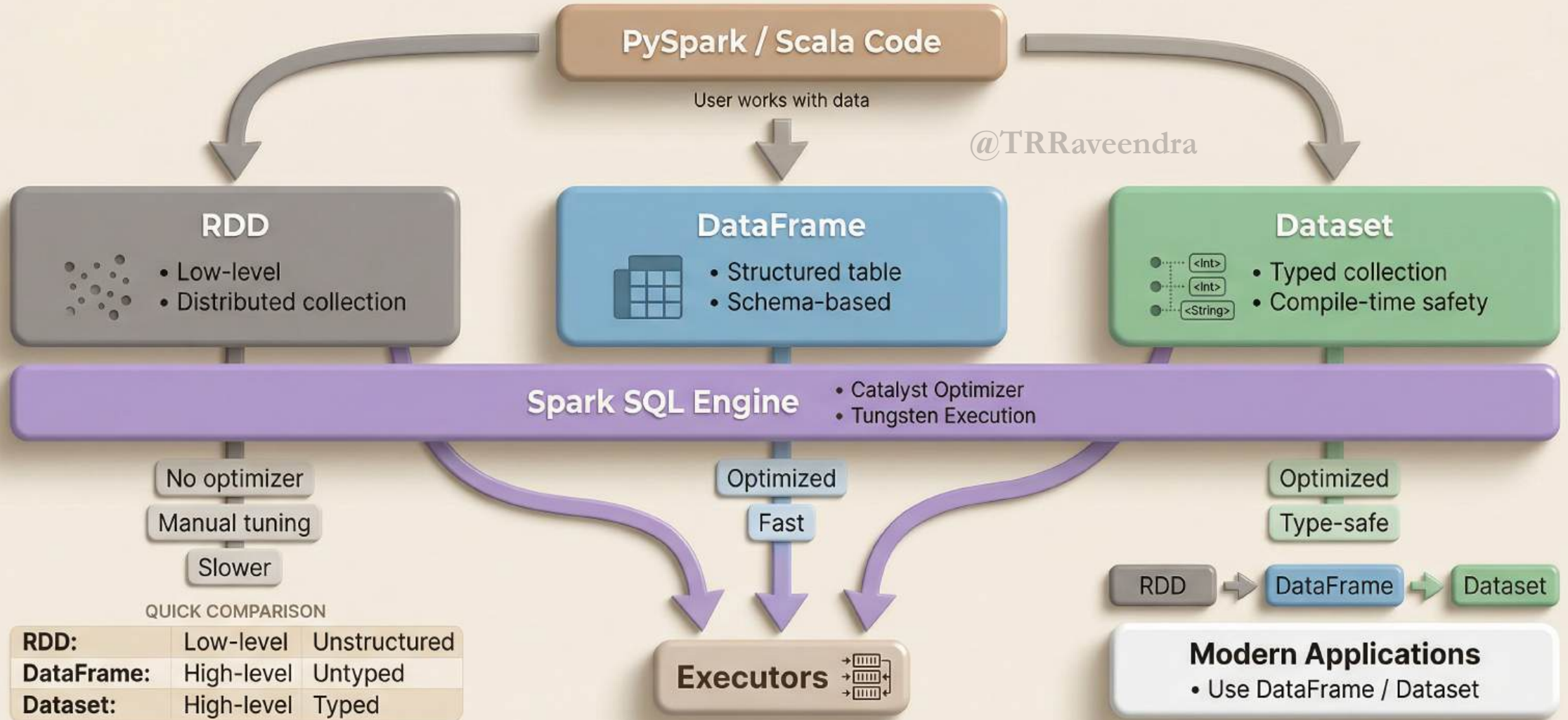
✓ Know different context types

✓ Know which one to use

✓ Understand execution gateway

RDD vs DataFrame vs Dataset

Data Abstractions in Spark

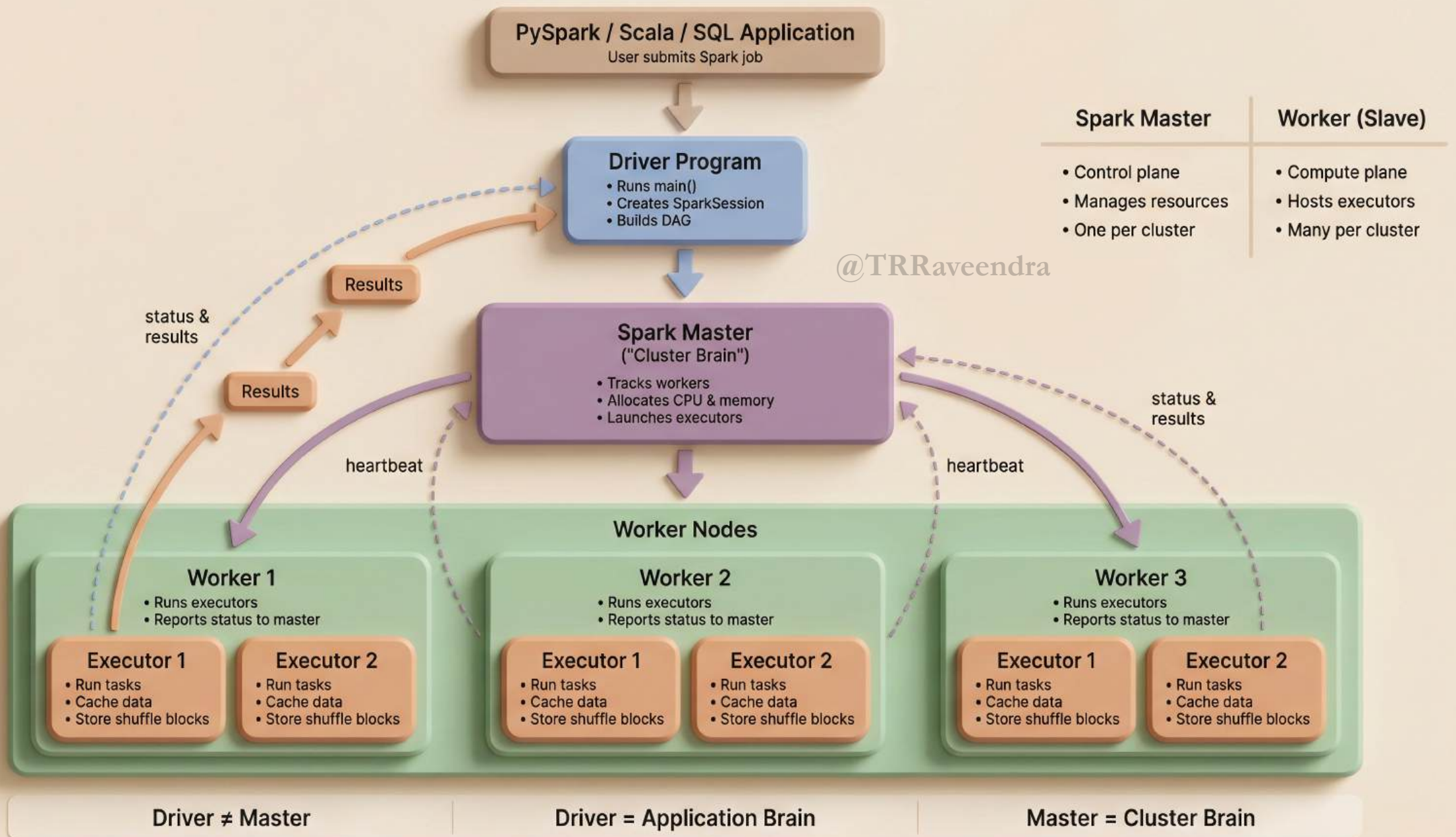


Understand abstractions

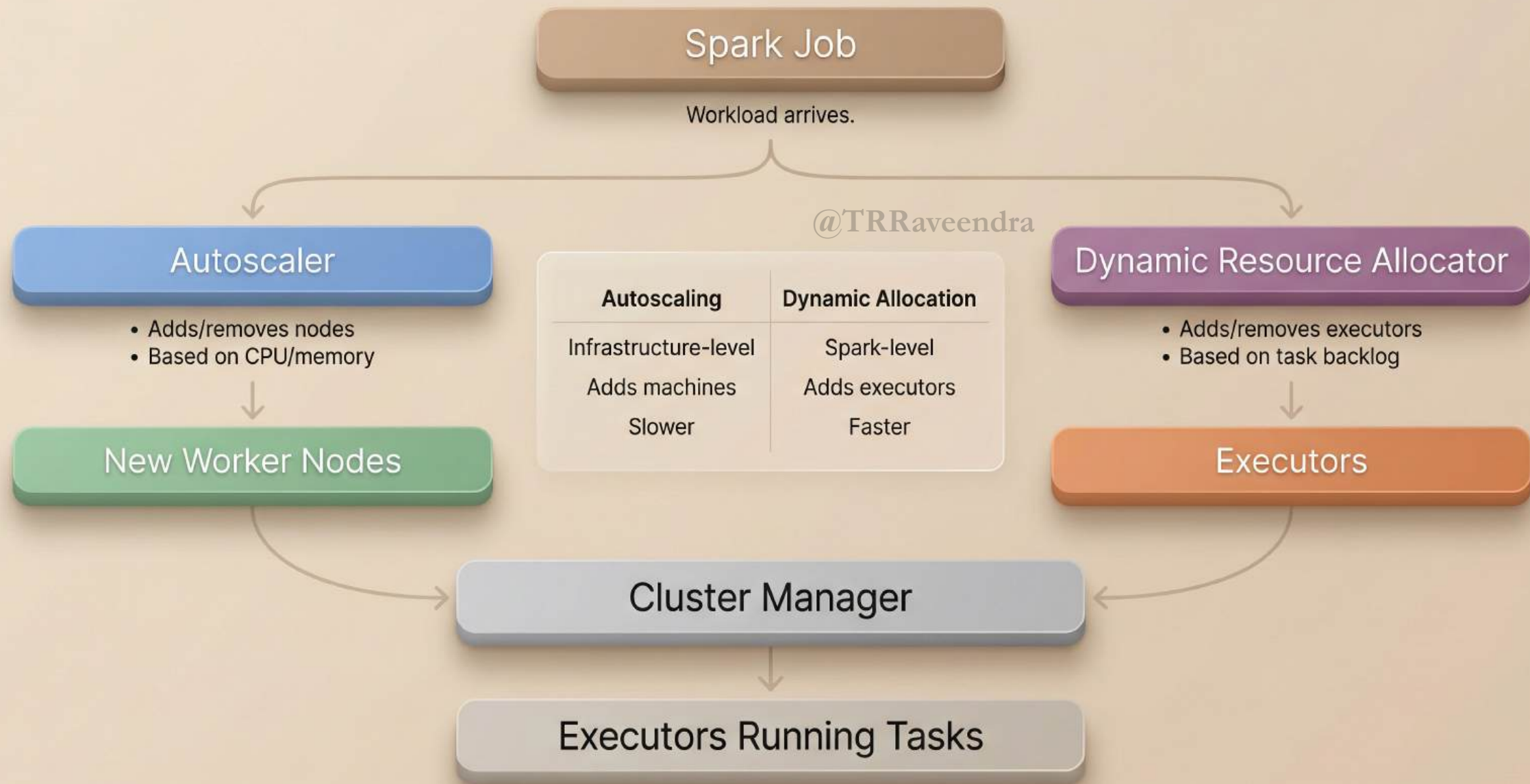
Know differences

Know what to choose

Understand performance impact



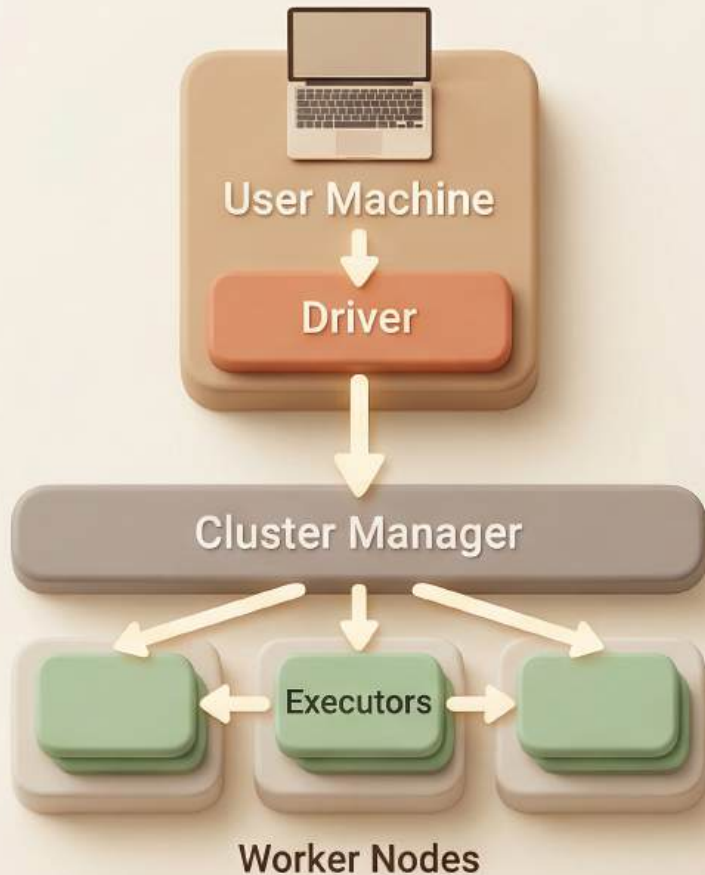
LEARNING GOALS • Understand master vs worker roles • See how cluster is controlled • Understand where computation runs • Understand interaction with driver



Autoscaling provides capacity → DRA uses capacity

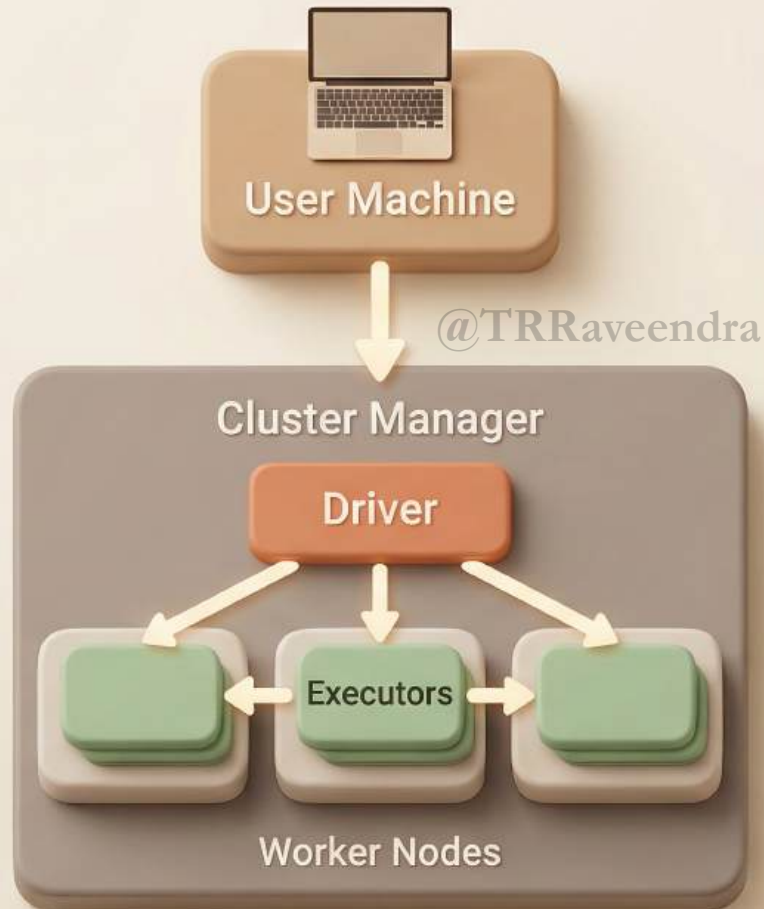
Understand difference • Know where each works • Know when to use • Understand complementarity

Client Mode



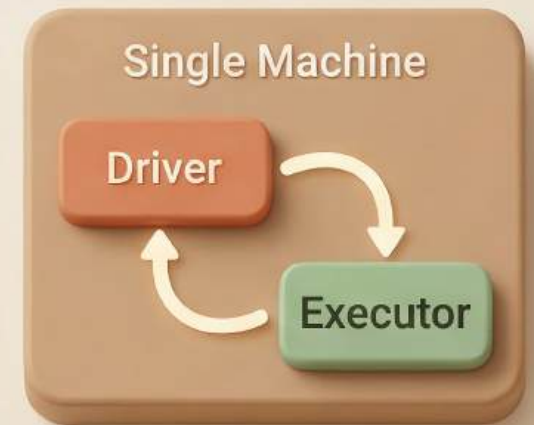
Client stays connected

Cluster Mode



Client can disconnect

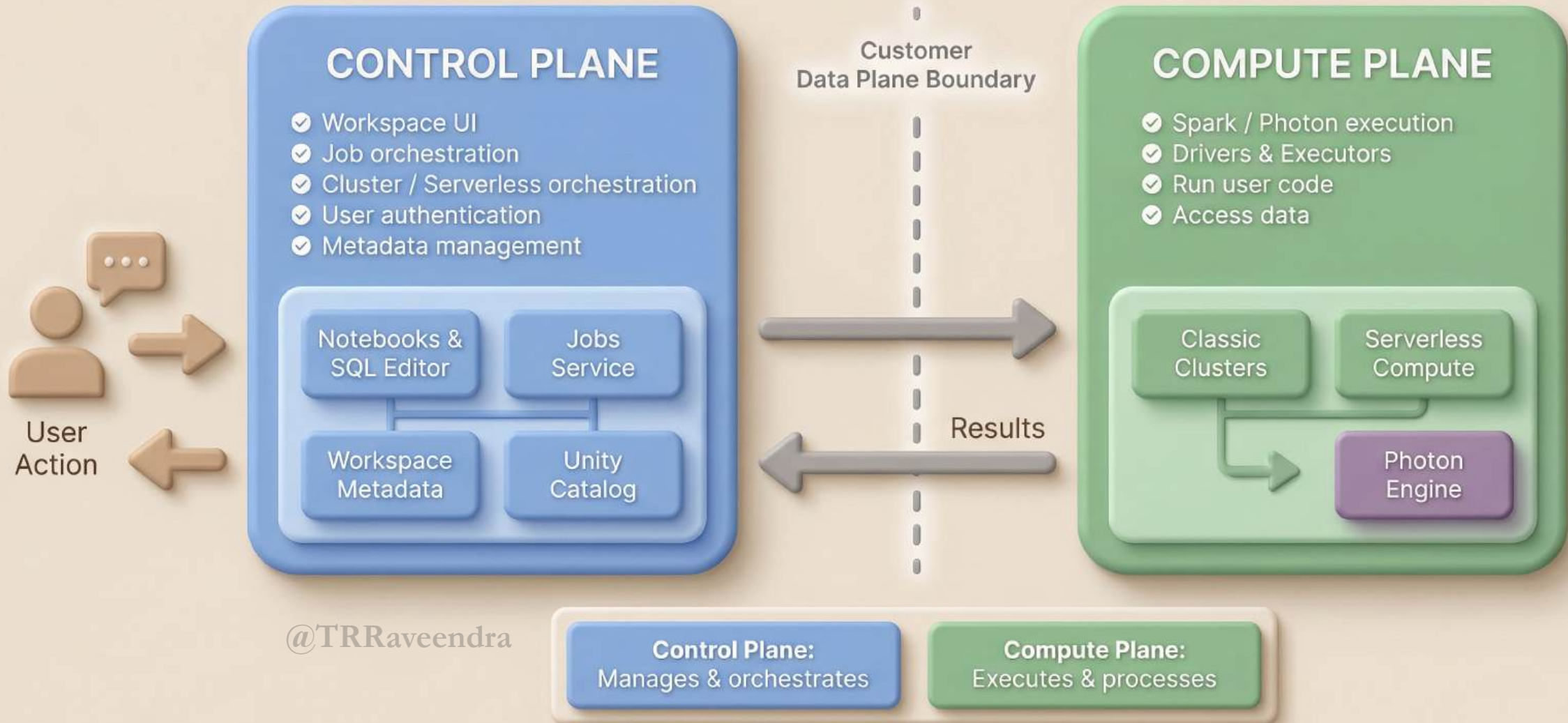
Local Mode



For testing & learning

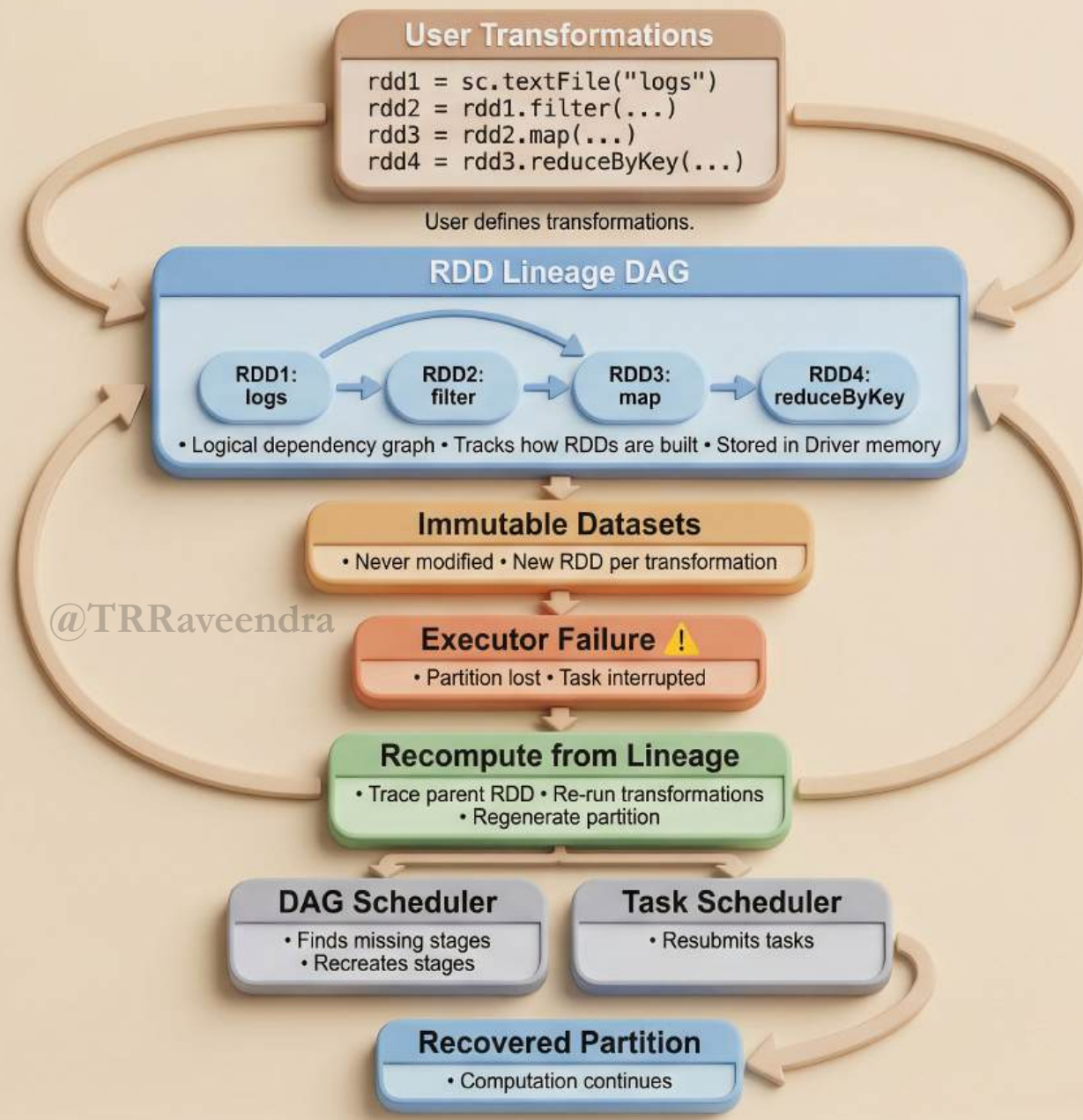
Understand client vs cluster mode | Know where driver runs | Know when to use each mode

Databricks Control Plane vs Compute Plane



Learning Goals:

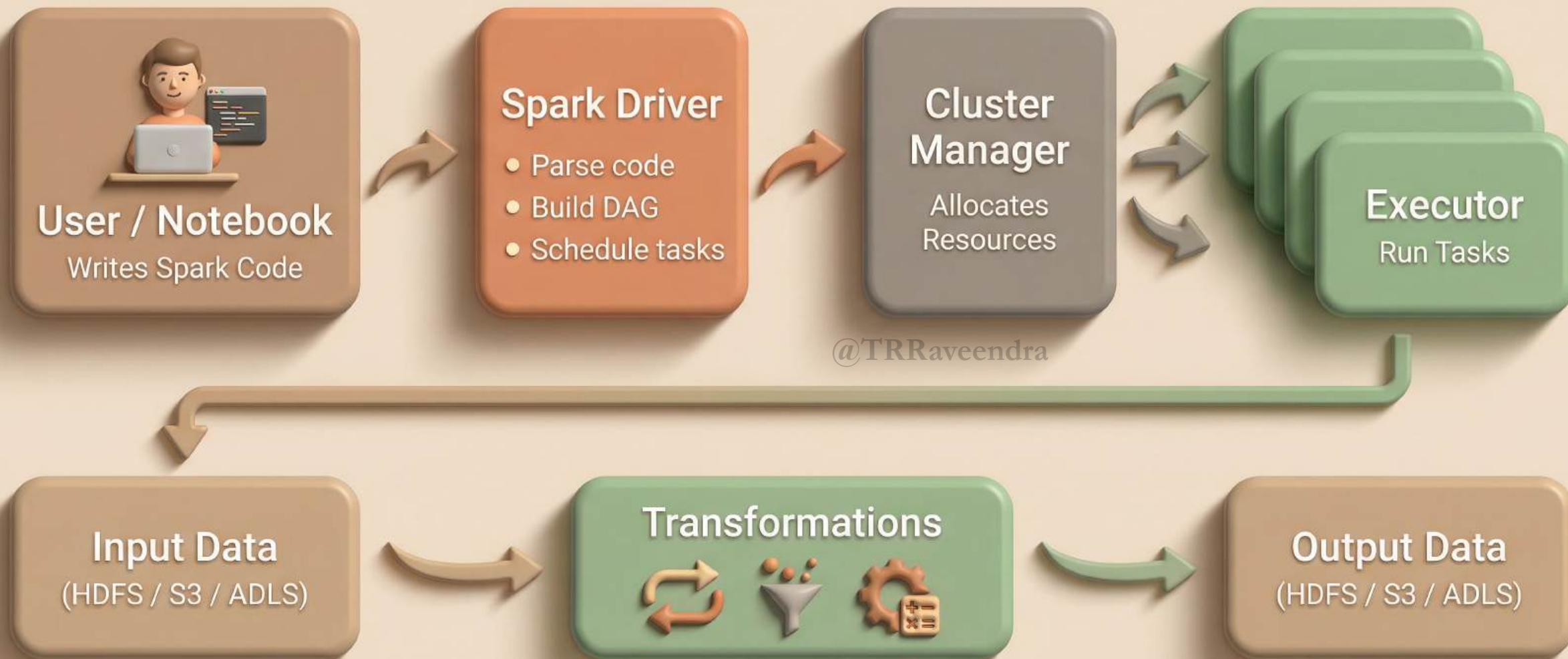
- ✓ Understand plane separation
- ✓ Know responsibilities
- ✓ Understand security boundary
- ✓ Understand execution flow



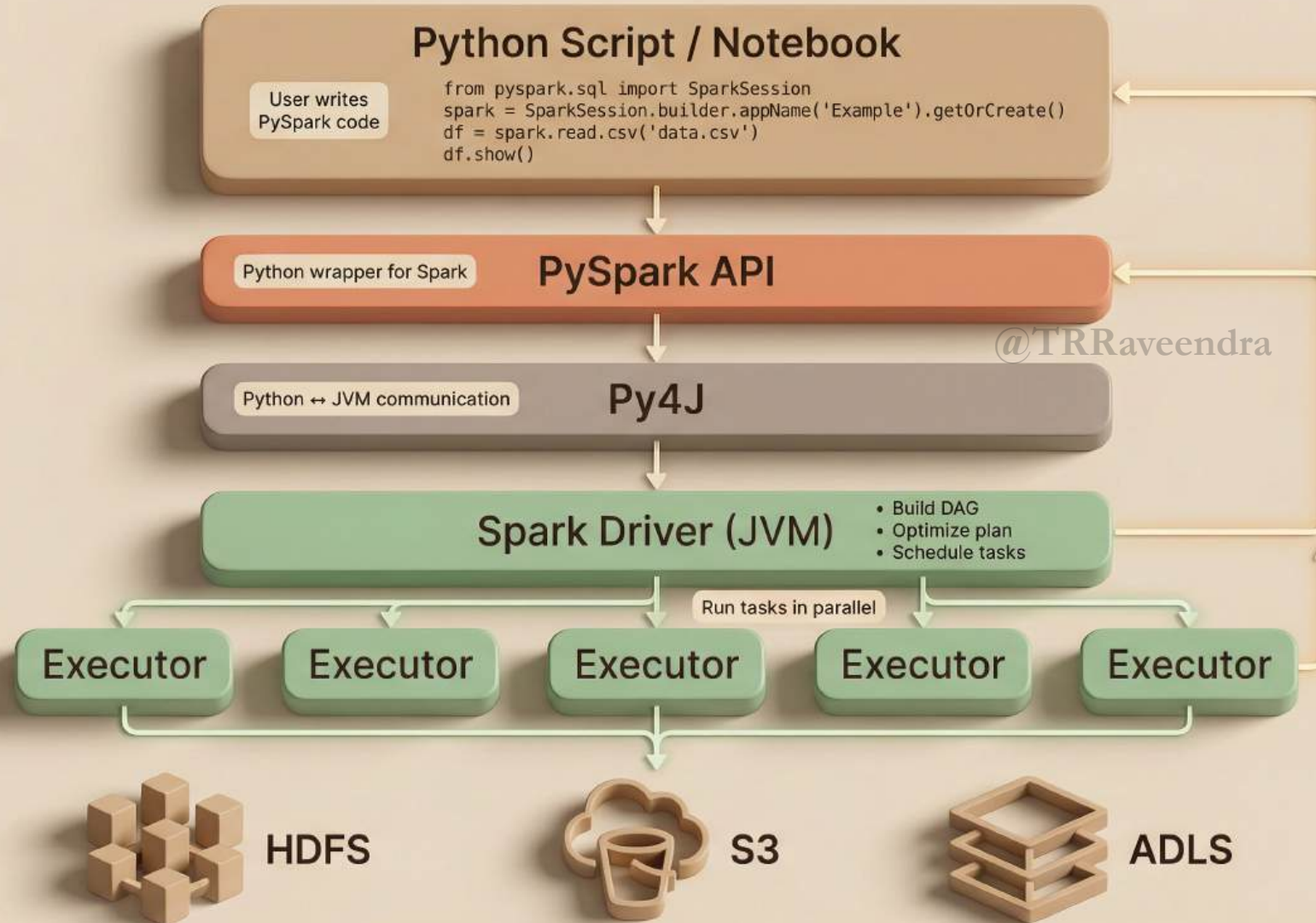
@TRRaveendra

Spark: Recomputes using lineage

Traditional Systems: Replicate data blocks



What is PySpark?



@TRRaveendra



Learning Goals:

- Understand what PySpark is
- See how Python connects to Spark
- Know where execution happens

RDD vs DataFrame vs Dataset — Clean 3D Spark APIs Comparison

LEFT COLUMN

RDD

- Low-level API
- No schema
- More control
- Slower than DF/DS

Use case section:
Custom transformations

MIDDLE COLUMN

DataFrame

- Schema-based
- Optimized
- SQL support
- High performance

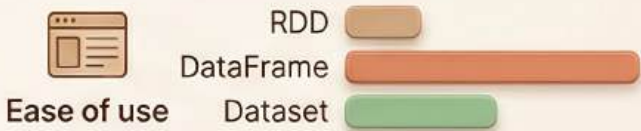
Use case section:
Analytics & ETL

RIGHT COLUMN

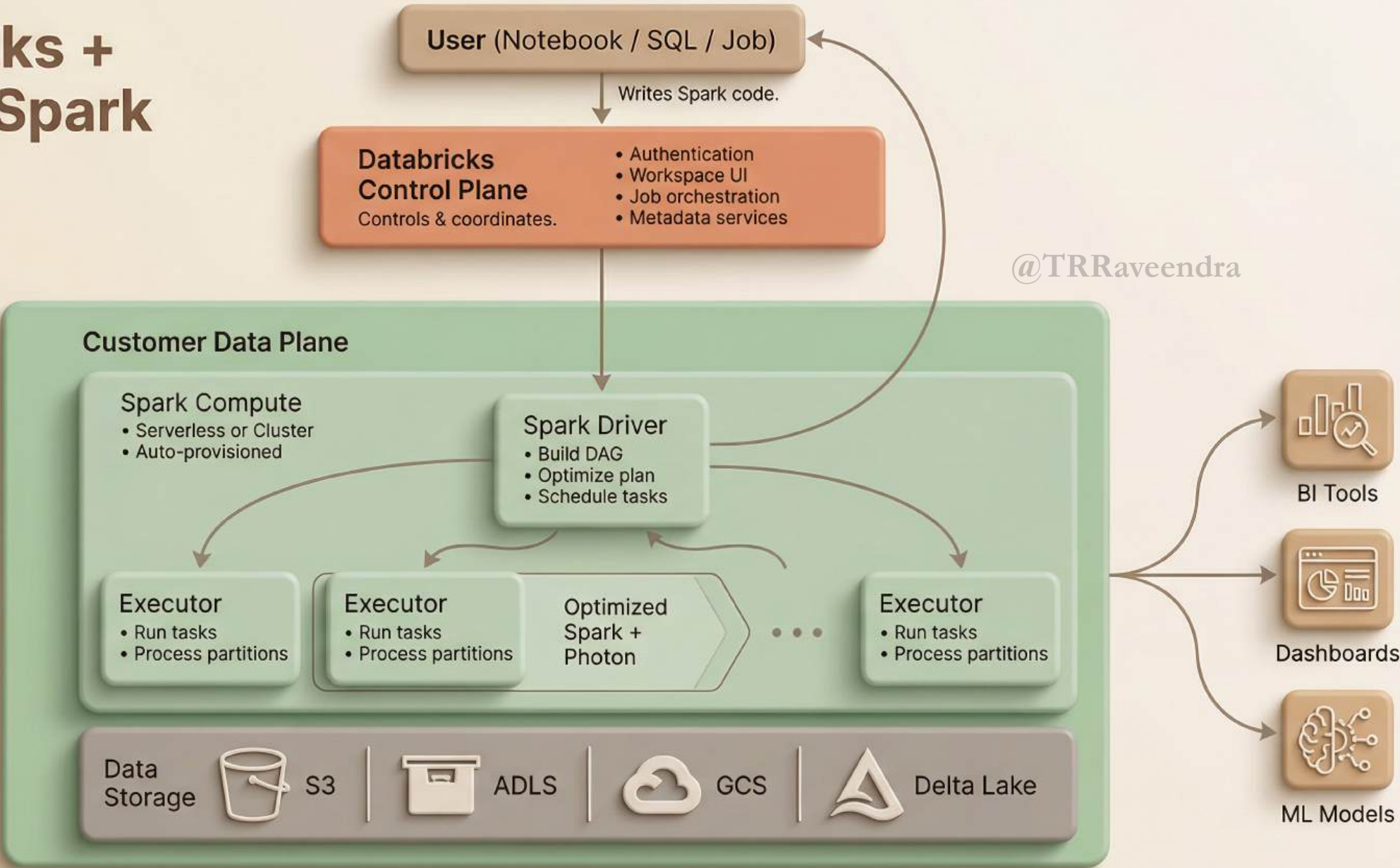
Dataset

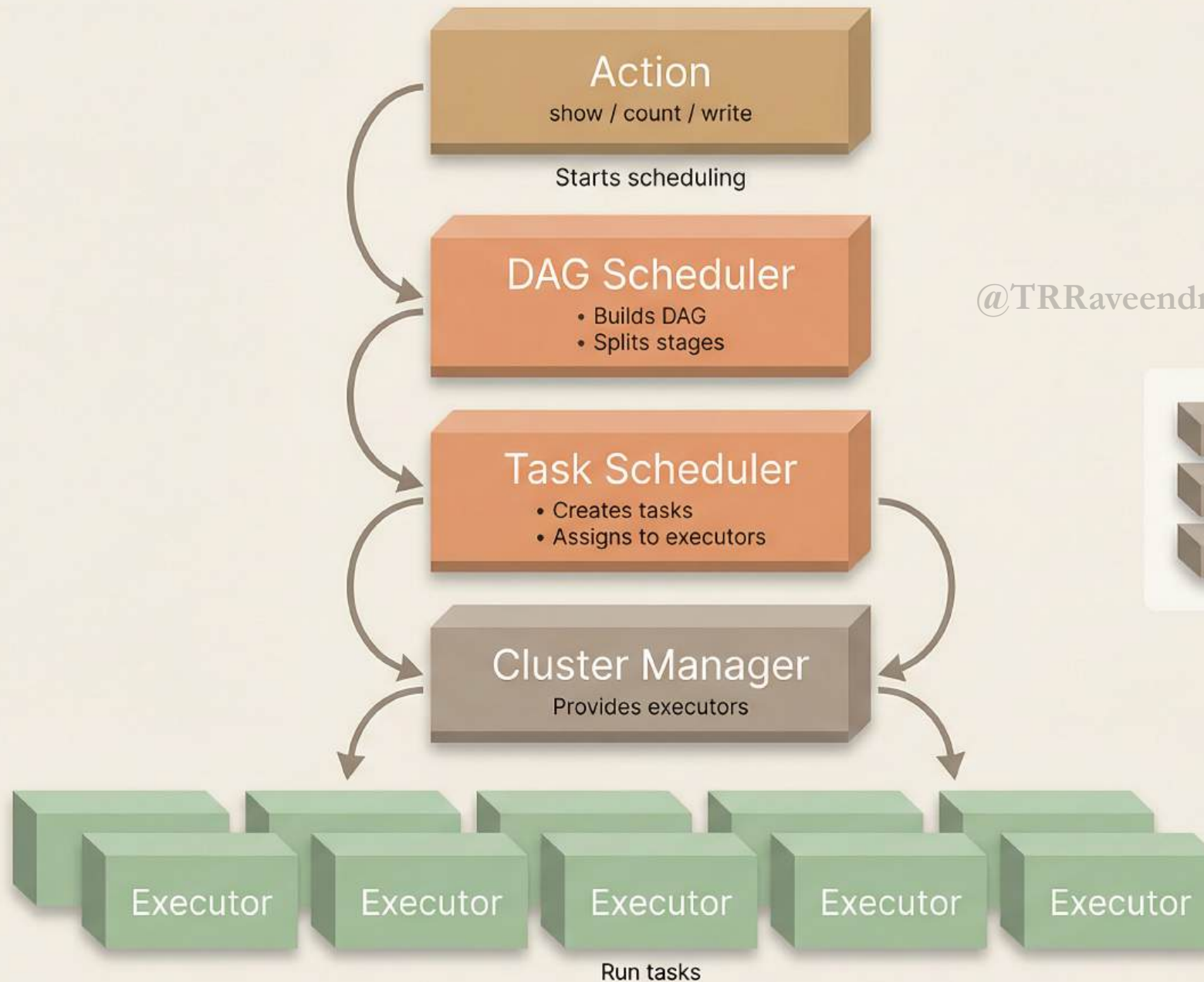
- Typed (Scala/Java)
- Compile-time safety
- Optimized

Use case section:
Complex business logic



Databricks + Apache Spark





@TRRaveendra

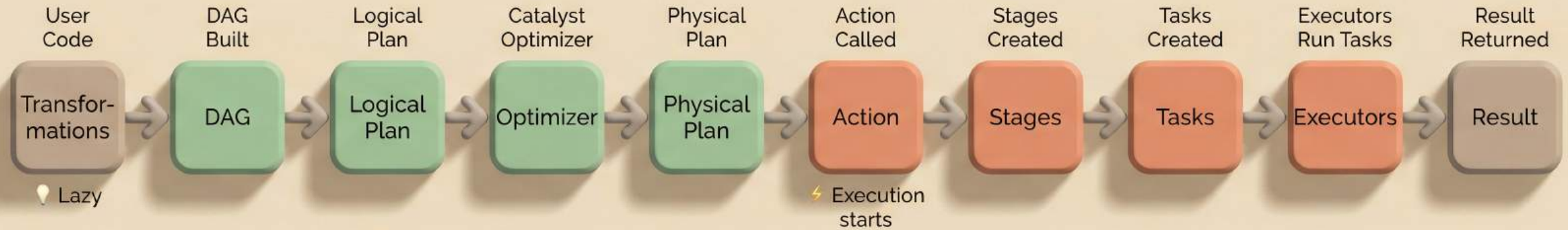


Failure Handling

- Retry task
- Relaunch executor

PySpark Execution Timeline

@TRRaveendra



Understand execution order • See where execution starts • Connect all internals



Transformations

- Lazy
- Create new RDD/DataFrame
- No execution

- map
- filter
- select
- join

@TRRaveendra

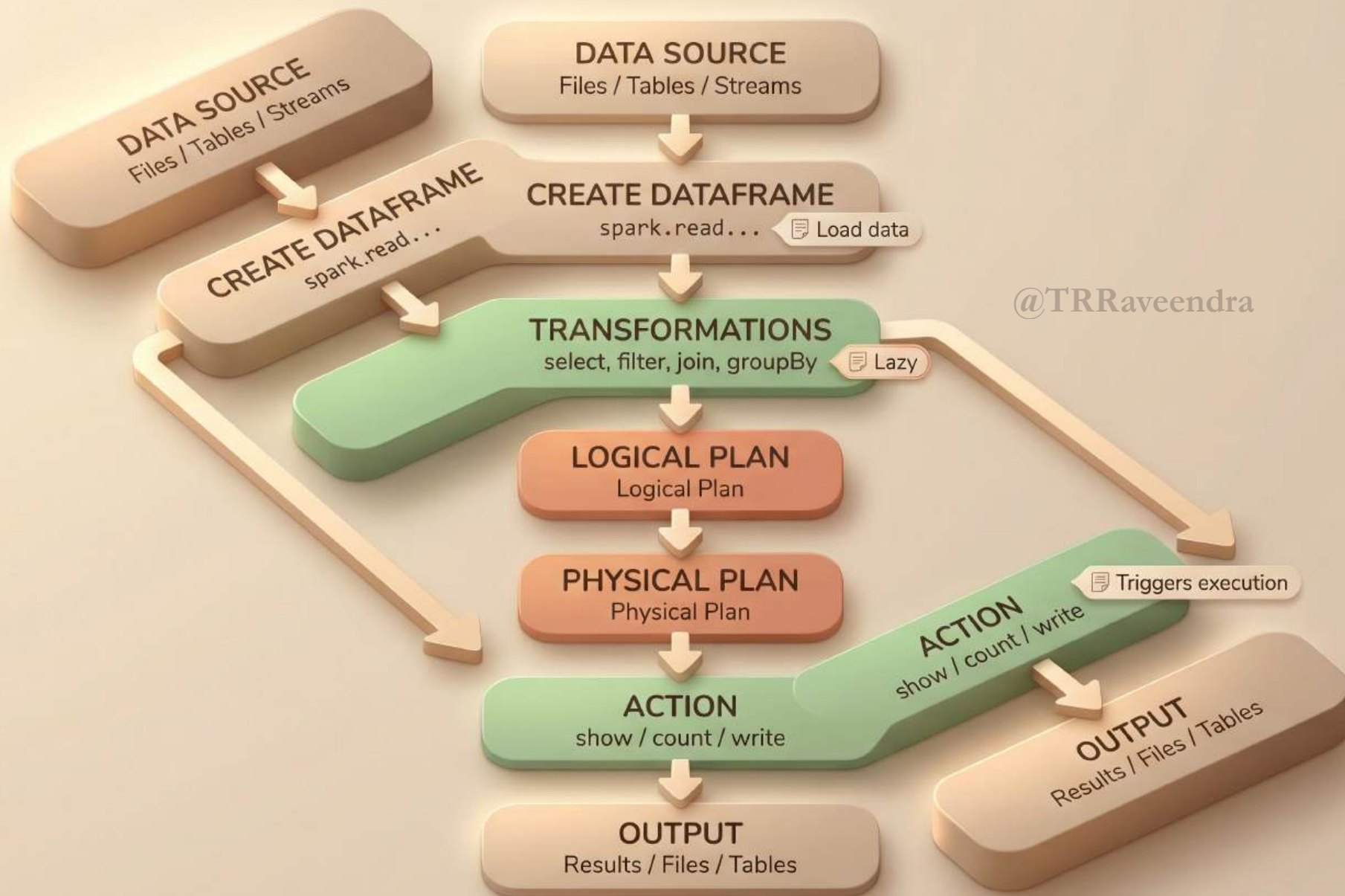


Actions

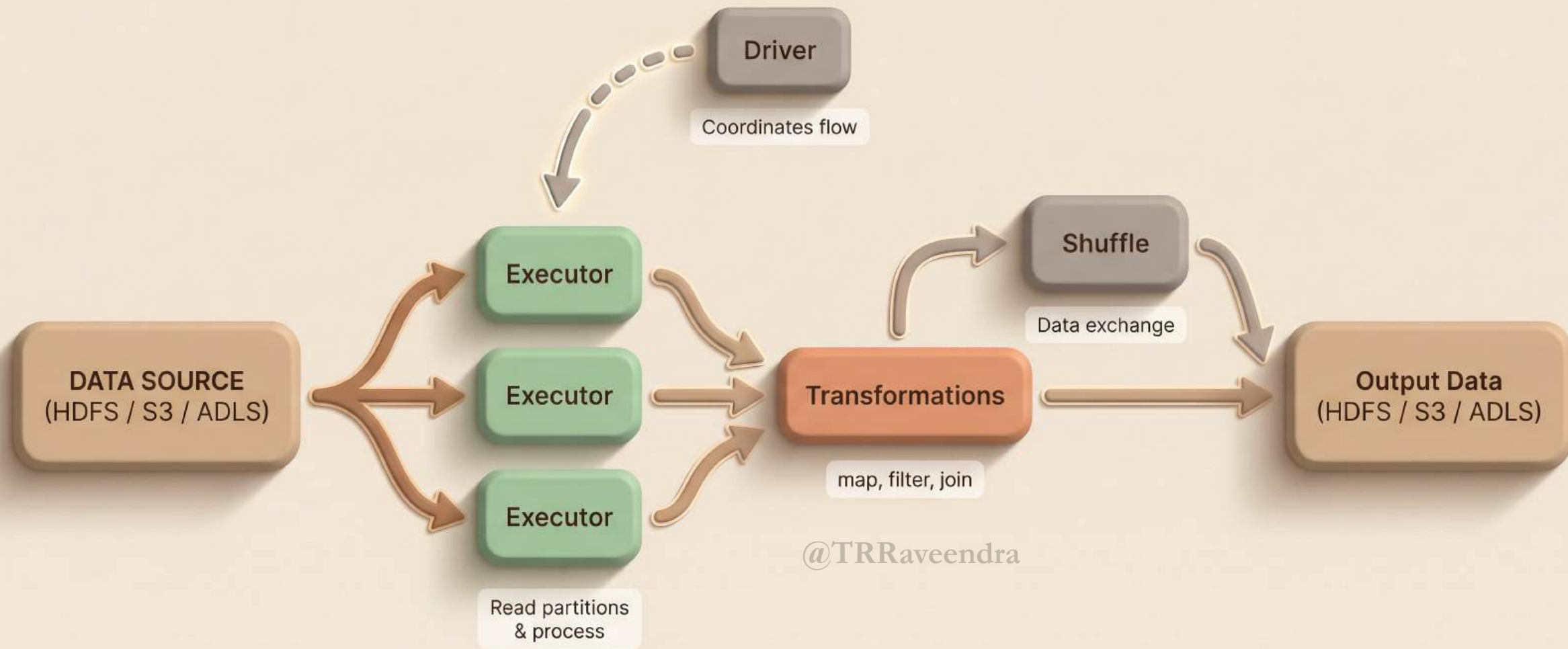
- Trigger execution
- Return result
- Start job

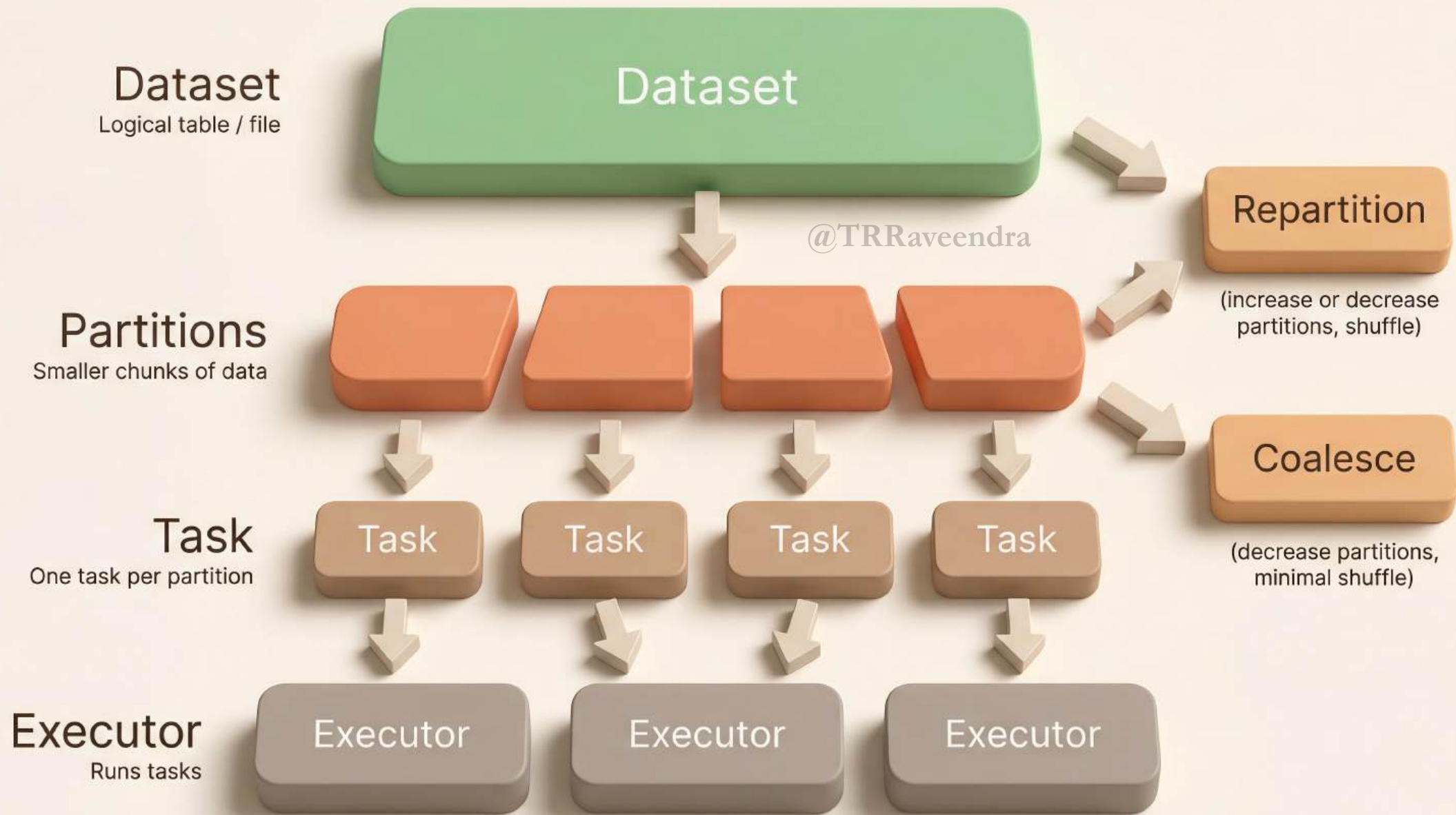
- count
- collect
- show
- save

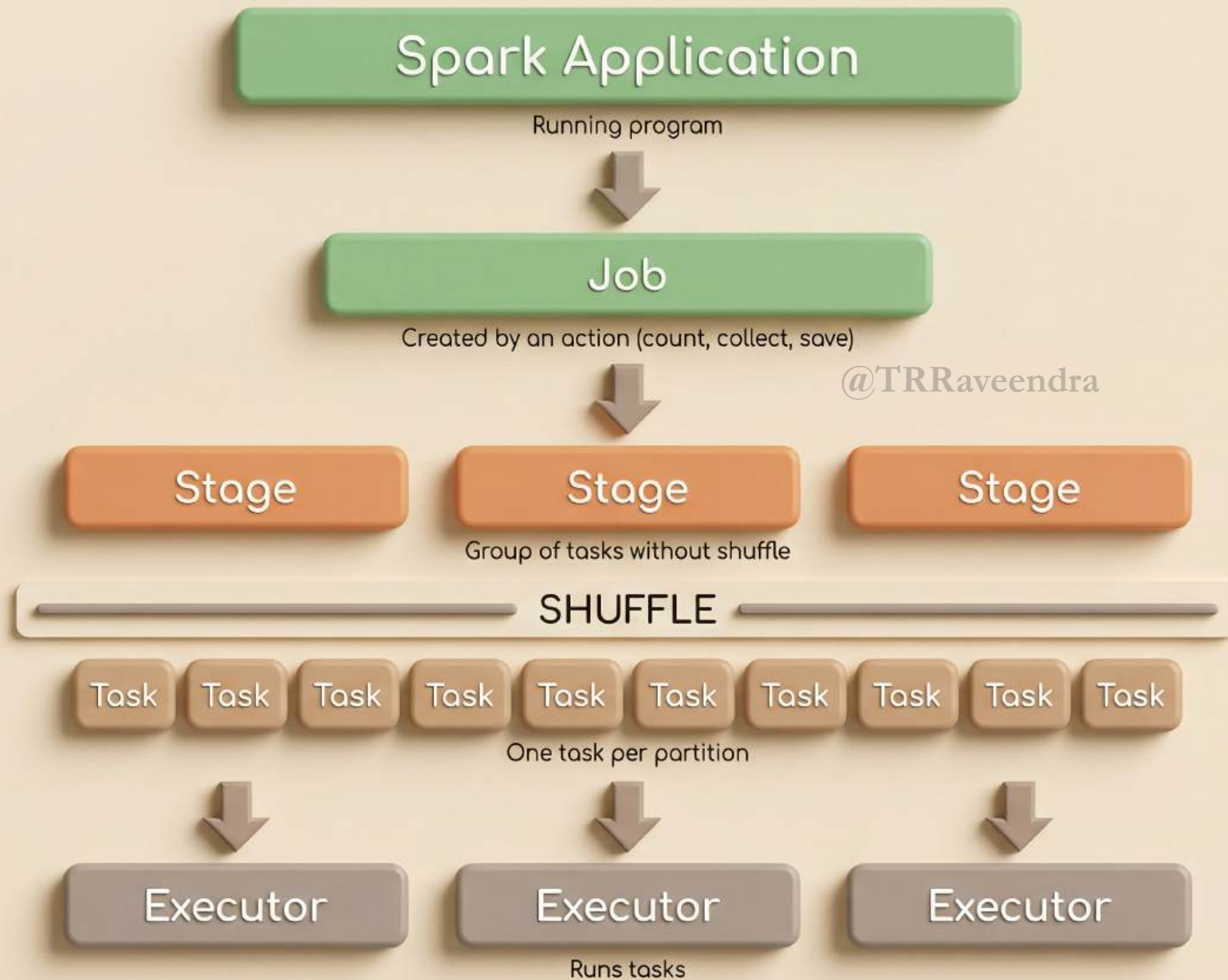




Understand lazy evaluation • See planning vs execution • Understand DataFrame lifecycle







@TRRaveendra

Understand hierarchy: Job → Stage → Task | Know what creates a job | Know what defines a stage | Know what a task is

spark.read



- Load data
- Create DataFrame
- Lazy

```
spark.read.csv(...)  
spark.read.parquet(...)  
spark.read.table(...)
```

@TRRaveendra

df.write



- Save DataFrame
- Output data
- Trigger action

```
df.write.parquet(...)  
df.write.saveAsTable(...)  
df.write.mode("overwrite")
```

**Same
DataFrame
Engine**

Transformations

- Lazy
- Build execution plan
- No job triggered

• `select()` • `join()`
• `filter()` • `groupBy()`

@TRRaveendra

Actions

- Trigger execution
- Return result
- Start job

• `show()` • `collect()`
• `count()` • `write()`



Learning Goals

Understand lazy evaluation • Know what triggers execution • Recognize transformations vs actions

PySpark DataFrame Save Modes

Append

Add new data

Overwrite

Replace existing data

@TRRaveendra

ErrorIfExists

Fail if data exists

Ignore

Do nothing if exists

```
df.write.mode("append").parquet(path)
```

FORMAT

`format('parquet / csv /
json / delta')`

File type

MODE

`mode('append / overwrite
/ ignore / error')`

Save behavior

PARTITION BY

`partitionBy('col')`

Organize output

@TRRaveendra

COMPRESSION

`option('compression','snappy')`

Reduce size

TABLE

`saveAsTable('table')`

Write to metastore

PATH

`save('path')`

Write to location

```
df.write.format('parquet') \  
  .mode('overwrite') \  
  .option('compression','snappy') \  
  .partitionBy('country') \  
  .save('/data/output')
```

Learning Goals: Understand common write options • Know what each option controls • Configure writes correctly

DataFrame Schema

Columns & data types

@TRRaveendra

Metadata Transformations

Change structure, not values



Column Rename

withColumnRenamed(), toDF()



Column Add/Replace

withColumn()



Explode/Struct Ops

explode(), select('struct.*')



Drop Column

drop()



Alias

alias()



Reorder Columns

select()



Change Data Type

cast(), selectExpr()

No heavy shuffle / no aggregation

```
df2 = df.withColumnRenamed('old', 'new') \
        .withColumn('age', col('age').cast('int')) \
        .drop('temp')
```

• Understand metadata vs data transformations • Know common schema APIs • Design cleaner DataFrames

What Is an Action?

PySpark Action

Action triggers execution of the accumulated transformation plan

⚡ Job starts here

Transformations → Action → Job Runs → Result

@TRRaveendra

👁 Viewing & Retrieving Data

Viewing & Retrieving

show() collect() head()
first() take(n)

Return data to driver

df.show() df.count()

df.count() df.write.mode("overwrite").parquet("/data/out")

📊 Calculation & Statistics

Calculation & Stats

count() sum() min()
avg() max()

Return computed value

💾 Writing / Saving Data

df.write cat

df.write
df.write.mode().save()
df.write.mode() saveAsTabl

⚠ Avoid *collect()* on large datasets

✓ Prefer show() or *limit()*

Each line triggers a Spark job

Understand what actions are · Know common action APIs · Recognize execution

PySpark Query Tuning Checklist — Clean 3D Educational Poster

Student-Focused Performance Optimization Guide

@TRRaveendra

✓ 1. Data Reading

- ✓ Use column pruning (.select() specific columns)
- ✓ Use predicate pushdown (filter at source)
- ✓ Prefer Parquet/Delta (columnar formats)

Why: Reduces I/O and memory usage

✓ 2. Transformations

- ✓ Use narrow transformations (map, filter)
- ✓ Avoid unnecessary repartition()
- ✓ Filter early (before joins/aggregations)

Why: Minimizes data movement

✓ 3. Joins

- ✓ Broadcast small tables (<200MB)
- ✓ Check join type (inner vs outer)
- ✓ Avoid Cartesian joins

Why: Prevents shuffle explosion

✓ 4. Shuffle & Partitioning

- ✓ Tune partitions (200MB per partition)
- ✓ Enable AQE (Adaptive Query Execution)
- ✓ Watch shuffle size in Spark UI

Why: Optimizes parallelism

✓ 5. Caching

- ✓ Cache reused DataFrames (.cache())
- ✓ Unpersist when done (.unpersist())
- ✓ Choose storage level wisely

Why: Avoids recomputation

✓ 6. Validation

- ✓ Check explain plan (.explain())
- ✓ Use Spark UI (localhost:4040)
- ✓ Measure runtime consistently

Why: Confirms optimizations work



🎓 **Learning Goals:** 🎓 Follow tuning steps • Prevent common mistakes • Build efficient pipelines



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame using spark session variable createDataFrame method  
# spark.createDataFrame(data,schema)
```

```
df = spark.createDataFrame([1,'ravi','2','Reshwanth',4,'Vikranth'],  
["id","name"])
```

```
# default minimum partitions 1
```

```
# displaying data can be done in three ways.
```

```
display(df) # html table format
```

```
df.show() # text table format
```

```
df.collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame using spark session variable range method  
# spark.range(n)
```

```
df = spark.range(10)
```

```
# default minimum partitions 1
```

```
# displaying data can be done in three ways.
```

```
display(df) # html table format
```

```
df.show() # text table format
```

```
df.collect() # ROW format (raw data in python collection list and  
tuple)
```

```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# converting RDD to DataFrame
```

```
rdd = spark.sparkContext.parallelize([(1,"ravi"),(2,"sindhu"),(3,"reshwanth"),  
(4,"vishnu")])  
df = rdd.toDF(["id","name"])
```

```
# default minimum partitions 1
```

```
# displaying data can be done in three ways.
```

```
display(df) # html table format
```

```
df.show() # text table format
```

```
df.collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# creating dataframe using SPARK READ API. spark.read.csv
```

```
df = spark.read.csv("dbfs://databricks-datasets/asa/airlines/1987.csv", header=True)
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# creating dataframe using SPARK READ API. spark.read.json  
# Reading json files
```

```
df = spark.read.json("dbfs:/databricks-datasets/structured-streaming/events/file-  
0.json")
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra

# creating dataframe using SPARK READ API. spark.read.text
# Reading text files

df = spark.read.text("dbfs:/databricks-datasets/credit-card-fraud/dbcl-10.txt")

# default minimum partitions 1
# Depends on data it will create N No of partitions and block size is 128MB
# Get Default Partition block size in bytes.
spark.conf.get("spark.sql.files.maxPartitionBytes")

# displaying data can be done in three ways. use limit function to get few rows.
display(df.limit(100)) # html table format
df.limit(10).show() # text table format
df.limit(10).collect() # ROW format (raw data in python collection list and
tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame from reading Parquet Files
```

```
# creating dataframe using SPARK READ API. spark.read.parquet  
# Reading parquet files
```

```
df = spark.read.parquet("/databricks-datasets/amazon/data20K")
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and  
tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame from reading delta table by path
```

```
# creating dataframe using SPARK READ API. spark.read.format("delta")
```

```
# Reading delta files
```

```
df = spark.read.format("delta").load("dbfs://databricks-datasets/nyctaxi-with-  
zipcodes/subsampled/")
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame from reading avro files
```

```
# creating dataframe using SPARK READ API. spark.read.format("avro")
```

```
# Reading avro files
```

```
df = spark.read.format("avro").load("/avrofiles_path/")
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame from reading orc files
```

```
# creating dataframe using SPARK READ API. spark.read.format("orc")
```

```
# Reading orc files
```

```
df = spark.read.orc("/orcfiles_path/")
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```

```
#LearningWithRavi https://youtube.com/@TRRaveendra

# Creating DataFrame from reading xml files

# Direct API is not available for xml and excel files.
# install spark-xml library at cluster level from maven central repository
# install spark-xml_2.12 version to avoid issues.

df = spark.read.format("com.databricks.spark.xml")\
    .option("rootTag", "emp")\
    .option("rowTag", "row")\
    .load("/FileStore/tables/emp_xml.xml")

# default minimum partitions 1
# Depends on data it will create N No of partitions and block size is 128MB
# Get Default Partition block size in bytes.
spark.conf.get("spark.sql.files.maxPartitionBytes")

# displaying data can be done in three ways. use limit function to get few rows.
display(df.limit(100)) # html table format
df.limit(10).show() # text table format
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```

```
#LearningWithRavi https://youtube.com/@TRRaveendra

# Creating DataFrame from reading excel files

# Direct API is not available for excel and excel files.
# install spark-excel library at cluster level from maven central repository
# install this version - com.crealytics:spark-excel_2.12:0.13.5

df =
spark.read.format("com.crealytics.spark.excel").load("/FileStore/tables/emp.xlsx", header=True)

# default minimum partitions 1
# Depends on data it will create N No of partitions and block size is 128MB
# Get Default Partition block size in bytes.
spark.conf.get("spark.sql.files.maxPartitionBytes")

# displaying data can be done in three ways. use limit function to get few rows.
display(df.limit(100)) # html table format
df.limit(10).show() # text table format
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra

# Creating DataFrame from reading image files

# use format as image for images.
# spark.read.format("image").load("path")

df = spark.read.format("image").load("/databricks-datasets/flower_photos/roses")

# default minimum partitions 1
# Depends on data it will create N No of partitions and block size is 128MB
# Get Default Partition block size in bytes.
spark.conf.get("spark.sql.files.maxPartitionBytes")

# displaying data can be done in three ways. use limit function to get few rows.
display(df.limit(100)) # html table format
df.limit(10).show() # text table format
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra

# Creating DataFrame from reading binary files like images, videos, audio files..

# spark.read.format("binaryFile").load("path")

df = spark.read.format("binaryFile").load("/databricks-
datasets/cctvVideos/mp4/train")
# default minimum partitions 1
# Depends on data it will create N No of partitions and block size is 128MB
# Get Default Partition block size in bytes.
spark.conf.get("spark.sql.files.maxPartitionBytes")

# displaying data can be done in three ways. use limit function to get few rows.
display(df.limit(100)) # html table format
df.limit(10).show() # text table format
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra

# Creating DataFrame from a table

# spark.table("table_name")

df = spark.table("table_name")

# default minimum partitions 1
# Depends on data it will create N No of partitions and block size is 128MB
# Get Default Partition block size in bytes.
spark.conf.get("spark.sql.files.maxPartitionBytes")

# displaying data can be done in three ways. use limit function to get few rows.
display(df.limit(100)) # html table format
df.limit(10).show() # text table format
df.limit(10).collect() # ROW format (raw data in python collection list and
tuple)
```



```
#LearningWithRavi https://youtube.com/@TRRaveendra
```

```
# Creating DataFrame from a sql query
```

```
# spark.sql("select * from table_name")
```

```
df = spark.sql("select * from emp")
```

```
# default minimum partitions 1
```

```
# Depends on data it will create N No of partitions and block size is 128MB
```

```
# Get Default Partition block size in bytes.
```

```
spark.conf.get("spark.sql.files.maxPartitionBytes")
```

```
# displaying data can be done in three ways. use limit function to get few rows.
```

```
display(df.limit(100)) # html table format
```

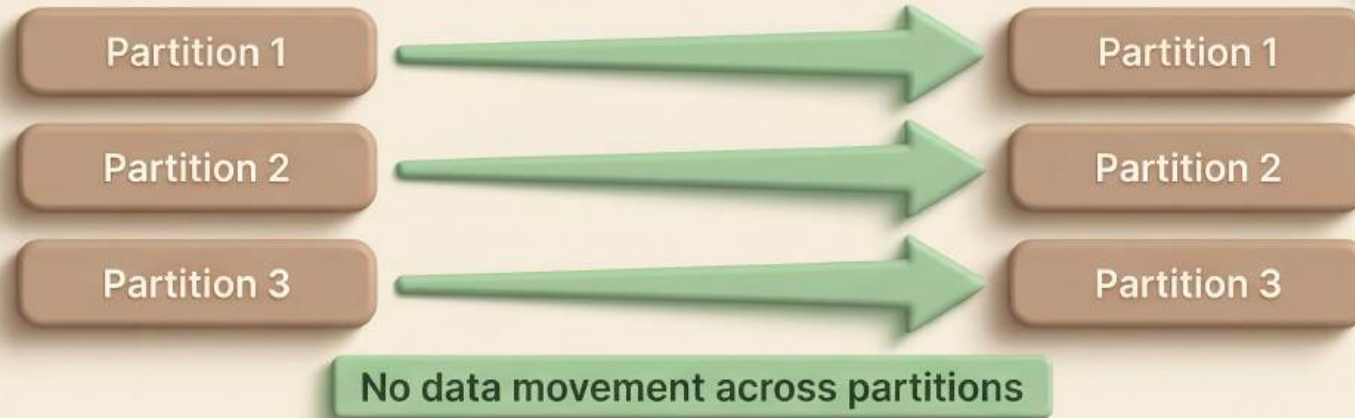
```
df.limit(10).show() # text table format
```

```
df.limit(10).collect() # ROW format (raw data in python collection list and tuple)
```

NARROW TRANSFORMATION

Each output partition depends on only one input partition

Note:
✓ No shuffle
✓ Fast execution



@TRRaveendra

Row-Level Transformations

- select()
- withColumn()
- drop()
- cast()

Row Filtering

- filter()
- where()

Projection & Renaming

- alias()
- withColumnRenamed()

Column Expressions

- selectExpr()
- when / otherwise()
- expr()

Explode

- explode()
 - posexplode()
- (as long as no repartition occurs)

What Narrow Transformations DO NOT Do

- ✗ No shuffle
- ✗ No aggregation
- ✗ No repartition

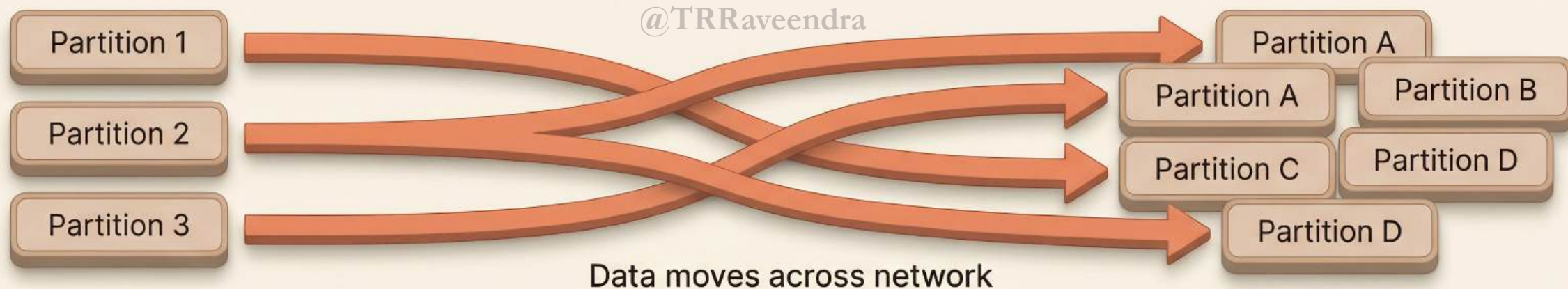
```
df2 = df.select("id", "name") \ .filter(col("age") > 30) \ .withColumn("age_plus_1", col("age") + 1)
```

All narrow transformations → fast execution

WIDE TRANSFORMATION

Output partitions depend on multiple input partitions

⚠ Shuffle required ⚠ Expensive operation



⌘ Aggregations

groupBy()
agg()

🔗 Joins

join()

↻ Repartitioning

repartition()
coalesce() (when
increasing partitions)

📊 Set Operations

distinct()
dropDuplicates()

🔍 Sorting

orderBy()
sort()

What Wide
Transformations
CAUSE

- ✓ Shuffle
- ✓ Network IO
- ✓ Disk spill
(possible)

```
df2 = df.groupBy("country") \
        .count() \
        .orderBy("count", ascending=False)
```

Wide transformations → shuffle

Shuffle

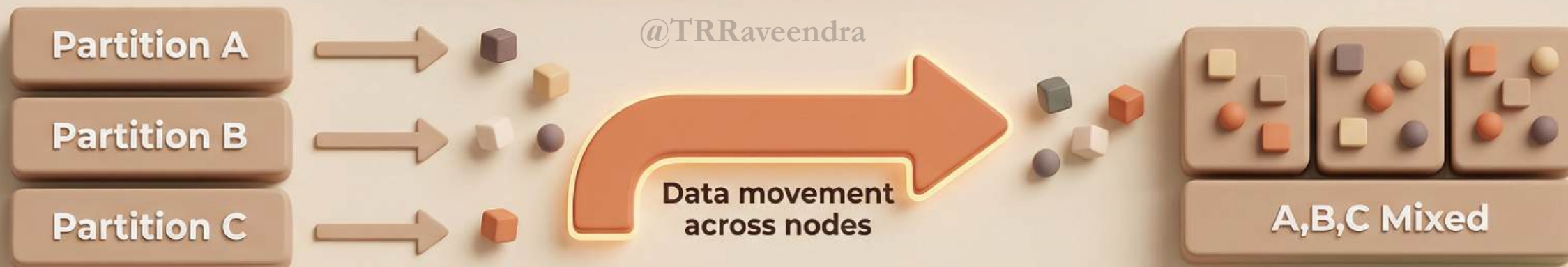
Data is redistributed across partitions over the network



Expensive



Causes network + disk IO



Joins:
"join()"



Aggregations:
"groupBy()", "agg()"



Distinct: "distinct()",
"dropDuplicates()"



Repartition:
"repartition()"



Sorting:
"orderBy()", "sort()"



Window Functions:
"over(Window...)"

```
df2 = df.groupBy('country').count()
```

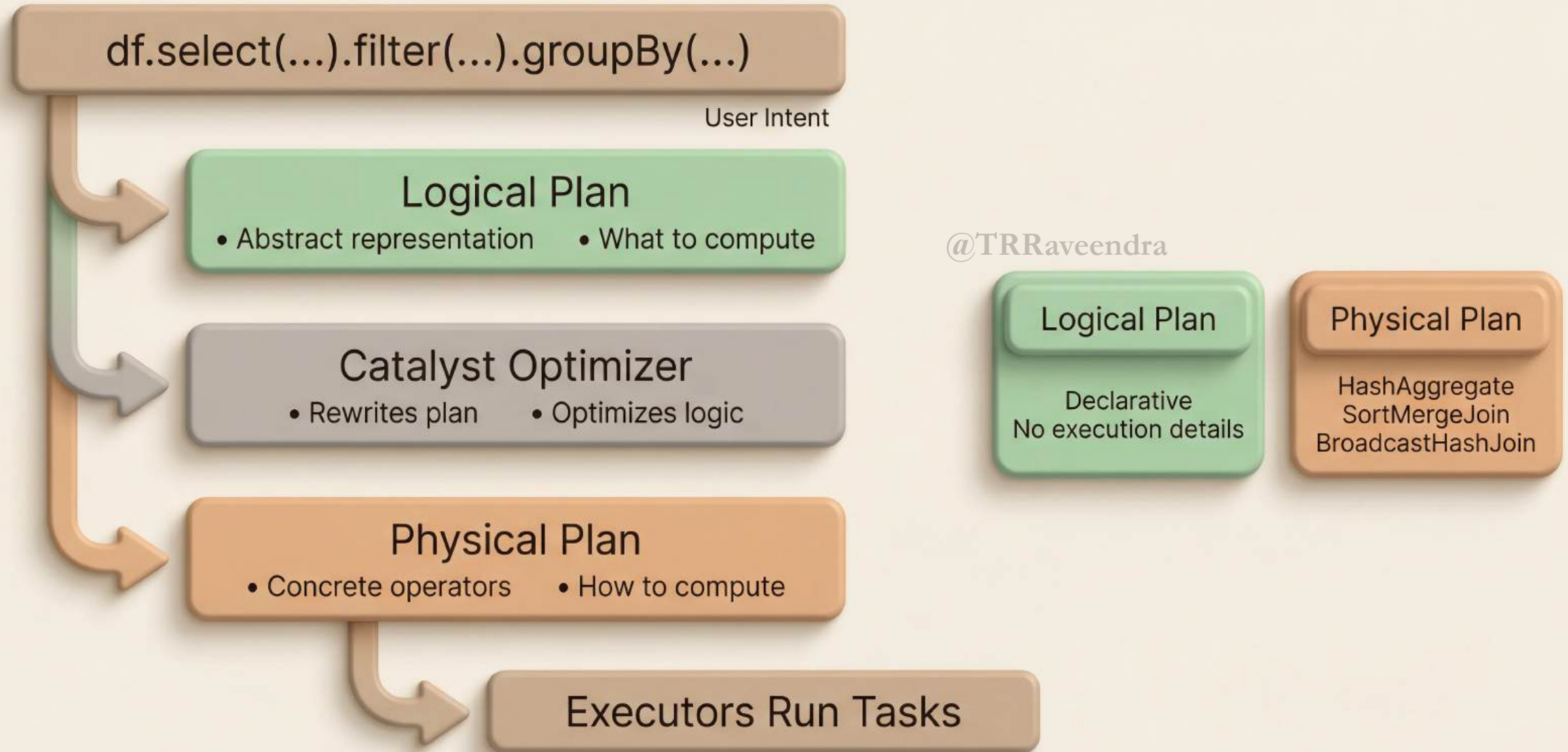
↑ Shuffle happens here

What Usually Does NOT Trigger Shuffle

- ✓ select
- ✓ filter
- ✓ withColumn
- ✓ drop

Learning Goals: "Recognize shuffle triggers", "Predict expensive operations", "Write optimized pipelines"

PySpark Logical Plan vs. Physical Plan



Learning Goals: Understand planning stages • Know difference between logical & physical • Learn where optimization happens



Partition
Chunk of distributed data



Default Partitions
Based on data source

Repartition

`repartition(n)`



Increase or decrease
partitions (shuffle)

Coalesce

`coalesce(n)`



Reduce partitions
(no shuffle)

Partition By

`partitionBy("col")`



Directory-based layout

Bucketing

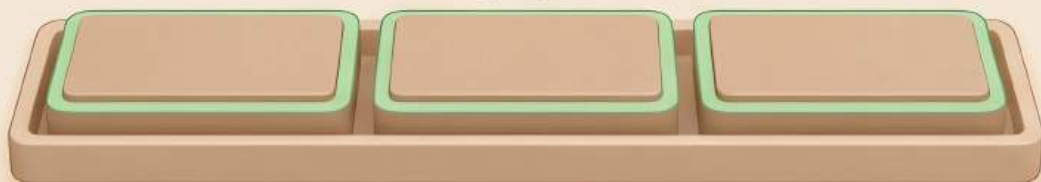
`bucketBy(n, "col")`



Optimize joins

@TRRaveendra

Few large partitions



Potential for data skew & idle executors

Many smaller partitions

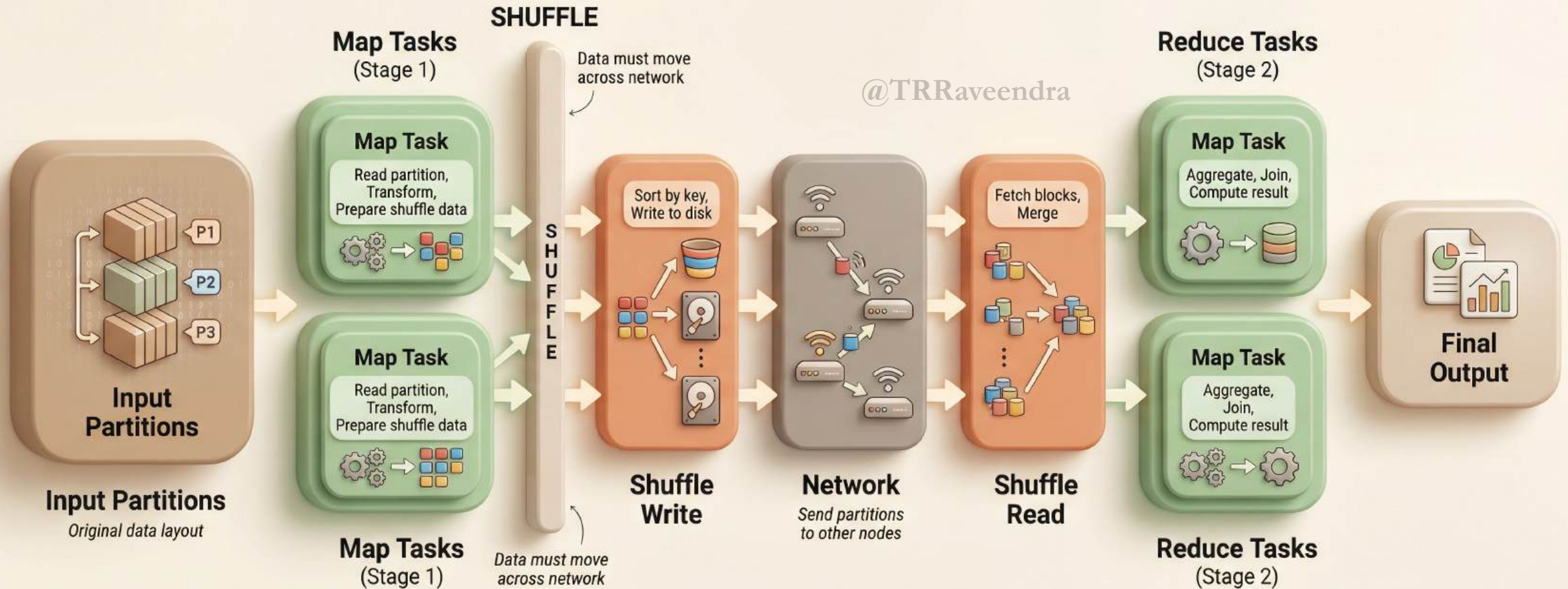


Better parallelism, but higher overhead

```
df = df.repartition(200, "country")
```

Understand partitioning • Choose correct strategy • Improve performance

Apache Spark Shuffle Overview — Clean 3D Educational Poster (Student-Focused)



✓ Understand what shuffle is • 👁 See where network transfer happens • 💰 Know why shuffle is expensive • ➡ Recognize shuffle boundary in Spark jobs

Reading Metadata And Data Functions

READ_METADATA	READ_DATA
df.columns	df.show()
df.printschema()	display(df)
df.dtypes	df.collect()
df.schema	df.describe("*").show()

Modifying Metadata And MetaData.

MODIFY_METADATA	MODIFY_DATA
df.WithColumnn()	df.fillna("val")
df.WithColumnRenamed()	df.dropna()
df.drop("column")	df.dropDuplicates()
df.toDF()	df.filter("col==condition")
	df.select("*")
	df.distinct()

Any Data Engineering Project Common Operations Are **Metadata Validations** and **Data Validations**.
 Like validating **Columns**, **Data Types**,
 Validating **Data** like **Duplicate Data**, **Null Data** using above functions.

https://github.com/raveendratal/PysparkTelugu/blob/master/emp_duplicate.csv

```
1 dbutils.fs.put("/FileStore/tables/emp_duplicate.csv", ""EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO
2 7369,SMITH,CLERK,7902,17-12-1980,800,null,20
3 7499,ALLEN,SALESMAN,7698,20-02-1981,1600,300,30
4 7521,WARD,SALESMAN,7698,22-02-1981,1250,500,30
5 7566,JONES,MANAGER,7839,04-02-1981,2975,null,20
6 7654,MARTIN,SALESMAN,7698,21-09-1981,1250,1400,30
7 7698,SGR,MANAGER,7839,05-01-1981,2850,null,30
8 7782,RAVI,MANAGER,7839,06-09-1981,2450,null,10
9 7788,SCOTT,ANALYST,7566,19-04-1987,3000,null,20
10 7839,KING,PRESIDENT,null,01-11-1981,5000,null,10
11 7844,TURNER,SALESMAN,7698,09-08-1981,1500,0,30
12 7876,ADAMS,CLERK,7788,23-05-1987,1100,null,20
13 7900,JAMES,CLERK,7698,12-03-1981,950,null,30
14 7902,FORD,ANALYST,7566,12-03-1981,3000,null,20
15 7934,MILLER,CLERK,7782,01-03-1982,1300,null,10
16 1234,SEKHAR,doctor,7777,null,667,78,80
17 7369,SMITH,CLERK,7902,17-12-1980,800,null,20
18 7499,ALLEN,SALESMAN,7698,20-02-1981,1600,300,30
19 7521,WARD,SALESMAN,7698,22-02-1981,1250,500,30
20 7566,JONES,MANAGER,7839,04-02-1981,2975,null,20
21 7654,MARTIN,SALESMAN,7698,21-09-1981,1250,1400,30
22 7698,SGR,MANAGER,7839,05-01-1981,2850,null,30
23 7782,RAVI,MANAGER,7839,06-09-1981,2450,null,10
24 7788,SCOTT,ANALYST,7566,19-04-1987,3000,null,20
25 7839,KING,PRESIDENT,null,01-11-1981,5000,null,10
26 7844,TURNER,SALESMAN,7698,09-08-1981,1500,0,30
27 7876,ADAMS,CLERK,7788,23-05-1987,1100,null,20
28 7900,JAMES,CLERK,7698,12-03-1981,950,null,30
29 7902,FORD,ANALYST,7566,12-03-1981,3000,null,20
30 7934,MILLER,CLERK,7782,01-03-1982,1300,null,10
31 1234,RAM,CLERK,7457,null,494,588,80
32 null,null,null,null,null,null,null,null
33 null,null,null,null,null,null,null,null
34 null,null,null,null,null,null,null,null""",True)
```

Creating DataFrame Reading CSV File

Cmd 15

```
1 # Creating DataFrame reading csv file using spark.read.csv API
2 # header=True if file is having first row is header
3 # inferSchema=True if we want datatypes based on data
4 # nullValue='null' if file is having null value and treating as null instead of string.
5 df_emp_csv = spark.read.csv("/FileStore/tables/emp_duplicate.csv",header=True,inferSchema=True,nullValue="null")
```

► (2) Spark Jobs

▼  df_emp_csv: pyspark.sql.dataframe.DataFrame

```
EMPNO: integer
ENAME: string
JOB: string
MGR: integer
HIREDATE: date
SAL: integer
COMM: integer
DEPTNO: integer
```

Creating DataFrame Reading CSV File

```
df = spark.read.csv("path/to/your/file.csv")
```

```
df = spark.read \
    .format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
    .load("path/to/your/csv/file.csv")
```

```
# Now you can manipulate the DataFrame using PySpark operations
df.show() # Display the DataFrame content
df.printSchema() # View the DataFrame schema
```

Purpose:

- Reads a CSV (Comma-Separated Values) file into a PySpark DataFrame, a distributed tabular data structure.
- Enables parallel processing and efficient handling of large datasets.

Usage:

There are two main ways to use **spark.read.csv** or **spark.read.format("csv").load()**

Key Options:

- **header (bool)**: Set to **True** if the first row contains column names (default: **False**).
- **delimiter (str)**: The character separating values in the CSV file (default: comma **,**).
- **inferSchema (bool)**: Automatically infer data types for columns (default: **False**).
- **mode (str)**: Handling mode for malformed rows (e.g., **PERMISSIVE**, **DROPMALFORMED**, **FAILFAST**). See Spark documentation for details.
- **nullValue (str)**: The string representing null values in the CSV file (default varies based on data source).
- **quote (str)**: Character used for quoting values (default: **"**).
- **escape (str)**: Character used for escaping special characters within quotes (default: **"**).
- **charset (str)**: Character encoding of the CSV file (default: **utf-8**).
- **compression (str)**: Compression codec used in the CSV file (e.g., **gzip**, **bzip2**).

READ_METADATA

`df.columns`

`df.printschema()`

`df.dtypes`

`df.schema`

Cmd 16

```
1 #Displaying columns using df.columns Action
2 df_emp_csv.columns
```

```
Out[4]: ['EMPNO', 'ENAME', 'JOB', 'MGR', 'HIREDATE', 'SAL', 'COMM', 'DEPTNO']
```

`df.columns` in PySpark

In PySpark, `df.columns` is a property of a DataFrame that returns a list of all column names present in the DataFrame. This list is essential for various data manipulation and analysis tasks.

READ_METADATA

`df.columns`

`df.printSchema()`

`df.dtypes`

`df.schema`

Cmd 17

```
1 #Displaying Printable schema using df.printSchema Action
2 df_emp_csv.printSchema()
```

```
root
|-- EMPNO: integer (nullable = true)
|-- ENAME: string (nullable = true)
|-- JOB: string (nullable = true)
|-- MGR: integer (nullable = true)
|-- HIREDATE: date (nullable = true)
|-- SAL: integer (nullable = true)
|-- COMM: integer (nullable = true)
|-- DEPTNO: integer (nullable = true)
```

`df.printSchema()` in PySpark

In PySpark, `df.printSchema()` is a method used to display the schema of a DataFrame in a hierarchical format. The schema provides information about the structure of the DataFrame, including:

- **Column names:** The names of each column in the DataFrame.
- **Data types:** The data type of each column (e.g., `StringType`, `IntegerType`, `DoubleType`, `TimestampType`, etc.).
- **Nullability:** Whether the column allows null values (nullable or not nullable).

Why Use `df.printSchema()`?

- **Understanding Data Structure:** It helps you visualize the organization of your data, especially when dealing with complex DataFrames.
- **Validating Data Transformations:** After applying transformations like `select`, `filter`, or `join`, you can use `printSchema()` to verify the resulting DataFrame's structure.
- **Debugging Data Issues:** If you encounter unexpected behavior or errors, inspecting the schema can help identify potential problems.

READ_METADATA

`df.columns`

`df.printschema()`

`df.dtypes`

`df.schema`

Cmd 18

```
1 #Displaying column with data type using df.dtypes action
2 df_emp_csv.dtypes
```

```
Out[6]: [('EMPNO', 'int'),
         ('ENAME', 'string'),
         ('JOB', 'string'),
         ('MGR', 'int'),
         ('HIREDATE', 'date'),
         ('SAL', 'int'),
         ('COMM', 'int'),
         ('DEPTNO', 'int')]
```

`df.dtypes` in PySpark

In PySpark, `df.dtypes` is a property that returns a list of tuples, where each tuple contains a column name and its corresponding data type.

This is useful for inspecting the schema of a DataFrame and understanding the data types of different columns.

Common Use Cases:

1. **Checking Data Types:** Verify the data types of columns before performing operations to ensure compatibility and avoid errors.
2. **Data Cleaning and Preparation:** Identify columns that need type conversions or handling of missing values.
3. **Feature Engineering:** Determine appropriate data types for new features created during data preprocessing.
4. **Data Validation:** Check if data types align with expected values and business rules.

READ_METADATA

df.columns

df.printschema()

df.dtypes

df.schema

```
Cmd 18
Python
1 #Displaying pyspark internal schema using df.schema action
2 df_emp_csv.schema

Out[9]: StructType([StructField('EMPNO', IntegerType(), True), StructField('ENAME', StringType(), True), StructField('JOB', StringType(), True), StructField('MGR', IntegerType(), True), StructField('HIREDATE', DateType(), True), StructField('SAL', IntegerType(), True), StructField('COMM', IntegerType(), True), StructField('DEPTNO', IntegerType(), True)])
```

```
1 #importing all data types from
2 from pyspark.sql.types import *
3 # creating pyspark schema
4 emp_schema = StructType([StructField('EMPNO', IntegerType(), True),
5                             StructField('ENAME', StringType(), True),
6                             StructField('JOB', StringType(), True),
7                             StructField('MGR', IntegerType(), True),
8                             StructField('HIREDATE', DateType(), True),
9                             StructField('SAL', IntegerType(), True),
10                            StructField('COMM', IntegerType(), True),
11                            StructField('DEPTNO', IntegerType(), True)])
```

df.schema in PySpark

In PySpark, `df.schema` is a property that returns the schema of a DataFrame as a `StructType` object. This schema provides detailed information about the structure of the DataFrame, including:

- **Column names:** The names of each column in the DataFrame.
- **Data types:** The data type of each column (e.g., `StringType`, `IntegerType`, `DoubleType`, `TimestampType`, etc.).
- **Nullability:** Whether the column allows null values (nullable or not nullable).

Common Use Cases:

1. **Inspecting Data Structure:** Understanding the schema helps you visualize the organization of your data and identify potential issues.
2. **Validating Data Transformations:** After applying transformations like `select`, `filter`, or `join`, you can use `df.schema` to verify the resulting DataFrame's structure.
3. **Debugging Data Issues:** If you encounter unexpected behavior or errors, inspecting the schema can help identify potential problems.
4. **Creating New DataFrames:** You can use the schema to create new DataFrames with specific structures.
5. **Data Quality Checks:** Validate data types and nullability constraints to ensure data integrity.

Additional Considerations:

- **Accessing Individual Fields:** You can access individual fields of the `StructType` to get specific information about columns.
- **Modifying Schema:** While you can't directly modify the schema of an existing DataFrame, you can create a new DataFrame with a modified schema using techniques like `withColumn`, `drop`, or `select`.

READ_DATA

`df.show()`

`display(df)`

`df.collect()`

`df.describe("*").show()`

`df.show()` in PySpark:

Purpose:

- Displays the contents of a PySpark DataFrame in a human-readable tabular format, similar to a spreadsheet.
- Provides a quick way to inspect the data and gain insights into its structure and values.

Optional Arguments:

- `n (int)`: Specify the number of rows to display. Defaults to 20.
- `truncate (bool)`: Whether to truncate long strings (default: `True`). If `False`, they might wrap to the next line.
- `vertical (bool)`: Display the DataFrame vertically (one record per line, default: `False`).

Tips and Considerations:

- Use `df.count()` to get the total number of rows in the DataFrame before using `show(n)`.
- For very large DataFrames, displaying all rows might be slow or impractical. Consider using `head()` or `tail()` to view specific sections of the data.
- Explore alternative visualization libraries like `matplotlib` or `seaborn` for more advanced visualizations in PySpark environments that support them.

Cmd 20

```
1 # displaying data in text table format using df.show()
2 df_emp_csv.show()
```

► (1) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800	null	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-02-04	2975	null	20
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30
7698	SGR	MANAGER	7839	1981-01-05	2850	null	30
7782	RAVI	MANAGER	7839	1981-09-06	2450	null	10
7788	SCOTT	ANALYST	7566	1987-04-19	3000	null	20
7839	KING	PRESIDENT	null	1981-11-01	5000	null	10
7844	TURNER	SALESMAN	7698	1981-08-09	1500	0	30
7876	ADAMS	CLERK	7788	1987-05-23	1100	null	20
7900	JAMES	CLERK	7698	1981-03-12	950	null	30
7902	FORD	ANALYST	7566	1981-03-12	3000	null	20
7934	MILLER	CLERK	7782	1982-03-01	1300	null	10
1234	SEKHAR	doctor	7777	null	667	78	80
7369	SMITH	CLERK	7902	1980-12-17	800	null	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30

READ_DATA

`df.show()`

`display(df)`

`df.collect()`

`df.describe("*").show()`

`df.collect()` in PySpark

In PySpark, `df.collect()` is an action that gathers all the rows of a DataFrame and returns them as a list of Row objects to the driver program. This is useful for small datasets where you want to perform further processing on the data locally.

Key Points:

- **Data Transfer:** `collect()` triggers a data shuffle, moving all data from the distributed cluster to the driver node.
- **Memory Considerations:** For large datasets, using `collect()` can lead to `OutOfMemoryError` on the driver node.
- **Use with Caution:** It's generally recommended to avoid `collect()` on large datasets, as it can significantly impact performance.

Common Use Cases:

- **Debugging:** Inspecting a small sample of data to understand its structure and content.
- **Small Data Processing:** Performing simple operations on a small dataset that can be handled on the driver node.
- **Exporting Data:** Writing the data to a local file or database.

Alternatives to `collect()`:

- **take(n):** Returns the first n rows of the DataFrame.
- **Iterating over Partitions:** Use `foreachPartition` to process data in chunks, reducing memory pressure.
- **Writing to a File:** Use `write.format("parquet").save()` or other file formats to save the DataFrame to disk.

```
Cmd 20:
1 # displaying data in row array format using df.collect()
2 df_emp_csv.collect()

▶ (1) Spark Jobs

Out[13]: [Row(EMPNO=7369, ENAME='SMITH', JOB='CLERK', MGR=7902, HIREDATE=datetime.date(1980, 12, 17), SAL=800, COMM=None, DEPTNO=20),
Row(EMPNO=7499, ENAME='ALLEN', JOB='SALESMAN', MGR=7698, HIREDATE=datetime.date(1981, 2, 20), SAL=1600, COMM=300, DEPTNO=30),
Row(EMPNO=7521, ENAME='WARD', JOB='SALESMAN', MGR=7698, HIREDATE=datetime.date(1981, 2, 22), SAL=1250, COMM=500, DEPTNO=30),
Row(EMPNO=7566, ENAME='JONES', JOB='MANAGER', MGR=7839, HIREDATE=datetime.date(1981, 2, 4), SAL=2975, COMM=None, DEPTNO=20),
Row(EMPNO=7654, ENAME='MARTIN', JOB='SALESMAN', MGR=7698, HIREDATE=datetime.date(1981, 9, 21), SAL=1250, COMM=1400, DEPTNO=30),
Row(EMPNO=7698, ENAME='SGR', JOB='MANAGER', MGR=7839, HIREDATE=datetime.date(1981, 1, 5), SAL=2850, COMM=None, DEPTNO=30),
Row(EMPNO=7782, ENAME='RAVI', JOB='MANAGER', MGR=7839, HIREDATE=datetime.date(1981, 9, 6), SAL=2450, COMM=None, DEPTNO=10),
Row(EMPNO=7788, ENAME='SCOTT', JOB='ANALYST', MGR=7566, HIREDATE=datetime.date(1987, 4, 19), SAL=3000, COMM=None, DEPTNO=20),
Row(EMPNO=7839, ENAME='KING', JOB='PRESIDENT', MGR=None, HIREDATE=datetime.date(1981, 11, 1), SAL=5000, COMM=None, DEPTNO=10),
Row(EMPNO=7844, ENAME='TURNER', JOB='SALESMAN', MGR=7698, HIREDATE=datetime.date(1981, 8, 9), SAL=1500, COMM=0, DEPTNO=30),
Row(EMPNO=7876, ENAME='ADAMS', JOB='CLERK', MGR=7788, HIREDATE=datetime.date(1987, 5, 23), SAL=1100, COMM=None, DEPTNO=20),
Row(EMPNO=7900, ENAME='JAMES', JOB='CLERK', MGR=7698, HIREDATE=datetime.date(1981, 3, 12), SAL=950, COMM=None, DEPTNO=30)]
```

READ_DATA

`df.show()`

`display(df)`

`df.collect()`

`df.describe("*").show()`

`df.display()` in PySpark

`df.display()` is a function typically used within the Databricks environment to provide a more interactive and visually appealing way to display a PySpark DataFrame. It offers several advantages over the standard `df.show()` method:

Key Features and Benefits:

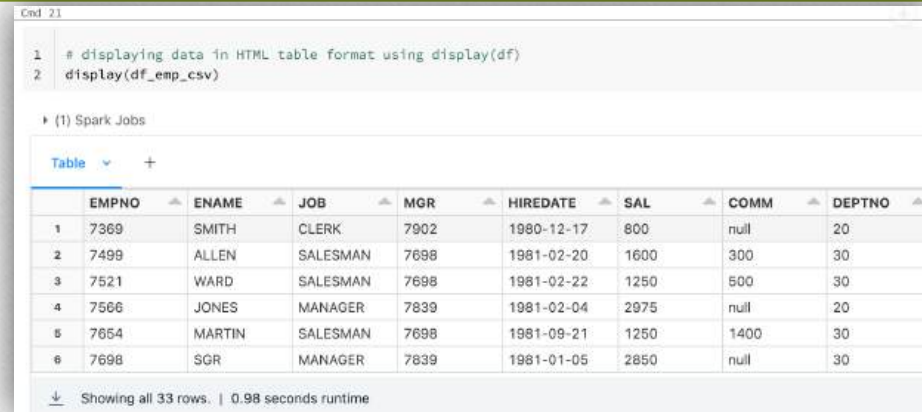
- Interactive Visualization:**
 - Allows you to explore the DataFrame directly within the notebook interface.
 - Provides features like filtering, sorting, and zooming.
- Customizable Display:**
 - Offers options to customize the table's appearance, such as column widths, formatting, and color schemes.
- Integration with Other Visualization Tools:**
 - Can be used in conjunction with other visualization libraries like Plotly or Bokeh to create more sophisticated visualizations.

How to Use:

- In Databricks Notebooks:**
 - Simply call `display(df)` to display the DataFrame in a tabular format.
- In Standard PySpark Environments:**
 - While `display()` is primarily a Databricks feature, you might be able to use it in other environments if you have the necessary libraries and configurations. However, it's not a standard PySpark function.

Important Note:

- The specific behavior and capabilities of `display()` can vary depending on the environment (e.g., Databricks, local PySpark setup).
- For basic DataFrame display in standard PySpark environments, `df.show()` is the primary method.



	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	1980-12-17	800	null	20
2	7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
3	7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
4	7566	JONES	MANAGER	7839	1981-02-04	2975	null	20
5	7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30
6	7698	SGR	MANAGER	7839	1981-01-05	2850	null	30

READ_DATA

`df.show()`

`display(df)`

`df.collect()`

`df.describe("*").show()`

```
Cmd 22
```

```
1 # displaying dataframe stats using df.describe()
2 df_emp_csv.describe("*").show()
```

► (2) Spark Jobs

summary	EMPNO	ENAME	JOB	MGR	SAL	COMM	DEPTNO
count	30	30	30	28	30	10	30
mean	7293.733333333334	null	null	7730.571428571428	1973.7	506.6	26.0
stddev	1655.846630690118	null	null	111.74422797966989	1182.2016908848655	514.8238533712283	16.525841832592434
min	1234	ADAMS	ANALYST	7457	494	0	10
max	7934	WARD	doctor	7902	5000	1400	80

`df.describe()` function to compute basic statistics for numeric and string columns in your DataFrame.

`df.describe()`

This function calculates the following statistics:

- **count:** Number of non-null values
- **mean:** Average value
- **stddev:** Standard deviation
- **min:** Minimum value
- **max:** Maximum value

summary: Contains the summary statistics (count, mean, stddev, min, max)

MODIFY_DATA

`df.fillna("val")`

`df.dropna()`

`df.dropDuplicates()`

`df.filter("col==condition")`

`df.select("*")`

`df.distinct()`

Cmd 23

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 df_emp_csv.dropna(how='all').show()
```

▶ (1) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800	null	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-02-04	2975	null	20
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30
7698	SGR	MANAGER	7839	1981-01-05	2850	null	30
7782	RAVI	MANAGER	7839	1981-09-06	2450	null	10

```
# Drop rows with any null value
df_dropped_any = df.dropna()
```

```
# Drop rows with all null values
df_dropped_all = df.dropna(how='all')
```

```
# Drop rows with less than 2 non-null values
df_dropped_thresh = df.dropna(thresh=2)
```

```
# Drop rows with null values in specific columns
df_dropped_subset = df.dropna(subset=["Age"])
```

`df.dropna()` in PySpark

In PySpark, `df.dropna()` is a function used to remove rows or columns containing null values from a DataFrame. It helps clean and prepare data for further analysis.

Key Parameters:

- **how:**
 - 'any': Drops a row if any value in that row is null. (Default)
 - 'all': Drops a row only if all values in that row are null.
- **thresh:**
 - Specifies the minimum number of non-null values required in a row to keep it.
- **subset:**
 - A list of column names to consider for dropping rows.

MODIFY_DATA

`df.fillna("val")`

`df.dropna()`

`df.dropDuplicates()`

`df.filter("col==condition")`

`df.select("*")`

`df.distinct()`

Cmd 24

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 df_emp_csv.dropna(how='all').distinct().show()
```

▶ (2) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7900	JAMES	CLERK	7698	1981-03-12	950	null	30
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30
7566	JONES	MANAGER	7839	1981-02-04	2975	null	20
7369	SMITH	CLERK	7902	1980-12-17	800	null	20
7839	KING	PRESIDENT	null	1981-11-01	5000	null	10
7844	TURNER	SALESMAN	7698	1981-08-09	1500	0	30
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7902	FORD	ANALYST	7566	1981-03-12	3000	null	20
1234	SEKHAR	doctor	7777	null	667	78	80

`df.distinct()` in PySpark

In PySpark, `df.distinct()` is a transformation that returns a new DataFrame containing distinct rows from the original DataFrame. It removes duplicate rows based on all columns

Key Points:

- **All Columns:** `distinct()` considers all columns when determining duplicates. If any combination of values is identical in two rows, one of them will be removed.
- **Performance Considerations:** For large datasets, `distinct()` can be computationally expensive, especially if there are many columns. In such cases, consider using techniques like partitioning or filtering to optimize performance.

By using `df.distinct()`, you can efficiently remove duplicate rows from your PySpark DataFrames and ensure accurate data analysis.

MODIFY_DATA

`df.fillna("val")`

`df.dropna()`

`df.dropDuplicates()`

`df.filter("col==condition")`

`df.select("*")`

`df.distinct()`

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 # df.dropDuplicates() -- remove duplicates based on selected Unique Column Combination
6 df_emp_csv.dropna(how='all').distinct().dropDuplicates(["EMPNO"]).show()
```

▶ (2) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1234	SEKHAR	doctor	7777	null	667	78	80
7369	SMITH	CLERK	7902	1980-12-17	800	null	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-02-04	2975	null	20
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30
7698	SGR	MANAGER	7839	1981-01-05	2850	null	30

`df.dropDuplicates()` in PySpark

This function is used to remove duplicate rows from a PySpark DataFrame. Unlike `df.distinct()`, which removes duplicates based on all columns, `df.dropDuplicates()`

Parameters:

- **subset:** A list of column names to consider for duplicate checking. If not specified, all columns are considered.

Key Points:

- **Column-Based Duplicates:** `dropDuplicates()` provides flexibility to remove duplicates based on specific columns, which can be useful in various data cleaning and preprocessing scenarios.
- **Performance Considerations:** Similar to `distinct()`, `dropDuplicates()` can be computationally expensive for large datasets. Consider using techniques like partitioning or filtering to optimize performance.

MODIFY_DATA

`df.fillna("val")`

`df.dropna()`

`df.dropDuplicates()`

`df.filter("col==condition")`

`df.select("*")`

`df.distinct()`

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 # df.dropDuplicates() -- remove duplicates based on selected Unique Column Combination
6 # df.fillna({key:val,}) Converting null values to actual values
7 df_emp_csv.dropna(how='all').distinct().dropDuplicates(["EMPNO"]).fillna({"COMM":0,"MGR":9999,"HIREDATE":"9999-12-31"}).show()
```

▶ (2) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1234	SEKHAR	doctor	7777	9999-12-31	667	78	80
7369	SMITH	CLERK	7902	1980-12-17	800	0	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-02-04	2975	0	20
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30
7698	SGR	MANAGER	7839	1981-01-05	2850	0	30
7782	PAVT	MANAGER	7839	1981-09-06	2450	0	10

`df.fillna()` in PySpark

The `df.fillna()` function in PySpark is used to replace missing values (null values) in a DataFrame with a specified value or another column. It's a valuable tool for data cleaning and preparation.

Parameters:

- **value:** The value to replace null values with. It can be a scalar value or a dictionary mapping column names to values.
- **subset:** A list of column names to apply the filling operation on.

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

df.withColumn() in PySpark

The `df.withColumn()` function in PySpark is used to add a new column to an existing DataFrame or to modify an existing column. It provides a flexible way to transform and enrich your data.

Parameters:

- **colName:** The name of the new column to be added or the name of the column to be modified.
- **colExpr:** A column expression that defines the value or transformation for the new or modified column. This can be a simple literal, a column reference, or a complex expression involving arithmetic operations, functions, and other DataFrame operations.

Key Points:

- **Column Creation:** You can create new columns based on existing columns or literal values.
- **Column Modification:** You can modify existing columns by applying transformations or calculations.
- **Complex Expressions:** The `colExpr` can involve various functions and operators, allowing for sophisticated data manipulation.

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 # df.dropDuplicates() -- remove duplicates based on selected Unique Column Combination
6 # df.fillna({key:val,}) Converting null values to actual values
7 df_emp_quality = df_emp_csv.dropna(how='all').distinct().dropDuplicates(["EMPNO"]).fillna({"COMM":0,"MGR":9999,"HIREDATE":"9999-12-31"})
8 # df.withColumn for adding a new column
9 from pyspark.sql.functions import * # import all functions
10 df_emp_quality = df_emp_quality.withColumn("CREATED_BY",lit("RAVI")).withColumn("TOTAL_SALARY",col("SAL")+col("COMM"))
11 df_emp_quality.show()
```

► (2) Spark Jobs

► df_emp_quality: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 8 more fields]

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	CREATED_BY	TOTAL_SALARY
1234	SEKHAR	doctor	7777	9999-12-31	667	78	80	RAVI	745
7369	SMITH	CLERK	7902	1980-12-17	800	0	20	RAVI	800
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30	RAVI	1900
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30	RAVI	1750
7566	JONES	MANAGER	7839	1981-02-04	2975	0	20	RAVI	2975
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30	RAVI	2650
7698	SGR	MANAGER	7839	1981-01-05	2850	0	30	RAVI	2850
7782	RAVI	MANAGER	7839	1981-09-06	2450	0	10	RAVI	2450
7788	SCOTT	ANALYST	7566	1987-04-19	3000	0	20	RAVI	3000
7839	KING	PRESIDENT	9999	1981-11-01	5000	0	10	RAVI	5000
7844	TURNER	SALESMAN	7698	1981-08-09	1500	0	30	RAVI	1500
7876	ADAMS	CLERK	7788	1987-05-23	1100	0	20	RAVI	1100
7900	JAMES	CLERK	7698	1981-03-12	950	0	30	RAVI	950
7902	FORD	ANALYST	7566	1981-03-12	3000	0	20	RAVI	3000
7934	MILLER	CLERK	7782	1982-03-01	1300	0	10	RAVI	1300

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

df.withColumnRenamed () in PySpark

The `df.withColumnRenamed ( )` function in PySpark is used to rename a specific column in a DataFrame. It takes the old column name and the new column name as arguments.

Parameters:

- **existingColumnName:** The name of the existing column to be renamed.
- **newColumnName:** The new name for the column.

Key Points:

- **Simple Renaming:** It's a straightforward way to change column names for better readability or consistency.
- **Data Preparation:** Renaming columns can be useful in preparing data for analysis or machine learning tasks.
- **Avoid Naming Conflicts:** Ensure that the new column name doesn't conflict with existing column names.

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 # df.dropDuplicates() -- remove duplicates based on selected Unique Column Combination
6 # df.fillna({key:val,}) Converting null values to actual values
7 df_emp_quality = df_emp_csv.dropna(how='all').distinct().dropDuplicates(["EMPNO"]).fillna({"COMM":0,"MGR":9999,"HIREDATE":"9999-12-31"})
8 # df.withColumn for adding a new column
9 from pyspark.sql.functions import * # import all functions
10 df_emp_quality = df_emp_quality.withColumn("CREATED_BY",lit("RAVI")).withColumn("TOTAL_SALARY",col("SAL")+col("COMM"))
11 # df.withColumnRenamed for renaming a column
12 df_emp_quality1 = df_emp_quality.withColumnRenamed("SAL","SALARY")
13 df_emp_quality1.show()
```

▶ (2) Spark Jobs

▶ df_emp_quality: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 8 more fields]
▶ df_emp_quality1: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 8 more fields]

EMPNO	ENAME	JOB	MGR	HIREDATE	SALARY	COMM	DEPTNO	CREATED_BY	TOTAL_SALARY
1234	SEKHAR	doctor	7777	9999-12-31	667	78	80	RAVI	745
7369	SMITH	CLERK	7902	1980-12-17	800	0	20	RAVI	800
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30	RAVI	1900
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30	RAVI	1750
7566	JONES	MANAGER	7839	1981-02-04	2975	0	20	RAVI	2975
7654	MARTIN	SALESMAN	7698	1981-09-21	1250	1400	30	RAVI	2650
7698	SGR	MANAGER	7839	1981-01-05	2850	0	30	RAVI	2850
7782	RAVI	MANAGER	7839	1981-09-06	2450	0	10	RAVI	2450
7788	SCOTT	ANALYST	7566	1987-04-19	3000	0	20	RAVI	3000
7839	KING	PRESIDENT	9999	1981-11-01	5000	0	10	RAVI	5000
7844	TURNER	SALESMAN	7698	1981-08-09	1500	0	30	RAVI	1500
7876	ADAMS	CLERK	7788	1987-05-23	1100	0	20	RAVI	1100
7900	JAMES	CLERK	7698	1981-03-12	950	0	30	RAVI	950
7902	FORD	ANALYST	7566	1981-03-12	3000	0	20	RAVI	3000
7934	MILLER	CLERK	7782	1982-03-01	1300	0	10	RAVI	1300

MODIFY_METADATA

`df.WithColumn()`

`df.WithColumnRenamed()`

`df.drop("column")`

`df.toDF()`

`df.drop()` in PySpark

The `df.drop()` function in PySpark is used to remove one or more columns from a DataFrame. It returns a new DataFrame with the specified columns removed.

Parameters:

- `cols`: A list of column names to be dropped.

Key Points:

- **Column Removal:** The `drop()` function removes the specified columns from the DataFrame, creating a new DataFrame without those columns.
- **Multiple Columns:** You can drop multiple columns by providing a list of column names to the `drop()` function.
- **Data Cleaning:** Dropping unnecessary columns can help simplify your data and improve performance.

By effectively using `df.drop()`, you can clean and prepare your data for further analysis and modeling in PySpark.

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 # df.dropDuplicates() -- remove duplicates based on selected Unique Column Combination
6 # df.fillna({key:val,}) Converting null values to actual values
7 df_emp_quality = df_emp_csv.dropna(how='all').distinct().dropDuplicates(["EMPNO"]).fillna({"COMM":0,"MGR":9999,"HIREDATE":"9999-12-31"})
8 # df.withColumn for adding a new column
9 from pyspark.sql.functions import * # import all functions
10 df_emp_quality = df_emp_quality.withColumn("CREATED_BY",lit("RAVI")).withColumn("TOTAL_SALARY",col("SAL")+col("COMM"))
11 # df.withColumnRenamed for renaming a column
12 df_emp_quality1 = df_emp_quality.withColumnRenamed("SAL","SALARY")
13 # df.drop() for removing a column/columns
14 df_emp_quality1 = df_emp_quality1.drop("MGR","HIREDATE")
15 df_emp_quality1.show()
```

► (2) Spark Jobs

► df_emp_quality: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 8 more fields]

► df_emp_quality1: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 6 more fields]

EMPNO	ENAME	JOB	SALARY	COMM	DEPTNO	CREATED_BY	TOTAL_SALARY
1234	SEKHAR	doctor	667	78	80	RAVI	745
7369	SMITH	CLERK	800	0	20	RAVI	800
7499	ALLEN	SALESMAN	1600	300	30	RAVI	1900
7521	WARD	SALESMAN	1250	500	30	RAVI	1750
7566	JONES	MANAGER	2975	0	20	RAVI	2975
7654	MARTIN	SALESMAN	1250	1400	30	RAVI	2650
7698	SGR	MANAGER	2850	0	30	RAVI	2850
7782	RAVI	MANAGER	2450	0	10	RAVI	2450
7788	SCOTT	ANALYST	3000	0	20	RAVI	3000
7839	KING	PRESIDENT	5000	0	10	RAVI	5000
7844	TURNER	SALESMAN	1500	0	30	RAVI	1500
7876	ADAMS	CLERK	1100	0	20	RAVI	1100
7900	JAMES	CLERK	950	0	30	RAVI	950
7902	FORD	ANALYST	3000	0	20	RAVI	3000
7934	MILLER	CLERK	1300	0	10	RAVI	1300

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

df.toDF() in PySpark

The `toDF()` function in PySpark is primarily used to:

1. **Rename Columns:**

- You can provide a list of new column names to rename the existing columns in a DataFrame.
- This is useful for standardizing column names or making them more descriptive.

2. **Create a New DataFrame from an RDD:**

- When working with RDDs (Resilient Distributed Datasets), you can convert them into DataFrames using `toDF()`. This allows you to leverage the powerful DataFrame API for data manipulation and analysis.

```
# Rename columns
new_df = df.toDF("FirstName", "YearsOld")

new_df.show()
```

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

```
1 # remove null rows using df.dropna(how='all')
2 # df.dropna(how='all') -- remove rows if all columns are having null values
3 # df.dropna(how='any') -- remove rows if any column is having a null value
4 # df.distinct() -- remove duplicate rows for all column combination
5 # df.dropDuplicates() -- remove duplicates based on selected Unique Column Combination
6 # df.fillna({key:val,}) Converting null values to actual values
7 df_emp_quality = df_emp_csv.dropna(how='all').distinct().dropDuplicates(["EMPNO"]).fillna({"COMM":0,"MGR":9999,"HIREDATE":"9999-12-31"})
8 # df.withColumn for adding a new column
9 from pyspark.sql.functions import * # import all functions
10 df_emp_quality = df_emp_quality.withColumn("CREATED_BY",lit("RAVI")).withColumn("TOTAL_SALARY",col("SAL")+col("COMM"))
11 # df.withColumnRenamed for renaming a column
12 df_emp_quality1 = df_emp_quality.withColumnRenamed("SAL","SALARY")
13 # df.drop() for removing a column/columns
14 df_emp_quality1 = df_emp_quality1.drop("MGR","HIREDATE")
15 # df.toDF() for renaming all columns
16 df_emp_quality1.toDF("EMP_NUMBER","EMP_NAME","EMP_JOB","EMP_SALARY","EMP_DEPTNO","EMP_COMM","EMP_CREATED_DATE","EMP_TOTAL_SALARY").show()
```

► (2) Spark Jobs

► df_emp_quality: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 8 more fields]

► df_emp_quality1: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 6 more fields]

EMP_NUMBER	EMP_NAME	EMP_JOB	EMP_SALARY	EMP_DEPTNO	EMP_COMM	EMP_CREATED_DATE	EMP_TOTAL_SALARY
1234	SEKHAR	doctor	667	78	80	RAVI	745
7369	SMITH	CLERK	800	0	20	RAVI	800
7499	ALLEN	SALESMAN	1600	300	30	RAVI	1900
7521	WARD	SALESMAN	1250	500	30	RAVI	1750
7566	JONES	MANAGER	2975	0	20	RAVI	2975
7654	MARTIN	SALESMAN	1250	1400	30	RAVI	2650
7698	SGR	MANAGER	2850	0	30	RAVI	2850
7782	RAVI	MANAGER	2450	0	10	RAVI	2450
7788	SCOTT	ANALYST	3000	0	20	RAVI	3000
7839	KING	PRESIDENT	5000	0	10	RAVI	5000
7844	TURNER	SALESMAN	1500	0	30	RAVI	1500
7876	ADAMS	CLERK	1100	0	20	RAVI	1100
7900	JAMES	CLERK	950	0	30	RAVI	950
7902	FORD	ANALYST	3000	0	20	RAVI	3000
7934	MILLER	CLERK	1300	0	10	RAVI	1300

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

`df.select()` in PySpark DataFrames:

Purpose:

- The `select()` method is a core function in PySpark DataFrames used for column selection and manipulation.
- It allows you to create a new DataFrame containing a subset of columns from the original DataFrame, including:
 - Specific columns by name
 - All columns
 - Columns with renamed aliases
 - New columns created from expressions involving existing columns

Key Points:

- `select()` is a transformation, returning a new DataFrame that doesn't modify the original one.
- You can use column names as strings or directly reference columns from `df`.
- To select all columns, use `*`.
- To create new columns with aliases, use `col.alias('alias_name')`.
- Expressions can include various PySpark functions for data manipulation and analysis.

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

```
1 #Note: renaming a column in multiple ways.
2 #Renaming all columns -> use toDF(col1,col2,col3...)
3 #Renaming single column -> use withColumnRenamed(oldcolumn,new_column)
4 #Renaming Selected Columns using SELECT or SELECTEXPR transformations using alias.
5 #use selectExpr("colname as new_name","colname2 as new_name2")
6 #use select(col("colname").alias("new_name"),col("colname2").alias("new_name"))
7 df_emp_csv.select(col("EMPNO").alias("EMP_NUMBER"),col("ENAME").alias("EMP_NAME")).show(2)
8 df_emp_csv.selectExpr("empno as EMP_NUMBER","ename as EMP_NAME").show(2)
```

► (2) Spark Jobs

```
+-----+-----+
|EMP_NUMBER|EMP_NAME|
+-----+-----+
|      7369|   SMITH|
|      7499|   ALLEN|
+-----+-----+
```

only showing top 2 rows

```
+-----+-----+
|EMP_NUMBER|EMP_NAME|
+-----+-----+
|      7369|   SMITH|
|      7499|   ALLEN|
+-----+-----+
```

only showing top 2 rows

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

df.selectExpr() in PySpark

The `df.selectExpr()` function in PySpark is a powerful tool for selecting and transforming columns in a DataFrame using SQL-like expressions. It allows you to create new columns, apply complex transformations, and perform various data manipulations.

Parameters:

- `expr1, expr2, ...`: SQL-like expressions that define the columns to be selected or created.

Key Points:

- **SQL-like Syntax:** You can use SQL-like syntax to express complex transformations.
- **Column Creation:** You can create new columns using expressions involving existing columns.
- **Flexibility:** `selectExpr()` offers a high degree of flexibility for data manipulation.

By understanding `df.selectExpr()`, you can efficiently manipulate and transform your DataFrames in PySpark, making your data analysis and processing tasks more concise and expressive.

MODIFY_METADATA

df.WithColumn()

df.WithColumnRenamed()

df.drop("column")

df.toDF()

```
1 #Note: renaming a column in multiple ways.
2 #Renaming all columns -> use toDF(col1,col2,col3...)
3 #Renaming single column -> use withColumnRenamed(oldcolumn,new_column)
4 #Renaming Selected Columns using SELECT or SELECTEXPR transformations using alias.
5 #use selectExpr("colname as new_name","colname2 as new_name2")
6 #use select(col("colname").alias("new_name"),col("colname2").alias("new_name"))
7 df_emp_csv.select(col("EMPNO").alias("EMP_NUMBER"),col("ENAME").alias("EMP_NAME")).show(2)
8 df_emp_csv.selectExpr("empno as EMP_NUMBER","ename as EMP_NAME").show(2)
```

► (2) Spark Jobs

```
+-----+-----+
|EMP_NUMBER|EMP_NAME|
+-----+-----+
|      7369|   SMITH|
|      7499|   ALLEN|
+-----+-----+
```

only showing top 2 rows

```
+-----+-----+
|EMP_NUMBER|EMP_NAME|
+-----+-----+
|      7369|   SMITH|
|      7499|   ALLEN|
+-----+-----+
```

only showing top 2 rows

SET Operators

Set Operators in PySpark

PySpark provides several set operators to manipulate DataFrames, allowing you to perform operations like union, intersection, and difference. These operators are useful for combining and filtering data from different DataFrames.

1. Union:

- Combines two DataFrames, preserving duplicates.
- **Syntax:** `df1.union(df2)`

2. UnionAll:

- Similar to `union`, but explicitly preserves all rows, including duplicates.
- **Syntax:** `df1.unionAll(df2)`

3. Intersection:

- Returns the intersection of two DataFrames, i.e., rows that exist in both DataFrames.
- **Syntax:** `df1.intersect(df2)`

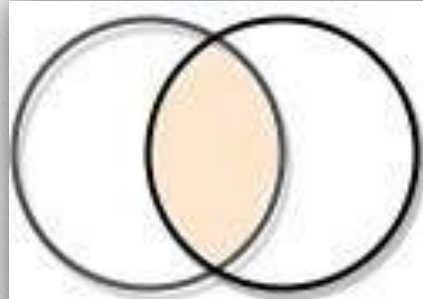
4. Except:

- Returns rows in the first DataFrame that are not present in the second DataFrame.
- **Syntax:** `df1.except(df2)`

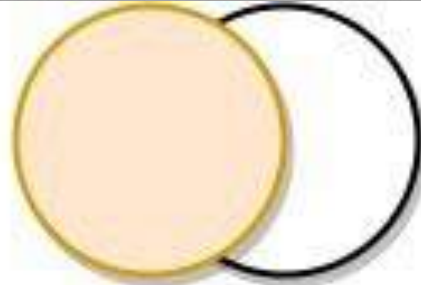
Important Considerations:

- **Schema Compatibility:** The DataFrames being combined must have compatible schemas. This means they should have the same number of columns, and the corresponding columns should have compatible data types.
- **Performance:** For large datasets, these operations can be computationally expensive. Consider partitioning and caching DataFrames to improve performance.
- **Null Values:** Be aware of how null values are handled in set operations. Different databases and systems may have different behaviors.

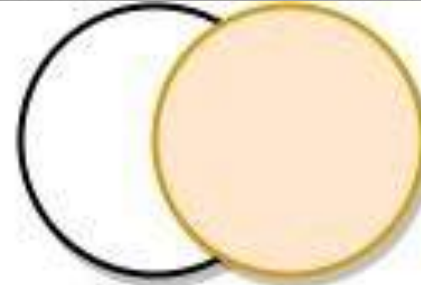
Types of Joins.



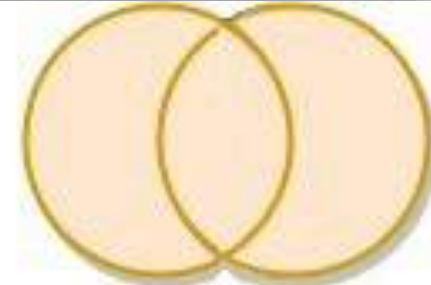
Inner



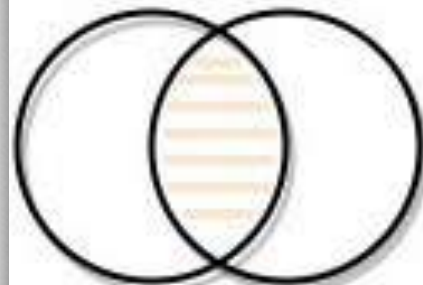
Left Outer



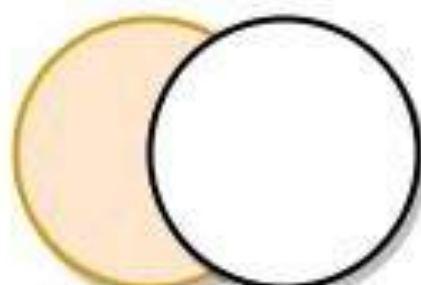
Right Outer



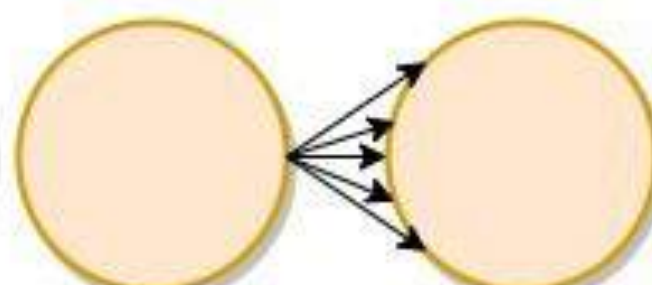
Full Outer



Left Semi



Left Anti



Cross

Types of Joins.

Different Types of Joins in PySpark DataFrames

In PySpark, joins are used to combine rows from two or more DataFrames based on a common column or set of columns. Here are the common types of joins:

1. Inner Join:

- Returns rows that have matching values in both DataFrames.

2. Outer Join:

- Returns all rows from both DataFrames, joining them based on the specified join condition. There are three types of outer joins:
 - **Left Outer Join:**
 - Returns all rows from the left DataFrame, and the matched rows from the right DataFrame. If there's no match in the right DataFrame, null values are filled.
 - **Right Outer Join:**
 - Returns all rows from the right DataFrame, and the matched rows from the left DataFrame. If there's no match in the left DataFrame, null values are filled.
 - **Full Outer Join:**
 - Returns all rows from both DataFrames, joining them based on the specified join condition. If there's no match in either DataFrame, null values are filled.

3. Self-Join:

- Joins a DataFrame with itself, treating it as two different DataFrames. This is useful for comparing rows within the same DataFrame.

Parameters:

- `df1`: The first DataFrame.
- `df2`: The second DataFrame.
- `on`: The join condition, specifying the columns to join on.
- `how`: The type of join (inner, outer, left, right, full).

EMPLOYEE- SCHEMA

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	17-12-1980	800	null	20
7499	ALLEN	SALESMAN	7698	20-02-1981	1600	300	30
7521	WARD	SALESMAN	7698	22-02-1981	1250	500	30
7566	JONES	MANAGER	7839	04-02-1981	2975	null	20
7654	MARTIN	SALESMAN	7698	21-09-1981	1250	1400	30
7698	SGR	MANAGER	7839	05-01-1981	2850	null	30
7782	RAVI	MANAGER	7839	06-09-1981	2450	null	10
7788	SCOTT	ANALYST	7566	19-04-1987	3000	null	20
7839	KING	PRESIDENT	null	01-11-1981	5000	null	10
7844	TURNER	SALESMAN	7698	09-08-1981	1500	0	30
7876	ADAMS	CLERK	7788	23-05-1987	1100	null	20
7900	JAMES	CLERK	7698	12-03-1981	950	null	30
7902	FORD	ANALYST	7566	12-03-1981	3000	null	20
7934	MILLER	CLERK	7782	01-03-1982	1300	null	10
1234	SEKHAR	doctor	7777	null	667	78	80
1234	RAM	CLERK	7457	null	494	588	80

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

```
1 df_emp_csv = spark.read.csv("/FileStore/tables/emp.csv",header=True,inferSchema=True,nullValue="null")
2 df_dept_csv = spark.read.csv("/FileStore/tables/dept.csv",header=True,inferSchema=True,nullValue="null")
```

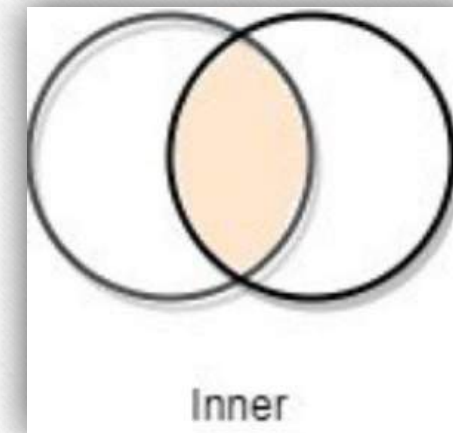
► (4) Spark Jobs

- df_emp_csv: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 6 more fields]
- df_dept_csv: pyspark.sql.dataframe.DataFrame = [Deptno: integer, Dname: string ... 1 more field]

```
1 # inner join
2 df_emp_csv.join(df_dept_csv,"DEPTNO","inner").show()
```

► (2) Spark Jobs

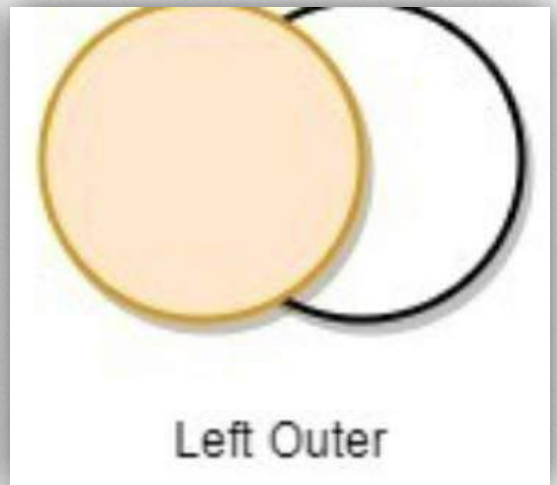
DEPTNO	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	Dname	Loc
20	7369	SMITH	CLERK	7902	2080-12-17	800	null	RESEARCH	DALLAS
30	7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	SALES	CHICAGO
30	7521	WARD	SALESMAN	7698	2081-02-22	1250	500	SALES	CHICAGO
20	7566	JONES	MANAGER	7839	2081-04-02	2975	null	RESEARCH	DALLAS
30	7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	SALES	CHICAGO
30	7698	SGR	MANAGER	7839	2081-05-01	2850	null	SALES	CHICAGO
10	7782	RAVI	MANAGER	7839	2081-06-09	2450	null	ACCOUNTING	NEW YORK
20	7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	RESEARCH	DALLAS
10	7839	KING	PRESIDENT	null	2081-11-17	5000	null	ACCOUNTING	NEW YORK
30	7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	SALES	CHICAGO
20	7876	ADAMS	CLERK	7788	2087-05-23	1100	null	RESEARCH	DALLAS
30	7900	JAMES	CLERK	7698	2081-12-03	950	null	SALES	CHICAGO
20	7902	FORD	ANALYST	7566	2081-12-03	3000	null	RESEARCH	DALLAS
10	7934	MILLER	CLERK	7782	2082-01-23	1300	null	ACCOUNTING	NEW YORK



```
1 # Left Outer join
2 df_emp_csv.join(df_dept_csv,df_emp_csv["DEPTNO"]==df_dept_csv["Deptno"],"left").show()
```

► (2) Spark Jobs

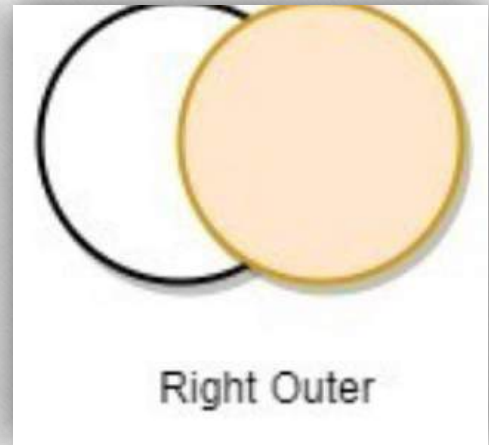
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	Deptno	Dname	Loc
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	20	RESEARCH	DALLAS
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	30	SALES	CHICAGO
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	30	SALES	CHICAGO
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	20	RESEARCH	DALLAS
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	30	SALES	CHICAGO
7698	SGR	MANAGER	7839	2081-05-01	2850	null	30	30	SALES	CHICAGO
7782	RAVI	MANAGER	7839	2081-06-09	2450	null	10	10	ACCOUNTING	NEW YORK
7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	20	20	RESEARCH	DALLAS
7839	KING	PRESIDENT	null	2081-11-17	5000	null	10	10	ACCOUNTING	NEW YORK
7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	30	30	SALES	CHICAGO
7876	ADAMS	CLERK	7788	2087-05-23	1100	null	20	20	RESEARCH	DALLAS
7900	JAMES	CLERK	7698	2081-12-03	950	null	30	30	SALES	CHICAGO
7902	FORD	ANALYST	7566	2081-12-03	3000	null	20	20	RESEARCH	DALLAS
7934	MILLER	CLERK	7782	2082-01-23	1300	null	10	10	ACCOUNTING	NEW YORK
1234	SEKHAR	doctor	7777	null	667	78	80	null	null	null



```
1 # Right Outer join
2 df_emp_csv.join(df_dept_csv,df_emp_csv["DEPTNO"]==df_dept_csv["Deptno"],"right").show()
```

► (2) Spark Jobs

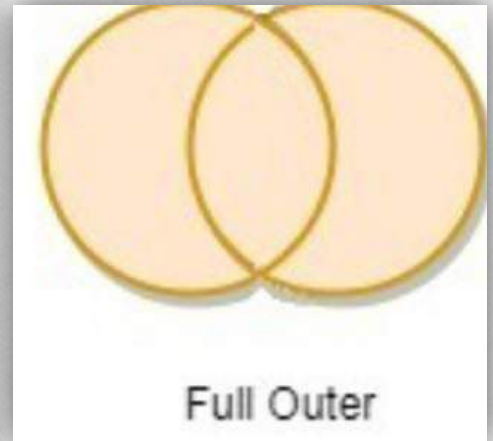
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	Deptno	Dname	Loc
7934	MILLER	CLERK	7782	2082-01-23	1300	null	10	10	ACCOUNTING	NEW YORK
7839	KING	PRESIDENT	null	2081-11-17	5000	null	10	10	ACCOUNTING	NEW YORK
7782	RAVI	MANAGER	7839	2081-06-09	2450	null	10	10	ACCOUNTING	NEW YORK
7902	FORD	ANALYST	7566	2081-12-03	3000	null	20	20	RESEARCH	DALLAS
7876	ADAMS	CLERK	7788	2087-05-23	1100	null	20	20	RESEARCH	DALLAS
7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	20	20	RESEARCH	DALLAS
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	20	RESEARCH	DALLAS
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	20	RESEARCH	DALLAS
7900	JAMES	CLERK	7698	2081-12-03	950	null	30	30	SALES	CHICAGO
7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	30	30	SALES	CHICAGO
7698	SGR	MANAGER	7839	2081-05-01	2850	null	30	30	SALES	CHICAGO
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	30	SALES	CHICAGO
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	30	SALES	CHICAGO
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	30	SALES	CHICAGO
null	null	null	null	null	null	null	null	40	OPERATIONS	BOSTON



```
1 # Full Outer join
2 df_emp_csv.join(df_dept_csv,df_emp_csv["DEPTNO"]==df_dept_csv["Deptno"],"full").show()
```

► (3) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	Deptno	Dname	Loc
7782	RAVI	MANAGER	7839	2081-06-09	2450	null	10	10	ACCOUNTING	NEW YORK
7839	KING	PRESIDENT	null	2081-11-17	5000	null	10	10	ACCOUNTING	NEW YORK
7934	MILLER	CLERK	7782	2082-01-23	1300	null	10	10	ACCOUNTING	NEW YORK
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	20	RESEARCH	DALLAS
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	20	RESEARCH	DALLAS
7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	20	20	RESEARCH	DALLAS
7876	ADAMS	CLERK	7788	2087-05-23	1100	null	20	20	RESEARCH	DALLAS
7902	FORD	ANALYST	7566	2081-12-03	3000	null	20	20	RESEARCH	DALLAS
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	30	SALES	CHICAGO
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	30	SALES	CHICAGO
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	30	SALES	CHICAGO
7698	SGR	MANAGER	7839	2081-05-01	2850	null	30	30	SALES	CHICAGO
7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	30	30	SALES	CHICAGO
7900	JAMES	CLERK	7698	2081-12-03	950	null	30	30	SALES	CHICAGO
null	null	null	null	null	null	null	null	40	OPERATIONS	BOSTON
1234	SEKHAR	doctor	7777	null	667	78	80	null	null	null



Semi-Join and Anti-Join in PySpark DataFrames

Semi-joins and anti-joins are specialized join operations in PySpark that are useful for filtering and selecting data based on specific conditions.

1. Semi-Join:

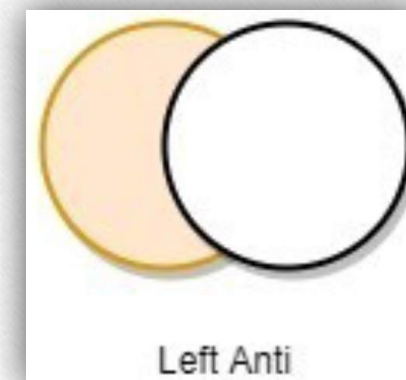
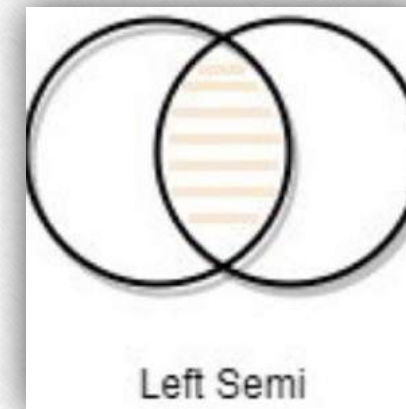
- **Purpose:** Returns rows from the left DataFrame that have a matching key in the right DataFrame.

2. Anti-Join:

- **Purpose:** Returns rows from the left DataFrame that do not have a matching key in the right DataFrame.

Key Points:

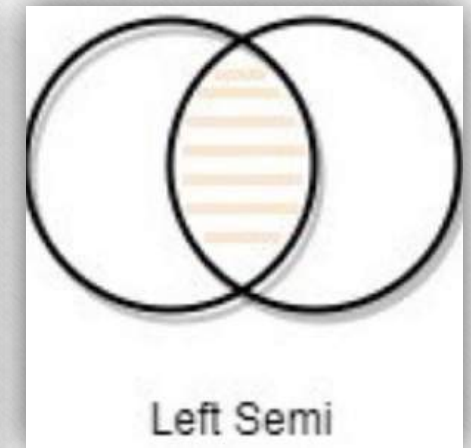
- **Efficiency:** Semi-joins and anti-joins are often more efficient than full joins, especially when the right DataFrame is significantly smaller.
- **Filtering and Selection:** These joins are useful for filtering data based on conditions or selecting specific rows from one DataFrame based on the existence of matches in another.



```
1 # left semi join
2 df_emp_csv.join(df_dept_csv,df_emp_csv["DEPTNO"]==df_dept_csv["Deptno"],"semi").show()
```

► (2) Spark Jobs

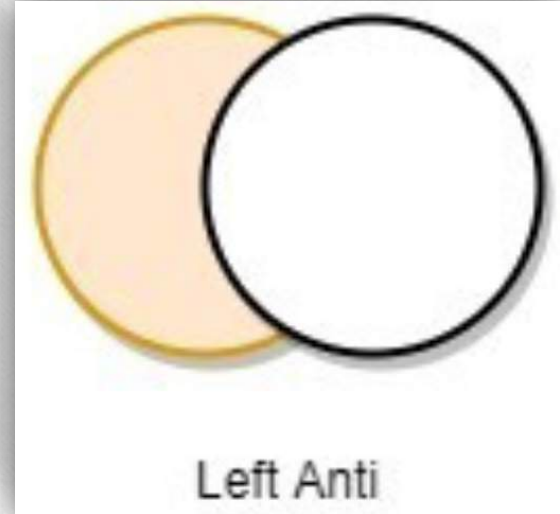
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	2080-12-17	800	null	20
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30
7698	SGR	MANAGER	7839	2081-05-01	2850	null	30
7782	RAVI	MANAGER	7839	2081-06-09	2450	null	10
7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	20
7839	KING	PRESIDENT	null	2081-11-17	5000	null	10
7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	30
7876	ADAMS	CLERK	7788	2087-05-23	1100	null	20
7900	JAMES	CLERK	7698	2081-12-03	950	null	30
7902	FORD	ANALYST	7566	2081-12-03	3000	null	20
7934	MILLER	CLERK	7782	2082-01-23	1300	null	10



```
1 # left anti join
2 df_emp_csv.join(df_dept_csv,df_emp_csv["DEPTNO"]==df_dept_csv["Deptno"],"anti").show()
```

► (2) Spark Jobs

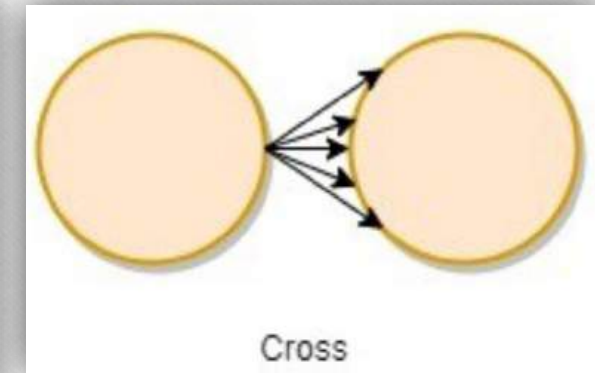
```
+-----+-----+-----+-----+-----+-----+-----+
|EMPNO| ENAME|  JOB| MGR|HIREDATE| SAL| COMM|DEPTNO|
+-----+-----+-----+-----+-----+-----+-----+
| 1234|SEKHAR|doctor|7777|    null|667|  78|   80|
+-----+-----+-----+-----+-----+-----+-----+
```



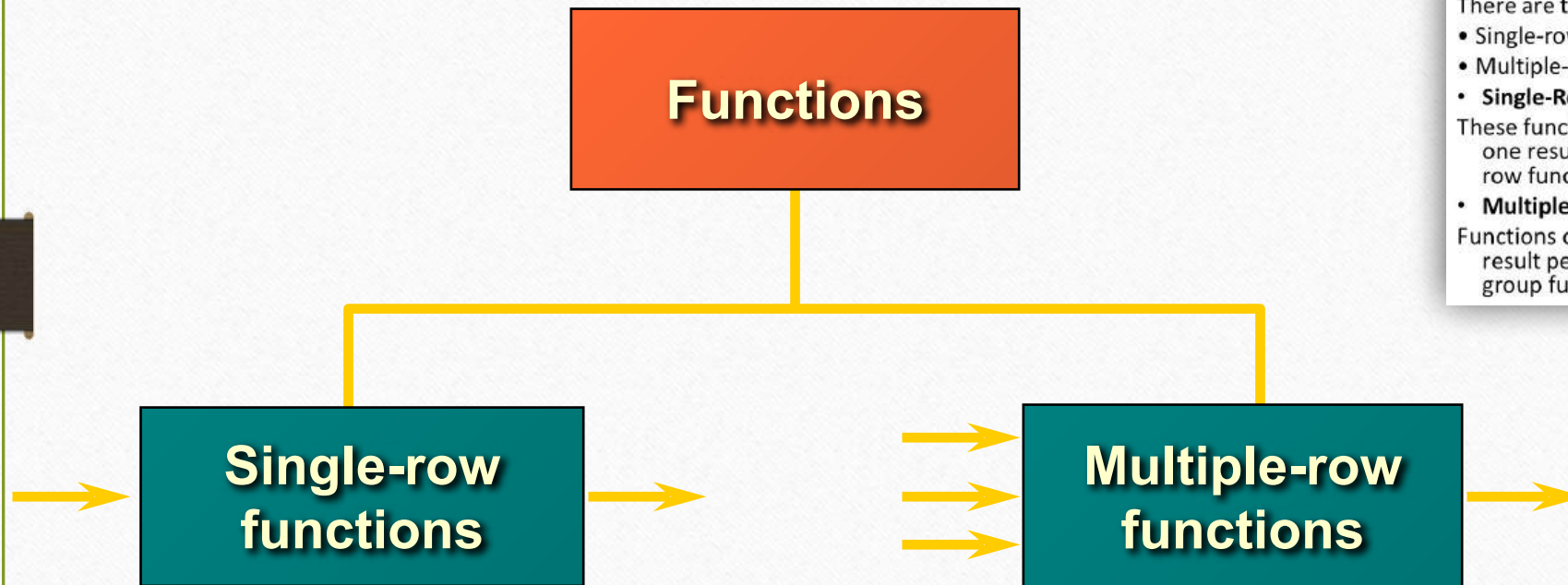
```
1 # Cross join |
2 df_emp_csv.crossJoin(df_dept_csv).show()
```

► (2) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	Deptno	Dname	Loc
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	10	ACCOUNTING	NEW YORK
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	20	RESEARCH	DALLAS
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	30	SALES	CHICAGO
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	40	OPERATIONS	BOSTON
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	10	ACCOUNTING	NEW YORK
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	20	RESEARCH	DALLAS
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	30	SALES	CHICAGO
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	40	OPERATIONS	BOSTON
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	10	ACCOUNTING	NEW YORK
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	20	RESEARCH	DALLAS
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	30	SALES	CHICAGO
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	40	OPERATIONS	BOSTON
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	10	ACCOUNTING	NEW YORK
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	20	RESEARCH	DALLAS
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	30	SALES	CHICAGO
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	40	OPERATIONS	BOSTON
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	10	ACCOUNTING	NEW YORK
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	20	RESEARCH	DALLAS



Two Types of SQL Functions



Two Types of SQL Functions

There are two distinct types of functions:

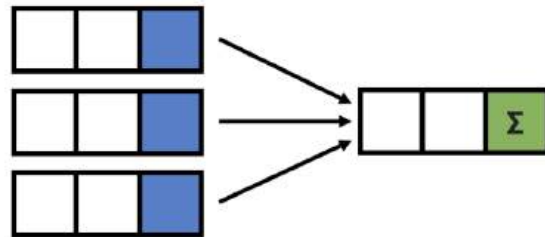
- Single-row functions
- Multiple-row functions
- **Single-Row Functions**

These functions operate on single rows only and return one result per row. There are different types of single-row functions

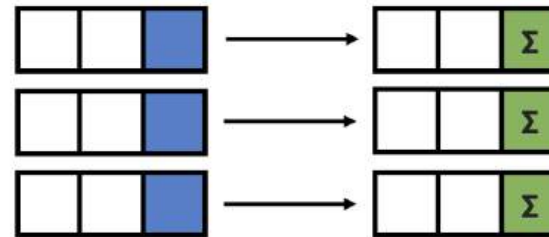
- **Multiple-Row Functions**

Functions can manipulate groups of rows to give one result per group of rows. These functions are known as group functions

Aggregate Functions (SUM, AVG, etc.)



Window Functions



```
1 # df.count() taking count as no of rows
2 print('Total No of Rows : ', df_emp_csv.count())
3 # Taking count at deptno wise
4 df_emp_csv.groupby("DEPTNO").count().show()
5
```

► (4) Spark Jobs

Total No of Rows : 15

DEPTNO count	
20	5
10	3
80	1
30	6

```
1 # more than one aggregation using AGG Functions
2 # import functions
3 from pyspark.sql.functions import sum,min,max,avg,variance,stddev
4 (df_emp_csv.groupby("DEPTNO")
5  .agg(sum("SAL").alias("TOTAL_SAL")
6       ,min("SAL").alias("MIN_SAL")
7       ,avg("SAL").alias("AVG_SAL")
8       ,max("SAL").alias("MAX_SAL")
9       ,variance("SAL").alias("VARIANCE_SAL")
10      ,stddev("SAL").alias("DEVIATION_SAL")).show())
11
```

► (2) Spark Jobs

DEPTNO	TOTAL_SAL	MIN_SAL	AVG_SAL	MAX_SAL	VARIANCE_SAL	DEVIATION_SAL
20	10875	800	2175.0	3000	1261875.0000000002	1123.3320969330487
10	8750	1300	2916.6666666666665	5000	3585833.3333333333	1893.6296716447314
80	667	667	667.0	667	null	null
30	9400	950	1566.6666666666667	2850	446666.6666666667	668.3312551921141

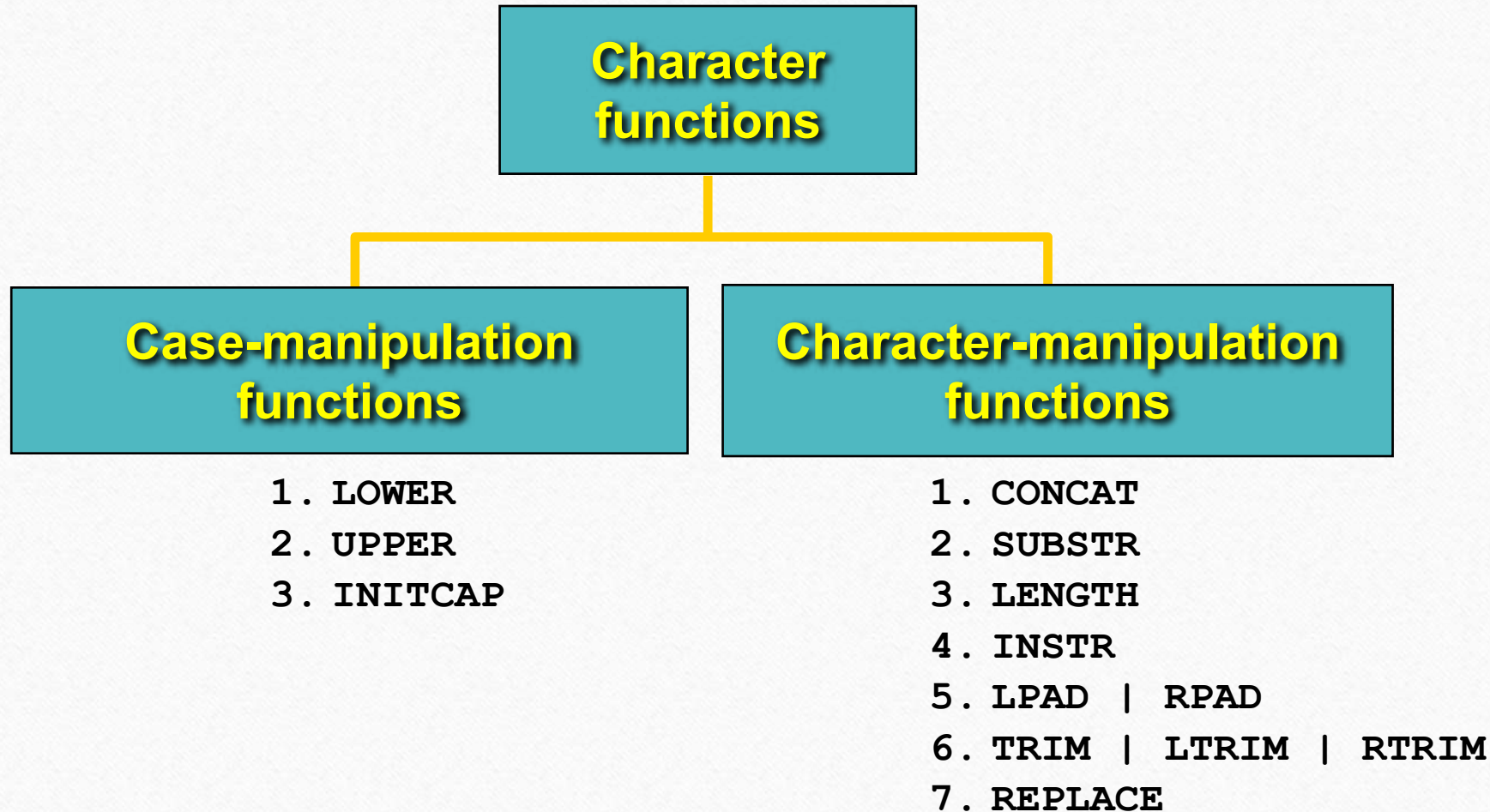
```

1 # Window Ranking functions
2 #import window function
3 from pyspark.sql.window import Window
4 # window partition declaration
5 part = Window.partitionBy("DEPTNO")
6 ( df_emp_csv.withColumn("RANK",rank().over(part.orderBy(col("SAL").desc()))
7   .withColumn("DENSE_RANK",dense_rank().over(part.orderBy(col("SAL").desc()))
8   .withColumn("ROW_NUMBER",row_number().over(part.orderBy(col("SAL").desc()))
9   .withColumn("PERCENT_RANK",percent_rank().over(part.orderBy(col("SAL").desc()))
10  .withColumn("NTILE",ntile(2).over(part.orderBy(col("SAL").desc()))
11  .show())
    
```

► (2) Spark Jobs

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	RANK	DENSE_RANK	ROW_NUMBER	PERCENT_RANK	NTILE
7839	KING	PRESIDENT	null	2081-11-17	5000	null	10	1	1	1	0.0	1
7782	RAVI	MANAGER	7839	2081-06-09	2450	null	10	2	2	2	0.5	1
7934	MILLER	CLERK	7782	2082-01-23	1300	null	10	3	3	3	1.0	2
7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	20	1	1	1	0.0	1
7902	FORD	ANALYST	7566	2081-12-03	3000	null	20	1	1	2	0.0	1
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	3	2	3	0.5	1
7876	ADAMS	CLERK	7788	2087-05-23	1100	null	20	4	3	4	0.75	2
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	5	4	5	1.0	2
7698	SGR	MANAGER	7839	2081-05-01	2850	null	30	1	1	1	0.0	1
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	2	2	2	0.2	1
7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	30	3	3	3	0.4	1
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	4	4	4	0.6	2
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	4	4	5	0.6	2
7900	JAMES	CLERK	7698	2081-12-03	950	null	30	6	5	6	1.0	2
1234	SEKHAR	doctor	7777	null	667	78	80	1	1	1	0.0	1

Character Functions



```

1 #trim for removing white spaces in both side
2 # ltrim for removing white spaces in left side
3 # rtrim for removing white space in right side
4 # lpad for padding a character in left side
5 # rpad for padding a character in right side
6 from pyspark.sql.functions import trim,ltrim,rtrim,lpad,rpad,upper,lower
7 ( df_emp_csv.select(trim("ENAME")
8                     ,ltrim("ENAME")
9                     ,rtrim("ENAME")
10                    ,lpad("ENAME",10,"*")
11                    ,rpad("ENAME",10,"*")
12                    ,rpad(lpad("ENAME",10,"*"),20,"*")
13                    ,upper("ENAME")
14                    ,lower("ENAME"))
15 .show())

```

► (1) Spark Jobs

trim(ENAME)	ltrim(ENAME)	rtrim(ENAME)	lpad(ENAME, 10, *)	rpad(ENAME, 10, *)	rpad(lpad(ENAME, 10, *), 20, *)	upper(ENAME)	lower(ENAME)
SMITH	SMITH	SMITH	*****SMITH	SMITH*****	*****SMITH*****	SMITH	smith
ALLEN	ALLEN	ALLEN	*****ALLEN	ALLEN*****	*****ALLEN*****	ALLEN	allen
WARD	WARD	WARD	*****WARD	WARD*****	*****WARD*****	WARD	ward
JONES	JONES	JONES	*****JONES	JONES*****	*****JONES*****	JONES	jones
MARTIN	MARTIN	MARTIN	****MARTIN	MARTIN****	****MARTIN*****	MARTIN	martin
SGR	SGR	SGR	*****SGR	SGR*****	*****SGR*****	SGR	sgr
RAVI	RAVI	RAVI	*****RAVI	RAVI*****	*****RAVI*****	RAVI	ravi
SCOTT	SCOTT	SCOTT	*****SCOTT	SCOTT*****	*****SCOTT*****	SCOTT	scott
KING	KING	KING	*****KING	KING*****	*****KING*****	KING	king
TURNER	TURNER	TURNER	****TURNER	TURNER****	****TURNER*****	TURNER	turner
ADAMS	ADAMS	ADAMS	*****ADAMS	ADAMS*****	*****ADAMS*****	ADAMS	adams
JAMES	JAMES	JAMES	*****JAMES	JAMES*****	*****JAMES*****	JAMES	james
FORD	FORD	FORD	*****FORD	FORD*****	*****FORD*****	FORD	ford
MILLER	MILLER	MILLER	****MILLER	MILLER****	****MILLER*****	MILLER	miller
SEKHAR	SEKHAR	SEKHAR	****SEKHAR	SEKHAR****	****SEKHAR*****	SEKHAR	sekhar

Creating bucketing while writing data into target

- while bucketing can be used with both `saveAsTable` when using the Dataset APIs.
- bucketing does not support `save` file option

What is Bucketing?

- Bucketing is a technique in both Spark and Hive used to optimize the performance of the task.
- In bucketing buckets (clustering columns) determine data partitioning and prevent data shuffle.
- Based on the value of one or more bucketing columns, the data is allocated to a predefined number of buckets.

```
%sql
CREATE TABLE default.emp_csv_part_bucket (
  EMPNO INT,
  ENAME STRING,
  JOB STRING,
  MGR INT,
  HIREDATE DATE,
  SAL INT,
  COMM INT,
  DEPTNO INT)
USING csv
PARTITIONED BY (DEPTNO)
CLUSTERED BY (JOB)
SORTED BY (EMPNO)
INTO 4 BUCKETS
```

Writing CSV files in Spark Write API

```
1 df_emp.write.format("csv").mode("append").save("/target/empcsv/",header=True)
2 df_emp.write.format("csv").mode("append").saveAsTable("emp_csv",header=True)
```

Creating Partition while writing into target

- while partitioning can be used with both `save` and `saveAsTable` when using the Dataset APIs.

Cmd 21

```
1 df_emp.write.format("csv").partitionBy("DEPTNO").save("/FileStore/tables/empcsv_part/")
2 df_emp.write.format("csv").partitionBy("DEPTNO").saveAsTable("emp_csv_part")
```

Writing json files in Spark Write API

Cmd 7

```
1 df_emp.write.format("json").mode("append").save("/target/empjson/")
2 df_emp.write.format("json").mode("append").saveAsTable("emp_json")
```

Cmd 8

Writing orc files in Spark Write API

Cmd 9

```
1 df_emp.write.format("orc").mode("append").save("/target/emporc/")
2 df_emp.write.format("orc").mode("append").saveAsTable("emp_orc")
```

Cmd 10

Writing parquet files in Spark Write API

Cmd 11

```
1 df_emp.write.format("parquet").mode("append").save("/target/empparquet/")
2 df_emp.write.format("parquet").mode("append").saveAsTable("emp_parquet")
```

Writing avro files in Spark Write API

Cmd 13

```
1 df_emp.write.format("avro").mode("append").save("/target/empavro/")
2 df_emp.write.format("avro").mode("append").saveAsTable("emp_avro")
```

Cmd 14

Writing xml files in Spark Write API

Cmd 15

```
1 df_emp.write.format("com.databricks.spark.xml").mode("append").save("/target/empxml/")
2 df_emp.write.format("com.databricks.spark.xml").mode("append").saveAsTable("emp_xml")
```

Cmd 16

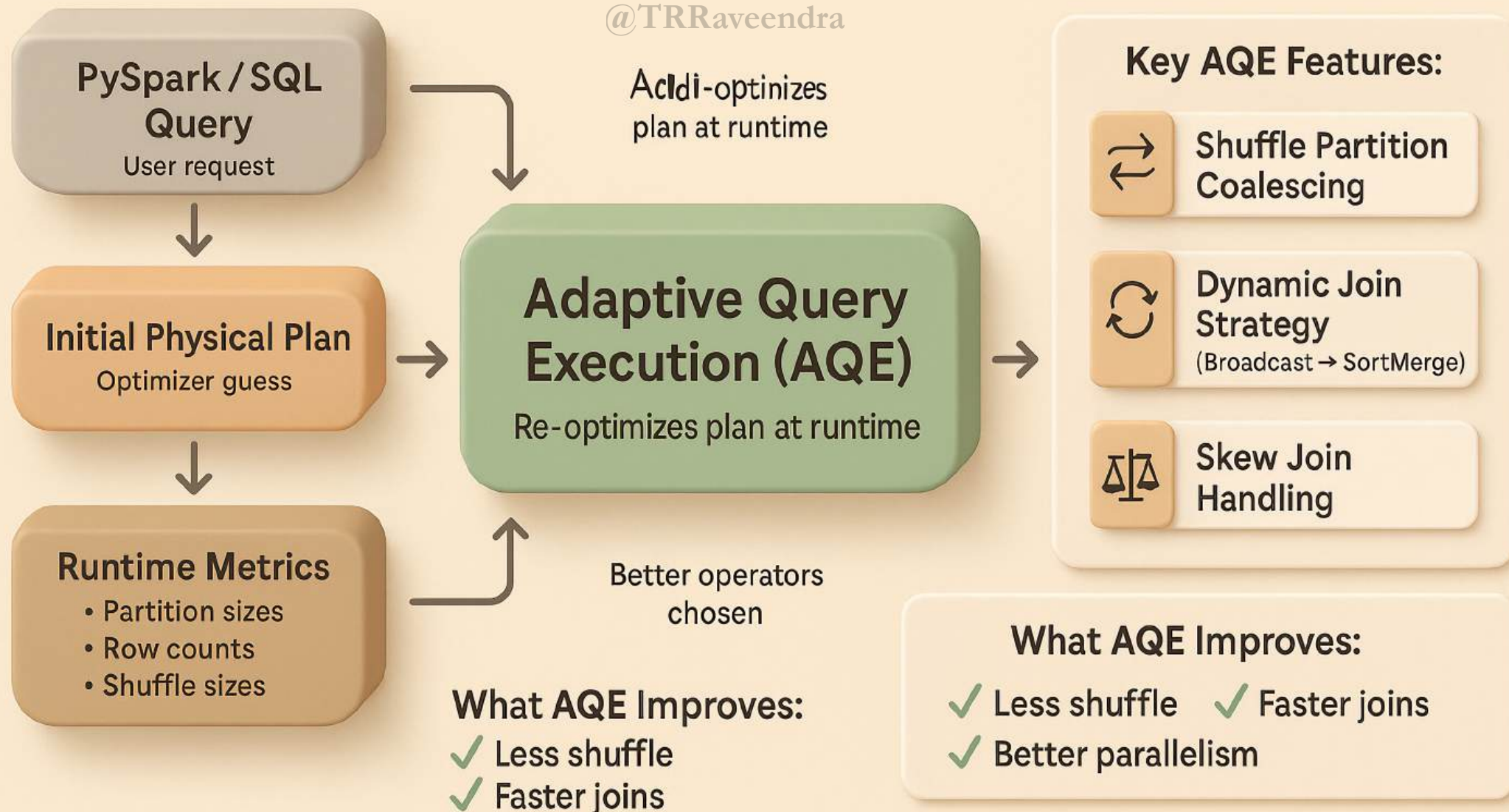
Writing excel files in Spark Write API

Cmd 17

```
1 df_emp.write.format("com.crealytics.spark.excel").mode("append").save("/target/empexcel/")
2 df_emp.write.format("com.crealytics.spark.excel").mode("append").saveAsTable("emp_excel")
```

PySpark Adaptive Query Execution (AQE)

@TRRaveendra



Understand what AQE is • See how plans change at runtime • Know why AQE improves performance

PySpark Join Types & Join Optimization

@TRRaveendra

Join Types

Inner Join

Left Outer Join

Right Outer Join

Full Outer Join

Left Semi Join

Left Anti Join

Defines which rows are kept

Join Execution Strategies



Broadcast Hash Join

Small table broadcasted



Sort Merge Join

Large tables, sorted



Shuffle Hash Join

Hash-based, shuffle

Chosen by optimizer

Join



Optimizer



Choose Strategy

Optimization Techniques

Use Broadcast

`broadcast(df)`

Filter early

**Select only
needed columns**

**Repartition
by join key**

Enable AQE

**Avoid
Cartesian join**

- Understand join semantics
- Know how Spark executes joins
- Optimize joins effectively

Join Request → **Optimizer** →

```
from pyspark.sql.functions import broadcast
df.join(broadcast(dim), 'id')
```

PySpark Execution Troubleshooting

Common Problems

Long running job

Job stuck at stage

Out of memory

Too many small files

Executor lost

Symptoms

Large shuffle

Skewed task times

GC overhead

Many retries

Low CPU usage

Typical Fixes

Enable AQE

Broadcast small tables

Increase partitions

Cache reused data

Increase
executor memory

@TRRaveendra



Quickly identify issues • Map symptom to fix • Build troubleshooting mindset



Understand CBO concept • Know importance of statistics • See how plans improve

PYSPARK MEMORY TUNING

WHERE MEMORY IS USED

Execution Memory

For computations and joins

Storage Memory

For caching and broadcasts

User Memory

For user data structures

COMMON MEMORY ISSUES



**Out of
Memory**



**requent
GC**



**Spill to
Disk**



Disk

^{@TRRaveendra}KEY TUNING KNOBS

spark.executor.memory

Main memory setting

spark.memory.fraction

Execution vs. storage split

spark.executor.memoryOverhead

Off-heap memory

spark.sql.shuffle.partitions

Parallelism control

Measure

Tune

Validate

BEST PRACTICES

Cache Selectively

Only frequently used data

Use Column Pruning

Select only needed columns

Avoid collect()

Use take() or show() instead

Use Broadcast Joins

For small tables

LEARNING GOALS

Understand memory layout
Know key configs
Tune memory safely

PySpark Performance Anti-Patterns

@TRRaveendra

❌ Anti-Pattern



collect() on Big Data

► Pulls all data to driver, causing OOM errors.



Too Many repartition()

► Excessive shuffling overloads network and disk I/O.



Python UDF Overuse

► Slows down execution; cannot be optimized by Catalyst.



Late Filtering

► Processes unnecessary data before filtering, increasing resource usage.



No Partitioning Strategy

► Results in data skew and inefficient data access.

✅ Better Practice



Use limit() / take()

► Retrieve only necessary rows for preview or debugging.



Let AQE Handle Partitions

► Adaptive Query Execution optimizes partition numbers automatically.



Built-in Functions

► Optimized by Catalyst for faster execution; avoid Python UDFs.



Filter at Source

► Apply filters as early as possible to reduce data volume.



Partition by Key

► Distribute data evenly across cluster for parallel processing.



Avoid Anti-Pattern



Apply Best Practice



Faster Jobs



Recognize Mistakes • Apply Better Practices • Build Efficient Pipelines

PySpark DataFrame Transformations — Clean 3D Deep Dive Educational Poster (Student-Focused)

Senior Big Data Instructor & PySpark Educator

What Is a Transformation?

PySpark Transformation — Builds logical execution plan (DAG)

Executed only when an **Action** is called

 **Lazy Evaluation**



A Basic Transformations

```
select()           withColumnRenamed()
filter() / where() limit()
withColumn()       sample()
drop()
```

B Deduplication & Ordering

```
distinct()
dropDuplicates()
sort()
orderBy() @TRRaveendra
```

C Aggregation & Grouping

```
groupBy()
agg()
rollup()
cube()
```

D Joining & Combining

```
join()
union()
unionByName()
intersect()
exceptAll()
```

E Column & Expression Transformations

```
selectExpr()
cast()
when() / otherwise()
alias()
expr()
```

F Nested / Complex Data Transformations

```
explode()
posexplode()
inline()
getField()
```

G Partitioning & Layout

```
repartition()
coalesce()
```

⚠ WIDE VS NARROW HINT:
✓ Narrow → select, filter, withColumn
⚠ Wide → groupBy, join, distinct, repartition

```
df2 = df.select("id","country") \
        .filter(col("age") > 25) \
        .groupBy("country") \
        .count()
```

All above are transformations (**lazy**)

Learning Goals • Understand what transformations are • Know major transformation APIs • Recognize lazy evaluation

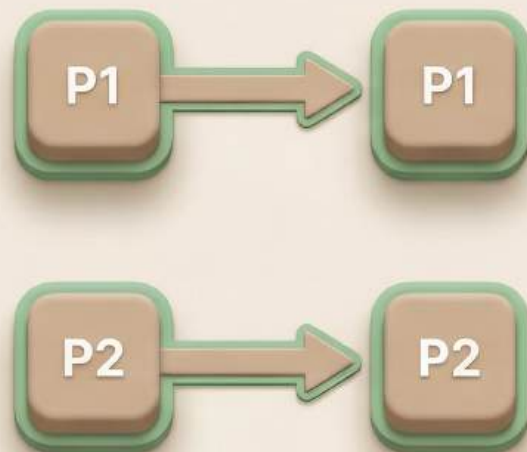
NARROW TRANSFORMATIONS

Each output partition depends on one input partition

✓ No shuffle

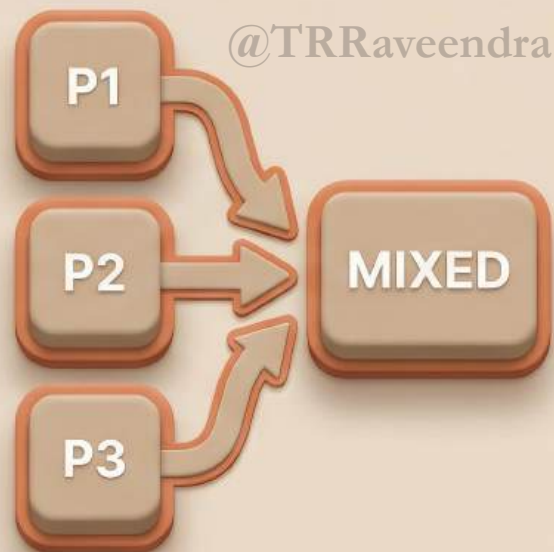
✓ Fast

COMMON APIS:
select(), filter(),
withColumn(),
drop(), limit()



P1 → P1, P2 → P2

DATA MOVEMENT COMPARISON



P1 ↱, P2 ↴ MIXED, P3 ↵

WIDE TRANSFORMATIONS

Output partitions depend on multiple input partitions

⚠ Shuffle required

⚠ Expensive

COMMON APIS:
groupBy(), join(),
distinct(), orderBy(),
repartition()

EXAMPLE STRIP

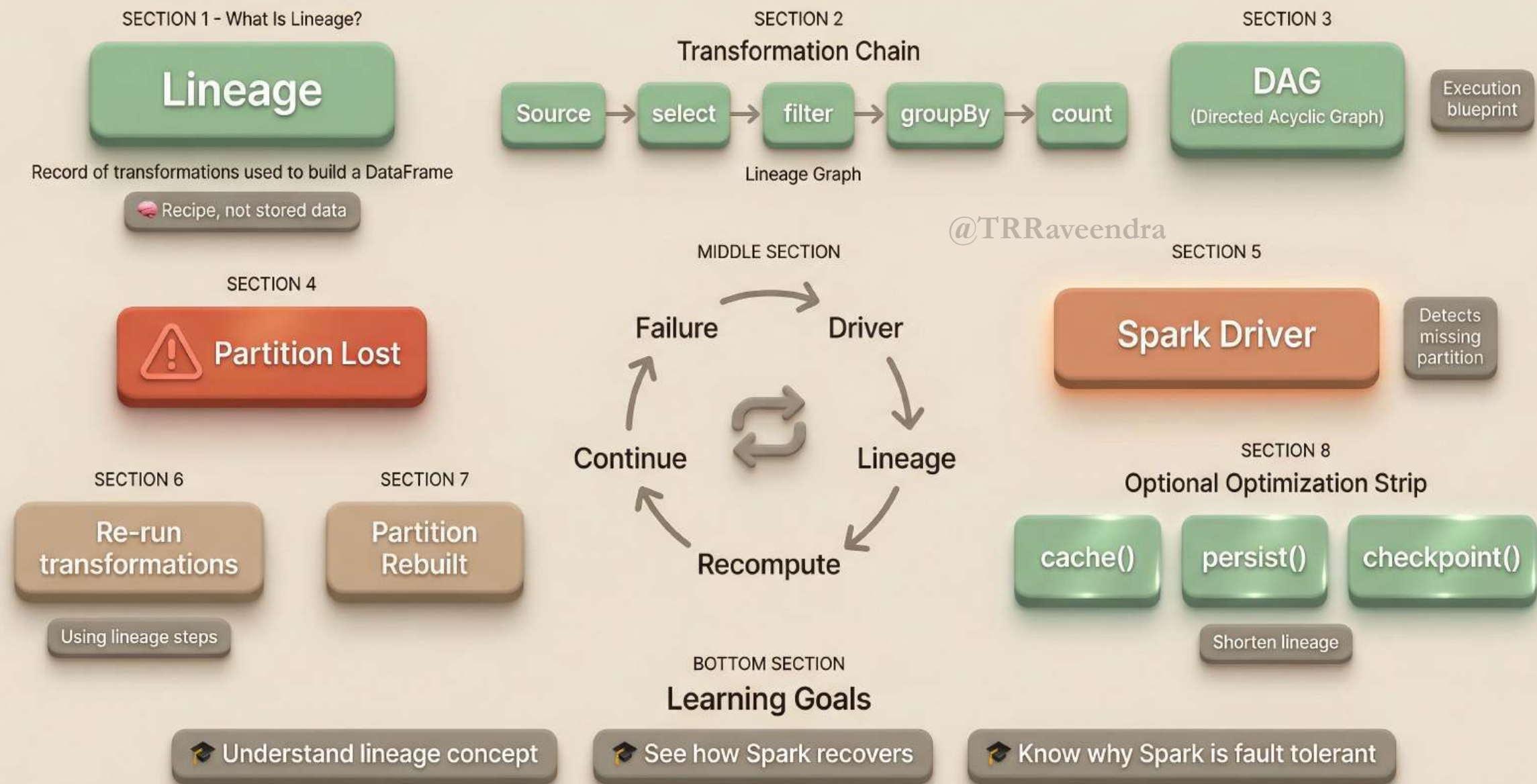
```
df.filter(col("age")>30) # Narrow
```

```
df.groupBy("country").count() # Wide
```

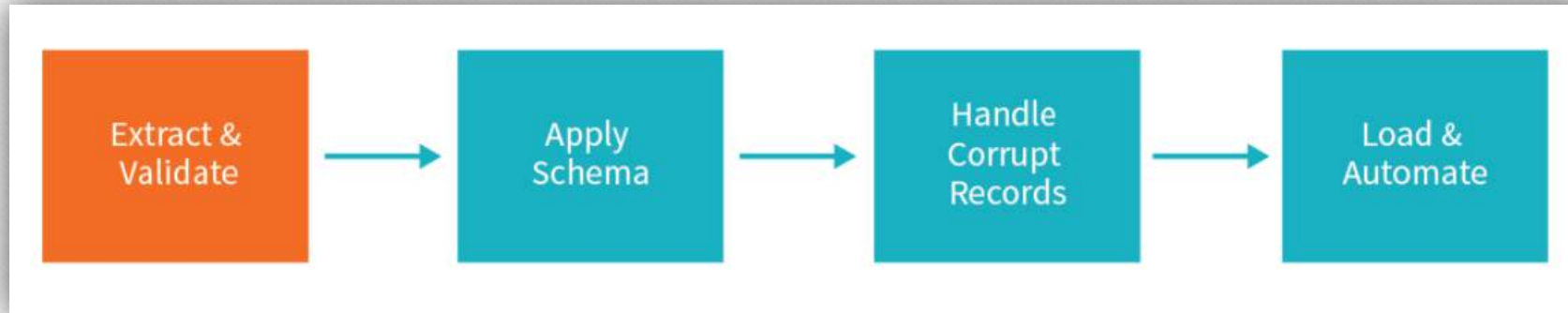
LEARNING GOALS STRIP

Distinguish narrow vs wide • Predict shuffle • Write efficient pipelines

PySpark Lineage & Fault Tolerance — Clean 3D Deep Dive Educational Poster (Student-Focused)



Spark End to End operations





Why Delta?

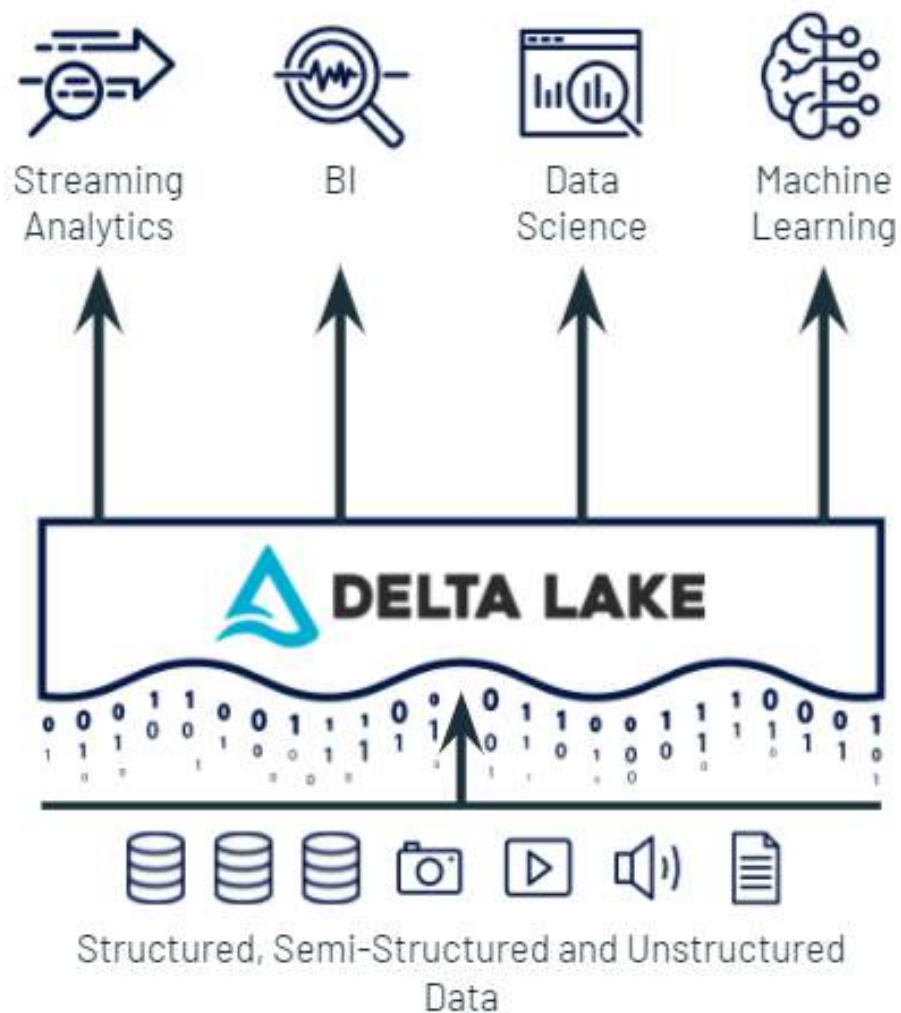
- Challenges in implementation of a data lake.
- Missing ACID properties.
- Lack of Schema enforcement.
- Lack of Consistency.
- Lack of Data Quality.
- Too many small files.
- Corrupted data due to Frequent job failures in prod

Delta Lake Features

An open-source storage format that brings ACID transactions to Apache Spark™ and big data workloads.

- **Open format:** Stored as Parquet format in blob storage.
- **ACID Transactions:** Ensures data integrity and read consistency with complex, concurrent data pipelines.
- **Schema Enforcement and Evolution:** Ensures data cleanliness by blocking writes with unexpected.
- **Audit History:** History of all the operations that happened in the table.
- **Time Travel:** Query previous versions of the table by time or version number.
- **Deletes and upserts:** Supports deleting and upserting into tables with programmatic APIs.
- **Scalable Metadata management:** Able to handle millions of files are scaling the metadata operations with Spark.
- **Unified Batch and Streaming Source and Sink:** A table in Delta Lake is both a batch table, as well as a streaming source and sink. Streaming data ingest, batch historic backfill, and interactive queries all just work out of the box.

Delta Storage Layer



One platform for every use case

Structured transactional layer

Data Lake for all your data

How Delta Lake Works

- Delta lake provides a storage layer on top of existing storage data lake. It acts as a middle layer between Spark runtime and storage
- Delta Lake will generate delta logs for each committed transactions
- Delta logs will have delta files stored as JSON which has information about the operations occurred
- Delta files are sequentially increasing named JSON files and together make up the log of all changes that have occurred to a table

Get Started with Delta using Spark APIs

Add Spark Package

```
pyspark --packages io.delta:delta-core_2.12:0.1.0
bin/spark-shell --packages io.delta:delta-core_2.12:0.1.0
```

Maven

```
<dependency>
  <groupId>io.delta</groupId>
  <artifactId>delta-core_2.12</artifactId>
  <version>0.1.0</version>
</dependency>
```

Instead of **parquet**...

```
dataframe
  .write
  .format("parquet")
  .save("/data")
```

... simply say **delta**

```
dataframe
  .write
  .format("delta")
  .save("/data")
```

```
df.write.format("delta").saveAsTable("events")      # create table in the metastore
```

```
df.write.format("delta").save("/delta/events")      # create table by path
```

```
df.write.format("delta").partitionBy("date").saveAsTable("events")      # create table in the metastore
```

```
df.write.format("delta").partitionBy("date").save("/delta/events")      # create table by path
```

```
-- Create table in the metastore
```

```
CREATE TABLE events (  
  date DATE,  
  eventId STRING,  
  eventType STRING,  
  data STRING)  
USING DELTA
```

```
-- Create table in the metastore
```

```
CREATE TABLE events (  
  date DATE,  
  eventId STRING,  
  eventType STRING,  
  data STRING)  
USING DELTA  
PARTITIONED BY (date)  
LOCATION '/delta/events'
```

If your source files are in Parquet format, you can use the SQL Convert to Delta statement to convert files in place to create an unmanaged table:

```
CONVERT TO DELTA parquet.`/mnt/delta/events`
```



databricks[®]
DELTA

=

Scalable storage
+

Transactional log

Databricks Delta

Next-generation engine built on top of Spark



Versioned
Parquet Files



Transactional
Delta Log



Indexes &
Stats

156

Tables created with a specified LOCATION are considered unmanaged by the metastore. Unlike a managed table, where no path is specified, an unmanaged table's files are not deleted when you DROP the table.

```
CREATE TABLE events
USING DELTA
LOCATION '/delta/events'
```

the table in the Hive metastore automatically inherits the schema, partitioning, and table properties of the existing data. This functionality can be used to “import” data into the metastore.

Reading Delta Table in SQL & Python

```
SELECT * FROM events    -- query table in the metastore

SELECT * FROM delta.`/delta/events`  -- query table by path
```

```
spark.table("events")    # query table in the metastore

spark.read.format("delta").load("/delta/events")  # query table by path
```

Scalable storage

table data stored as Parquet files
on HDFS, AWS S3, Azure Blob Stores

Transactional log

sequence of metadata files to track
operations made on the table

stored in scalable storage along with table

pathToTable/

+---- 000.parquet

+---- 001.parquet

+---- 002.parquet

+
|
...

+---- _delta_log/

+---- 000.json

+---- 001.json

...

Log Structured Storage

Changes to the table are stored as *ordered, atomic* commits

Each commit is a set of actions file in directory `_delta_log`

`_delta_log/`

`000.json`

`001.json`

INSERT actions

Add 001.parquet

Add 002.parquet

UPDATE actions

Remove 001.parquet

Remove 002.parquet

Add 003.parquet

Breaking Down Transactions Into Atomic Commits

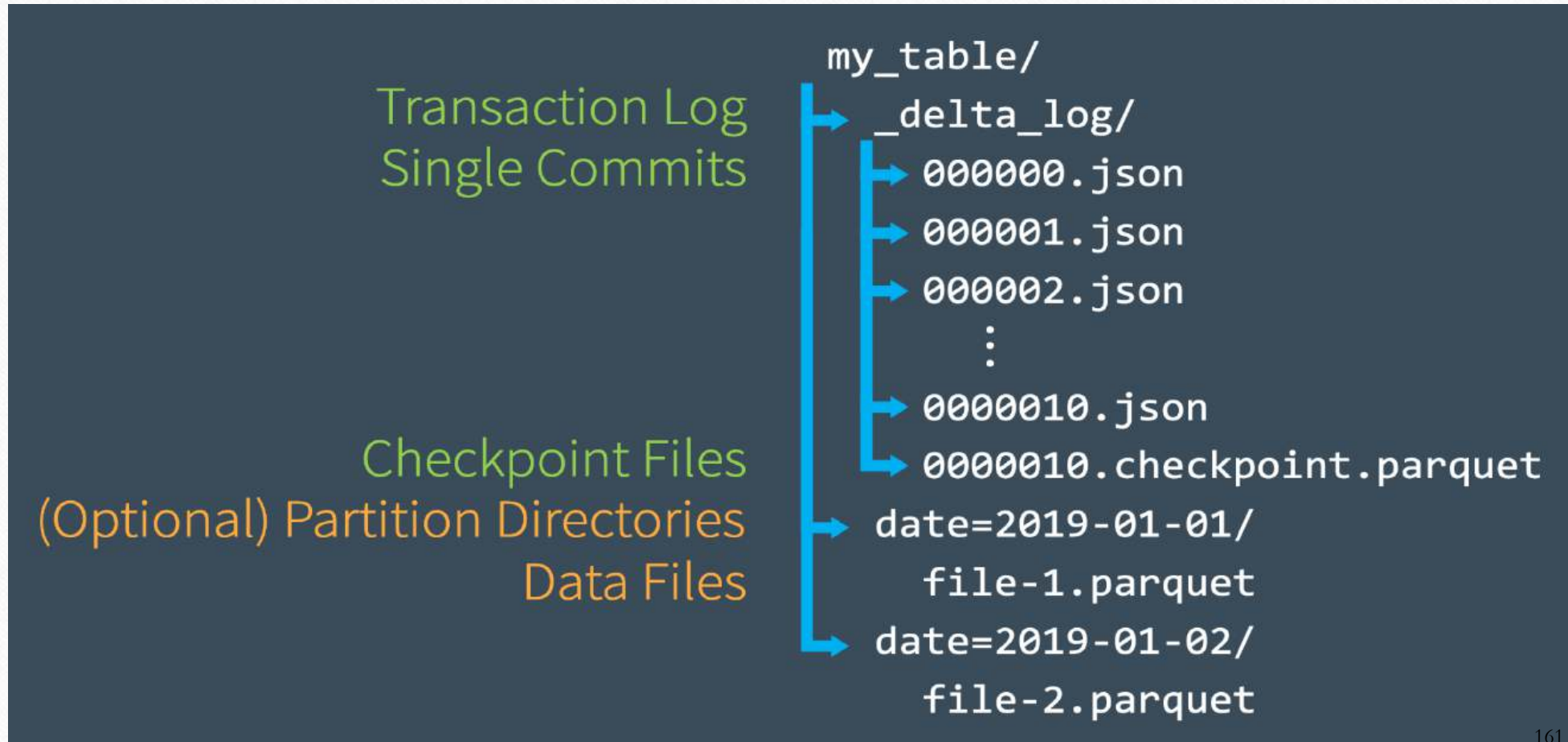
Whenever a user performs an operation to modify a table (such as an INSERT, UPDATE or DELETE), Delta Lake breaks that operation down into a series of discrete steps composed of one or more of the **actions** below.

- **Add file** – adds a data file.
- **Remove file** – removes a data file.
- **Update metadata** – Updates the table's metadata (e.g., changing the table's name, schema or partitioning).
- **Set transaction** – Records that a structured streaming job has committed a micro-batch with the given ID.
- **Change protocol** – enables new features by switching the Delta Lake transaction log to the newest software protocol.
- **Commit info** – Contains information around the commit, which operation was made, from where and at what time.

Those actions are then recorded in the transaction log as ordered, atomic units known as **commits**.

Quickly Recomputing State With Checkpoint Files

Once we've made a total of 10 commits to the transaction log, Delta Lake saves a checkpoint file in Parquet format in the same `_delta_log` subdirectory. Delta Lake automatically generates checkpoint files every 10 commits.





Delta Partitioned Table Structure

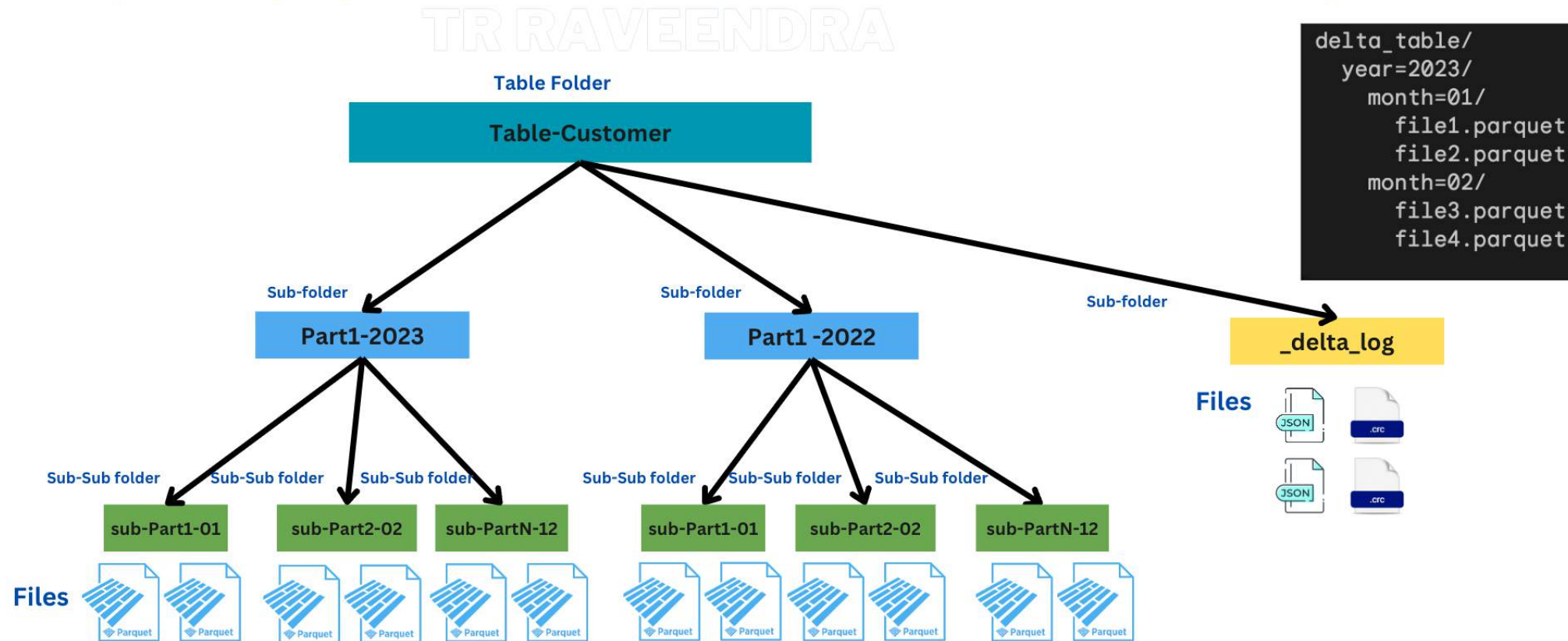


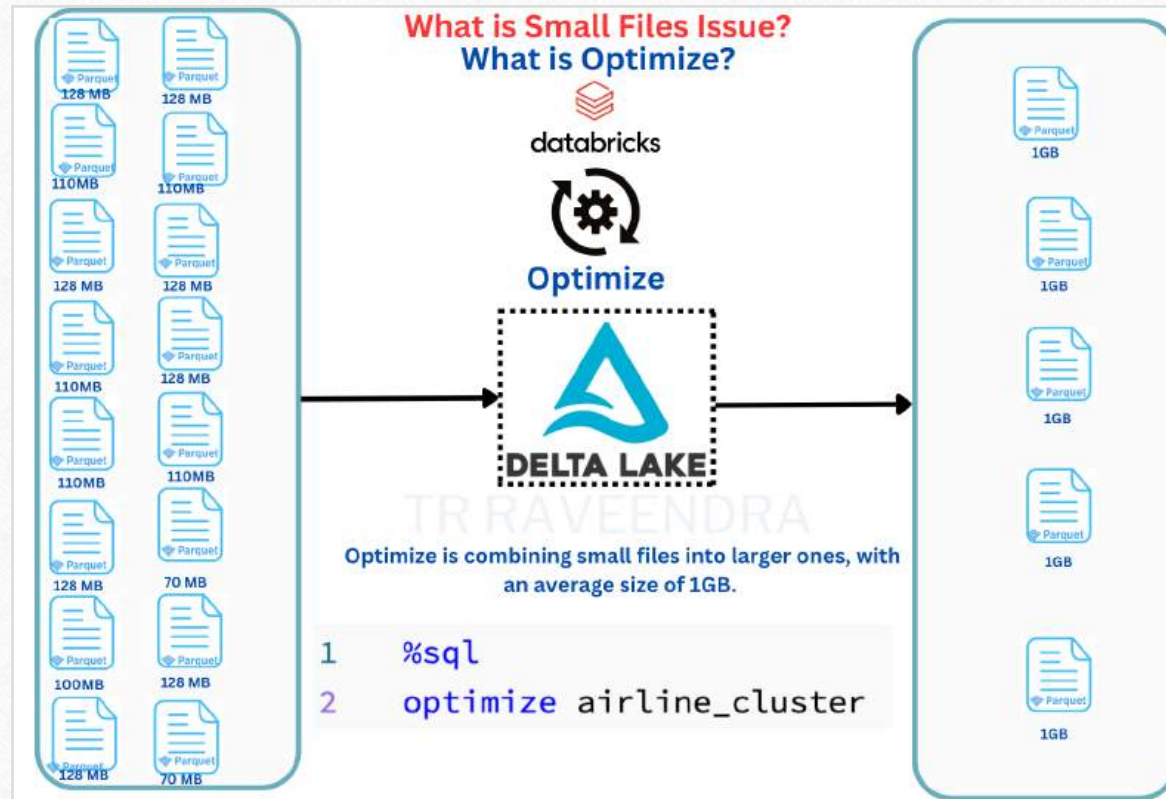
Delta Lake manages files in partitioned tables by storing them in a directory structure that is based on the partition columns.

For example :

```
1 df.write.format("delta").partitionBy("YEAR","MONTH").outputMode("append").saveAsTable("customer")
```

if a table is partitioned by the year and month columns, then the files for that table would be stored in a directory structure like this





The small files issue in Delta Tables is a performance problem that can occur when a Delta Table contains a large number of small files. This can happen for a variety of reasons, such as:

1. Streaming workloads that write data in small batches
2. Hive-style partitioning that creates a separate file for each partition value
3. Appending data to a Delta Table in small amounts over time

Optimize performance with file management

To improve query speed, Delta Lake on Databricks supports the ability to optimize the layout of data stored in cloud storage. Delta Lake on Databricks supports few algorithms: **bin-packing** , **Data-Skipping** And **Z-Ordering**.

Compaction (bin-packing)

Delta Lake on Databricks can improve the speed of read queries from a table by coalescing small files into larger ones. You trigger compaction by running the **OPTIMIZE** command

SQL

```
OPTIMIZE delta.`/data/events`
```

or

SQL

```
OPTIMIZE events
```

If you have a large amount of data and only want to optimize a subset of it, you can specify an optional partition predicate using **WHERE**

SQL

```
OPTIMIZE events WHERE date >= '2017-01-01'
```

Data Skipping

Databricks' Data Skipping feature takes advantage of the multi-file structure. As new data is inserted into a Databricks Delta table, file-level min/max statistics are collected for **all** columns. Then, when there's a lookup query against the table, Databricks Delta first consults these statistics in order to determine which files can safely be skipped. The picture below illustrates the process:

1. Keep track of simple statistics such as minimum and maximum values at a certain granularity that's correlated with I/O granularity.
2. Leverage those statistics at query planning time in order to avoid unnecessary I/O.

```
SELECT input_file_name() as "file_name",  
       min(col) AS "col_min",  
       max(col) AS "col_max"  
FROM table  
GROUP BY input_file_name()
```

file_name	col_min	col_max
data_file_1	6	8
data_file_2	3	10
data_file_3	1	4

Data Skipping Example

In this example, we skip file 1 because its minimum value is higher than our desired value.

```
SELECT * FROM table WHERE col <= 5
```



```
SELECT file_name FROM index  
WHERE col_min <= 5
```

file_name	col_min	col_max
data_file_1	6	8
data_file_2	3	10
data_file_3	1	4

Similarly, we skip file 3 based on its maximum value. File 2 is the only one we need to access.

```
SELECT * FROM table WHERE col == 5
```



```
SELECT file_name FROM index  
WHERE col_min <= 5 AND col_max >= 5
```

file_name	col_min	col_max
data_file_1	6	8
data_file_2	3	10
data_file_3	1	4

Liquid clustering

Liquid clustering in Databricks is a new approach to data layout that addresses the limitations of traditional partitioning. Liquid clustering is a dynamic data layout technique that automatically adjusts the data layout based on clustering keys. This means that liquid clustering can optimize for a variety of query patterns, including queries that filter on high cardinality columns and queries that access data in a specific order.

Liquid clustering is also incremental, meaning that data is only rewritten as necessary to accommodate data that needs to be clustered. This makes liquid clustering very efficient, especially for tables that are updated frequently.

- ✓ **Improved query performance:** Liquid clustering can significantly improve the performance of queries that filter on high cardinality columns and queries that access data in a specific order.
- ✓ **Reduced maintenance overhead:** Liquid clustering is incremental, so you don't need to worry about manually rewriting your data to maintain optimal performance.
- ✓ **Flexibility:** Liquid clustering allows you to easily change the clustering keys for your table without rewriting the data. This makes it easy to adapt your data layout to changing query patterns.

Z-Ordering (multi-dimensional clustering)

Z-Ordering is another Databricks enhancement to Delta Lake.

What is Z-Ordering?

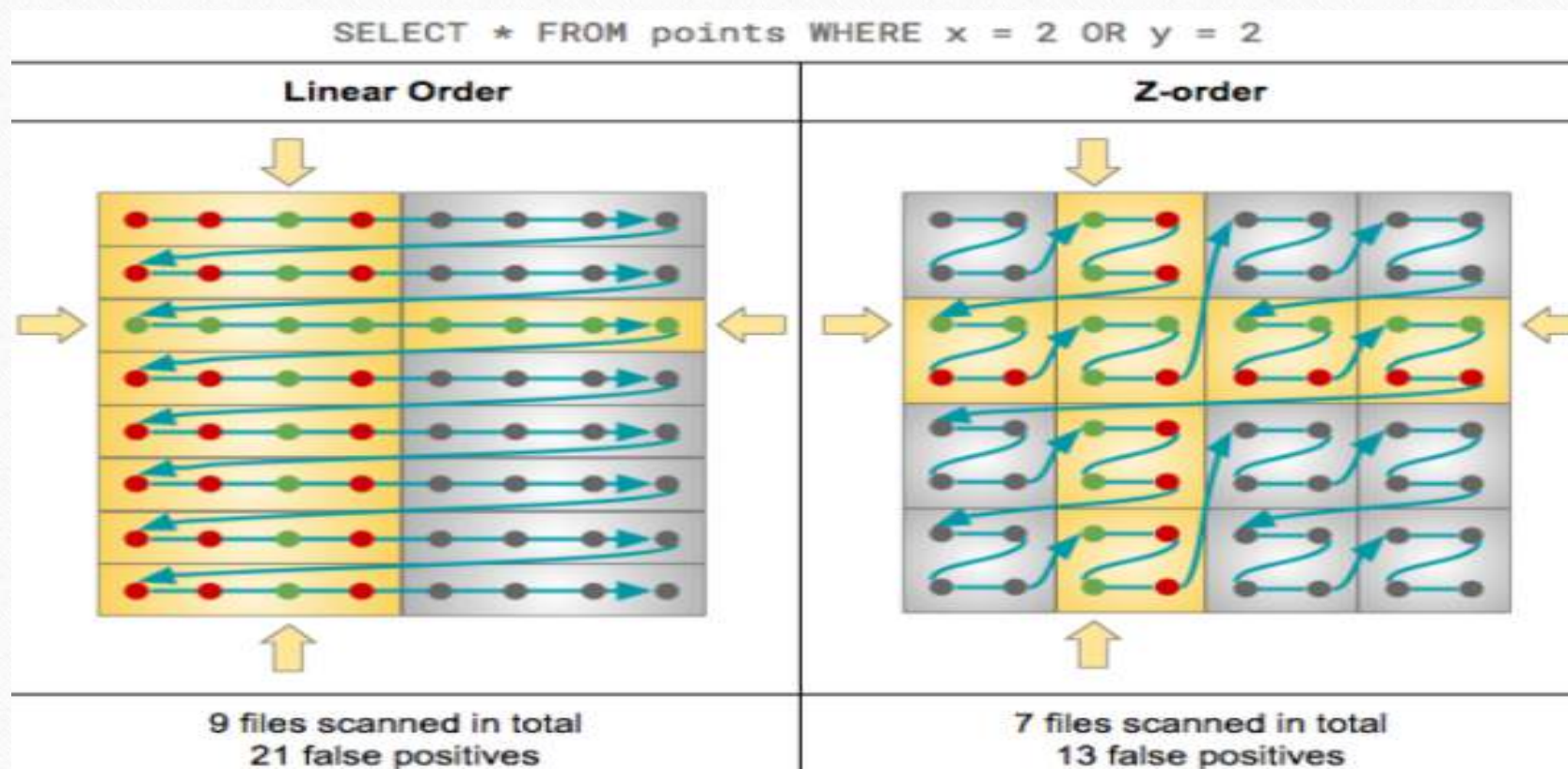
Relational databases use *secondary indexes* to help speed queries. However, indexing becomes impractical with Big Data; the indexes themselves become very large, and they cannot be updated at write time in a performant manner. Z-ordering is a technique that replaces indexes by placing data columns that are "close" in value into the same physical files within a partition.

Z-Ordering is a technique to colocate related information in the same set of files. This co-locality is automatically used by Delta Lake on Databricks **data-skipping** algorithms to dramatically reduce the amount of data that needs to be read. To Z-Order data, you specify the columns to order on in the ZORDER BY clause:

If we interleave the bits of each dimension's values, and sort by the result, we can write files where the 2-dimensional neighbors are also close in 1 dimension.

Even better, Z-ordering works even if the values are of different lengths. It also works across more than 2 dimensions, although it breaks down quickly after 4-5.

Z-ordering lets us skip more files, and get fewer false positives in the files we do read.



ZORDER - Examples..

```
%sql
-- NOTE: this cell may take several minutes to run
-- If time is tight, you may want to just read through the rest of this section

DROP TABLE IF EXISTS flights_delta_z;

-- create a Delta table
CREATE TABLE IF NOT EXISTS flights_delta_z
  USING DELTA
  PARTITIONED BY (Origin)
  AS
  SELECT *
  FROM flights_temp;

-- Now add Z-Ordering
-- In this example, we'll just Z-Order by one column, TailNum,
-- to see how it improves queries using TailNum with the partition key, Origin
OPTIMIZE flights_delta_z ZORDER BY (TailNum)
```

SQL

```
OPTIMIZE events
WHERE date >= current_timestamp() - INTERVAL 1 day
ZORDER BY (eventType)
```

What Is Schema Evolution?

Schema evolution is a feature that allows users to easily change a table's current schema to accommodate data that is changing over time. Most commonly, it's used when performing an append or overwrite operation, to automatically adapt the schema to include one or more new columns.

Following up on the example from the previous section, developers can easily use schema evolution to add the new columns that were previously rejected due to a schema mismatch. Schema evolution is activated by adding `.option('mergeSchema', 'true')` to your `.write` or `.writeStream` Spark command.

```
# Add the mergeSchema option
loans.write.format("delta") \
    .option("mergeSchema", "true") \
    .mode("append") \
    .save(DELTALAKE_SILVER_PATH)
```

How Does Schema Enforcement Work?

Delta Lake uses schema validation *on write*, which means that all new writes to a table are checked for compatibility with the target table's schema at write time. If the schema is not compatible, Delta Lake cancels the transaction altogether (no data is written), and raises an exception to let the user know about the mismatch.

To determine whether a write to a table is compatible, Delta Lake uses the following rules. The DataFrame to be written:

- **Cannot contain any additional columns that are not present in the target table's schema.** Conversely, it's OK if the incoming data doesn't contain every column in the table – those columns will simply be assigned null values.
- **Cannot have column data types that differ from the column data types in the target table.** If a target table's column contains StringType data, but the corresponding column in the DataFrame contains IntegerType data, schema enforcement will raise an exception and prevent the write operation from taking place.
- **Can not contain column names that differ only by case.** This means that you cannot have columns such as 'Foo' and 'foo' defined in the same table. While Spark can be used in case sensitive or insensitive (default) mode, Delta Lake is case-preserving but insensitive when storing the schema. Parquet is case sensitive when storing and returning column information. To avoid potential mistakes, data corruption or loss issues (which we've personally experienced at Databricks), we decided to add this restriction.

172

Query an older snapshot of a table (time travel)

Delta Lake time travel allows you to query an older snapshot of a Delta table. Time travel has many use cases, including:

1. Re-creating analyses, reports, or outputs (for example, the output of a machine learning model). This could be useful for debugging or auditing, especially in regulated industries.
2. Writing complex temporal queries.
3. Fixing mistakes in your data.
4. Providing snapshot isolation for a set of queries for fast changing tables.

Python

```
df1 = spark.read.format("delta").option("timestampAsOf", timestamp_string).load("/delta/events")  
df2 = spark.read.format("delta").option("versionAsOf", version).load("/delta/events")
```

Data Recovery Based on Snapshots

Fix accidental deletes to a table for the user 111:

```
INSERT INTO my_table  
  SELECT * FROM my_table TIMESTAMP AS OF date_sub(current_date(), 1)  
  WHERE userId = 111
```

Fix accidental incorrect updates to a table:

```
MERGE INTO my_table target  
  USING my_table TIMESTAMP AS OF date_sub(current_date(), 1) source  
  ON source.userId = target.userId  
  WHEN MATCHED THEN UPDATE SET *
```

Query the number of new customers added over the last

```
SELECT count(distinct userId) - (  
  SELECT count(distinct userId)  
  FROM my_table TIMESTAMP AS OF date_sub(current_date(), 7))
```

Query an earlier version of the table (time travel)

```
SELECT * FROM events VERSION AS OF 0
```

```
SELECT * FROM events TIMESTAMP AS OF '2019-01-29 00:37:58'
```

Clean up snapshots

Delta Lake provides snapshot isolation for reads, which means that it is safe to run OPTIMIZE even while other users or jobs are querying the table. Eventually however, you should clean up old snapshots. You can do this by running the VACUUM command:

```
VACUUM events
```

You control the age of the latest retained snapshot by using the RETAIN <N> HOURS option:

```
VACUUM events RETAIN 24 HOURS
```

```
VACUUM eventsTable -- vacuum files not required by versions older than the default retention period

VACUUM '/data/events' -- vacuum files in path-based table

VACUUM delta.`/data/events/`

VACUUM delta.`/data/events/` RETAIN 100 HOURS -- vacuum files not required by versions more than 100 hours old

VACUUM eventsTable DRY RUN -- do dry run to get the list of files to be deleted
```

Delta Lake Data retention

By default, Delta tables retain the commit history for 30 days. This means that you can specify a version from 30 days ago. However, there are some caveats:

vacuum deletes only data files, not log files. Log files are deleted automatically and asynchronously after checkpoint operations. The default retention period of log files is 30 days, configurable through the **delta.logRetentionPeriod** property which you set with the ALTER TABLE SET TBLPROPERTIES SQL method.

delta.logRetentionDuration = "interval <interval>": controls how long the history for a table is kept. Each time a checkpoint is written

delta.deletedFileRetentionDuration = "interval <interval>": controls how long ago a file must have been deleted before being a candidate for VACUUM. The default is interval 7 days

NOTE: VACUUM doesn't clean up log files; log files are automatically cleaned up after checkpoints are written.

Describe Detail History

You can retrieve more information about the table (for example, number of files, data size) using DESCRIBE DETAIL.

SQL

```
DESCRIBE DETAIL '/data/events/'
```

```
DESCRIBE HISTORY delta.`/data/events/`
```

```
DESCRIBE HISTORY eventsTable
```

Clone a Delta table

You can create a copy of an existing Delta table at a specific version using the clone command. Clones can be either deep or shallow.

A deep clone is a clone copies the source table data to the clone target in addition to the metadata of the existing table. Additionally, stream metadata is also cloned such that a stream that writes to the Delta table can be stopped on a source table and continued on the target of a clone from where it left off.

A shallow clone is a clone that does not copy the data files to the clone target. The table metadata is equivalent to the source. These clones are cheaper to create.

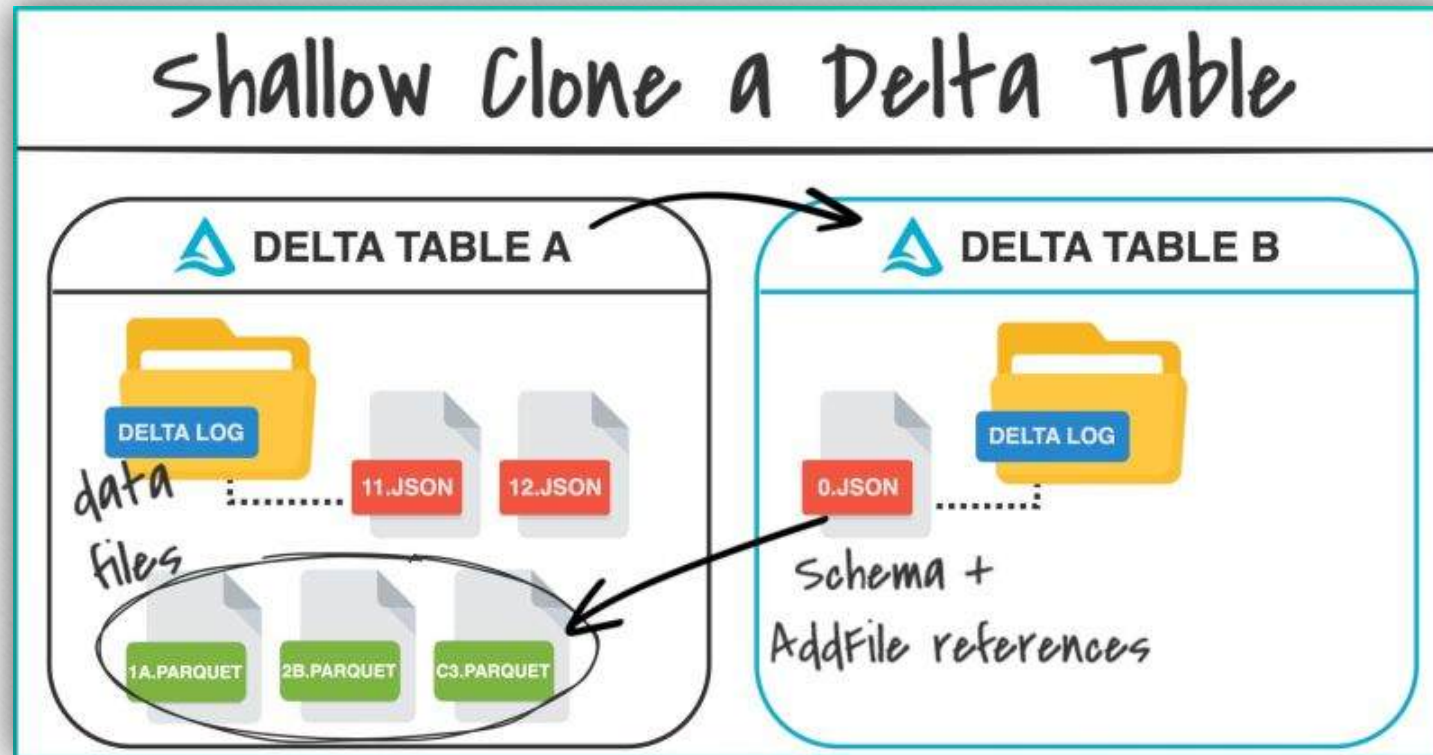
Note: Any changes made to either deep or shallow clones affect only the clones themselves and not the source table.

Clone a Delta table

This is a great way to create a copy of a table **without copying** all of the **data files**.

Instead, only a portion of **the metadata is copied**, including schema & partitioning information, invariants, and column nullability.

Also important to note is that the cloned table will have an **independent history** from its source. As a result, time travel queries on a cloned table will not work the same as they do on the source table.



```
%sql
-- Create a deep clone of /data/source at /data/target
CREATE TABLE delta.`/data/target/` CLONE delta.`/data/source/` |
-- Replace the target
CREATE OR REPLACE TABLE db.target_table CLONE db.source_table
-- No-op if the target table exists
CREATE TABLE IF NOT EXISTS TABLE delta.`/data/target/` CLONE db.source_table

CREATE TABLE db.target_table SHALLOW CLONE delta.`/data/source`

CREATE TABLE db.target_table SHALLOW CLONE delta.`/data/source` VERSION AS OF version
-- timestamp can be like "2019-01-01" or like date_sub(current_date(), 1)
CREATE TABLE db.target_table SHALLOW CLONE delta.`/data/source` TIMESTAMP AS OF timestamp_expression
```

Clone use cases

Data archiving: Data may need to be kept for longer than is feasible with time travel or for disaster recovery. In these cases, you can create a deep clone to preserve the state of a table at a certain point in time for archival. Incremental archiving is also possible to keep a continually updating state of a source table for disaster recovery.

SQL

```
-- Every month run  
CREATE OR REPLACE TABLE delta.`/some/archive/path` CLONE my_prod_table
```

Machine learning flow reproduction: When doing machine learning, you may want to archive a certain version of a table on which you trained an ML model. Future models can be tested using this archived data set.

SQL

```
-- Trained model on version 15 of Delta table  
CREATE TABLE delta.`/model/dataset` CLONE entire_dataset VERSION AS OF 15
```

Clone use cases Continue...

Short-term experiments on a production table: In order to test out a workflow on a production table without corrupting the table, you can easily create a shallow clone. This allows you to run arbitrary workflows on the cloned table that contains all the production data but does not affect any production workloads.

SQL

```
-- Perform shallow clone
CREATE OR REPLACE TABLE my_test SHALLOW CLONE my_prod_table;

UPDATE my_test WHERE user_id is null SET invalid=true;
-- Run a bunch of validations. Once happy:

-- This should leverage the update information in the clone to prune to only
-- changed files in the clone if possible
MERGE INTO my_prod_table
USING my_test
ON my_test.user_id <=> my_prod_table.user_id
WHEN MATCHED AND my_test.user_id is null THEN UPDATE *;

DROP TABLE my_test;
```

Clone use cases Continue...

Data sharing: Other business units within a single organization may want to access the same data but may not require the latest updates. Instead of giving access to the source table directly, clones with different permissions can be provided for different business units. The performance of the clone can exceed that of a simple view as well.

SQL

```
-- Perform deep clone
CREATE OR REPLACE TABLE shared_table CLONE my_prod_table;

-- Grant other users access to the shared table
GRANT SELECT ON shared_table TO `<user-name>@<user-domain>.com`;
```

Run VACUUM Regularly

To ensure that concurrent readers can continue reading a stale snapshot of a table, Databricks Delta leaves deleted files on DBFS for a period of time. The VACUUM command helps save on storage costs by cleaning up these invalid files. It can, however, interrupt users querying a Delta table similar to when partitions are re-written. VACUUM should be run regularly to clean up expired snapshots that are no longer required.

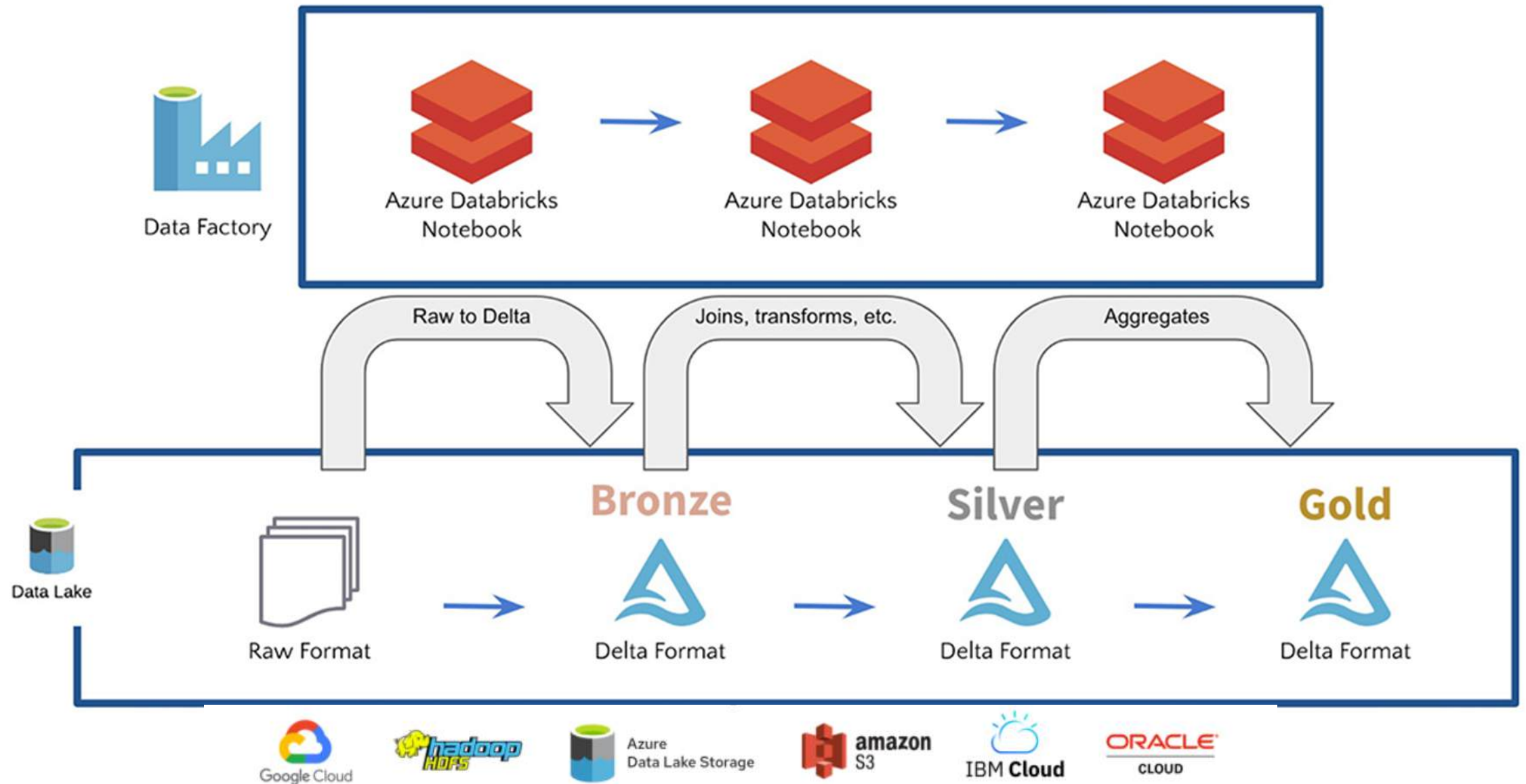
Batch Modifications

Parquet files, that form the underpinning of Delta, are immutable and thus need to be rewritten completely to reflect changes regardless of the extent of the change. Use MERGE INTO to batch changes to amortize costs.

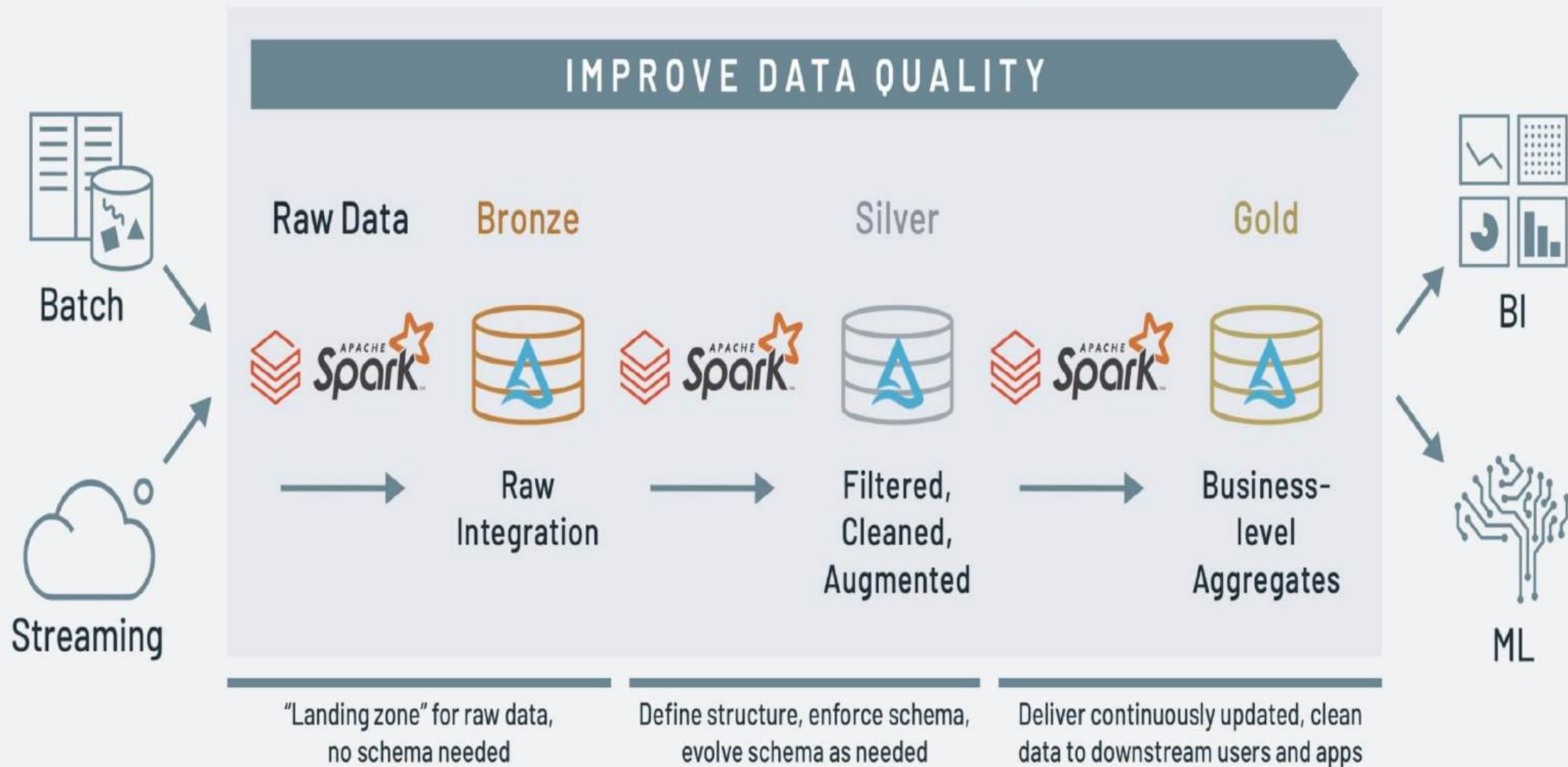
Use DELETES

Manually deleting files from the underlying storage is likely to break the Delta table so instead you should use DELETE commands to ensure proper progression of the change.

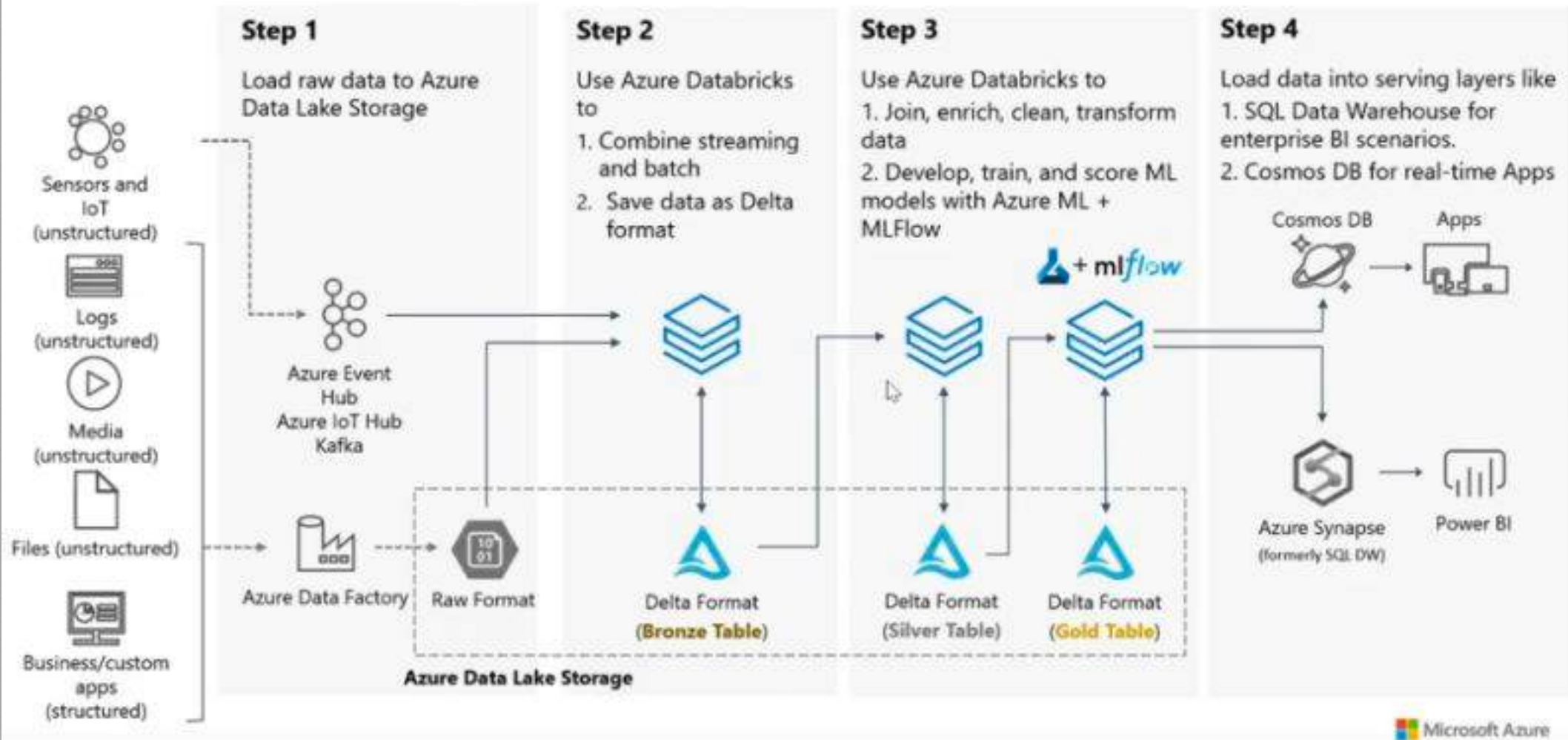
Lakehouse Architecture



Building reliable, performant data pipelines with DELTA LAKE

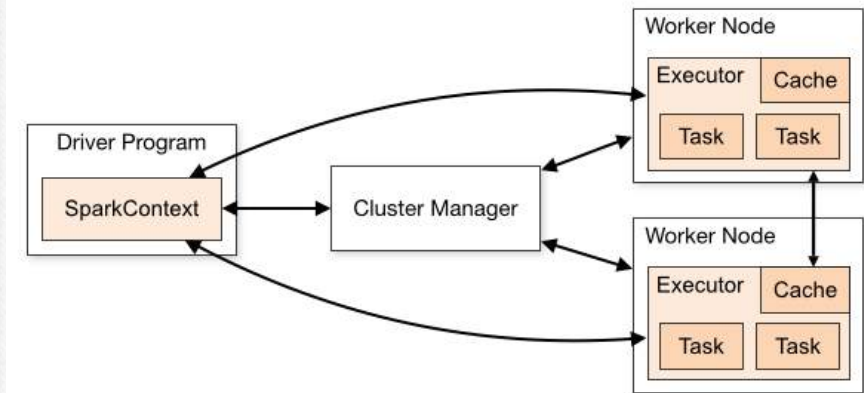
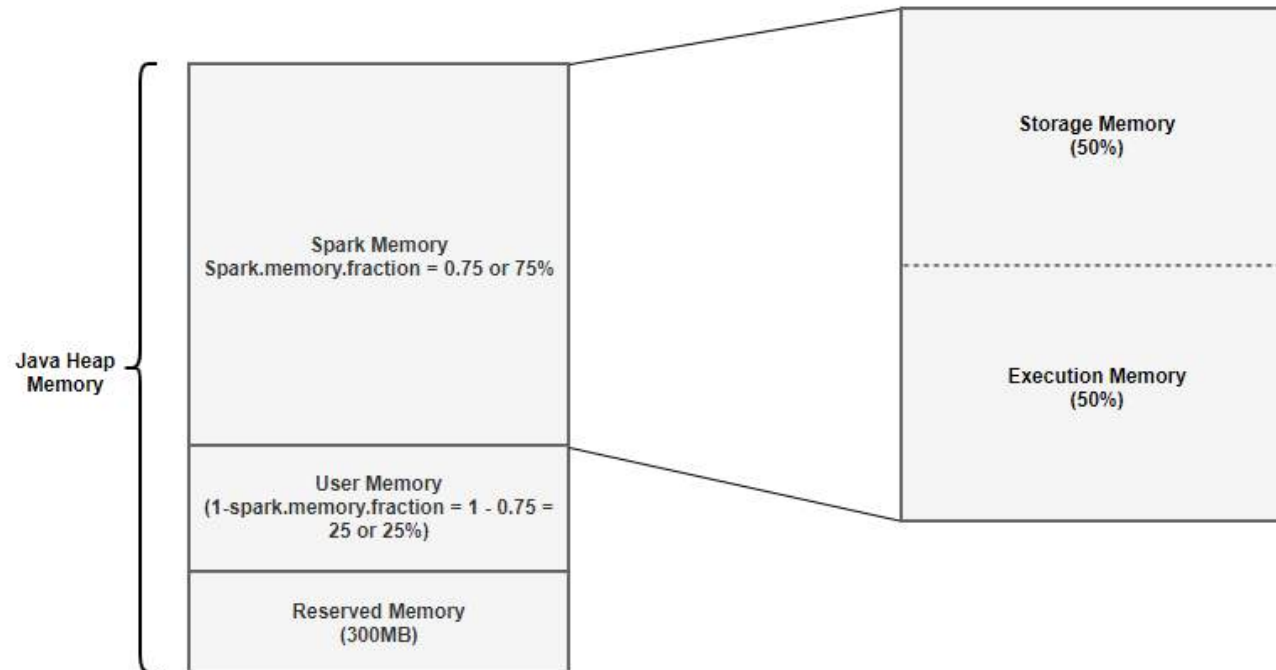


Azure Databricks – Delta Lake at Scale on Azure



Performance Tuning

- ☑ Data Format Selection
- ☑ Partitioning
- ☑ Compression and splittability
- ☑ Shuffle Partitions
- ☑ Cache and Persist
- ☑ Number of Partitions and Partition Size in PySpark
- ☑ File Format selection
- ☑ Join Strategies
- ☑ Memory Management
- ☑ Small Files issue
- ☑ DataSkew issue



Executor memory allocation example

If the executor memory settings are as in the table below, calculate the actual size allocated to each memory area.

conf	value
spark.executor.memory	4g
spark.memory.fraction	0.75
spark.memory.storageFraction	0.5

- Reserved Memory : **300MB**
- User Memory : $(4096\text{MB} - 300\text{MB}) * (1 - 0.75) = \mathbf{949\text{MB}}$
- Spark Memory : $(4096\text{MB} - 300\text{MB}) * 0.75 = \mathbf{2847\text{MB}}$
- Storage Memory : $(4096\text{MB} - 300\text{MB}) * 0.75 * 0.5 = \mathbf{1423\text{MB}}$
- Execution Memory : $(4096\text{MB} - 300\text{MB}) * 0.75 * (1 - 0.5) = \mathbf{1423\text{MB}}$

cache() is an Apache Spark transformation that can be used on a DataFrame, Dataset, or RDD when you want to perform more than one action. cache() caches the specified DataFrame, Dataset, or RDD in the memory of your cluster's workers. Since cache() is a transformation, the caching operation takes place only when a Spark action (for example, count(), show(), take(), or write()) is also used on the same DataFrame, Dataset, or RDD in a single action.

```
df=spark.table("input_table_name")
df.cache.take(5)           # Call take(5) on the DataFrame df, while also caching it
df.count()                 # Call count() on the DataFrame df
```

```
df1=spark.read.parquet(input_path1)
df2=spark.read.parquet(input_path2)
df1.cache()                # Cache DataFrame df1

joined_df = df1.join(df2, df1.id==df2.id, 'inner') # Join DataFrame df1 and df2
filtered_df = joined_df.filter("name == 'John'")  # Filter the joined DataFrame for the name "John"
df1.count()                                       # Call count() on the cached DataFrame
filtered_df.show()                              # Show the filtered DataFrame filtered_df
```

```
df=spark.table("input_table_name")
df.cache.count()           # Call count() on the DataFrame df, while also caching it
df.count()                 # Call count() on the DataFrame df
df.filter("name=='John']").count()
```

You should call count() or write() immediately after calling cache() so that the entire DataFrame is processed and cached in memory. If you only cache part of the DataFrame, the entire DataFrame may be recomputed when a subsequent action is performed on the DataFrame.

Determine Desired Partition Size: Consider the desired partition size based on the available resources, cluster configuration, and the nature of your workload. A common guideline is to aim for partitions that are 128 MB to 1 GB in size, as this range generally balances performance and resource utilization. However, this can vary depending on your specific use case and cluster setup.

Default partition size parameter value is `spark.sql.files.maxPartitionBytes. = 128MB`

We can increase or decrease based on incoming file sizes using `spark.conf.set("spark.sql.files.maxPartitionBytes","1G")`

Table size	Target file size	Approximate number of files in table
10GB	256MB	40
1TB	256MB	4096
2.56TB	256MB	10240
3TB	307MB	12108
5TB	512MB	17339
7TB	716MB	20784
10TB	1GB	24437
20TB	1GB	34437
50TB	1GB	64437
100TB	1GB	114437

Optimize performance with file management

To improve query speed, Delta Lake supports the ability to optimize the layout of data in storage. There are various ways to optimize the layout.

Compaction (bin-packing)

Delta Lake can improve the speed of read queries from a table by coalescing small files into larger ones.

```
OPTIMIZE '/path/to/delta/table' -- Optimizes the path-based Delta Lake table
OPTIMIZE delta_table_name;
OPTIMIZE delta.`/path/to/delta/table`;
-- If you have a large amount of data and only want to optimize a subset of it,
--you can specify an optional partition predicate using `WHERE`:
OPTIMIZE delta_table_name WHERE date >= '2017-01-01'
```

What is Z-ordering?

Z-ordering is a technique to colocate related information in the same set of files. This co-locality is automatically used by Delta Lake on Databricks data-skipping algorithms. This behavior dramatically reduces the amount of data that Delta Lake on Databricks needs to read. To Z-order data, you specify the columns to order on in the ZORDER BY clause:

```
OPTIMIZE events
WHERE date >= current_timestamp() - INTERVAL 1 day
ZORDER BY (eventType)
```

Repartitioning: Use the `repartition()` function to redistribute data across partitions. You can manually repartition the data using a different column or use the `repartition()` function along with a random or semi-random value to shuffle the data evenly across partitions.

```
# Repartition the DataFrame using a random column
repartitioned_df = df.repartition("random_column")
```

Skew Join Optimization: Spark provides a built-in optimization technique called skew join optimization. It handles data skew during joins by redistributing the skewed data across multiple partitions and performing the join in a way that minimizes the impact of skewness.

```
# Enable skew join optimization
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Broadcast Join: If one side of the join operation is small enough to fit in memory, you can use a broadcast join. This technique broadcasts the smaller DataFrame to all worker nodes, avoiding data shuffling. However, this approach may not be suitable if the skewed key is on the larger DataFrame.

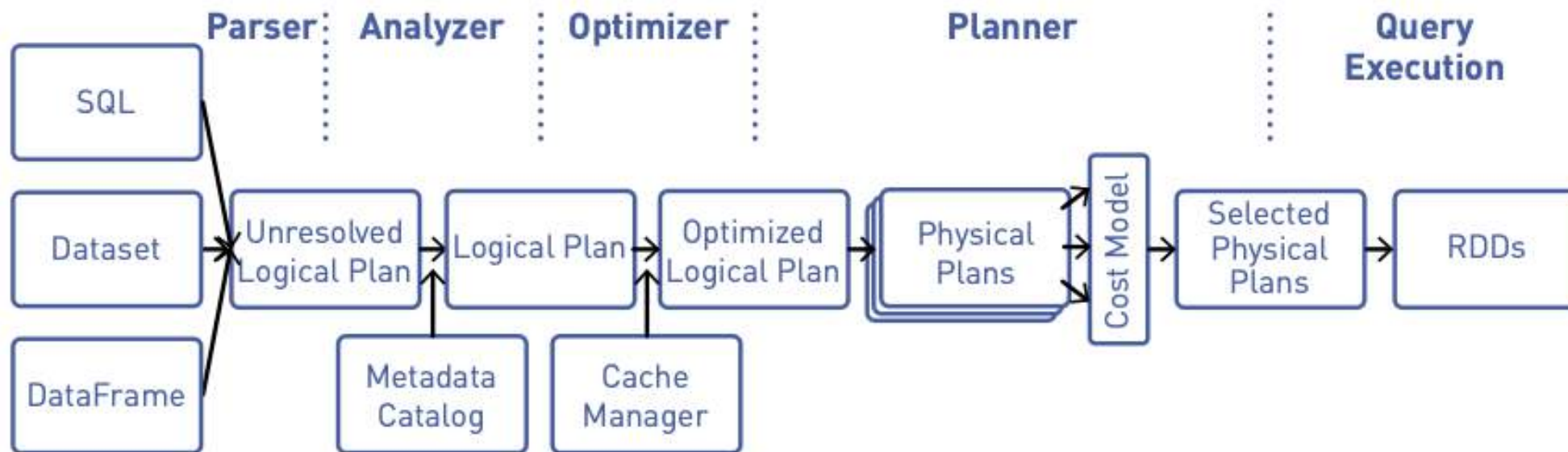
```
from pyspark.sql.functions import broadcast

# Perform a broadcast join
joined_df = df1.join(broadcast(df2), "key_column")
```

Spark Execution Plan

Spark Execution Plan steps

- To illustrate all of these architectural and most relevantly transformations and actions - let's go through a more thorough example, this time using DataFrames and a csv file.
- The DataFrame and SparkSQL work almost exactly as we have described above, we're going to build up a plan for how we're going to access the data and then finally execute that plan with an action. We'll see this process in the diagram below. We go through a process of analyzing the query, building up a plan, comparing them and then finally executing it.



explain(extended=None, mode=None)

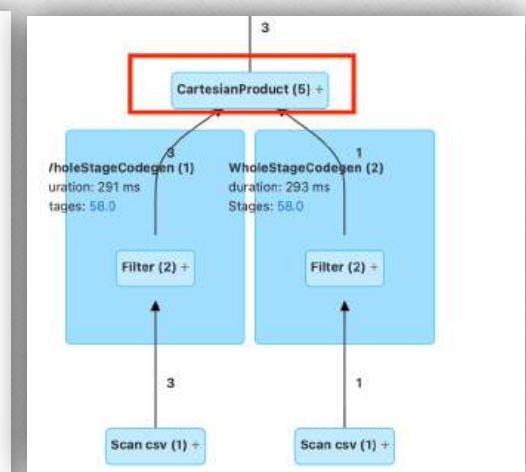
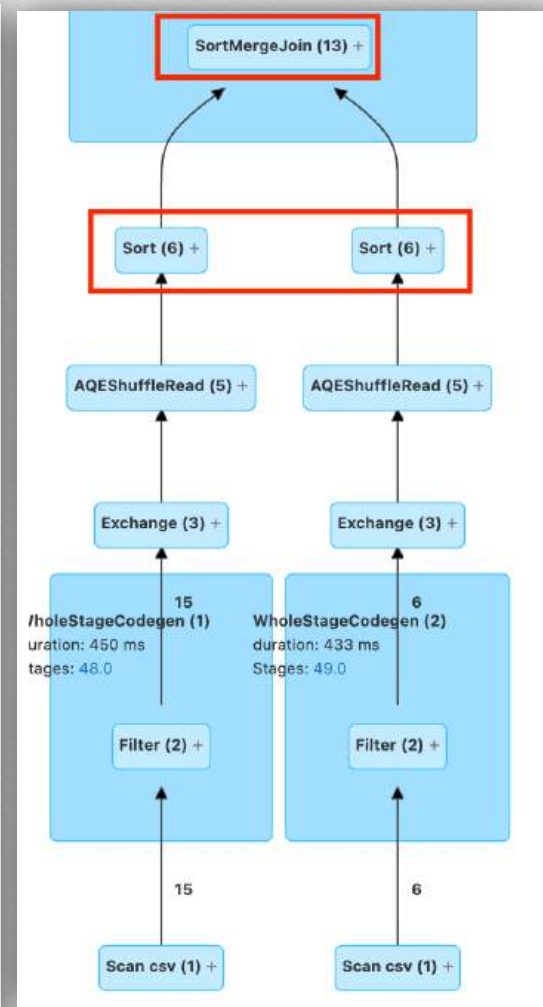
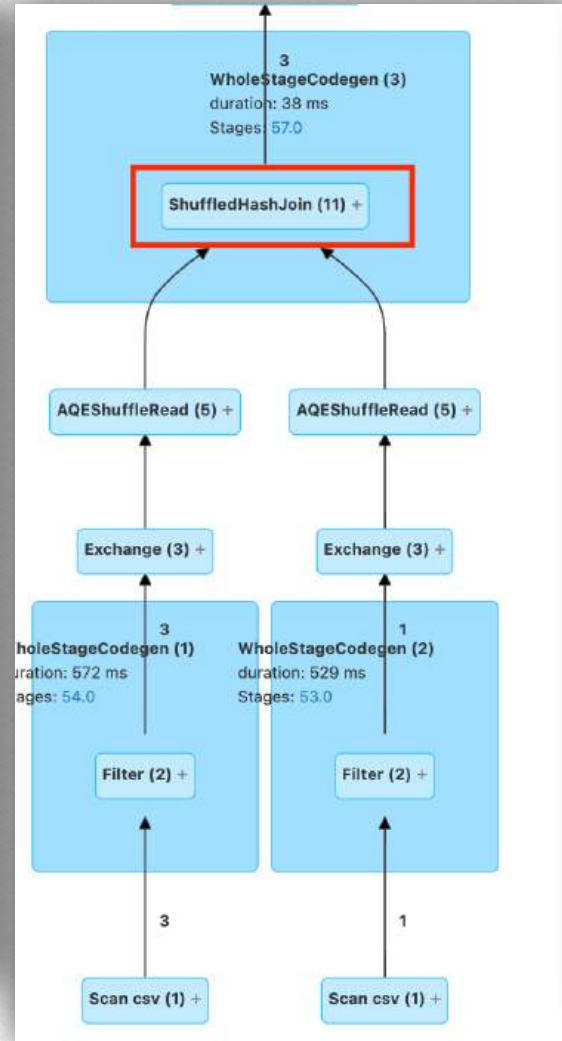
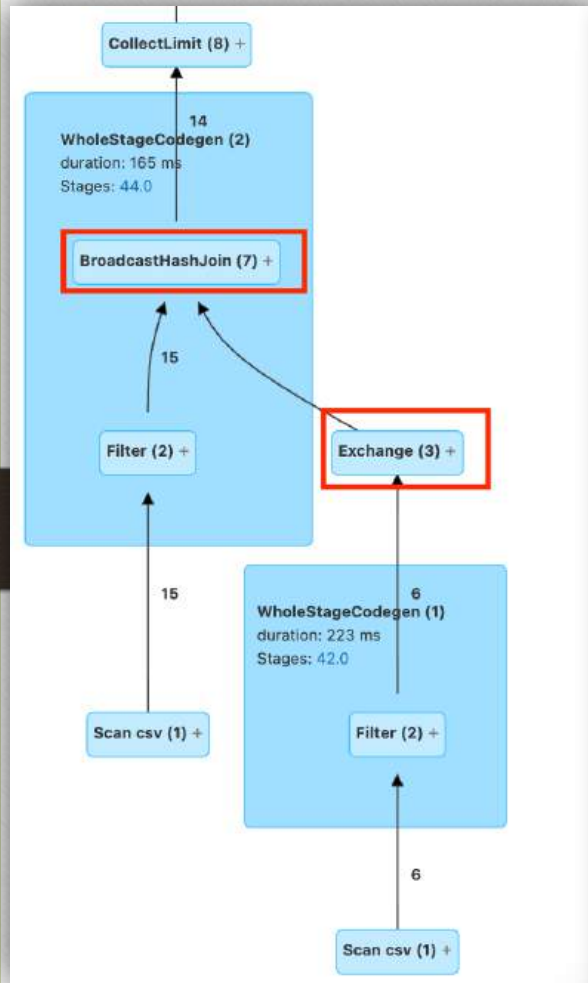
- Prints the (logical and physical) plans to the console for debugging purpose.

Parameters

- **extended** – boolean, default False. If False, prints only the physical plan. When this is a string without specifying the mode, it works as the mode is specified.

mode –

- specifies the expected output format of plans.
- **simple** : Print only a physical plan.
- **extended** : Print both logical and physical plans.
- **codegen** : Print a physical plan and generated codes if they are available.
- **cost** : Print a logical plan and statistics if they are available.
- **formatted** : Split explain output into two sections: a physical plan outline and node details.



Partitioning

Creating Partition using year and month column.

- first we need to add two columns.
- year (yyyy) and month (mm) for creating partitions

Cmd 27

```
1 from pyspark.sql.functions import date_format
2 #creating two YEAR and MONTH new columns based on hiredate date field
3 df_emp_csv = df_emp.withColumn("YEAR",date_format("HIREDATE",'yyyy')).withColumn("MONTH",date_format("HIREDATE",'MM'))
4 df_emp_csv.show()
```

df_emp_csv: pyspark.sql.dataframe.DataFrame = [EMPNO: integer, ENAME: string ... 8 more fields]

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	YEAR	MONTH
7369	SMITH	CLERK	7902	2080-12-17	800	null	20	2080	12
7499	ALLEN	SALESMAN	7698	2081-02-20	1600	300	30	2081	02
7521	WARD	SALESMAN	7698	2081-02-22	1250	500	30	2081	02
7566	JONES	MANAGER	7839	2081-04-02	2975	null	20	2081	04
7654	MARTIN	SALESMAN	7698	2081-09-28	1250	1400	30	2081	09
7698	SGR	MANAGER	7839	2081-05-01	2850	null	30	2081	05
7782	RAVI	MANAGER	7839	2081-06-09	2450	null	10	2081	06
7788	SCOTT	ANALYST	7566	2087-04-19	3000	null	20	2087	04
7839	KING	PRESIDENT	null	2081-11-17	5000	null	10	2081	11
7844	TURNER	SALESMAN	7698	2081-09-08	1500	0	30	2081	09
7876	ADAMS	CLERK	7788	2087-05-23	1100	null	20	2087	05
7900	JAMES	CLERK	7698	2081-12-03	950	null	30	2081	12
7902	FORD	ANALYST	7566	2081-12-03	3000	null	20	2081	12
7934	MILLER	CLERK	7782	2082-01-23	1300	null	10	2082	01
1234	SEKHAR	doctor	7777	null	667	78	80	null	null

```
1 # using PartitionBy with two columns creating partitions
2 df_emp_csv.write.format("delta").partitionBy("YEAR","MONTH").mode("overwrite").saveAsTable("emp_part")
```

Compression Types

GZIP compression uses more CPU resources than Snappy or LZ0, but provides a higher compression ratio.

GZip is often a good choice for cold data, which is accessed infrequently. Snappy or LZ0 are a better choice for hot data, which is accessed frequently for analytics.

BZip2 can also produce more compression than GZip for some types of files, at the cost of some speed when compressing and decompressing.

HBase does not support BZip2 compression.

Snappy often performs better than LZ0. It is worth running tests to see if you detect a significant difference.

CSV Compressions..

- bzip2, deflate, lz4, gzip, snappy,

Cmd 21

```
1 df_airlines.coalesce(1).write.format("csv").mode("overwrite").option("compression","gzip").save("/bigdata/csv_gz/",header=True)
2 df_airlines.coalesce(1).write.format("csv").mode("overwrite").option("compression","snappy").save("/bigdata/csv_snappy/",header=True)
3 df_airlines.coalesce(1).write.format("csv").mode("overwrite").option("compression","deflate").save("/bigdata/csv_deflate/",header=True)
4 df_airlines.coalesce(1).write.format("csv").mode("overwrite").option("compression","lz4").save("/bigdata/csv_lz4/",header=True)
5 df_airlines.coalesce(1).write.format("csv").mode("overwrite").option("compression","bzip2").save("/bigdata/csv_bz2/",header=True)
```

Parquet Compressions

- brotli, uncompressed, lz4, gzip, lzo, snappy, none, zstd

Cmd 26

This lzo library is not available by default in databricks. need to install manually if its required.

Cmd 27

```
1 df_airlines.coalesce(1).write.format("parquet").mode("overwrite").option("compression","snappy").save("/bigdata/parquet_snappy/",header=True)
2 df_airlines.coalesce(1).write.format("parquet").mode("overwrite").option("compression","gzip").save("/bigdata/parquet_gz/",header=True)
3 df_airlines.coalesce(1).write.format("parquet").mode("overwrite").option("compression","zstd").save("/bigdata/parquet_zstd/",header=True)
4 df_airlines.coalesce(1).write.format("parquet").mode("overwrite").option("compression","lz4").save("/bigdata/parquet_lz4/",header=True)
5 df_airlines.coalesce(1).write.format("parquet").mode("overwrite").option("compression","lzo").save("/bigdata/parquet_lzo/",header=True)
```

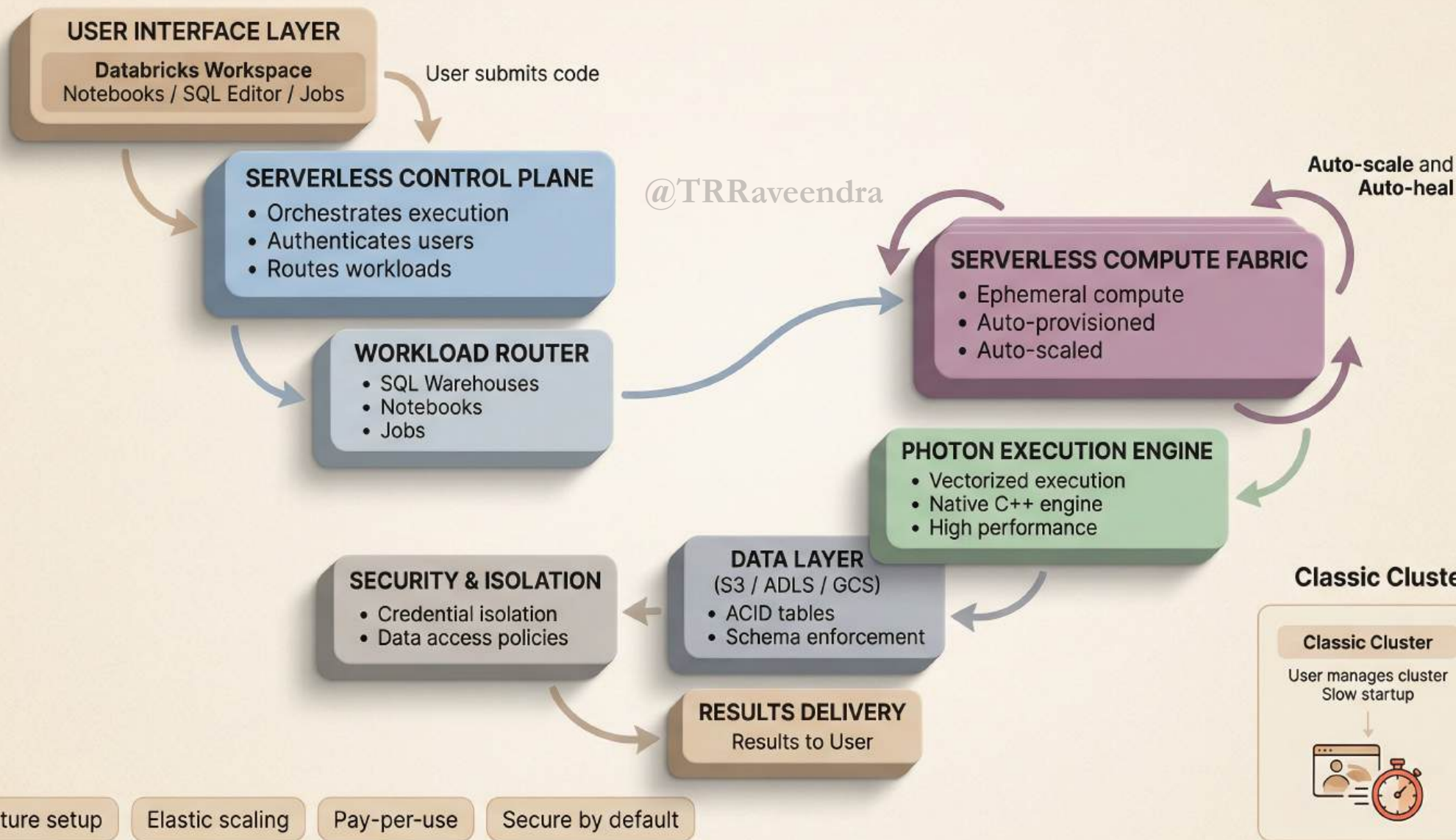
Property	CSV	Json	Parquet	Avro	ORC
Human Readable	YES	YES	NO	NO	NO
Compressable	YES	YES	YES	YES	YES
Splittable	YES*	YES*	YES	YES	YES
Complex data structure	NO	YES	YES	YES	YES
Schema evolution	NO	NO	YES	YES	YES
Columnar	NO	NO	YES	NO	YES

- Join Hints Types
 - `BROADCAST`
 - Suggests that Spark use broadcast join. The join side with the hint will be broadcast regardless of `autoBroadcastJoinThreshold`. If both sides of the join have the broadcast hints, the one with the smaller size (based on stats) will be broadcast. The aliases for `BROADCAST` are `BROADCASTJOIN` and `MAPJOIN`.
 - `MERGE`
 - Suggests that Spark use shuffle sort merge join. The aliases for `MERGE` are `SHUFFLE_MERGE` and `MERGEJOIN`.
 - `SHUFFLE_HASH`
 - Suggests that Spark use shuffle hash join. If both sides have the shuffle hash hints, Spark chooses the smaller side (based on stats) as the build side.
 - `SHUFFLE_REPLICATE_NL`
 - Suggests that Spark use shuffle-and-replicate nested loop join.



```
1 -- how to use hints for performance tuning
2 -- Join Hints for broadcast join
3 select /*+ BROADCAST(d) */ * from v_df_emp_csv as e
4 inner join v_df_dept_csv as d on e.deptno = d.deptno
5 -- Join Hints for shuffle sort merge join
6 select /*+ SHUFFLE_MERGE(e) */ * from v_df_emp_csv as e
7 inner join v_df_dept_csv as d on e.deptno = d.deptno
8 -- Join Hints for shuffle hash join
9 select /*+ SHUFFLE_HASH(e) */ * from v_df_emp_csv as e
10 inner join v_df_dept_csv as d on e.deptno = d.deptno
11 -- Join Hints for shuffle-and-replicate nested loop join
12 select /*+ SHUFFLE_REPLICATE_NL(e) */ * from v_df_emp_csv as e
13 inner join v_df_dept_csv as d on e.deptno = d.deptno
```

Databricks Serverless Architecture — Deep Dive



Learning Goals: Understand Serverless concept • Know internal components • See execution flow • Understand benefits vs classic clusters

Classic Cluster

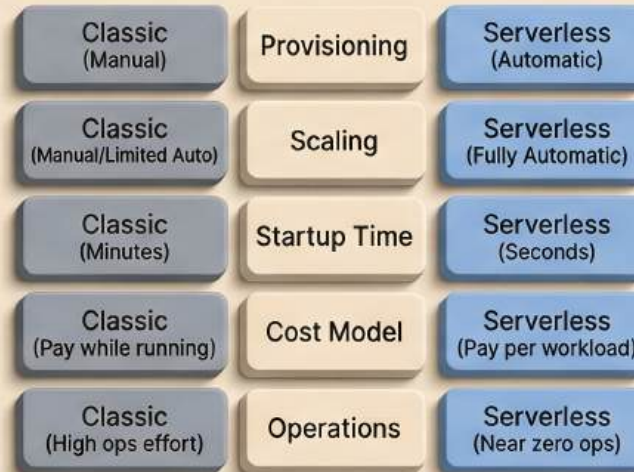
- User creates cluster
- User selects VM size
- User manages scaling
- Long-running nodes



Classic Cluster Architecture



Serverless Architecture



Need control & customization
→ Classic Cluster

Need simplicity & speed
→ Serverless

Serverless Compute

- No cluster creation
- Auto-provisioned compute
- Auto-scaled
- Ephemeral compute

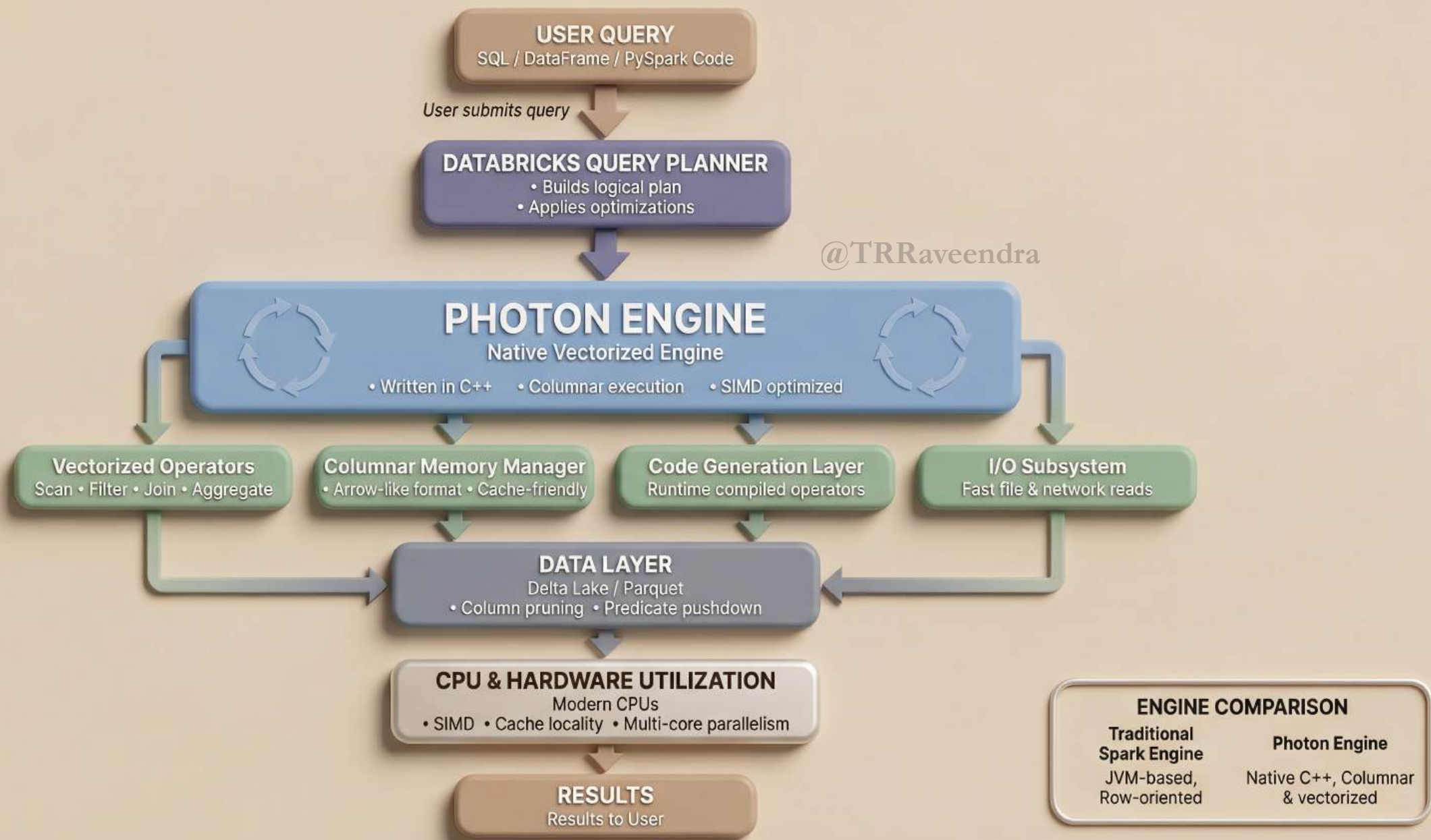


• Understand deployment models

• Know operational differences

• Know cost implications

• Choose right compute type



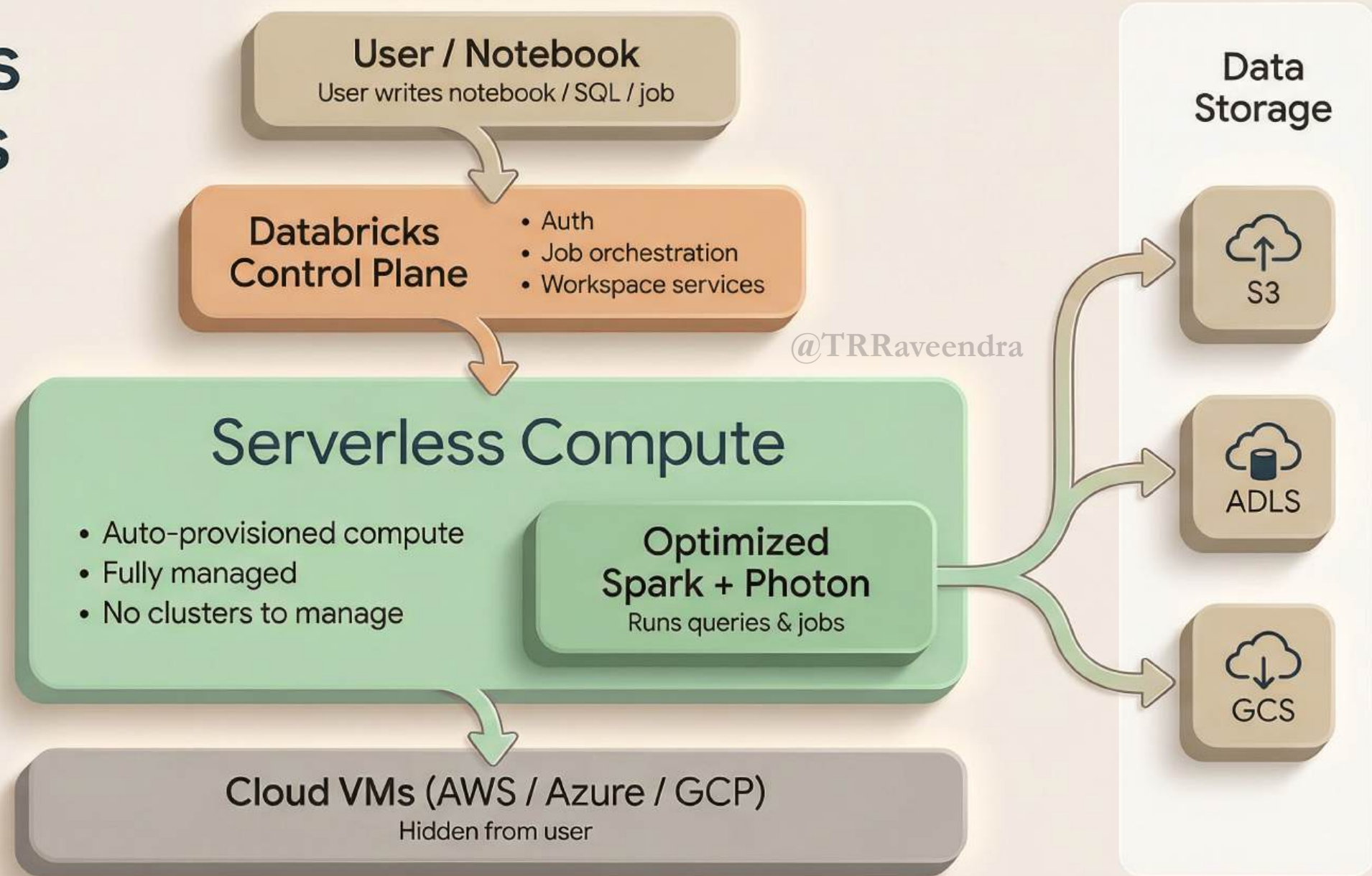
PERFORMANCE GAINS

Lower latency, Higher throughput, Better CPU utilization

LEARNING GOALS

Understand Photon architecture, Know why it is fast, See execution flow, Understand hardware efficiency

Databricks Serverless Compute



✓ Understand serverless concept

✓ Know where compute runs

✓ See difference from traditional clusters

Databricks Control Plane

- Workspace UI
- Job orchestration
- Authentication & metadata
- Cluster management

No customer data processed

Send commands
& configurations



Status & metrics

Customer Data Plane

- Compute (clusters / serverless)
- Spark & Photon execution
- Reads/writes customer data

@TRRaveendra

Customer data stays here

Understand separation of planes • Know where data lives • Understand security boundary

Users & Applications



Analysts



Data Engineers



Data Scientists



BI Tools

Databricks Compute

Spark + Photon

@TRRaveendra

Unity Catalog

Security & governance

Delta Lake

ACID data lake



S3 / ADLS / GCS



Workload Types



SQL
Analytics



Streaming

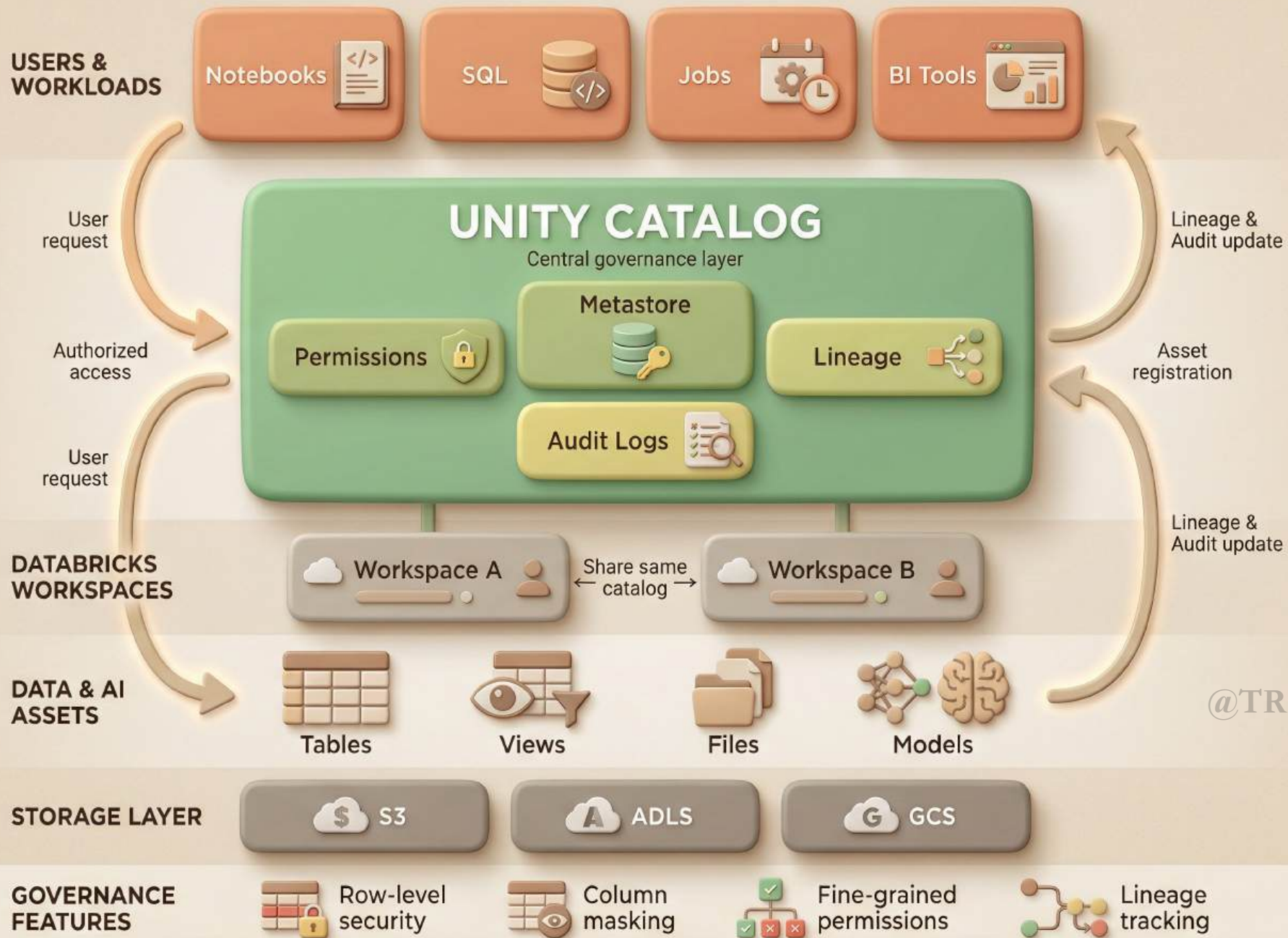


Machine
Learning



BI

Understand Lakehouse concept • See how components fit together • Know role of Delta & Unity Catalog



LEARNING GOALS: Understand role of Unity Catalog • Know how governance is centralized • See how access is controlled

Users & Queries

SQL / DataFrame / BI Queries

Spark
Planner

Spark SQL Planner

Generates physical plan

Results

Execution
Flow

Photon Engine (C++ Vectorized)

- Columnar execution
- SIMD
- Cache-aware

Data Source

Delta Lake / Parquet

Learning Goals

Cloud Storage

S3 / ADLS / GCS

@TRRaveendra

Why Photon
Is Faster



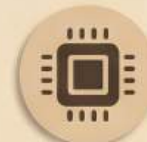
Vectorized
processing



Off-heap
memory



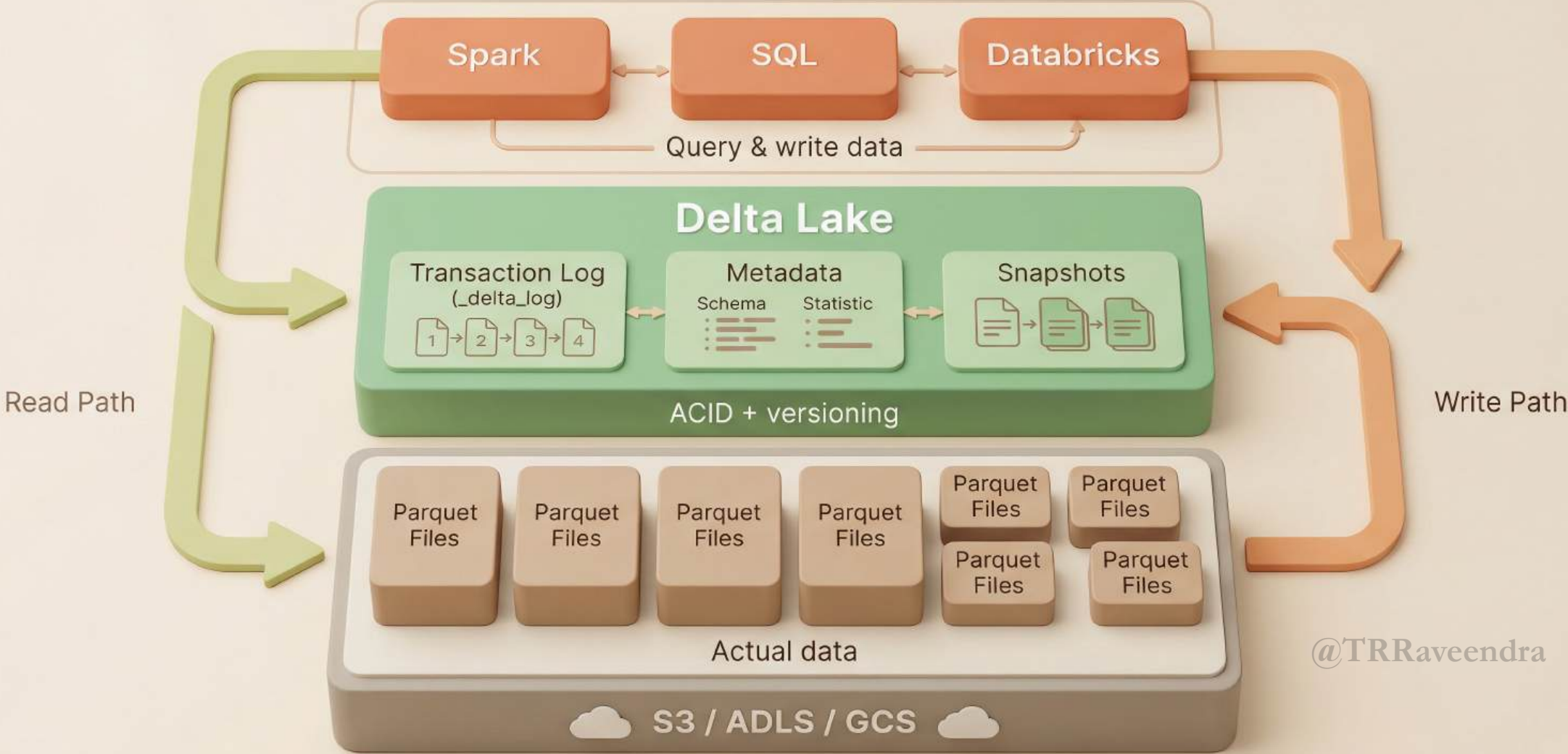
Reduced JVM
overhead



CPU-optimized

Understand Photon's role • See where acceleration happens • Know why it improves performance

DELTA LAKE ARCHITECTURE



Delta Features



ACID Transactions



Time Travel



Schema Enforcement



Schema Evolution

Learning Goals



Understand Delta Lake components

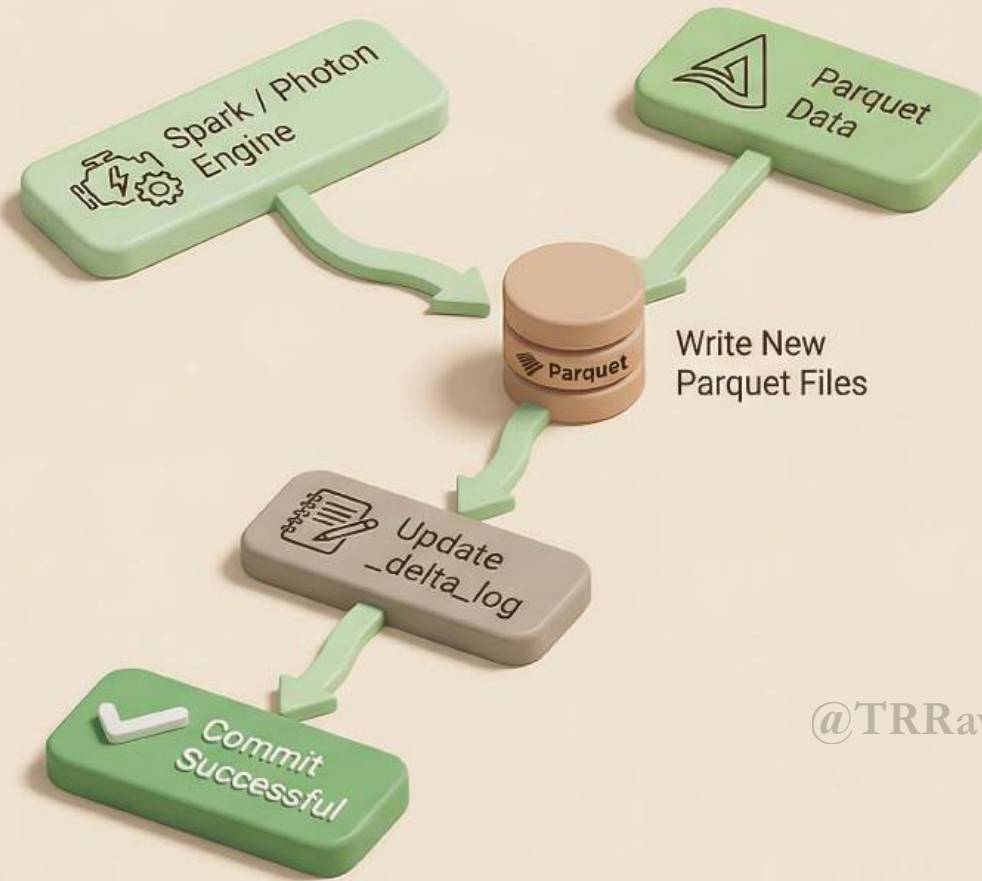


Know role of transaction log

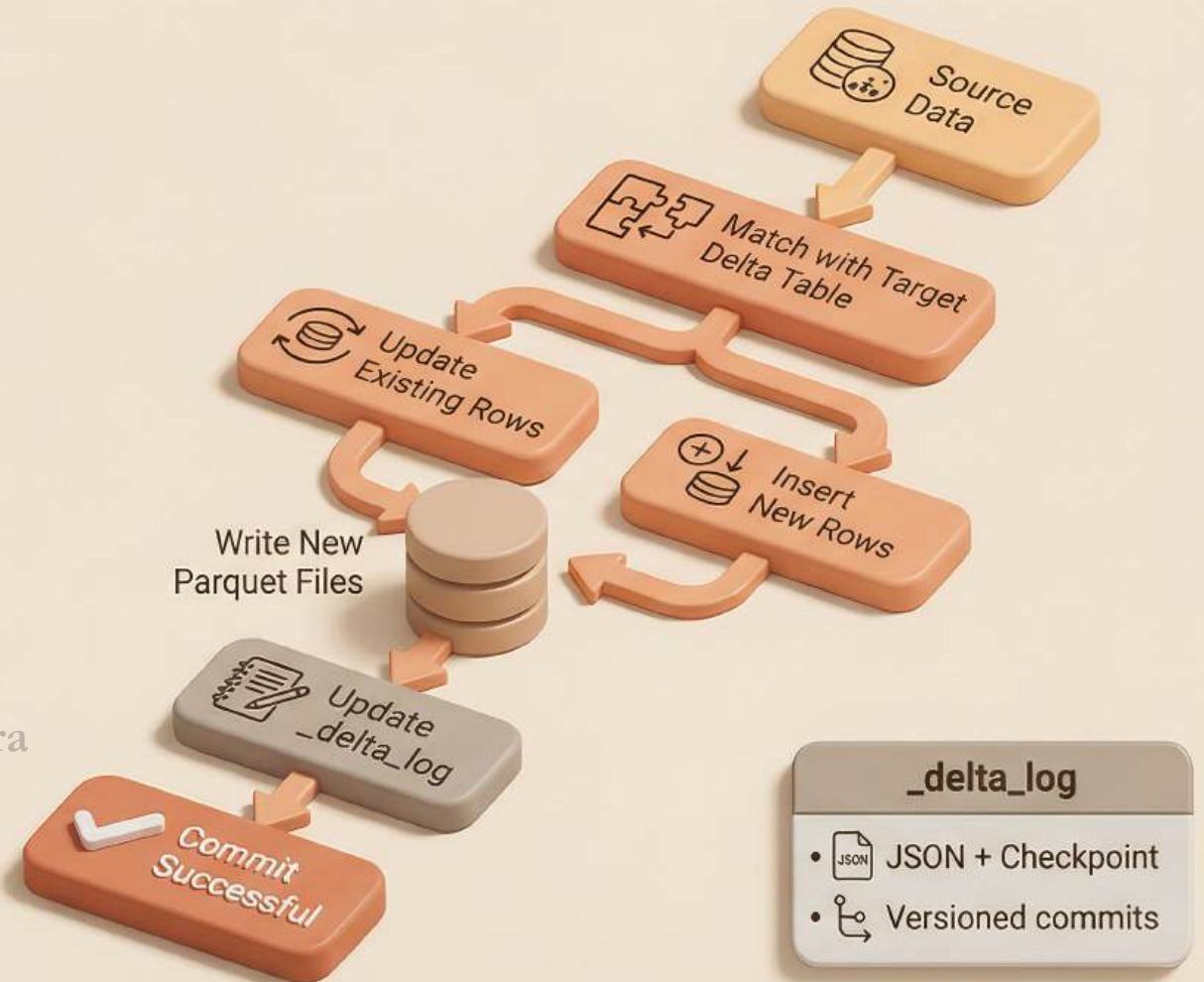


See how ACID is achieved

Delta Write (Append) Flow



Delta MERGE (Upsert) Flow



@TRRaveendra



Learning Goals: Understand Delta writes • Understand MERGE mechanics • See how ACID is maintained

Delta Lake Time Travel



```
SELECT *  
FROM table  
VERSION AS OF 2
```

```
SELECT *  
FROM table  
TIMESTAMP AS OF  
'2025-01-10'
```

@TRRaveendra

Read flow

Query



Delta Log

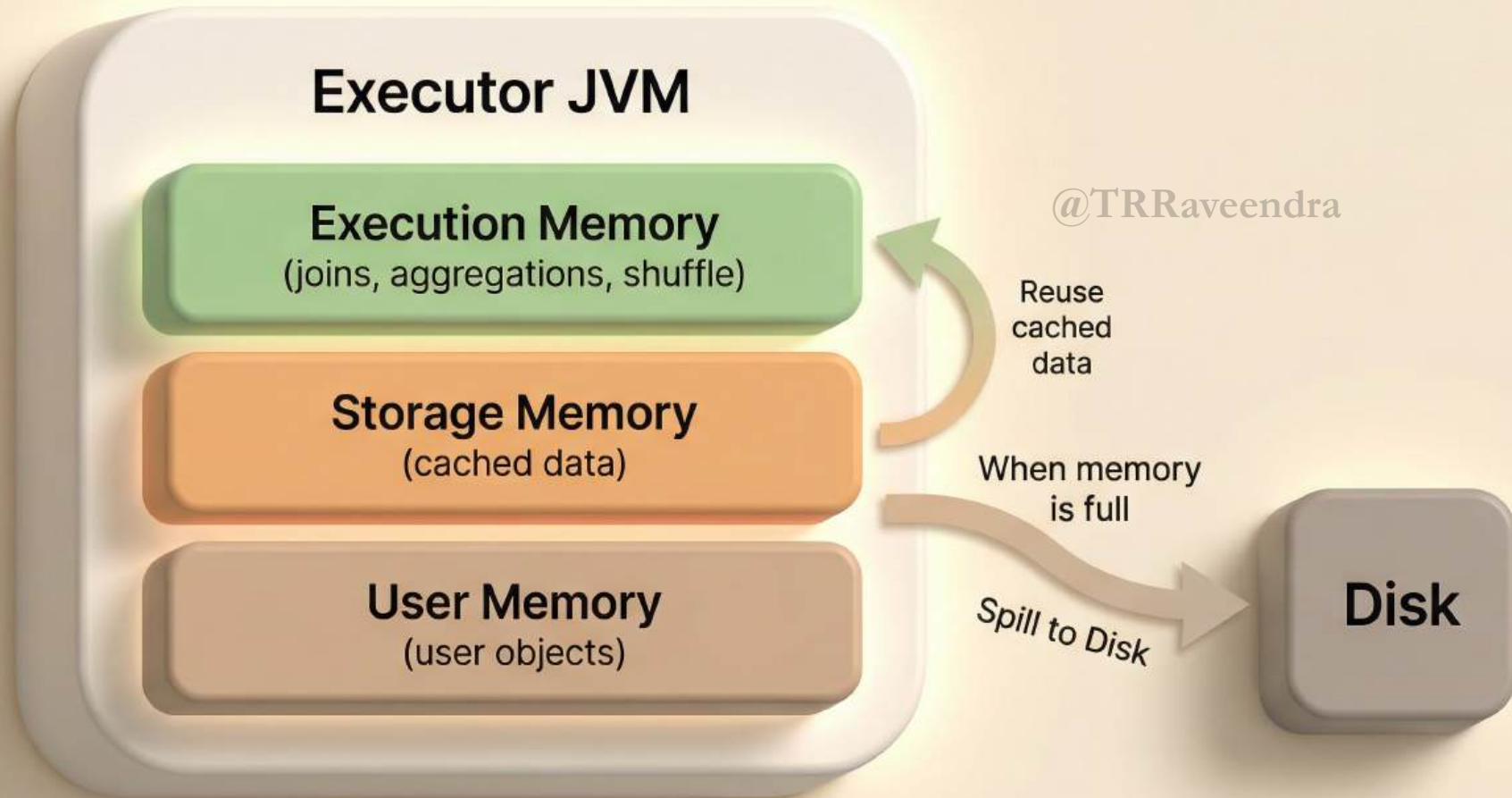


Specific
Version Files



Learning Goals: • Understand table versioning • Know how to query history • Understand rollback capability

Apache Spark Memory Flow



- Understand executor memory layout
- Know difference between execution & storage memory
- Understand spilling
- See how caching works

Apache Spark Memory Management Internals — Unified Memory Architecture Ultra-Deep Dive

Enterprise-Grade Technical Reference for Data Engineers & Performance Specialists

MIDDLE LAYERS - CONFIGURATION & SCENARIOS

`spark.executor.memory`
Total heap size per executor

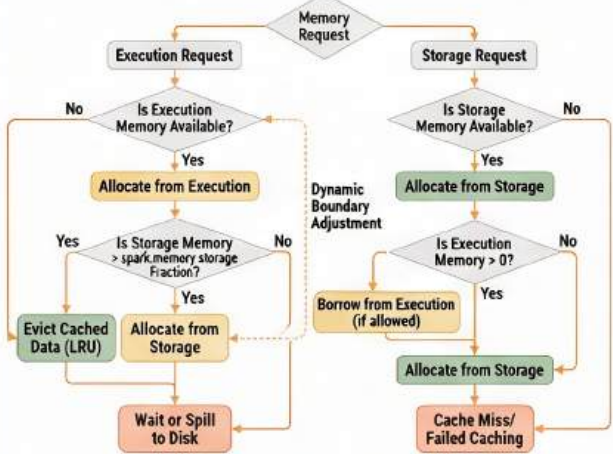
`spark.memory.fraction`
Fraction of heap used for Unified Memory Pool

`spark.memory.storageFraction`
Fraction of Unified Pool immune to eviction

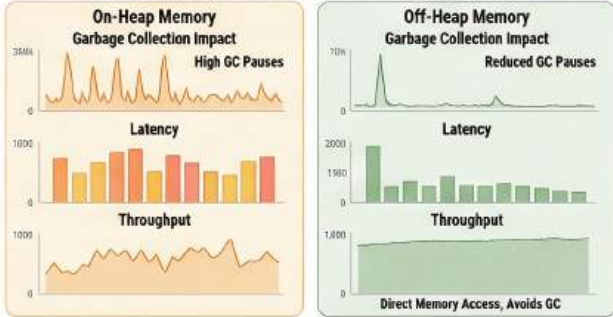
`spark.memory.offHeap.enabled`
Enable off-heap memory storage

`spark.memory.offHeap.size`
Total off-heap memory size

Unified Memory Manager Internals

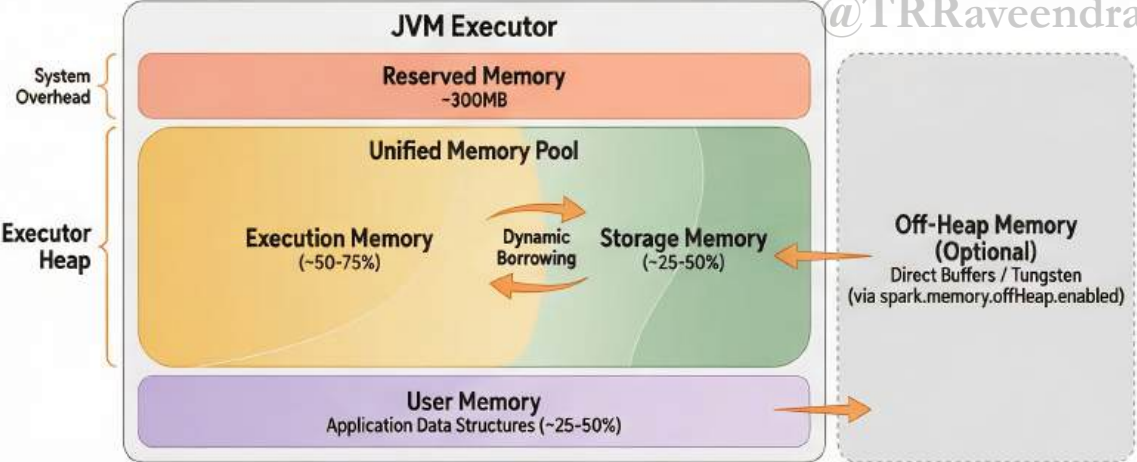


On-Heap vs Off-Heap Comparison



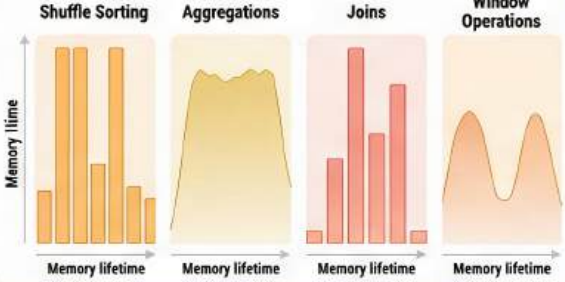
BOTTOM LAYERS - ADVANCED CONCEPTS

TOP LAYER - EXECUTOR MEMORY LANDSCAPE

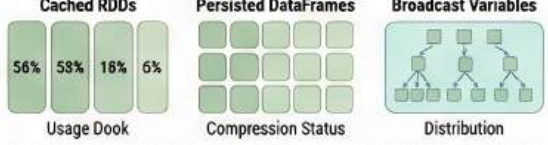


@TRRaveendra

Execution Memory Consumers



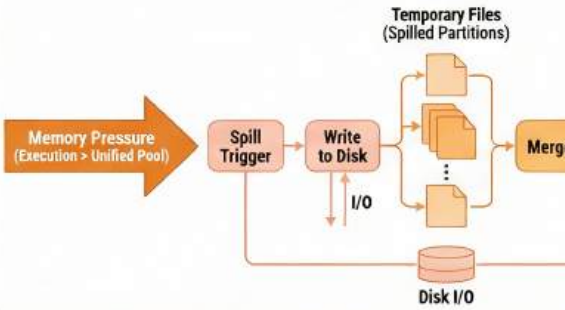
Storage Memory Consumers



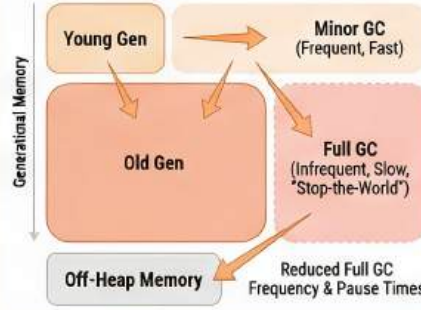
LRU Eviction Policy



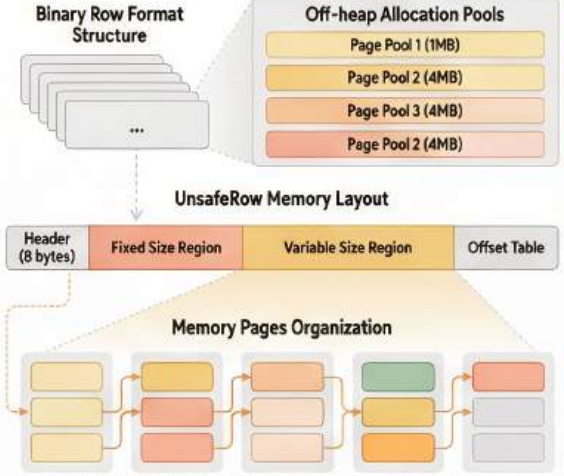
Spill-to-Disk Path



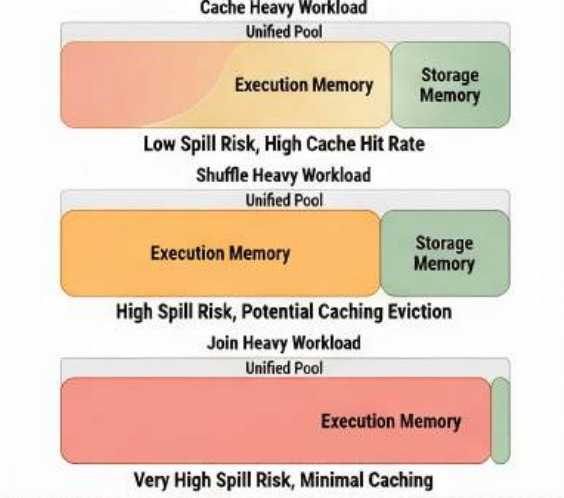
JVM Garbage Collection Interaction



Tungsten Memory Model



Memory Pressure Scenarios



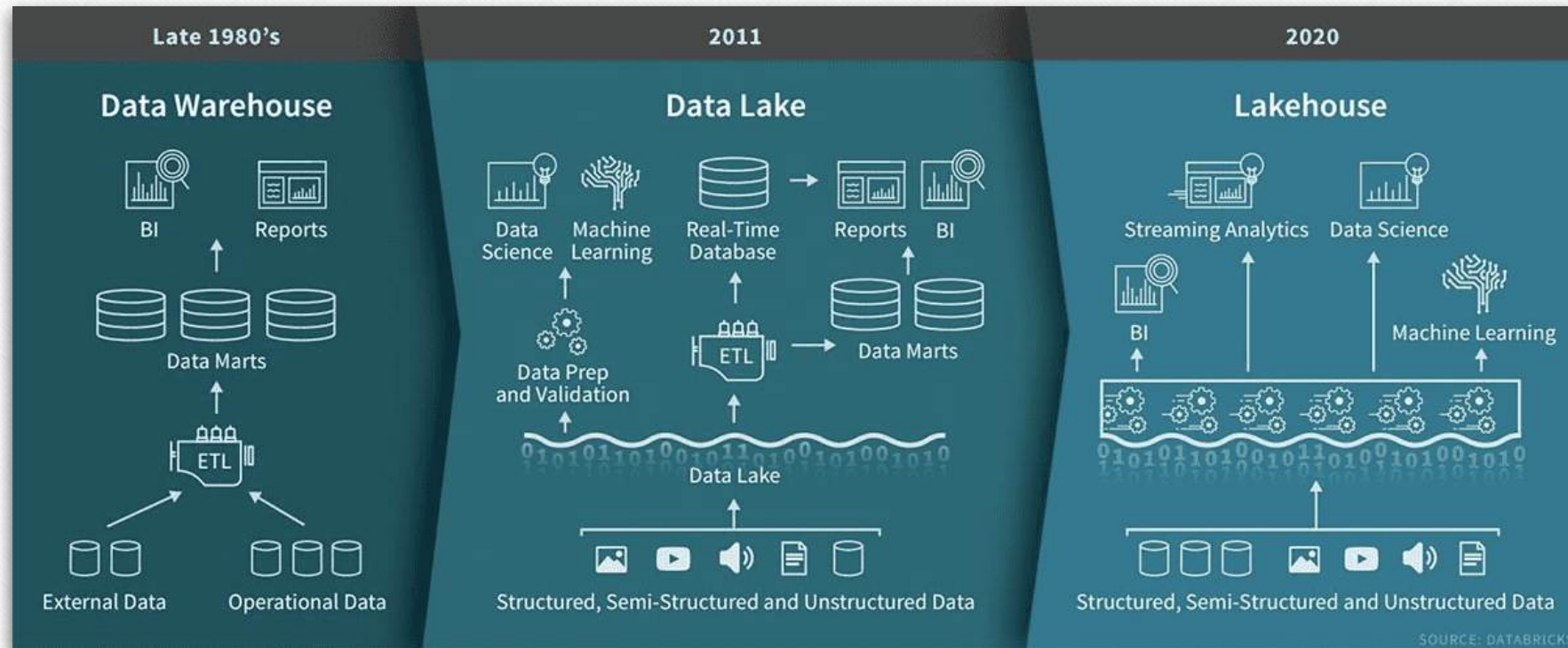
BOTTOM PANEL - LEARNING OUTCOMES

- Understand Spark's unified memory design for efficient resource utilization
- Know where memory goes during execution across different operations
- Learn how spilling happens under memory pressure and its impact
- Identify tuning strategies using configuration parameters and monitoring

Databricks Spark + Delta + Data Lake = LakeHouse



Different Types of Architectures from 1980 To Till



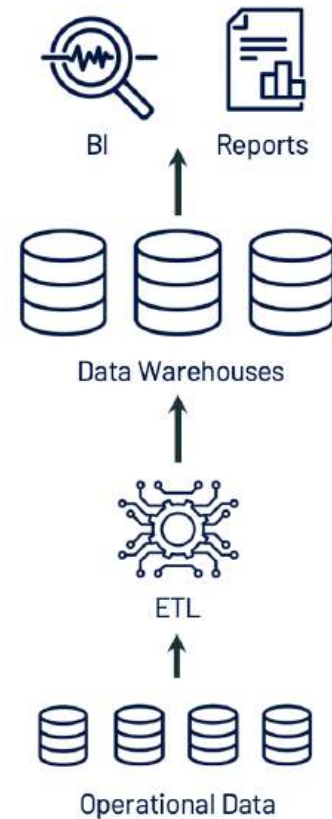
Data Warehouse

- ☑ The Data Warehouse is your company's central data store.
- ☑ A Data Warehouse is required for all companies that wish to make data-driven choices since it serves as the "Single Source of Truth" for all data in the company.
- ☑ Analyzing Existing Business Data
- ☑ New Business Models Launched Based on Existing Data Analysis
- ☑ Enhance customer service and interaction
- ☑ boost quality and resource utilisation

Are there any gaps in the Data Warehouse?

- ☑ Data Warehouses only support Structured Data.
- ☑ There is no support for video, audio, or text.
- ☑ There is no support for data science or machine learning, and there is only limited support for streaming.
- ☑ As a result, the majority of commercial and social media data is kept in data lakes and blob storage.

1990's Traditional Data Warehouses

ORACLE®
DATABASE

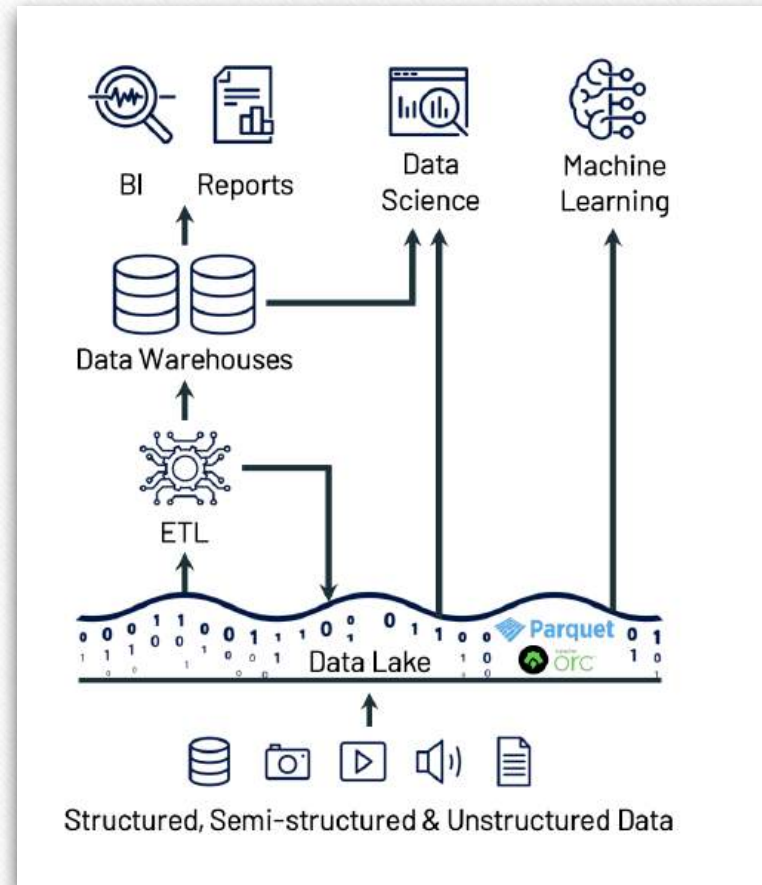
teradata.

IBM DB2

Microsoft
SQL Server

MySQL

Big Data Data Lake with Data Warehouses



Big Data Data Lake with Data Warehouses

- ✓ A datalake is a centralized repository for storing, processing, and securing massive volumes of organized, semistructured, and unstructured data. It can store data in its native format and handle any type of data, regardless of size.

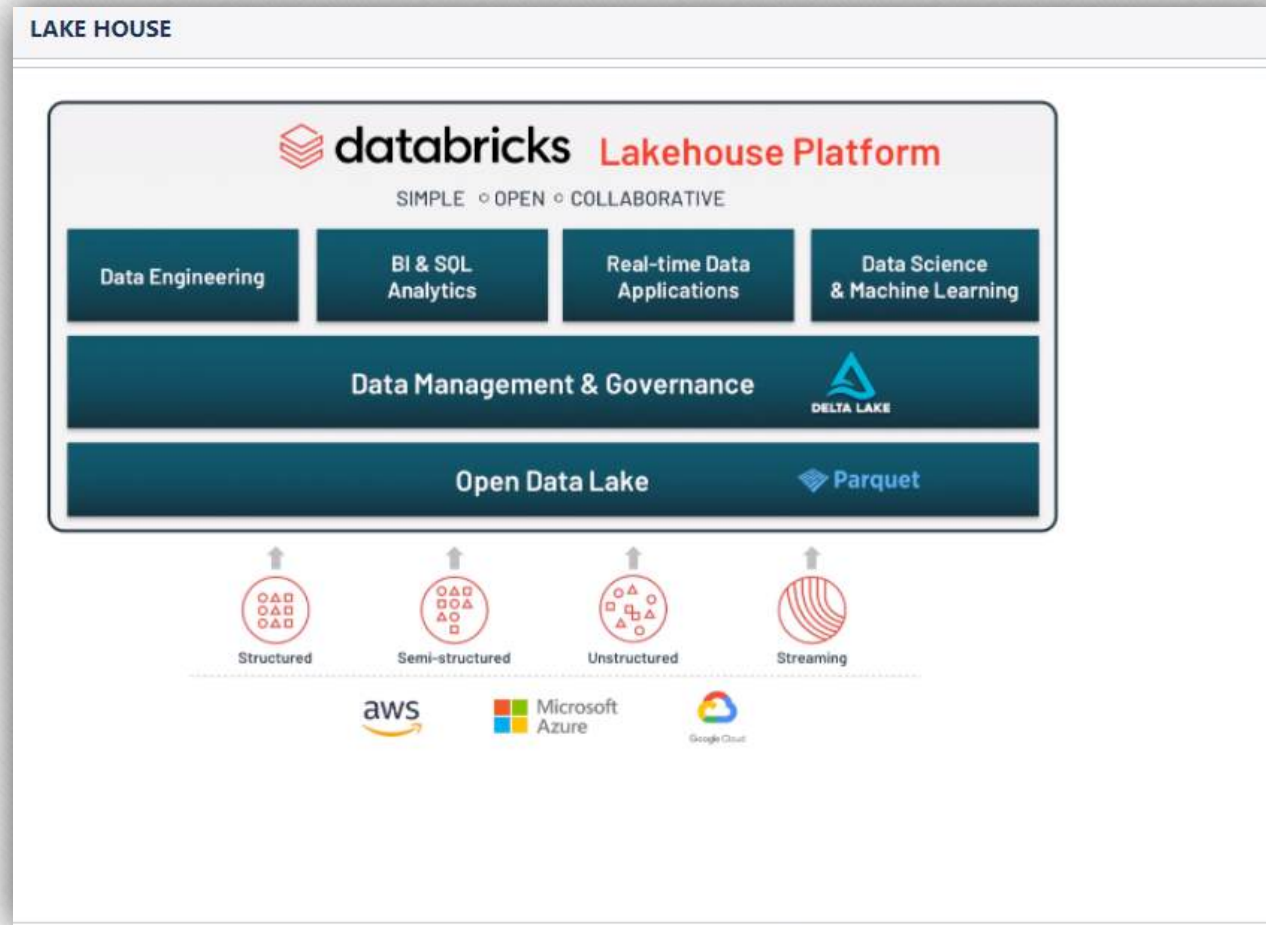
✓ ADVANTAGES OF DATA LAKE

- ✓ The primary benefit of data lakes is the concentration of several information sources.
- ✓ At a cheaper cost, distributed and infinite storage
- ✓ There are image, video, audio, json, csv, and other data storage types accessible.
- ✓ **Volume and Variety:** A data lake can hold the massive amounts of data required by Big Data, artificial intelligence, and machine learning. Data lakes can manage the volume, diversity, and velocity of data absorbed in any format from numerous sources.
- ✓ **Ingest Speed:** Format is unimportant during ingest. It employs schema-on-read rather than schema-on-write, which implies that data is not processed for use until it is required. Data can be swiftly written.
- ✓ **Cost savings:** In terms of storage expenses, a data lake can be substantially less expensive than a data warehouse.
- ✓ This enables businesses to acquire a broader range of data, such as unstructured data like rich media, sensor data from the Internet of Things (IoT), email, or social media.
- ✓ **Greater Accessibility:** Data stored in a data lake makes it simple to open copies or subsets of data for multiple users or user groups to access. Data access might be restricted, while businesses can allow more accessibility.
- ✓ **Advanced Algorithms:** Data lakes enable businesses to run complicated queries and deep learning algorithms to identify trends.

Disadvantages of DataLake

1. Data appending is difficult.
Adding additional data results in improper readings.
2. It is tough to modify existing data.
GDPR/CCPA necessitates fine-grained adjustments to current data lakes.
3. Jobs that fail in the middle result in corrupted files.
Half of the data is present in the data lake, but the other half is missing.
4. Operation in real time
Inconsistency results from combining streaming and batch.
5. Expensive to preserve older data versions.
Reproducibility, auditing, and governance are all required in regulated situations.
6. Difficulty in dealing with vast amounts of metadata
The metadata itself becomes challenging to handle in vast data lakes.
7. "Too many files" issues
Data lakes struggle to handle millions of little files.
8. It is difficult to obtain excellent performance.
Data partitioning for performance is error-prone and difficult to modify.
9. Problems with data quality
It is a continual challenge to guarantee that all data is correct and of high quality.
10. Incomplete ACID Transactions
Insert, update, and delete operations are not possible due to a lack of ACID Properties.
- 11 We require extra Warehouse and ETL processes to load from DataLake to DataWarehouse in order to satisfy Incomplete ACID Transactions and Metadata Catalog maintenance.

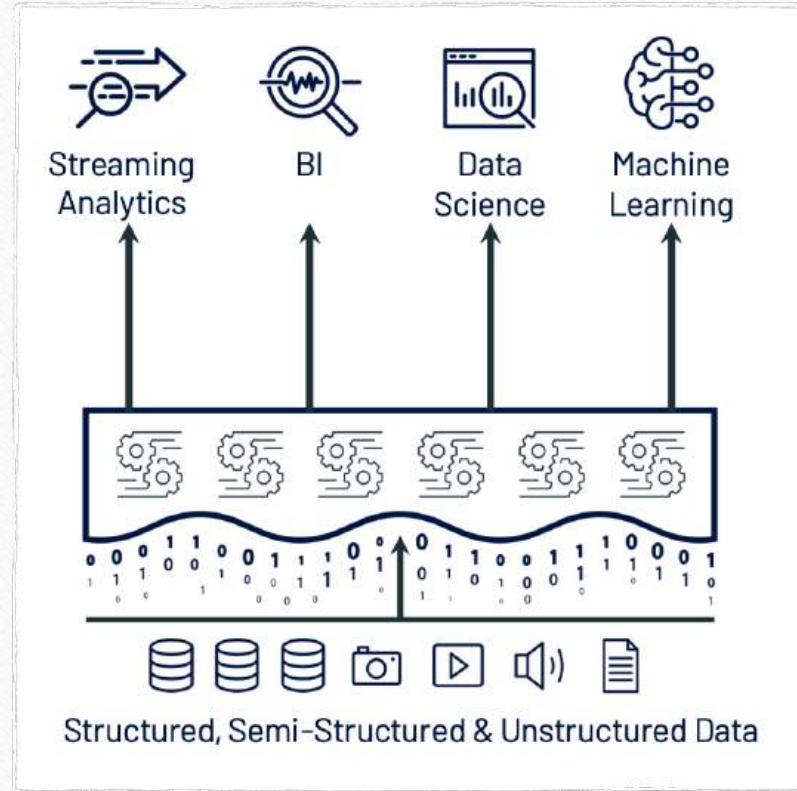
Modern Lakehouse Architecture



	Data Warehouse	Hadoop M/R	APACHE Spark
Separate Compute & Storage	✗	✓	✓
More than SQL (i.e ML)	✗	✓	✓
Open Source at Scale	✗	✓	✓
SQL & Optimization	✓	✗	✓
Data Model & Catalog	✓	✗	✓
ACID Transactions	✓	✗	✓



DataWarehouse + Data Lake = Lakehouse



Delta Lake Alternative..... ICEBERG



Modern Lakehouse Features

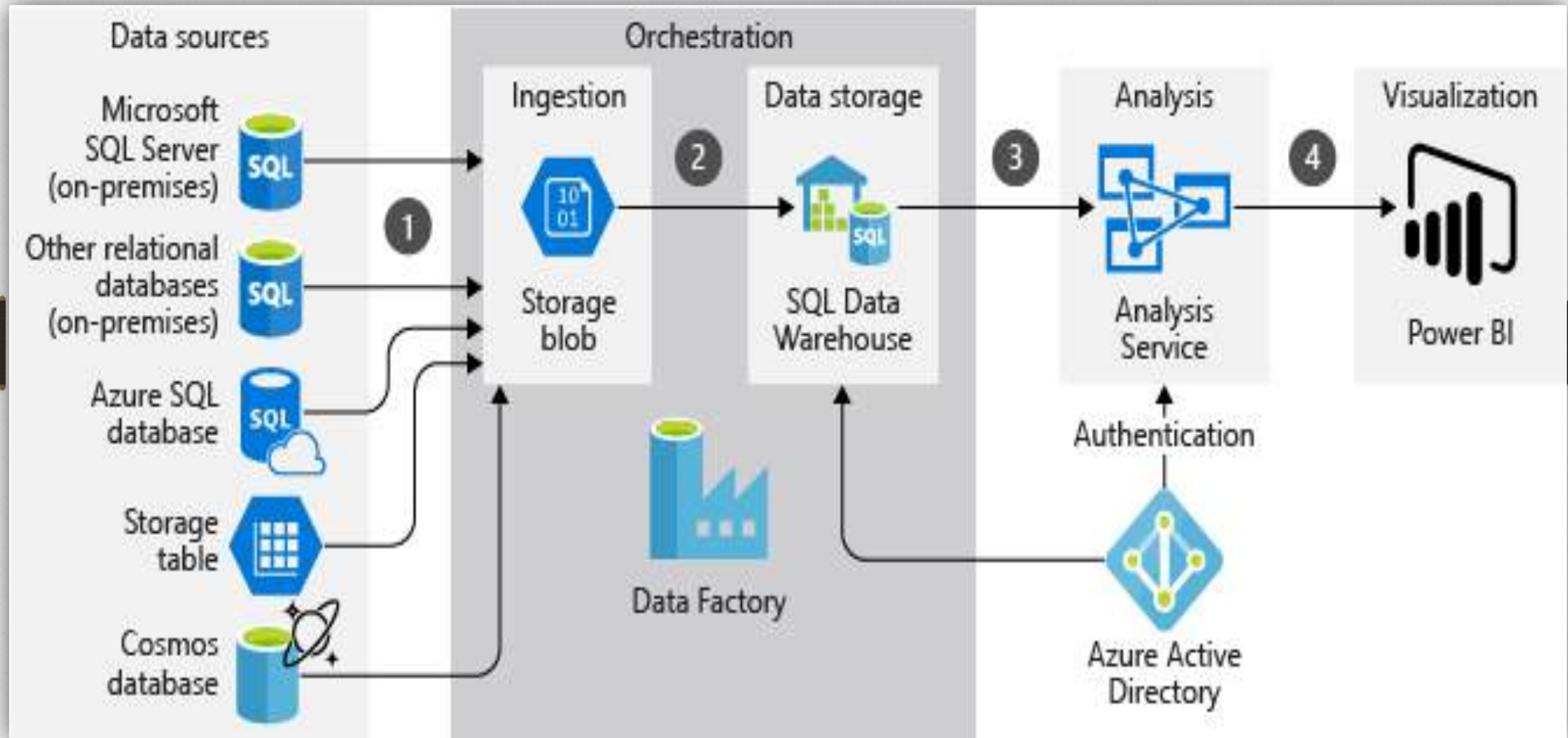
Delta Lake is an open-source storage framework that allows for the creation of a Lakehouse architecture with computation engines such as Spark. DeltaLake enables ACID transactions, scalable #metadata management, and the unification of streaming and batch data processing. Delta Lake is fully compatible with Apache Spark APIs and runs on top of your existing datalake.

- 1) Because it is ACID compliant, it provides guaranteed consistency.
- 2) Strong data storage. Data is stored in versioned parquet files.
- 3) It is intended to work with Apache Spark.
- 4) The audit logs will be kept in json format.
- 5) Time Travel Support

A lakehouse has the following features:

- 1) support for numerous data kinds and formats
- 2) data reliability and consistency help with a range of jobs (BI, data science, machine learning, and analytics)
- 3) direct use of BI techniques to source data
- 4) Metadata layers for data lakes
- 5) new query engine designs that enable high-performance SQL execution on data lakes
- 6) improved access to data science and machine learning tools
- 7) Data reading and writing at the same time.
- 8) Schema support combined with data governance methods.
- 9) Source data can be accessed directly.
- 10) Storage and computation resources are separated.
- 11) Storage formats that are standardised.
- 12) Structured and semi-structured data types, including IoT data, are supported.
- 13) End-To-End Batch and Streaming Loads.

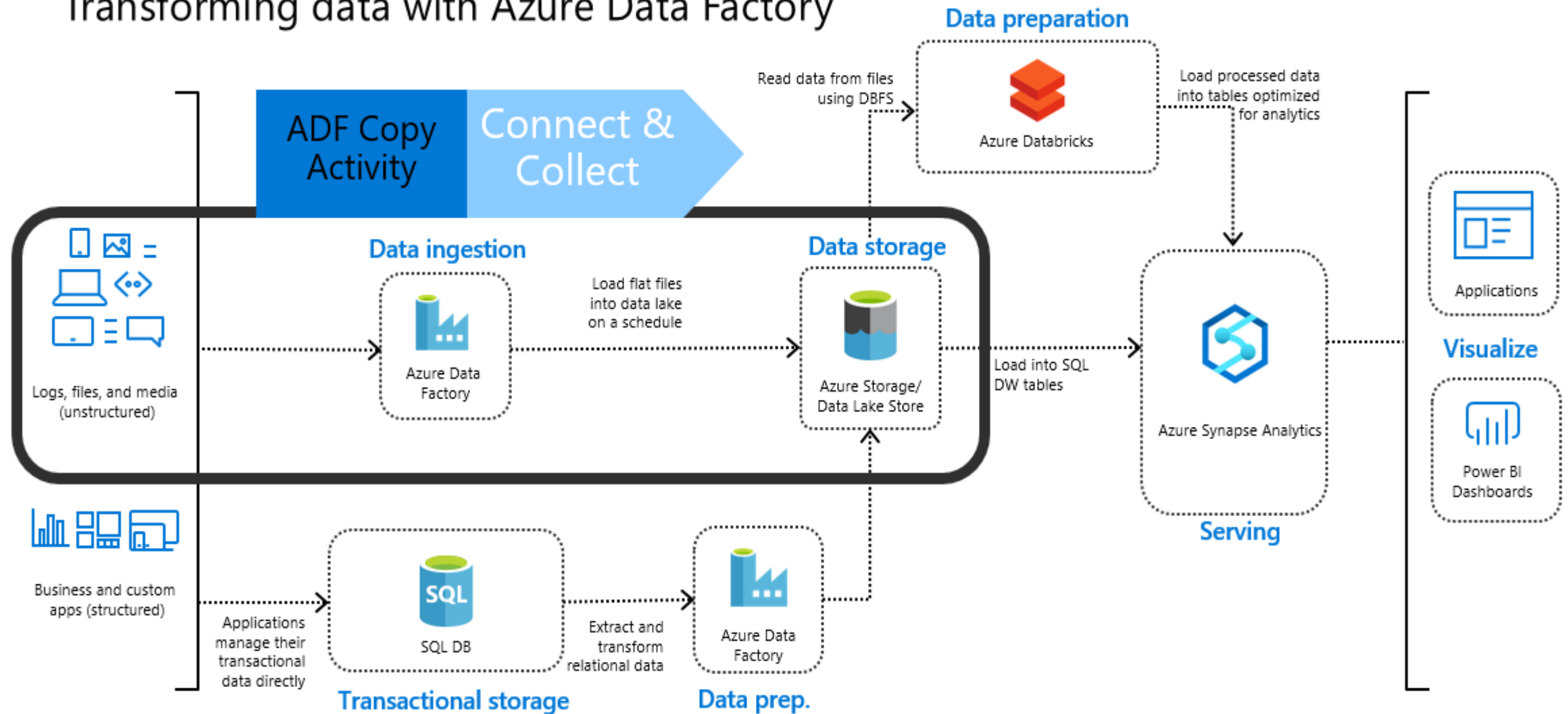
Cloud Data Warehouse Architecture

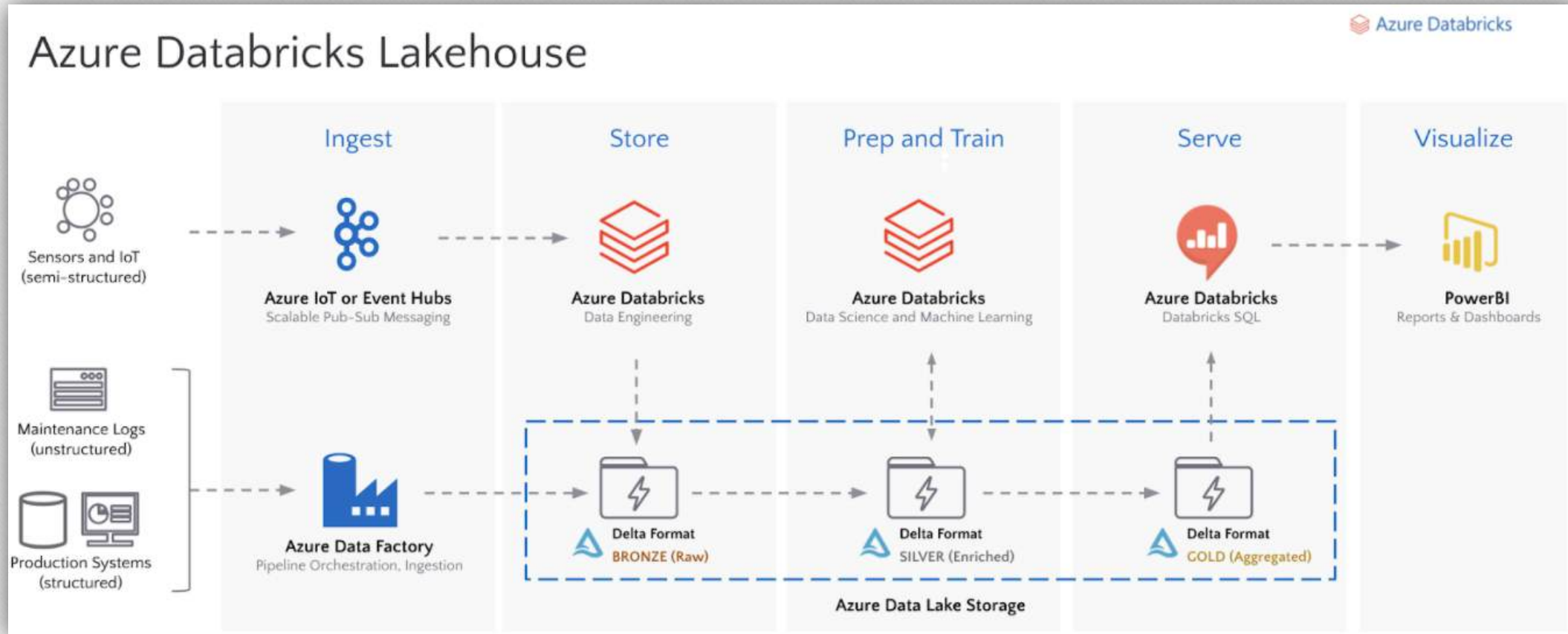


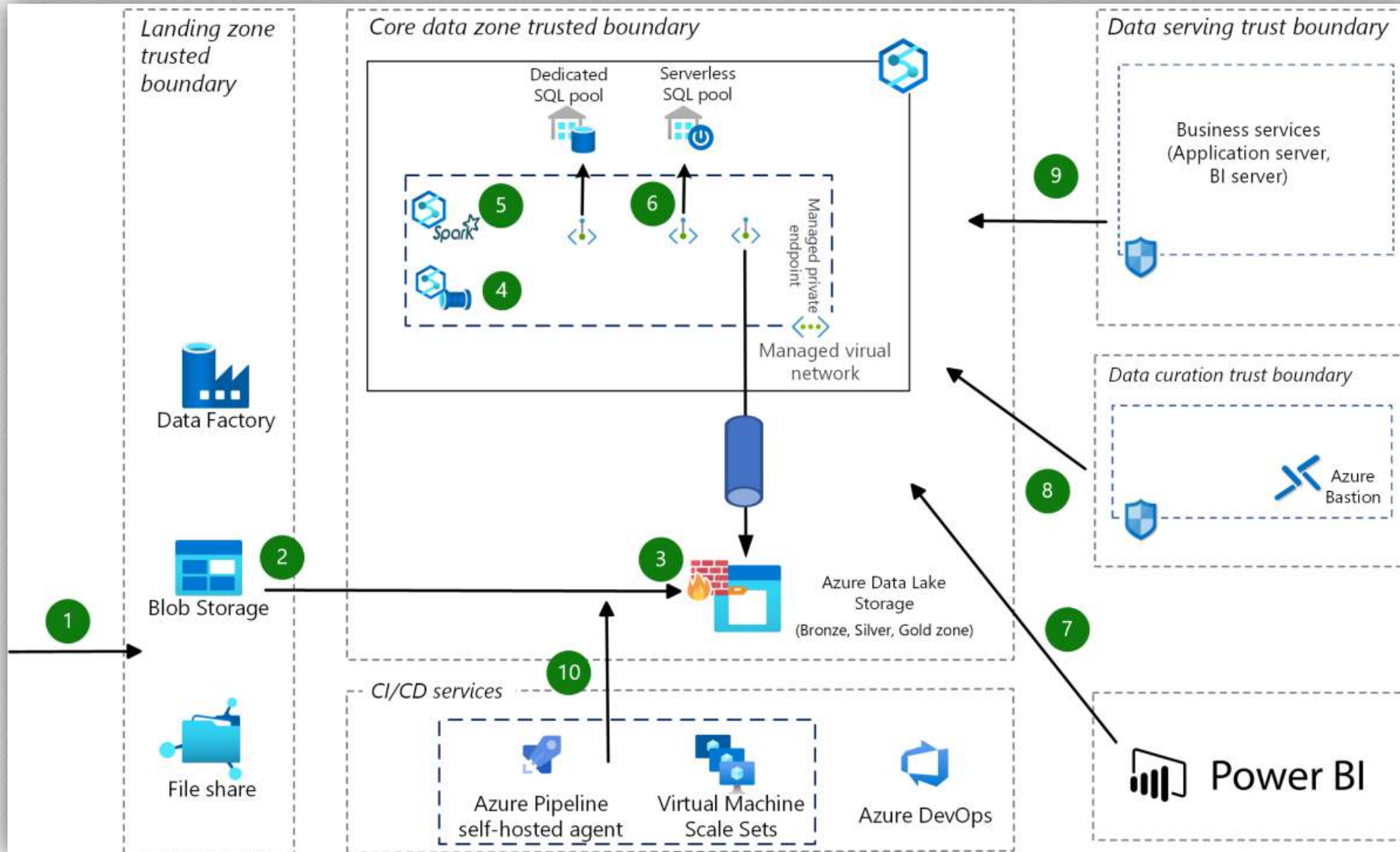
Cloud Big Data + Data Warehouse Architecture

Data transformation in Azure

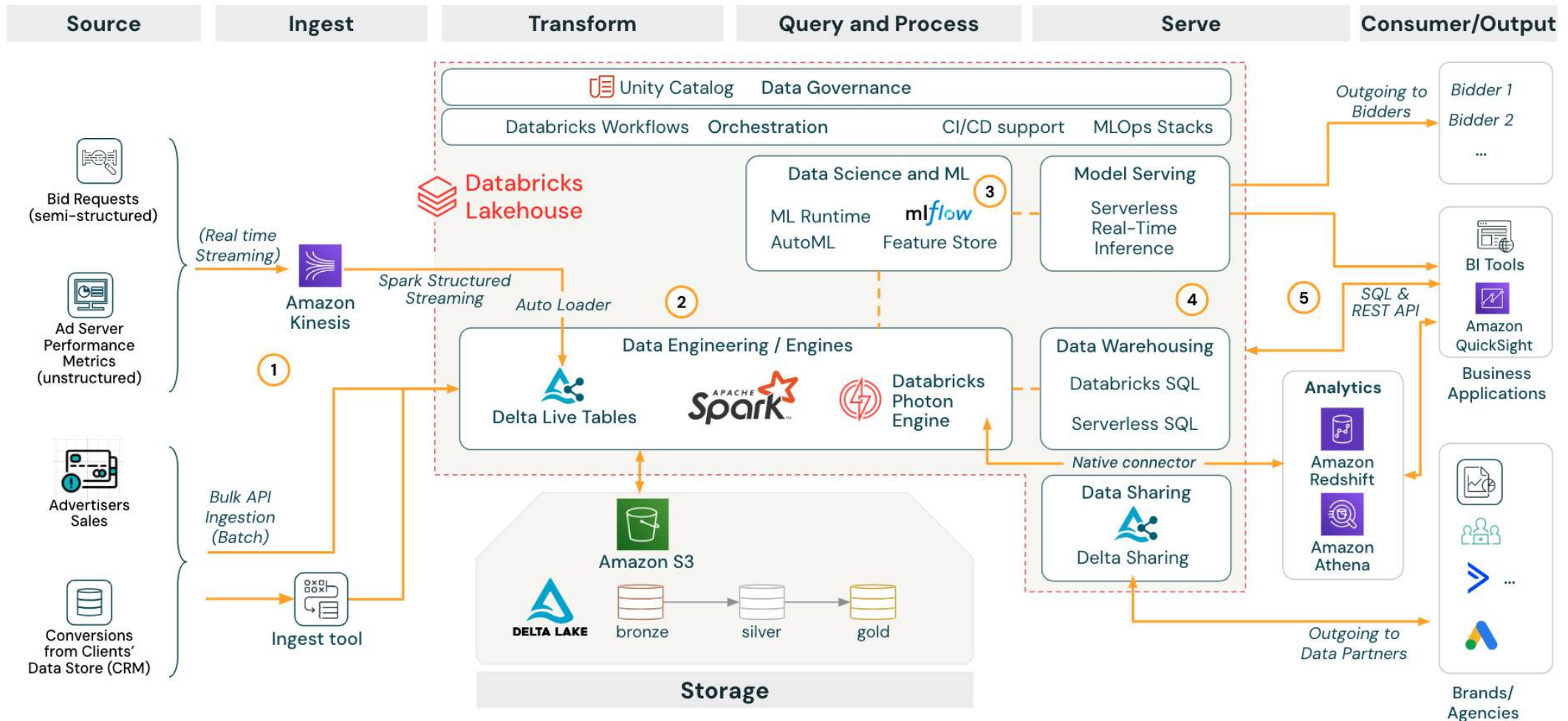
Transforming data with Azure Data Factory



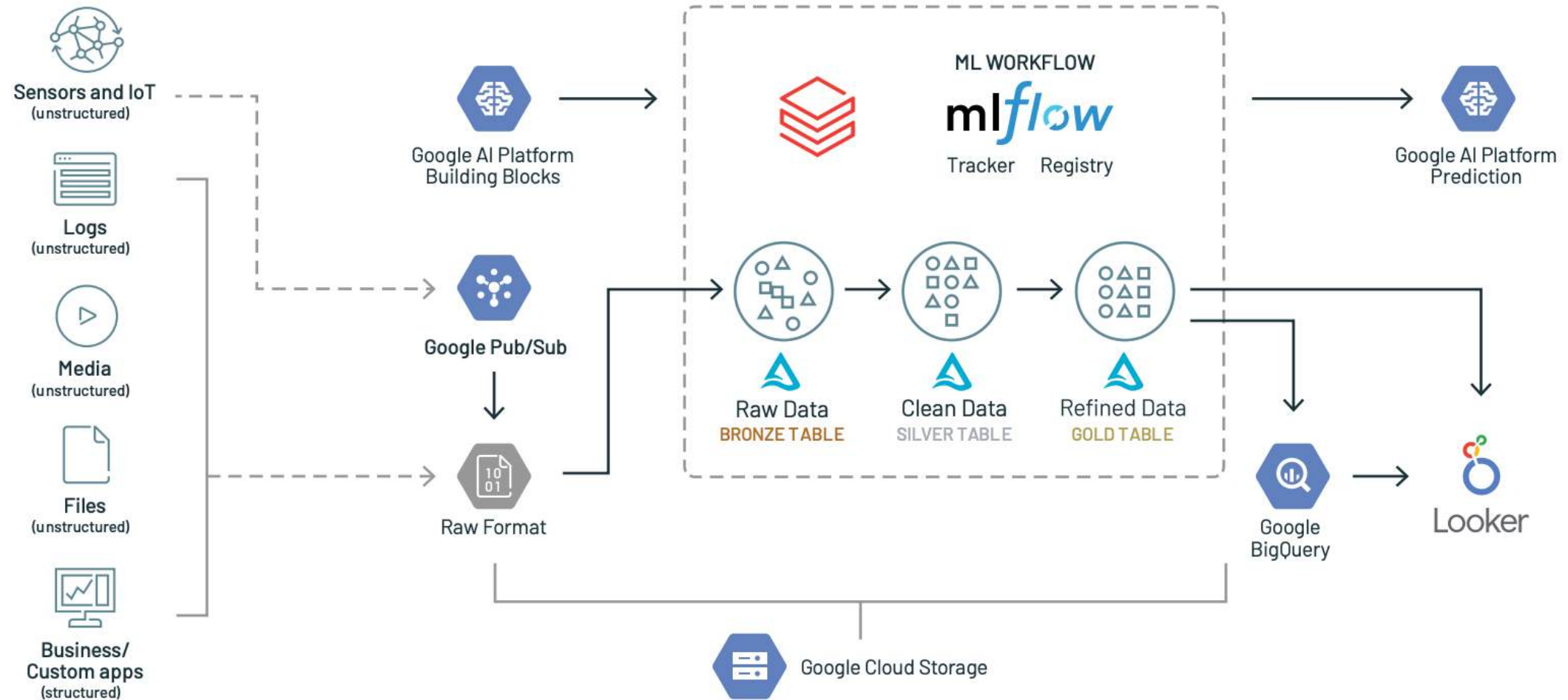




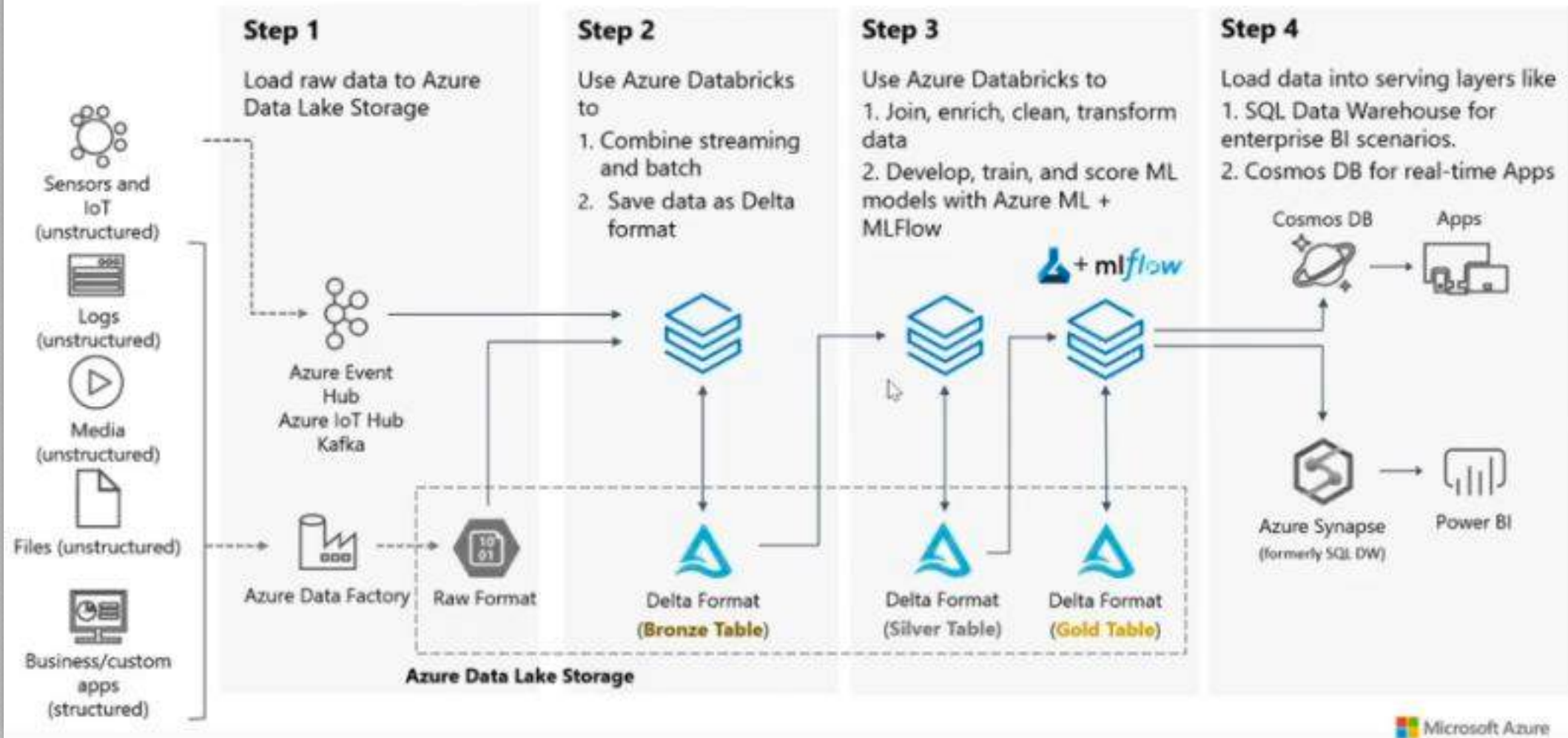
Databricks Lakehouse on AWS for Real Time Bidding Platforms



REFERENCE ARCHITECTURE



Azure Databricks – Delta Lake at Scale on Azure



THANK YOU